
ANALYST TO ARCHITECT · ABHISHEK TIWARI

Part IV

Machine Learning

From the math to an end-to-end project · Chapters 35-44

Part bundle · approved manuscript

25 September 2026
Every number checked

Contents

Chapter 35. The Math Under the Models

Why this matters

In plain English

35.1 Vectors: every customer is a point

35.2 The dot product: a number for "how aligned"

35.3 Matrices: predicting every row at once

35.4 Loss and the derivative: which way is downhill

35.5 Gradients: the slope in every direction

35.6 Gradient descent, by hand and in NumPy

35.7 Probability distributions: describing what's likely

35.8 Likelihood and maximum likelihood

35.9 Entropy, cross-entropy, and log loss

35.10 Principal component analysis (PCA)

35.11 Which math sits under which model

Common mistakes and how to spot them

In the real world: Meera and the 91% lead score

Tools

The project: gradient descent and PCA from scratch

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 36. The Machine Learning Workflow & Feature Engineering

Why this matters

In plain English

36.1 Framing the problem

36.2 One row per real enquiry

36.3 Train, validation, and test sets

36.4 Cross-validation

36.5 Data leakage

36.6 Feature engineering

36.7 Missing values in machine learning

36.8 scikit-learn pipelines

36.9 Baselines, and the final test

Common mistakes and how to spot them

In the real world: Vikram's 98% model

Tools

The project: a leakage-free feature pipeline for Riverstone lead scoring

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 37. Supervised Learning Algorithms

- Why this matters
- In plain English
- 37.0 The data for this chapter
- 37.1 Linear regression
- 37.2 Regularization: ridge, lasso, and elastic net
- 37.3 Logistic regression
- 37.4 k-nearest neighbors
- 37.5 Naive Bayes
- 37.6 Decision trees
- 37.7 Random forests
- 37.8 Gradient boosting
- 37.9 Support vector machines
- 37.10 Bias, variance, and learning curves
- 37.11 Hyperparameter tuning
- 37.12 Project result: lead scoring, baseline vs boosting
- Which algorithm when
- Common mistakes and how to spot them
- In the real world: "Should we buy the AutoML tool?"
- Tools
- The project: lead scoring, baseline vs boosting
- You've got it when...
- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 38. Unsupervised Learning

- Why this matters
- In plain English
- 38.0 The accounts, prepared
- 38.1 k-means
- 38.2 Choosing k, and whether the clusters are real
- 38.3 Profiling and naming clusters
- 38.4 Hierarchical clustering
- 38.5 DBSCAN
- 38.6 Seeing many dimensions: PCA, t-SNE, UMAP
- 38.7 Anomaly detection
- 38.8 Judging unsupervised results
- 38.9 Market basket analysis
- Common mistakes and how to spot them
- In the real world: the four segments that already existed
- Tools
- The project: customer segmentation and a product-bundle analysis
- You've got it when...
- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 39. Evaluation, Tuning, Interpretation & Honesty

Why this matters

In plain English

39.1 The confusion matrix

39.2 Curves: ROC and precision-recall

39.3 Regression metrics

39.4 Calibration: are the probabilities honest?

39.5 Choosing the threshold by business cost

39.6 Imbalanced data

39.7 Interpretation: SHAP and partial dependence

39.8 Fairness checks

39.9 Model cards

Common mistakes and how to spot them

In the real world: the rep who scored lower

Tools

The project: cost-based evaluation and explanation of the lead-scoring model

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 40. Time Series & Forecasting

Why this matters

In plain English

40.1 What a time series is made of

40.2 Stationarity

40.3 Baselines that are hard to beat

40.4 Exponential smoothing

40.5 ARIMA and SARIMA

40.6 Machine learning for forecasting

40.7 Backtesting

40.8 Forecast accuracy: MAPE, WAPE, MASE

40.9 Demand forecasting for a manufacturer

40.10 Anomaly detection in sensor data

Common mistakes and how to spot them

In the real world: the planner who already had a model

Tools

The project: forecast Riverstone's monthly demand by product category

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 41. NLP Foundations

Why this matters

In plain English

41.1 Text as data

41.2 Cleaning and tokenization

41.3 Bag of words and TF-IDF

- 41.4 Text classification
- 41.5 Sentiment analysis
- 41.6 Topic modeling
- 41.7 Word embeddings: the bridge to LLMs
- Common mistakes and how to spot them
- In the real world: the topics nobody had named
- Tools
- The project: classify and summarize Riverstone support tickets by topic
- You've got it when...
- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 42. Recommender Systems & Ranking

- Why this matters
- In plain English
- 42.1 The data, and a popularity baseline
- 42.2 Item-based collaborative filtering
- 42.3 Evaluating recommendations: precision@k and NDCG@k
- 42.4 Matrix factorization and implicit feedback
- 42.5 Content-based filtering
- 42.6 The cold-start problem
- 42.7 B2B cross-selling and ranking by segment
- Common mistakes and how to spot them
- In the real world: the recommendation that didn't need a model
- Tools
- The project: "customers who bought this also bought" for Riverstone
- You've got it when...
- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 43. A First Look at Deep Learning

- Why this matters
- In plain English
- 43.1 A neuron is logistic regression
- 43.2 Why one neuron isn't enough
- 43.3 A full forward pass, by hand
- 43.4 Backpropagation: the chain rule, automated
- 43.5 A network on tabular data
- 43.6 A first image classifier
- 43.7 Transfer learning
- 43.8 When to reach for deep learning
- Common mistakes and how to spot them
- In the real world: the deep learning pitch that needed a baseline
- Tools
- The project: a tabular network and an image classifier, evaluated honestly
- You've got it when...

- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 44. Capstone: An End-to-End Data Science Project

- Why this matters
- In plain English
- 44.1 The question, and the plan
- 44.2 Refitting the model (a recap, not a re-teach)
- 44.3 From probability to rupees
- 44.4 Building the call list: probability isn't the same as value
- 44.5 Explaining two accounts
- 44.6 One function, not six steps
- 44.7 The one-page summary
- 44.8 The Part IV toolkit, applied
- Common mistakes and how to spot them
- In the real world: what happened to the call list
- Tools
- The project: your own end-to-end pass
- You've got it when...
- Recap
- Practice exercises
- Key terms
- Where this leads
- Answers to practice exercises

Chapter 35. The Math Under the Models

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: treat each row of a dataset as a vector and measure distance and similarity between rows · multiply a matrix of features by a vector of weights to predict every row at once · read a derivative and a gradient as "which way is downhill, and how steep" · run gradient descent by hand for three steps, then in NumPy, and spot a bad learning rate or unscaled feature from the loss alone · describe data with the Bernoulli, binomial, Poisson, and normal distributions, and say when a distribution doesn't fit · find a maximum likelihood estimate and connect it to the loss a classifier minimizes · calculate entropy, cross-entropy, and log loss, and use them to judge a probability score · work through principal component analysis (PCA) on a small example by hand, then on all Riverstone customers.

Before you start: Chapter 4 (percentages, averages, compounding), Chapter 13 (the one-year database and the `sales_lines` view), Chapter 17 (running Python code, lists, loops), and Chapter 21 (mean, standard deviation, and basic probability). No calculus or linear algebra is assumed.

Time needed: 10–14 hours of reading and practice, spread over two weeks. Do the hand calculations with a pen before you run the code.

Tools: Python 3 with NumPy and pandas (Chapter 17 covers installation); scikit-learn for two cross-checks. A calculator or spreadsheet for the hand work.

Practice data: three small files built from the one-year Riverstone database (`riverstone_2025`) by `companion/ch35/make_ch35_data.py` : `customers_2025.csv` , `orders_2025.csv` , and `leads_2025.csv` . Every number in this chapter was calculated, and every output shown is real.

Why this matters

You can train a machine learning model in three lines of Python. The trouble starts when it misbehaves, and it will:

- The model's error stops falling after a few rounds of training, or shoots up to a number with 80 digits.
- A colleague's clustering puts every customer in the same group, because one column is measured in rupees and the rest in counts.
- A vendor tells your sales head their lead score is "91% accurate", and nobody in the room knows whether that is good.
- Two models have the same accuracy, and someone asks which to trust.
- An interviewer asks, "Why do we use log loss for logistic regression and not squared error?"

Each of these is a math question in disguise. You don't need proofs to answer them. You need six ideas, understood well enough to calculate by hand: **vectors**, **matrices**, **gradients**, **probability distributions**, **likelihood**, and **entropy**. This chapter teaches those six with Riverstone's numbers, one worked calculation at a time. After it, every algorithm in Chapters 36 to 43 will look like a variation on one theme, and you'll be able to reason about a model instead of guessing.

In plain English

Think of a tailor who makes uniforms for Riverstone's customers.

The tailor writes each customer's measurements on a card: chest, waist, sleeve, height. That list of numbers, always in the same order, is a **vector**. A drawer full of cards, one per customer, is a **matrix**. To compare two customers, the tailor lines their cards up side by side and sees how closely the measurements agree; that comparison is a **dot product**.

The tailor cuts a pattern, the customer tries it on, and the tailor notes how badly it fits: one number for "how wrong". That number is the **loss**. The tailor doesn't start over each time. They ask, *"If I let out the waist a little, does the fit get better or worse, and how quickly?"* That question is the **derivative**, and asking it for every seam at once gives the **gradient**. Adjusting every seam a small amount in the better direction, trying it on again, and repeating is **gradient descent**. Adjust too boldly and the jacket swings from too tight to too loose and never fits; that's a **learning rate** set too high.

When a new customer walks in and the tailor has no card yet, they guess the size from experience of who usually walks in. That's a **probability distribution**. Choosing the guess that makes the customers they've already seen least surprising is **maximum likelihood**, and the tailor's surprise when a guess is wrong is **cross-entropy**.

Finally, the tailor notices that chest, waist, and sleeve mostly move together, so a single "size" (S, M, L) captures most of what the card says. Finding that one summary direction is **principal component analysis**.

Vectors, matrices, gradients, distributions, likelihood, entropy, PCA: that's the whole chapter.

35.1 Vectors: every customer is a point

A row is a list of numbers

"How do I describe a customer so a computer can compare them with another?" Pick some measurements and write them in a fixed order. Here are two Riverstone customers from 2025, described by the number of orders they placed and their net revenue in ₹ lakh (₹100,000):

Customer	Orders	Revenue (₹ lakh)
Sharma Hardware	16	5.03
Metro Mart	16	3.29

As vectors: **Sharma** = [16, 5.03] and **Metro** = [16, 3.29]. A **vector** is an ordered list of numbers. Each number is a **component**, and the number of components is the vector's **dimension**. These are 2-dimensional vectors, so you can draw each customer as a point on a chart with orders across and revenue up. A customer described by seven measurements is a point in 7-dimensional space. You can't draw that, but the arithmetic is the same.

In machine learning, each component is usually called a **feature**. A model never sees "Sharma Hardware"; it sees the vector.

Distance between two customers

"How different are Sharma Hardware and Metro Mart?" Treat them as points and measure the straight-line distance, the **Euclidean distance**. Subtract component by component, square, add, and take the square root:

$$\text{distance} = \sqrt{(16 - 16)^2 + (5.03 - 3.29)^2} = \sqrt{0 + 1.74^2} = \sqrt{3.0276} = \mathbf{1.74}$$

The **length** (or **norm**) of a single vector is its distance from zero. For Sharma: $\sqrt{16^2 + 5.03^2} = \sqrt{256 + 25.30} = \sqrt{281.30} = \mathbf{16.77}$.

Here is the same arithmetic in NumPy, the Python library for working with arrays of numbers:

```
import numpy as np
import pandas as pd

sharma = np.array([16, 5.03])
metro = np.array([16, 3.29])

difference = sharma - metro
distance = np.sqrt(np.sum(difference ** 2))
print("difference:", difference)
print("distance: ", round(distance, 2))
print("same with np.linalg.norm:", round(np.linalg.norm(sharma - metro), 2))
print("length of sharma:", round(np.linalg.norm(sharma), 2))
```

```
difference: [0.  1.74]
distance:    1.74
same with np.linalg.norm: 1.74
length of sharma: 16.77
```

How it works:

- `np.array([16, 5.03])` makes a vector. Arithmetic on arrays happens **element by element**: `sharma - metro` subtracts 16 from 16 and 3.29 from 5.03 in one step, with no loop.
- `difference ** 2` squares each component; `np.sum` adds them; `np.sqrt` takes the root. That's the formula, line for line.
- `np.linalg.norm` does the same calculation in one call. Use it in real work, but know what it does.

Watch out: units decide distance. Measure revenue in rupees instead of lakh and Sharma becomes [16, 502,775]. Now a difference of 1 order counts as much as a difference of ₹1, so distance is driven almost entirely by revenue, and orders might as well not exist. Any method built on distance (k-nearest neighbors, k-means clustering, PCA) is affected. The fix is **scaling**: put every feature on a comparable scale before measuring distance. Section 35.10 shows the damage with real numbers, and Chapter 36 turns scaling into a standard step.

The whole customer table as vectors

Load the practice file and look at a few customers. Each row is a vector of features:

```
customers = pd.read_csv("customers_2025.csv")
print(len(customers), "customers who ordered in 2025")
view = customers[["customer_name", "orders", "revenue", "storage_share",
                  "kitchen_share", "industrial_share", "furniture_share"]]
view.columns = ["customer", "orders", "revenue", "storage", "kitchen", "industrial", "furniture"]
print(view.head(5).to_string(index=False))
```

```
23 customers who ordered in 2025
   customer  orders  revenue  storage  kitchen  industrial  furniture
Sharma Hardware      16   502775    0.656    0.310      0.000     0.034
Patel Kitchenware      7   174148    0.414    0.492      0.000     0.094
Green Leaf Hotels     17   331389    0.386    0.614      0.000     0.000
Coastal Foods        12   323631    0.481    0.000      0.519     0.000
Metro Mart           16   329298    0.726    0.274      0.000     0.000
```

The four share columns (named `storage_share`, `kitchen_share`, and so on in the file, shortened here to fit the page) give each customer's revenue split across Riverstone's four product categories. Sharma Hardware's split is 65.6% Storage, 31.0% Kitchen, 0% Industrial, and 3.4% Furniture. Check: $0.656 + 0.310 + 0.000 + 0.034 = 1.000$. ✓ That split is a 4-dimensional vector, and it's the one the next section uses to ask which customers *buy like* each other.

35.2 The dot product: a number for "how aligned"

Multiply and add

"Which customers buy the same mix of products as Sharma Hardware?" That question isn't about size. A small café and a large hotel chain can buy exactly the same mix. You need a measure of **direction**, not distance.

The tool is the **dot product**: multiply two vectors component by component, then add the results. Using product-mix vectors in the order [Storage, Kitchen, Industrial, Furniture]:

	Storage	Kitchen	Industrial	Furniture
Sharma Hardware	0.656	0.310	0.000	0.034
Metro Mart	0.726	0.274	0.000	0.000
Coastal Foods	0.481	0.000	0.519	0.000

$\text{Sharma} \cdot \text{Metro} = 0.656 \times 0.726 + 0.310 \times 0.274 + 0.000 \times 0.000 + 0.034 \times 0.000 = 0.476256 + 0.084940 + 0 + 0 = \mathbf{0.561196}$

$\text{Sharma} \cdot \text{Coastal} = 0.656 \times 0.481 + 0.310 \times 0.000 + 0.000 \times 0.519 + 0.034 \times 0.000 = \mathbf{0.315536}$

The dot product is large when two vectors have big numbers **in the same places**. Sharma and Metro both put most of their money into Storage and Kitchen, so their products line up. Coastal Foods spends half its money on Industrial Crates, where Sharma spends nothing, so that half contributes zero.

Spreadsheet link. You have already met the dot product: it's `SUMPRODUCT`. With Sharma's shares in B2:E2 and Metro's in B3:E3, `=SUMPRODUCT(B2:E2, B3:E3)` returns 0.561196 in both Excel and Google Sheets. Chapter 4's weighted average discount was a dot product too.

Cosine similarity: removing size

A raw dot product still grows when vectors get longer. To measure direction alone, divide by both lengths. The result is the **cosine similarity**, the cosine of the angle between the two vectors:

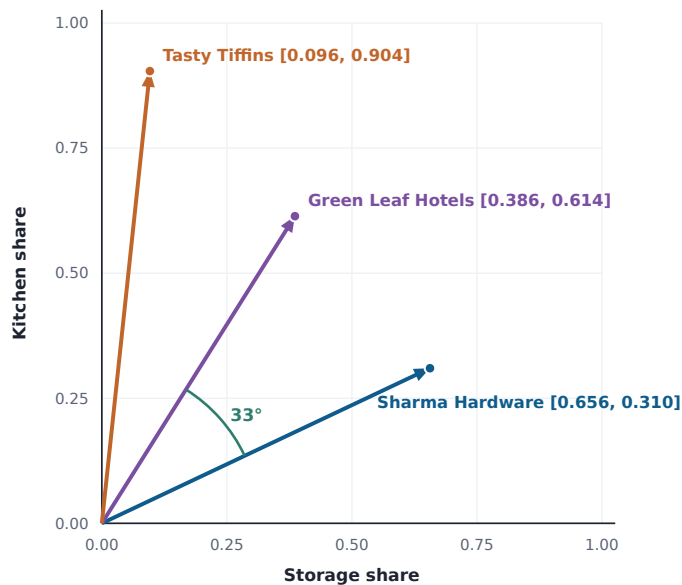
$$\text{cosine similarity} = (a \cdot b) \div (\text{length of } a \times \text{length of } b)$$

It runs from -1 (opposite directions) through 0 (at right angles, nothing in common) to 1 (pointing the same way). Revenue shares can't be negative, so here it runs from 0 to 1.

Work it for Sharma and Metro. Sharma's length is $\sqrt{(0.656^2 + 0.310^2 + 0^2 + 0.034^2)} = \sqrt{(0.430336 + 0.0961 + 0 + 0.001156)} = \sqrt{0.527592} = 0.72636$. Metro's is $\sqrt{(0.726^2 + 0.274^2)} = \sqrt{(0.527076 + 0.075076)} = \sqrt{0.602152} = 0.77598$. So:

$$\text{cosine(Sharma, Metro)} = 0.561196 \div (0.72636 \times 0.77598) = 0.561196 \div 0.563640 = \mathbf{0.9957}$$

For Sharma and Coastal, Coastal's length is $\sqrt{(0.481^2 + 0.519^2)} = 0.70762$, and the cosine is $0.315536 \div (0.72636 \times 0.70762) = \mathbf{0.6139}$. Sharma buys very much like Metro Mart and only partly like Coastal Foods.



Reading the picture

Each arrow is one customer's mix.
Its direction says what they buy;
its length doesn't matter for similarity.

Angle Sharma → Green Leaf: 33°
cosine in these 2 dimensions = 0.843

Angle Sharma → Tasty Tiffins: 59°
cosine in these 2 dimensions = 0.520

Smaller angle, higher cosine, more alike.

Industrial and Furniture shares are zero for these three customers, apart from Sharma's 3.4% Furniture.

Figure 35.1 – Product mixes as arrows (the Storage and Kitchen components only, two of the four). The smaller the angle between two arrows, the higher their cosine similarity, whatever their length.

Now let NumPy compare Sharma with all 23 customers at once:

```
share_cols = ["storage_share", "kitchen_share", "industrial_share", "furniture_share"]
mix = customers[share_cols].to_numpy()
names = customers["customer_name"].to_numpy()

sharma_mix = mix[0]
dots = mix @ sharma_mix
lengths = np.linalg.norm(mix, axis=1)
cosine = dots / (lengths * np.linalg.norm(sharma_mix))

ranking = pd.Series(cosine, index=names).sort_values(ascending=False)
print("Most similar mix to Sharma Hardware:")
print(ranking.head(5).round(4).to_string())
print("\nLeast similar:")
print(ranking.tail(3).round(4).to_string())
```

Most similar mix to Sharma Hardware:

Sharma Hardware	1.0000
Royal Banquets	0.9988
Evergreen Mart	0.9963
Metro Mart	0.9957
Om Sai Provisions	0.9873

Least similar:

Harbour Traders	0.5897
Tasty Tiffins	0.5198
Deccan Packaging	0.4676

How it works:

- `mix` is a 23×4 table of numbers: one row per customer, one column per category. `mix[0]` is the first row, Sharma Hardware.

- `@` is Python's operator for matrix multiplication. `mix @ sharma_mix` takes the dot product of **every row** with Sharma's vector, giving 23 numbers in one step. Section 35.3 explains why this works.
- `np.linalg.norm(mix, axis=1)` computes the length of each row (`axis=1` means "across the columns of each row").
- Dividing gives the 23 cosine similarities. Sharma's similarity with itself is 1, as it must be. ✓

Reading it. The customer whose mix most resembles Sharma Hardware's is not another hardware retailer but **Royal Banquets**, a hospitality customer in Lucknow. Segment labels and buying behavior are different things. The least similar are two wholesalers who spend heavily on Industrial Crates, and Tasty Tiffins, which buys almost nothing but kitchenware. This exact calculation, on real purchase histories, is the engine of the "customers who bought this also bought" recommender you build in Chapter 42, and dot products between word vectors are how the language models in Chapter 54 measure meaning.

35.3 Matrices: predicting every row at once

A dataset is a matrix

A **matrix** is a rectangular grid of numbers with rows and columns. Its **shape** is written rows × columns. `mix` above is a 23 × 4 matrix. In machine learning the feature matrix is traditionally called **X** (capital, because it's a matrix) and the thing you want to predict is **y** (lowercase, a vector with one value per row).

Matrix times vector

"If every order were worth the average, what revenue would each customer have brought in?" In 2025, Riverstone's 173 orders brought in ₹4,335,471, an average of ₹4,335,471 ÷ 173 = ₹25,060.53 per order. A very simple model says:

$$\text{predicted revenue} = ₹25,060.53 \times \text{orders} + ₹0$$

The two numbers the model uses, ₹25,060.53 and ₹0, are its **parameters** or **weights**: a slope w and an intercept b . To apply them to all 23 customers at once, write each customer's row as `[1, orders]`. The 1 is there so the intercept has something to multiply. Stack the rows into a matrix and put the weights in a vector `[b, w] = [0, 25,060.53]`. Then every prediction is the dot product of a row with the weight vector:

$$\text{Sharma: } [1, 16] \cdot [0, 25,060.53] = 1 \times 0 + 16 \times 25,060.53 = \text{₹400,968}$$

Matrix-times-vector means exactly this: take the dot product of each row of the matrix with the vector, and collect the answers. The number of columns in the matrix must equal the length of the vector (2 and 2 here), and the result has one number per row.

```

X = np.column_stack([np.ones(len(customers)), customers["orders"]])
y = customers["revenue"].to_numpy()
weights = np.array([0.0, 4335471 / 173]) # [intercept b, slope w]

predicted = X @ weights
print("X:", X.shape, " weights:", weights.shape, " predicted:", predicted.shape)
print("first three rows of X:\n", X[:3])
result = pd.DataFrame({"customer": names, "orders": customers["orders"],
                      "actual": y, "predicted": predicted.round(0).astype(int)})
result["error"] = result["predicted"] - result["actual"]
print(result.head(4).to_string(index=False))
print("total actual: ", y.sum())
print("total predicted:", round(predicted.sum()))

```

```

X: (23, 2) weights: (2,) predicted: (23,)
first three rows of X:
[[ 1. 16.]
 [ 1.  7.]
 [ 1. 17.]]

```

	customer	orders	actual	predicted	error
	Sharma Hardware	16	502775	400968	-101807
	Patel Kitchenware	7	174148	175424	1276
	Green Leaf Hotels	17	331389	426029	94640
	Coastal Foods	12	323631	300726	-22905
	total actual:		4335474		
	total predicted:		4335471		

How it works:

- `np.column_stack` glues columns side by side: a column of 23 ones and the column of orders, making the 23×2 matrix **X**.
- `X @ weights` multiplies a (23×2) matrix by a vector of length 2 and returns a vector of length 23: one prediction per customer.
- The **error** (prediction minus actual) is negative when the model under-predicts. Sharma Hardware is under-predicted by ₹101,807, because its orders are bigger than average.

Reconcile. The predictions add up to ₹4,335,471, exactly Riverstone's 2025 net revenue. ✓ That isn't luck: the slope is total revenue ÷ total orders, so multiplying back by total orders returns the total. (The `actual` column adds up to ₹4,335,474 because the practice file rounds each customer's revenue to the nearest rupee; three rupees of rounding across 23 customers.)

Spreadsheet link. `=MMULT(A2:B24, D2:D3)` multiplies a 23×2 range by a 2×1 range in both Excel and Google Sheets, returning 23 predictions (entered as a dynamic array in Microsoft 365 and Sheets). It's the same `@`.

Two more matrix words

- The **transpose** of a matrix, written **X^s** (`X.T` in NumPy), flips rows into columns. A 23×2 matrix becomes 2×23 . You'll see it in the gradient formula in section 35.5, and in PCA.
- **Mean-centering** subtracts each column's average from that column, so every column averages zero. PCA starts with it (section 35.10).

That's enough linear algebra for this part of the book. A neural network layer (Chapter 43) is this same $x @ weights$ with a bigger weight matrix, followed by a small twist.

35.4 Loss and the derivative: which way is downhill

A tiny dataset to learn on

The best way to understand how a model learns is to watch one learn from data small enough to check by hand. Take four Riverstone customers, with orders as the feature x and revenue in ₹ thousand as the target y :

Customer	Orders (x)	Revenue, ₹ thousand (y)
Om Sai Provisions	4	66
Patel Kitchenware	7	174
Evergreen Mart	8	198
Fresh Bowl Kitchens	12	297

(Revenue is rounded to the nearest thousand: Om Sai's ₹66,399 becomes 66. Small whole numbers keep the hand calculations readable.)

Start with the simplest possible model, a line through zero with one weight: **predicted revenue** = $w \times$ **orders**. The question is which w fits best.

The loss: one number for "how wrong"

For any w , you can measure how wrong the model is. The usual measure for predicting a number is the **mean squared error (MSE)**: for each row, take the error (prediction minus actual), square it, then average.

$$\text{loss}(w) = \text{average of } (w \times x - y)^2$$

Try $w = 20$ (₹20 thousand per order):

Customer	x	y	prediction $20x$	error	error ²
Om Sai	4	66	80	14	196
Patel	7	174	140	-34	1,156
Evergreen	8	198	160	-38	1,444
Fresh Bowl	12	297	240	-57	3,249

$$\text{loss}(20) = (196 + 1,156 + 1,444 + 3,249) \div 4 = 6,045 \div 4 = 1,511.25$$

A **loss function** turns "how good is this model?" into a single number where lower is better. Squaring does two jobs: it stops positive and negative errors cancelling out, and it punishes big misses much more than small ones (an error of 57 costs 3,249, more than an error of 14 does sixteen times over).

Training a model means finding the parameter values that make the loss as small as possible.

Everything else in this section is about how to find them without trying every value.

The derivative: the slope of the loss

Plot the loss for every w and you get a valley. You're standing at $w = 20$ and want to walk downhill. You need to know which direction is down, and how steep it is. That's what a **derivative** tells you: how much the loss changes for a tiny change in w . A negative derivative means increasing w lowers the loss; a positive one means increasing w raises it; zero means you're at the bottom (or a flat spot).

You can measure it without any calculus. Nudge w by a tiny amount and see what happens to the loss:

```
x = np.array([4, 7, 8, 12], dtype=float)
y = np.array([66, 174, 198, 297], dtype=float)

def loss(w):
    return np.mean((w * x - y) ** 2)

h = 0.001
print("loss(20)          =", round(loss(20), 4))
print("loss(20.001)     =", round(loss(20 + h), 4))
print("measured slope =", round((loss(20 + h) - loss(20)) / h, 2))
```

```
loss(20)          = 1511.25
loss(20.001)     = 1510.6651
measured slope = -584.93
```

Raising w by 0.001 lowered the loss by about 0.585, so the slope is about -585 per unit of w . Negative, so w should go **up**.

Calculus gives the exact slope as a formula. For mean squared error with one weight, the derivative is:

derivative of loss with respect to $w = 2 \times \text{average of } (\text{error} \times x)$

The error is multiplied by x because a weight's effect on a prediction is proportional to the feature it multiplies: nudging w moves Fresh Bowl's prediction (12 orders) three times as much as Om Sai's (4 orders). Check it at $w = 20$ using the error column above:

$$2 \times (14 \times 4 + (-34) \times 7 + (-38) \times 8 + (-57) \times 12) \div 4 = 2 \times (56 - 238 - 304 - 684) \div 4 = 2 \times (-1,170) \div 4 = -585 \checkmark$$

The formula and the nudge agree. (The nudge gave -584.93 because 0.001 is small but not zero.)

Simplification note. You don't need to derive derivative formulas yourself. Libraries such as scikit-learn and PyTorch calculate them for you (PyTorch does it automatically for any model you write, which Chapter 43 shows). What you need is the meaning: *the derivative is the local slope of the loss, and its sign says which way to move.*

At the bottom of the valley the slope is zero. For this one-weight model you can solve for that point directly: $w = (\text{sum of } x \times y) \div (\text{sum of } x^2) = 6,630 \div 273 = \mathbf{24.29}$, about ₹24,290 of revenue per order.

35.5 Gradients: the slope in every direction

A real model has many parameters. Give the line an intercept, **predicted revenue** = $w \times \text{orders} + b$, and the loss now depends on two numbers. The loss surface is a bowl instead of a curve, and "downhill" has a direction on a map.

A **partial derivative** is the slope in one parameter's direction, holding the others still. The **gradient** is the vector of all the partial derivatives: one component per parameter. It points in the direction of steepest **increase** in the loss, so you walk the opposite way.

For mean squared error with w and b :

slope for $w = 2 \times \text{average of } (\text{error} \times x)$

slope for $b = 2 \times \text{average of } (\text{error})$

The intercept adds 1 to every prediction when it rises by 1, so its partial derivative has no x in it.

Work out the gradient at the start, $w = 0$ and $b = 0$. Every prediction is 0, so the errors are -66, -174, -198, and -297:

slope for $w = 2 \times (-66 \times 4 - 174 \times 7 - 198 \times 8 - 297 \times 12) \div 4 = 2 \times (-264 - 1,218 - 1,584 - 3,564) \div 4 = 2 \times (-6,630) \div 4 = \mathbf{-3,315}$

slope for $b = 2 \times (-66 - 174 - 198 - 297) \div 4 = 2 \times (-735) \div 4 = \mathbf{-367.5}$

The gradient is **[-3,315, -367.5]**. Both components are negative, so both w and b should increase. The w component is nine times bigger: the loss is far more sensitive to the slope than to the intercept, because w is multiplied by orders of 4 to 12 and b by 1. Remember that; it causes trouble in section 35.6.

Python link. With a feature matrix **X** that includes a column of ones, the whole gradient is one line: `2 * X.T @ (X @ weights - y) / len(y)`. `X @ weights - y` is the error vector, and `X.T @ errors` takes the dot product of each column (the ones, then the orders) with the errors. That's why the transpose appears in machine learning formulas.

35.6 Gradient descent, by hand and in NumPy

The update rule

Gradient descent repeats one move: compute the gradient, then step every parameter a small amount in the opposite direction.

new parameter = old parameter – learning rate × slope for that parameter

The **learning rate** is the size of the step, a setting you choose. A setting chosen by you rather than learned from the data is a **hyperparameter**. Use a learning rate of **0.005**.

Three steps by hand

Step 1. Start at $w = 0$, $b = 0$. The gradient (section 35.5) is $[-3,315, -367.5]$, and the loss is $(66^2 + 174^2 + 198^2 + 297^2) \div 4 = (4,356 + 30,276 + 39,204 + 88,209) \div 4 = 162,045 \div 4 = \mathbf{40,511.25}$.

new $w = 0 - 0.005 \times (-3,315) = \mathbf{16.575}$

new $b = 0 - 0.005 \times (-367.5) = \mathbf{1.8375}$

Step 2. Predict with $w = 16.575$, $b = 1.8375$. Om Sai: $16.575 \times 4 + 1.8375 = 68.14$, an error of $+2.14$. Patel: $16.575 \times 7 + 1.8375 = 117.86$, error -56.14 . Evergreen: 134.44 , error -63.56 . Fresh Bowl: 200.74 , error -96.26 . The loss has fallen to **4,115.66**. The slopes are $2 \times (2.14 \times 4 - 56.14 \times 7 - 63.56 \times 8 - 96.26 \times 12) \div 4 = \mathbf{-1,024.03}$ for w and $2 \times (2.14 - 56.14 - 63.56 - 96.26) \div 4 = \mathbf{-106.91}$ for b (to two decimals; the code below carries full precision).

new $w = 16.575 + 0.005 \times 1,024.03 = \mathbf{21.695}$

new $b = 1.8375 + 0.005 \times 106.91 = \mathbf{2.372}$

Step 3. Predictions 89.15, 154.24, 175.93, 262.71; errors 23.15, -19.76 , -22.07 , -34.29 ; loss **647.26**; slopes -316.84 and -26.48 .

new $w = 21.695 + 0.005 \times 316.84 = \mathbf{23.279}$

new $b = 2.372 + 0.005 \times 26.48 = \mathbf{2.504}$

Step	w	b	Loss	Slope for w	Slope for b
1	0.000	0.000	40,511.25	$-3,315.00$	-367.50
2	16.575	1.838	4,115.66	$-1,024.03$	-106.91
3	21.695	2.372	647.26	-316.84	-26.48
after 3	23.279	2.504	316.36		

Three things to notice. The loss fell from 40,511 to 316 in three steps. The steps get **smaller on their own**, even though the learning rate is fixed, because the slope flattens as you approach the bottom. And *b* has barely moved.

The same three steps in NumPy

```
w, b = 0.0, 0.0
learning_rate = 0.005

for step in range(1, 4):
    error = (w * x + b) - y
    current_loss = np.mean(error ** 2)
    slope_w = 2 * np.mean(error * x)
    slope_b = 2 * np.mean(error)
    print(f"step {step}: w={w:7.3f} b={b:6.3f} loss={current_loss:10.2f} "
          f"slope_w={slope_w:9.2f} slope_b={slope_b:8.2f}")
    w = w - learning_rate * slope_w
    b = b - learning_rate * slope_b

print(f"after 3: w={w:7.3f} b={b:6.3f} loss={np.mean((w * x + b - y) ** 2):10.2f}")
```

```
step 1: w= 0.000 b= 0.000 loss= 40511.25 slope_w= -3315.00 slope_b= -367.50
step 2: w= 16.575 b= 1.838 loss= 4115.66 slope_w= -1024.03 slope_b= -106.91
step 3: w= 21.695 b= 2.372 loss= 647.26 slope_w= -316.84 slope_b= -26.48
after 3: w= 23.279 b= 2.504 loss= 316.36
```

How it works:

- Each pass through the loop does exactly what you did by hand: predict, find errors, compute the loss and both slopes, then update.
- `np.mean(error * x)` is "average of error $\times$ x", computed for all four customers at once.
- Printing *before* updating matches the hand table, where each row shows the parameters that produced that loss. Every number agrees with the table. ✓

Running it to the bottom

Let the same loop run for 1,000 steps, and compare with the exact best line. NumPy's `np.polyfit(x, y, 1)` finds the least-squares line directly, without gradient descent:

```
def descend(x_values, learning_rate, steps):
    w, b = 0.0, 0.0
    losses = []
    for _ in range(steps):
        error = (w * x_values + b) - y
        losses.append(np.mean(error ** 2))
        w -= learning_rate * 2 * np.mean(error * x_values)
        b -= learning_rate * 2 * np.mean(error)
    return w, b, losses

w, b, losses = descend(x, 0.005, 1000)
best_w, best_b = np.polyfit(x, y, 1)
best_loss = np.mean((best_w * x + best_b - y) ** 2)
print(f"gradient descent, 1000 steps: w={w:.2f} b={b:.2f} loss={losses[-1]:.2f}")
print(f"exact least squares: w={best_w:.2f} b={best_b:.2f} loss={best_loss:.2f}")
```

```
gradient descent, 1000 steps: w=27.12  b=-25.00  loss=109.48
exact least squares:          w=28.51  b=-37.21  loss=91.55
```

Reading it. After 1,000 steps the loss has fallen from 40,511 to 109.48, close to the best possible 91.55, but the parameters are further off than that suggests: w is 27.12 instead of 28.51, and b is -25.00 instead of -37.21 . The loss surface is a long, narrow valley. It's steep across the w direction and almost flat along the b direction, so the descent races down the steep wall and then creeps along the flat floor. A small loss doesn't guarantee the parameters have settled.

Watch out: a flat loss curve doesn't mean training has finished. If you judge convergence only by "the loss stopped changing much", you can stop with parameters that are still far from their best values. Here the intercept is off by ₹12,000. Check the parameters too, or better, fix the cause below.

The fix: scale the feature

The valley is narrow because orders (4 to 12) are much larger than the constant 1 that multiplies b . **Center** the feature by subtracting its average, 7.75 orders, and the two directions become equally steep. Now you can use a much bigger learning rate:

```
x_centered = x - x.mean()
w_c, b_c, losses_c = descend(x_centered, 0.05, 150)
print(f"centered, 150 steps: w={w_c:.2f}  b={b_c:.2f}  loss={losses_c[-1]:.2f}")
print(f"back on the original scale: b = {b_c - w_c * x.mean():.2f}")
```

```
centered, 150 steps: w=28.51  b=183.75  loss=91.55
back on the original scale: b = -37.21
```

With the feature centered, 150 steps land exactly on the least-squares answer: a slope of 28.51 and, converted back, an intercept of -37.21 . The intercept of the centered model, 183.75, has a plain meaning: the predicted revenue (₹183,750) for a customer with the average number of orders. About seven times fewer steps, and the right answer. This is why Chapter 36 treats scaling as a routine step before training.

What to tell the sales head. "Among these four customers, each extra order goes with about ₹28,500 of extra annual revenue." Don't read the intercept of -37.21 as "a customer with no orders brings in minus ₹37,210": no customer in the data has fewer than 4 orders, and a line is only trustworthy near the data it was fitted on. With four customers this is a teaching example, not a finding.

The learning rate

"How do I pick the learning rate?" Watch the loss. Run the original (uncentered) problem with three learning rates:

```
for lr in [0.0005, 0.005, 0.015]:
    _, _, run = descend(x, lr, 30)
    print(f"rate {lr:<6}: loss at steps 1-4 = "
          + ", ".join(f"{v:,.0f}" for v in run[:4])
          + f" ... step 30 = {run[-1]:,.0f}")
```

```
rate 0.0005: loss at steps 1-4 = 40,511, 35,141, 30,488, 26,456 ... step 30 = 913
rate 0.005 : loss at steps 1-4 = 40,511, 4,116, 647, 316 ... step 30 = 270
rate 0.015 : loss at steps 1-4 = 40,511, 46,679, 53,794, 61,999 ... step 30 = 2,521,040
```

- **0.0005 is too small.** The loss falls steadily but slowly: still 913 at step 30, more than three times the loss of the better rate.
- **0.005 is about right** for this problem. The loss falls from 40,511 to 316 in three steps and is at 270 by step 30.
- **0.015 is too large.** Each step overshoots the bottom of the valley and lands higher up the other side, so the loss **grows**. By step 30 it has passed 2.5 million, and it keeps climbing. A loss that shoots up to enormous numbers, or to `nan` ("not a number"), almost always means the learning rate is too high.

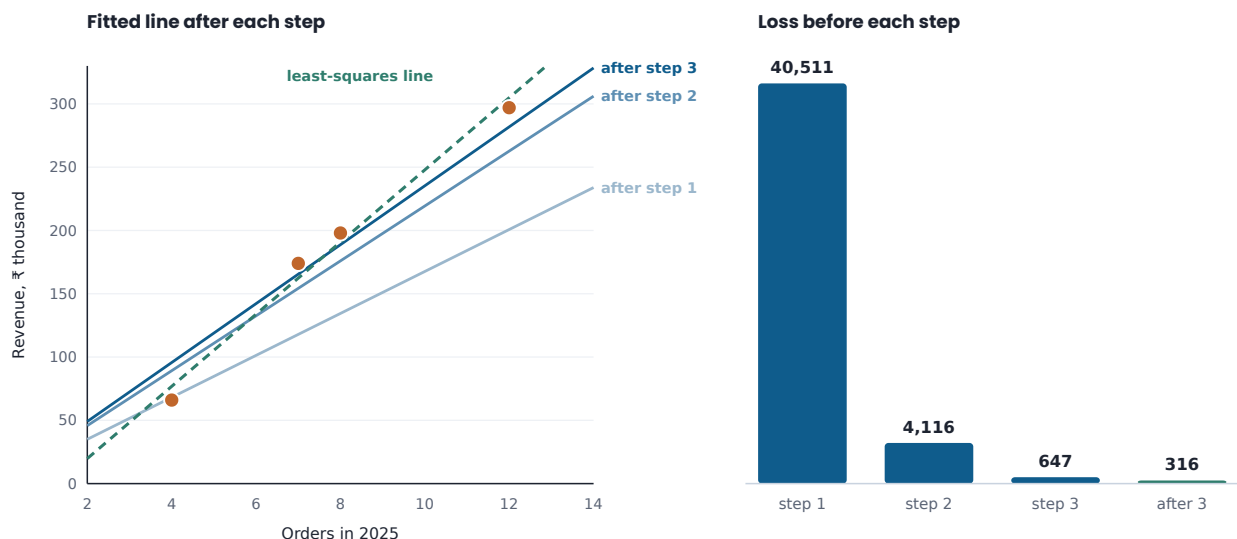


Figure 35.2 – Three steps of gradient descent by hand. Left: each step's line moves toward the least-squares line. Right: the loss before each step and after the third.

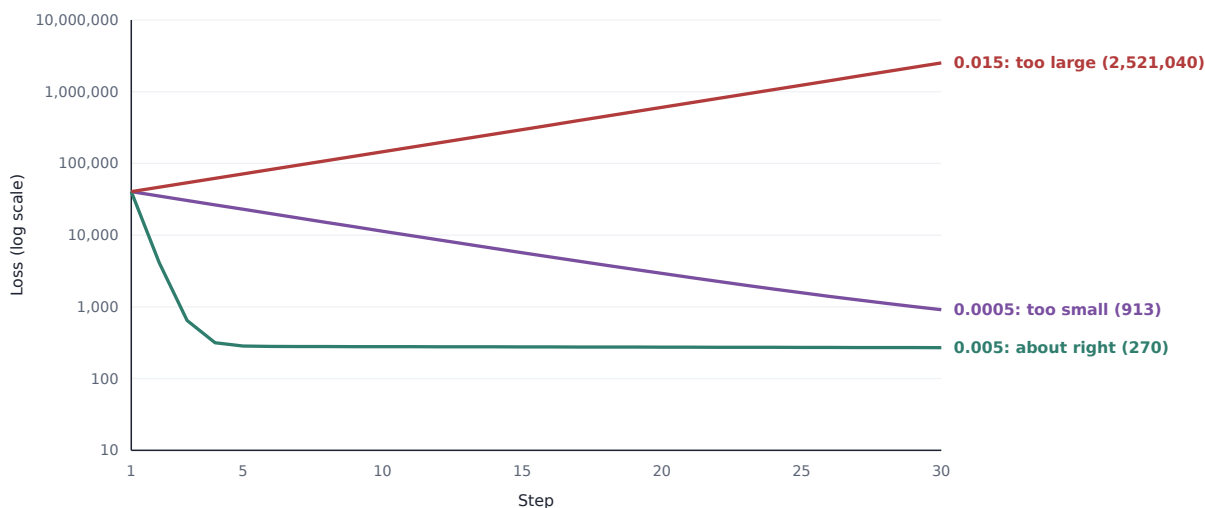


Figure 35.3 – The same problem with three learning rates. Too small crawls, about right drops fast, too large climbs away from the answer. The vertical axis is logarithmic: each gridline is ten times the one below.

Try it. In the learning-rate loop, try 0.014 and 0.0145. One of them converges, slowly, and one diverges, slowly. For this problem the dividing line is about 0.01447, which is 2 divided by the steepness of the valley's steepest direction. You don't need that formula at work; you need the habit of plotting the loss.

The pattern behind every model

You've now seen the three ingredients of almost every machine learning model:

1. **Parameters**, the numbers the model can adjust (w and b).
2. **A loss function** that scores how wrong the parameters are (mean squared error).
3. **An optimizer** that changes the parameters to lower the loss (gradient descent).

Linear regression (Chapter 37) uses exactly these, with more features. Logistic regression swaps in a different loss (section 35.9). A neural network (Chapter 43) has thousands or millions of parameters and computes its gradient automatically, but the loop is the one you wrote above. Real libraries add improvements such as **stochastic gradient descent** (computing each step on a random batch of rows instead of all of them, which is much faster on large data) and optimizers such as **Adam** that adjust the step size per parameter. They all follow the downhill slope.

35.7 Probability distributions: describing what's likely

A model that predicts a number with a straight line gives one answer. Many models instead give a **probability**: "this lead has a 20% chance of being won". To build and judge those, you need a way to describe how likely every possible outcome is. That description is a **probability distribution**. Chapter 21 introduces probability; this section adds the four distributions you'll meet most often in machine learning, each on real Riverstone data.

Bernoulli: one yes-or-no outcome

In 2025 Riverstone received **30** real sales enquiries (after removing duplicate web-form submissions, as in Chapter 13), and **6** were won. One lead is a single trial with two outcomes: won (1) or not won (0). A **Bernoulli distribution** describes it with one parameter, p , the probability of a 1.

$$P(\text{won}) = p \quad P(\text{not won}) = 1 - p$$

If $p = 0.2$, a lead is won with probability 0.2 and not won with probability 0.8. Every yes-or-no prediction in this book (will this customer churn, is this transaction fraud, will this lead convert) models the outcome as Bernoulli.

Binomial: how many yeses out of n

"If each lead has a 20% chance, how likely is it that exactly 6 of 30 are won?" Counting successes in n independent Bernoulli trials gives a **binomial distribution**:

$$P(\text{exactly } k \text{ won out of } n) = C(n, k) \times p^k \times (1 - p)^{(n - k)}$$

$C(n, k)$, read " n choose k ", counts how many different sets of k leads could be the winners. $C(30, 6) = 593,775$. So:

$$P(6 \text{ of } 30) = 593,775 \times 0.2^6 \times 0.8^{24} = 593,775 \times 0.000064 \times 0.0047224 = \mathbf{0.179}$$

Even when the true rate is exactly 20%, getting exactly 6 wins out of 30 happens only about 18% of the time. Five wins or seven are nearly as likely. Chapter 22 builds on this to ask whether a difference in win rates is real or noise.

Poisson: counts of events in a period

"How many orders should we expect in a week?" Riverstone's 173 orders in 2025 fell into 52 weeks, an average of $173 \div 52 = \mathbf{3.33}$ orders a week. When events arrive independently at a steady average rate, the number in a period follows a **Poisson distribution**, with one parameter, the average rate λ ("lambda"):

$$P(k \text{ events}) = \lambda^k \times e^{(-\lambda)} \div k!$$

Here e is the mathematical constant 2.71828, and $k!$ (" k factorial") is $1 \times 2 \times \dots \times k$. For zero orders in a week: $P(0) = 3.33^0 \times e^{(-3.33)} \div 0! = e^{(-3.33)} = 0.0359$. Over 52 weeks, that predicts $52 \times 0.0359 = \mathbf{1.9}$ weeks with no orders.

Does the model describe Riverstone's real weeks? Count them:


```
orders = pd.read_csv("orders_2025.csv", parse_dates=["order_date"])
week = (orders["order_date"].dt.dayofyear - 1) // 7 # week 0 = 1-7 January
counts = np.bincount(week, minlength=52)[:52]
rate = counts.mean()
print(f"{counts.sum()} orders in 52 weeks, average {rate:.2f} a week")

from math import exp, factorial
print("orders in a week | weeks observed | Poisson expects")
for k in range(9):
    expected = 52 * rate ** k * exp(-rate) / factorial(k)
    print(f"{k:>16} | {np.sum(counts == k):>14} | {expected:>15.1f}")
```

```
173 orders in 52 weeks, average 3.33 a week
orders in a week | weeks observed | Poisson expects
0 | 1 | 1.9
1 | 8 | 6.2
2 | 10 | 10.3
3 | 10 | 11.5
4 | 10 | 9.5
5 | 7 | 6.3
6 | 3 | 3.5
7 | 2 | 1.7
8 | 1 | 0.7
```

How it works: `dayofyear` numbers the days of 2025 from 1 to 365; subtracting 1 and dividing by 7 with `//` (whole-number division) gives a week number from 0. `np.bincount` counts how many orders fall into each week. The loop compares the number of weeks with k orders against $52 \times P(k)$.

Reading it. The Poisson model tracks the real weeks closely: it expects 10.3 weeks with two orders and 11.5 with three, and Riverstone had 10 of each. That's useful. A demand planner who knows orders are roughly Poisson with a rate of 3.3 a week knows that a week with 8 or more orders happens about once a year by chance alone, and doesn't need an explanation. (Riverstone's rate isn't really constant, since the festive season in October and November is busier; Chapter 40 handles seasonality properly.)

Normal: the bell curve, and when it lies

The **normal distribution** is the symmetric bell curve described by a mean and a standard deviation. Many methods assume it, often silently. Riverstone's order values have a mean of ₹25,060.53 and a standard deviation of ₹17,398.71. If order values were normal with those numbers, what would follow?

```
from statistics import NormalDist

values = orders["order_value"]
bell = NormalDist(mu=values.mean(), sigma=values.std())
print(f"mean ₹{values.mean():.2f} sd ₹{values.std():.2f} median ₹{values.median():.0f}")
above, below = 1 - bell.cdf(50000), bell.cdf(0)
print(f"above ₹50,000: normal predicts {above:.1%}, actual {np.mean(values > 50000):.1%}")
print(f"below ₹0: normal predicts {below:.1%}, actual {np.mean(values < 0):.1%}")
```

```
mean ₹25,060.53 sd ₹17,398.71 median ₹21,375
above ₹50,000: normal predicts 7.6%, actual 9.2%
below ₹0: normal predicts 7.5%, actual 0.0%
```


Reading it. The normal model says 7.5% of orders should have a **negative** value, which is impossible. It also underestimates large orders (7.6% predicted, 9.2% actual). Order values are **skewed**: most are moderate, a few are large, and none can be below zero, which is why the median (₹21,375) sits well below the mean. A normal distribution is a poor description of money amounts, counts of rare events, waiting times, and anything bounded at zero. Chapter 21 shows how to check the shape of data with a histogram before assuming.

Distribution	Describes	Parameters	Riverstone example	Used in
Bernoulli	one yes-or-no outcome	p	is this lead won?	logistic regression, classifiers (Ch 37)
Binomial	number of yeses in n tries	n, p	leads won out of 30	A/B tests (Ch 30), win-rate checks (Ch 22)
Poisson	count of events in a period	λ	orders per week	demand and anomaly models (Ch 40)
Normal	a measurement that clusters symmetrically	mean, standard deviation	<i>not</i> order values	linear regression errors (Ch 37), sampling (Ch 22)

35.8 Likelihood and maximum likelihood

Turning the question around

So far you've asked: *given* a parameter, how likely is the data? Machine learning asks the reverse: *given* the data, which parameter is most believable?

The **likelihood** of a parameter value is the probability of the data you actually observed, calculated as if that value were true. For Riverstone's 30 leads with 6 won, and a win rate p , the probability of that exact sequence of wins and losses is:

$$\text{likelihood}(p) = p^6 \times (1 - p)^{24}$$

(The $C(30, 6)$ from the binomial formula is the same for every p , so it doesn't change which p is best, and it's usually left out.)

Try three values:

p	$p^6 \times (1 - p)^{24}$
0.1	$0.000001 \times 0.079766 = 0.0000000798$
0.2	$0.000064 \times 0.0047224 = 0.000000302$
0.3	$0.000729 \times 0.00019158 = 0.000000140$

A win rate of 0.2 makes the observed data about 3.8 times as likely as 0.1 and about 2.2 times as likely as 0.3. The **maximum likelihood estimate (MLE)** is the parameter value that makes the observed data most likely.

Why everyone uses the log

Those likelihoods are tiny, and with 30,000 leads instead of 30 they'd be too small for a computer to store (they'd round to zero). So in practice you work with the **log-likelihood**: the natural logarithm of the likelihood. Logs turn multiplication into addition and keep numbers manageable, and because the log always rises when its input rises, the best p is the same.

$$\text{log-likelihood}(p) = 6 \times \ln(p) + 24 \times \ln(1 - p)$$

$$\text{at } p = 0.2: 6 \times \ln(0.2) + 24 \times \ln(0.8) = 6 \times (-1.60944) + 24 \times (-0.22314) = -9.65663 - 5.35545 = \mathbf{-15.0121}$$

Search every value of p from 0.001 to 0.999 and let NumPy find the best:

```
leads = pd.read_csv("leads_2025.csv")
won = leads["won"].to_numpy()
print(len(won), "leads,", won.sum(), "won")

p_grid = np.linspace(0.001, 0.999, 999)
log_likelihood = won.sum() * np.log(p_grid) + (len(won) - won.sum()) * np.log(1 - p_grid)
best = p_grid[np.argmax(log_likelihood)]
print(f"maximum likelihood estimate of p: {best:.3f}")
print(f"log-likelihood there: {log_likelihood.max():.4f}")
print(f"sample win rate: {won.mean():.3f}")
```

```
30 leads, 6 won
maximum likelihood estimate of p: 0.200
log-likelihood there: -15.0121
sample win rate: 0.200
```

How it works: `np.linspace(0.001, 0.999, 999)` makes 999 evenly spaced candidate values of p (0.001, 0.002, ..., 0.999). The log-likelihood is computed for all of them at once, and `np.argmax` returns the position of the largest.

The maximum likelihood estimate is **0.200**, exactly the sample win rate, $6 \div 30$. That's not a coincidence: for a Bernoulli outcome, the MLE is always the observed proportion, and for a Poisson count it's the observed average (3.33 orders a week). The MLE confirms the obvious in simple cases. Its value is that the same principle works for models where there's no obvious answer, like a lead-scoring model with twenty features.

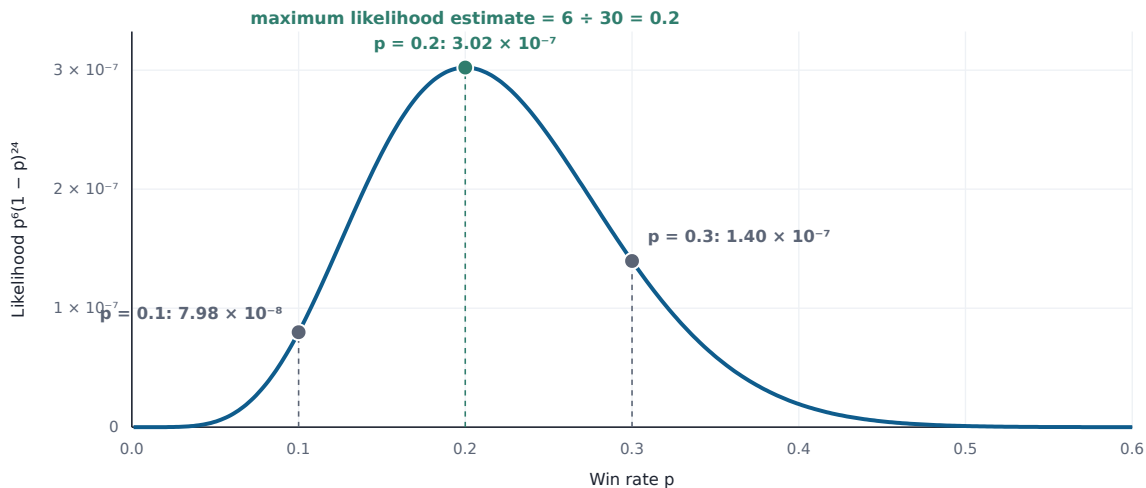


Figure 35.4 – The likelihood of each possible win rate, given 6 wins in 30 leads. The peak, the maximum likelihood estimate, is at 0.2. The curve is wide: rates from about 0.1 to 0.35 remain fairly plausible with only 30 leads.

Why this matters for models

Training a model is maximum likelihood estimation. A model's parameters decide the probabilities it assigns; fitting it means finding the parameters under which the training data is most likely. Two facts connect this to what you already know:

- For a model that predicts probabilities of yes-or-no outcomes, maximizing the log-likelihood is the same as minimizing **log loss** (section 35.9).
- For a model that predicts numbers, if you assume the errors are normally distributed, maximizing the likelihood is the same as minimizing **mean squared error**. That's where section 35.4's loss comes from.

35.9 Entropy, cross-entropy, and log loss

Entropy: how uncertain is an outcome?

"Before we look at anything about a lead, how uncertain are we about whether it will be won?"

Information theory measures uncertainty with **entropy**. For a yes-or-no outcome with probability p :

$$\text{entropy} = -(p \times \log_2 p + (1 - p) \times \log_2 (1 - p))$$

Measured with log base 2, the unit is the **bit**. A fair coin ($p = 0.5$) has entropy $-(0.5 \times (-1) + 0.5 \times (-1)) = 1$ **bit**, the most uncertain a yes-or-no outcome can be. Riverstone's lead outcome, at $p = 0.2$:

$$\begin{aligned} \text{entropy} &= -(0.2 \times \log_2 0.2 + 0.8 \times \log_2 0.8) = -(0.2 \times (-2.3219) + 0.8 \times (-0.3219)) = -(-0.4644 - 0.2575) \\ &= 0.722 \text{ bits} \end{aligned}$$

Less than 1 bit, because leads are usually not won, so you're less uncertain than with a coin. An outcome that's certain ($p = 0$ or 1) has entropy 0.

Information gain: the preview of decision trees

Entropy lets you measure how much a piece of information reduces uncertainty. Split Riverstone's leads by source:

Group	Leads	Won	Win rate	Entropy
All leads	30	6	20.0%	0.722
Website	14	1	7.1%	0.371
Every other source	16	5	31.3%	0.896

Check the website entropy: $-(1/14 \times \log_2(1/14) + 13/14 \times \log_2(13/14)) = -(0.0714 \times (-3.8074) + 0.9286 \times (-0.1069)) = 0.2720 + 0.0993 = 0.371$. ✓

After the split, the average uncertainty, weighted by group size, is $14/30 \times 0.371 + 16/30 \times 0.896 = 0.173 + 0.478 = \mathbf{0.651 \text{ bits}}$. Knowing whether a lead came through the website reduced uncertainty from 0.722 to 0.651 bits, an **information gain** of **0.071 bits**. A decision tree (Chapter 37) builds itself by trying every possible split and choosing the one with the biggest gain, then repeating inside each branch.

Cross-entropy and log loss: scoring probability predictions

Entropy measures uncertainty when you know the true probabilities. **Cross-entropy** measures how surprised you are, on average, when you use **predicted** probabilities and then see the real outcomes. For machine learning it's written with natural logs, averaged over rows, and called **log loss** (or **binary cross-entropy**):

$$\text{log loss} = -\text{average of } (y \times \ln(\text{predicted } p) + (1 - y) \times \ln(1 - \text{predicted } p))$$

For each row, only one term is active. If the lead was won ($y = 1$), the cost is $-\ln(\text{predicted } p)$. If not ($y = 0$), it's $-\ln(1 - \text{predicted } p)$.

Suppose a scoring tool gave five past leads these probabilities of being won, and here's what happened:

Lead	Predicted p	Won?	Cost
A	0.9	yes	$-\ln 0.9 = 0.105$
B	0.7	yes	$-\ln 0.7 = 0.357$
C	0.2	no	$-\ln 0.8 = 0.223$
D	0.1	no	$-\ln 0.9 = 0.105$
E	0.8	no	$-\ln 0.2 = 1.609$

$$\text{log loss} = (0.105 + 0.357 + 0.223 + 0.105 + 1.609) \div 5 = 2.399 \div 5 = \mathbf{0.480}$$

Look at lead E. The tool was 80% sure it would be won, and it wasn't. That one confident mistake costs more than the other four rows together. Log loss is harsh on confident wrong answers by design: a prediction of 0.99 for a lead that isn't won costs $-\ln 0.01 = 4.6$, and a prediction of exactly 1.0 costs infinity. A model trained on log loss learns to say "80%" only when it has earned it.

```
from sklearn.metrics import log_loss

y_true = np.array([1, 1, 0, 0, 0])
y_pred = np.array([0.9, 0.7, 0.2, 0.1, 0.8])
by_hand = -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
print(f"log loss by hand:      {by_hand:.3f}")
print(f"log loss, sklearn:    {log_loss(y_true, y_pred):.3f}")

base_rate = np.full(len(won), won.mean()) # predict 0.2 for every lead
print(f"always predicting 0.2 on the 30 leads: {log_loss(won, base_rate):.4f} nats"
      f" = {log_loss(won, base_rate) / np.log(2):.4f} bits")
print(f"-(log-likelihood at p=0.2) / 30:      {-log_likelihood.max() / 30:.4f}")
```

```
log loss by hand:      0.480
log loss, sklearn:    0.480
always predicting 0.2 on the 30 leads: 0.5004 nats = 0.7219 bits
-(log-likelihood at p=0.2) / 30:      0.5004
```

How it works: `np.log` is the natural log, so the hand formula gives log loss in **nats** (the natural-log unit); dividing by $\ln 2$ converts nats to bits. `np.full(30, 0.2)` builds a "model" that predicts the base rate for every lead.

Reconcile three ideas. The three last numbers are the same quantity seen three ways:

- A model that always predicts the base rate of 0.2 has a log loss of **0.5004 nats**.
- In bits, that's **0.7219**, exactly the entropy of the outcome from earlier in this section. ✓ When your predictions match the true rate, cross-entropy equals entropy; any worse predictions give a higher cross-entropy.
- It's also the negative log-likelihood at $p = 0.2$ (section 35.8), $-(-15.0121) \div 30 = \mathbf{0.5004}$. ✓

Minimizing log loss is maximizing likelihood.

That gives you a benchmark for any lead score at Riverstone: a model that doesn't beat a log loss of about 0.50 on these leads knows nothing beyond the overall win rate. Chapter 39 uses the same idea to evaluate a real lead-scoring model.

Watch out: accuracy hides what log loss shows. A "model" that says *not won* for every lead is $24 \div 30 = \mathbf{80\% \text{ accurate}}$ on Riverstone's leads, while being useless. Accuracy counts right and wrong answers; log loss scores how well the probabilities match reality, and punishes false confidence. When someone quotes an accuracy figure, ask for the base rate first.

35.10 Principal component analysis (PCA)

The idea: one direction that says most

"We track seven numbers for every customer. Can we summarize them with fewer?" When several columns move together, they carry overlapping information. **Principal component analysis (PCA)** finds new axes, called **principal components**, that point along the directions where the data spreads out most. The first component captures as much of the spread as any single direction can; the second captures the most of what's left, at right angles to the first; and so on. Keep the first few and you've compressed the data with little loss.

"Spread" here is **variance**, the average squared distance from the mean (Chapter 21). PCA ranks directions by how much variance the data has along them.

A small example by hand

Five customers, two features: orders and revenue in ₹ ten-thousands (so Sharma Hardware's ₹502,775 becomes 50). Revenue is measured in ten-thousands so that the two columns have spreads of a similar size; the full example below shows what happens when they don't.

Customer	Orders	Revenue (₹10k)
Sharma Hardware	16	50
Metro Mart	16	33
Harbour Traders	10	41
Patel Kitchenware	7	17
Blue Bay Cafe	4	5

Step 1: center. The averages are $53 \div 5 = 10.6$ orders and $146 \div 5 = 29.2$. Subtract them:

Customer	Orders – 10.6	Revenue – 29.2
Sharma	5.4	20.8
Metro	5.4	3.8
Harbour	–0.6	11.8
Patel	–3.6	–12.2
Blue Bay	–6.6	–24.2

Step 2: the covariance matrix. The **covariance** of two columns is the average product of their centered values (dividing by $n - 1 = 4$, as for a sample standard deviation). It's positive when they rise together. The **covariance matrix** holds every column's variance on the diagonal and each pair's covariance off it.

variance of orders = $(5.4^2 + 5.4^2 + 0.6^2 + 3.6^2 + 6.6^2) \div 4 = (29.16 + 29.16 + 0.36 + 12.96 + 43.56) \div 4 = 115.2 \div 4 = \mathbf{28.8}$

variance of revenue = $(20.8^2 + 3.8^2 + 11.8^2 + 12.2^2 + 24.2^2) \div 4 = (432.64 + 14.44 + 139.24 + 148.84 + 585.64) \div 4 = 1,320.8 \div 4 = \mathbf{330.2}$

covariance = $(5.4 \times 20.8 + 5.4 \times 3.8 + (-0.6) \times 11.8 + (-3.6) \times (-12.2) + (-6.6) \times (-24.2)) \div 4 = (112.32 + 20.52 - 7.08 + 43.92 + 159.72) \div 4 = 329.4 \div 4 = \mathbf{82.35}$

So the covariance matrix is:

	Orders	Revenue
Orders	28.8	82.35
Revenue	82.35	330.2

Step 3: find the direction of most variance. The principal components are the **eigenvectors** of the covariance matrix, and the variance along each is its **eigenvalue**. In words: an eigenvector is a direction that the matrix only stretches, without turning, and the eigenvalue is how much it stretches. For a 2×2 covariance matrix with variances a and d and covariance c , the larger eigenvalue has a formula:

largest eigenvalue = $(a + d) \div 2 + \sqrt{((a - d) \div 2)^2 + c^2}$
 $= (28.8 + 330.2) \div 2 + \sqrt{((28.8 - 330.2) \div 2)^2 + 82.35^2} = 179.5 + \sqrt{(22,710.49 + 6,781.52)} = 179.5 + \sqrt{29,492.01} = 179.5 + 171.73 = \mathbf{351.23}$

The total variance is $28.8 + 330.2 = 359.0$, so the first component holds $351.23 \div 359.0 = \mathbf{97.8\%}$ of it. The second holds $359.0 - 351.23 = 7.77$, or 2.2%.

Its direction is found by solving $(28.8 - 351.23) \times u + 82.35 \times v = 0$, which gives $v = 3.915 \times u$. Scaled to length 1, that's **[0.247, 0.969]**: a direction that is mostly revenue, with some orders. It's a "customer size" axis.

Step 4: project. Each customer's **score** on the first component is the dot product of its centered row with the direction. Sharma: $5.4 \times 0.247 + 20.8 \times 0.969 = 1.34 + 20.15 = \mathbf{21.49}$. Blue Bay: $-6.6 \times 0.247 + (-24.2) \times 0.969 = -1.63 - 23.45 = \mathbf{-25.08}$. One number per customer now says most of what the two columns said.

```

small = np.array([[16, 50], [16, 33], [10, 41], [7, 17], [4, 5]], dtype=float)
centered = small - small.mean(axis=0)
cov = centered.T @ centered / (len(small) - 1)
print("covariance matrix:\n", cov)

eigenvalues, eigenvectors = np.linalg.eigh(cov)    # returned smallest first
order = np.argsort(eigenvalues)[::-1]
eigenvalues, eigenvectors = eigenvalues[order], eigenvectors[:, order]
pc1 = eigenvectors[:, 0] * np.sign(eigenvectors[1, 0])    # point it toward higher revenue
print("eigenvalues:", eigenvalues.round(2))
print("share of variance:", (eigenvalues / eigenvalues.sum()).round(3))
print("first component direction:", pc1.round(3))
print("scores on PC1:", (centered @ pc1).round(2))

```

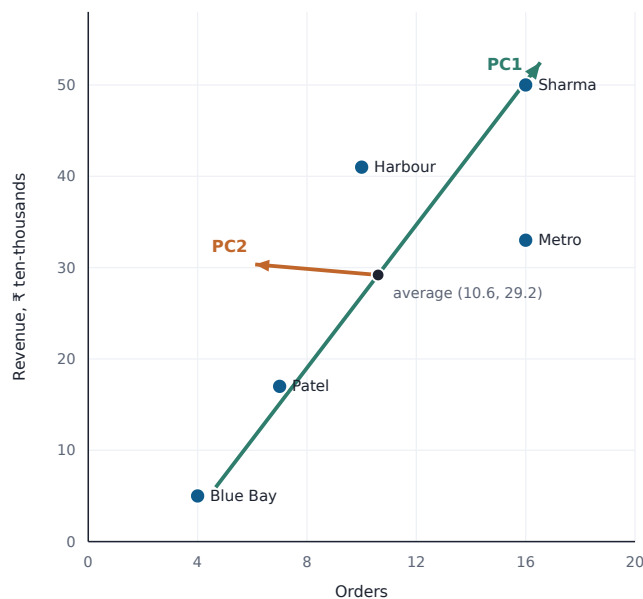
```

covariance matrix:
[[ 28.8  82.35]
 [ 82.35 330.2 ]]
eigenvalues: [351.23  7.77]
share of variance: [0.978 0.022]
first component direction: [0.247 0.969]
scores on PC1: [ 21.49  5.02 11.28 -12.71 -25.08]

```

How it works:

- `centered.T @ centered / 4` is the covariance matrix in one line: the dot product of each centered column with each other column, divided by $n - 1$. Compare it with the three hand calculations. ✓
- `np.linalg.eigh` finds eigenvalues and eigenvectors of a symmetric matrix (a covariance matrix is always symmetric). It returns them smallest first, so `argsort(...)[::-1]` reorders them largest first.
- An eigenvector's sign is arbitrary: $[0.247, 0.969]$ and $[-0.247, -0.969]$ describe the same axis. Multiplying by the sign of the revenue component makes "higher score = bigger customer" so the output is stable and readable.



Principal components

PC1 direction $[0.247, 0.969]$
variance $351.23 = 97.8\%$ of total

PC2 direction $[-0.969, 0.247]$
variance $7.77 = 2.2\%$ of total

The two components are at right angles.
 Orders are drawn 3× wider than revenue so the points are easier to see, which makes the right angle look slanted.

Figure 35.5 – The five customers and their principal components. The first component runs along the "customer size" direction and holds 97.8% of the variance; the short second component holds the rest.

All 23 customers, seven features

Now the real version: seven features per customer, which you can't draw. First, the mistake.

```
features = ["orders", "revenue", "avg_order_value", "avg_discount_pct",
            "storage_share", "kitchen_share", "industrial_share"]
F = customers[features].to_numpy(dtype=float)

def pca(matrix):
    centered = matrix - matrix.mean(axis=0)
    values, vectors = np.linalg.eigh(np.cov(centered, rowvar=False))
    order = np.argsort(values)::-1]
    return values[order] / values.sum(), vectors[:, order], centered @ vectors[:, order]

share_raw, vectors_raw, _ = pca(F)
print("UNSCALED share of variance, first 3 components:", share_raw[:3].round(4))
print("UNSCALED PC1 weights (size only):")
print(pd.Series(np.abs(vectors_raw[:, 0]), index=features).round(3).to_string())
```

```
UNSCALED share of variance, first 3 components: [0.9978 0.0022 0.    ]
UNSCALED PC1 weights (size only):
orders          0.000
revenue         1.000
avg_order_value 0.027
avg_discount_pct 0.000
storage_share   0.000
kitchen_share   0.000
industrial_share 0.000
```

Reading the wrong version. The first component "explains" 99.8% of the variance, which sounds wonderful and means nothing. Its weights are almost entirely on revenue. Revenue is measured in rupees and varies by hundreds of thousands, while shares vary between 0 and 1, so revenue's variance swamps everything. This unscaled PCA has rediscovered "revenue" and thrown away the other six features.

The fix is to **standardize** each column first: subtract the mean and divide by the standard deviation, so every feature has a standard deviation of 1 and an equal chance to matter.

```
F_std = (F - F.mean(axis=0)) / F.std(axis=0)
share, vectors, scores = pca(F_std)
signs = np.sign(vectors[features.index("industrial_share"), :])
signs[1] = np.sign(vectors[features.index("storage_share"), 1])
vectors = vectors * signs      # PC1 toward industrial buyers, PC2 toward storage
scores = F_std @ vectors
print("share of variance:", share.round(3))
print(f"first two components together: {share[:2].sum():.1%}")
loadings = pd.DataFrame(vectors[:, :2], index=features, columns=["PC1", "PC2"])
print(loadings.round(2).to_string())

from sklearn.decomposition import PCA
check = PCA().fit(F_std)
print("scikit-learn agrees:", np.allclose(check.explained_variance_ratio_, share))
```

```

share of variance: [0.535 0.205 0.154 0.084 0.02 0.002 0. ]
first two components together: 73.9%
           PC1  PC2
orders      0.30 -0.58
revenue     0.40 -0.47
avg_order_value 0.37 0.13
avg_discount_pct 0.44 0.15
storage_share 0.21 0.53
kitchen_share -0.45 -0.35
industrial_share 0.42 -0.06
scikit-learn agrees: True

```

How it works:

- `F.std(axis=0)` is each column's standard deviation; after dividing, every column has a spread of 1.
- `np.cov(centered, rowvar=False)` is the covariance matrix, with columns as variables.
- `np.allclose` checks that scikit-learn's `PCA` gives the same shares, allowing for tiny rounding differences. It does. (Its component signs can differ from ours, which is why the shares are compared, not the directions.)

Reading it. With every feature on the same scale, the first component holds 53.5% of the variance and the first two hold 73.9%. Read the component by its **weights** (also called **loadings**):

- **PC1** weighs heavily and positively on industrial share, average discount, revenue, and average order value, and negatively on kitchen share. It's a **"wholesale industrial buyer" axis**: big, discounted orders of crates at one end, kitchenware buyers at the other.
- **PC2** weighs positively on storage share and negatively on orders, revenue, and kitchen share. High scores mean **a few orders of storage products only**.

Plot every customer on these two axes, and the segments sort themselves, although PCA was never told which segment anyone belongs to:

```

plot_table = pd.DataFrame({"customer": names, "segment": customers["segment"],
                           "PC1": scores[:, 0].round(2), "PC2": scores[:, 1].round(2)})
print(plot_table.sort_values("PC1", ascending=False).head(5).to_string(index=False))
print(plot_table.groupby("segment")[["PC1", "PC2"]].mean().round(2).to_string())

```

```

      customer  segment  PC1  PC2
Harbour Traders Wholesale 3.75 -0.30
Coastal Foods Wholesale 3.04 -0.38
Deccan Packaging Wholesale 2.87 -0.51
Northgate Distributors Wholesale 2.76 -0.12
Western Logistics Wholesale 2.62 0.74
      PC1  PC2
segment
Hospitality -1.47 -0.48
Retail      -0.38 0.22
Wholesale   2.71 0.42

```

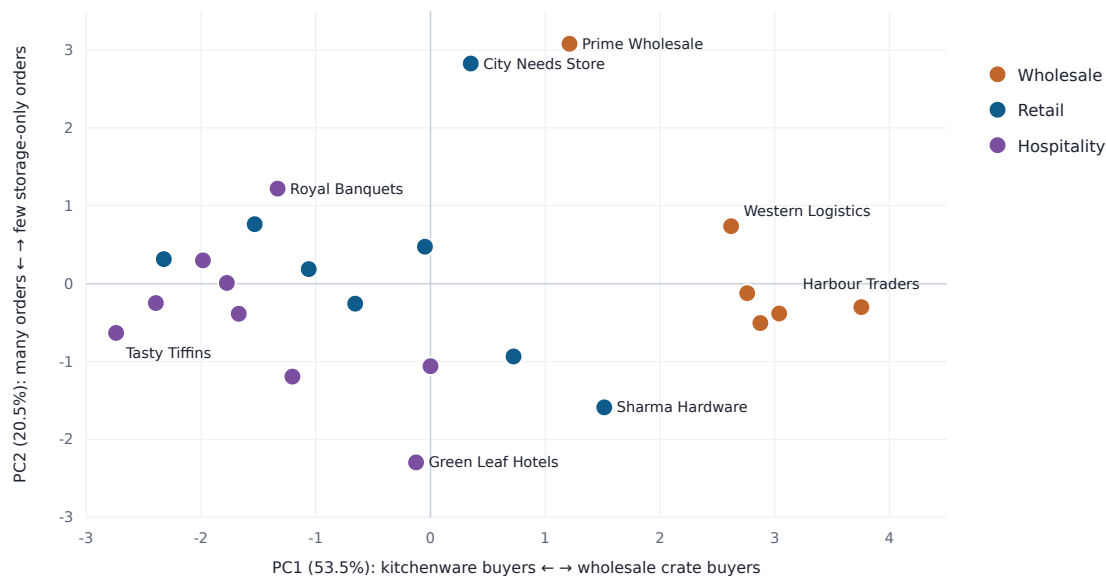


Figure 35.6 — All 23 customers on the first two principal components of seven standardized features. The five regular wholesale buyers sit together on the right; Prime Wholesale, a wholesaler with only two storage-only orders, sits apart, high on PC2.

What to tell the sales head. "Two combined measures describe about three-quarters of how our customers differ: how much they buy like a wholesale crate buyer, and whether they place a few storage-only orders. Prime Wholesale is labeled Wholesale but doesn't yet buy like one; it's worth a conversation." Chapter 38 uses PCA with clustering to build proper customer segments.

Watch out: PCA components are not features. "PC1" isn't a column anyone recorded; it's a weighted mix of all seven. It's useful for plotting, compressing, and feeding models many correlated columns, but it's harder to explain than the original features. If a manager needs to act on a result, show them the original columns behind it.

35.11 Which math sits under which model

Every idea in this chapter reappears. Use this table as a map for the rest of the book.

Math idea	What it does in a model	Where you'll see it next
Vectors and distance	Represent rows; find nearest neighbors	k-nearest neighbors (Ch 37), k-means and DBSCAN (Ch 38)
Dot product and cosine similarity	Measure similarity; weigh features	Linear models (Ch 37), recommenders (Ch 42), embeddings and attention (Ch 41, 54)
Matrix × vector	Predict every row at once	Linear and logistic regression (Ch 37), neural network layers (Ch 43)
Derivative and gradient	Say which way to change parameters	Every trained model; automatic in PyTorch (Ch 43)

Math idea	What it does in a model	Where you'll see it next
Gradient descent and learning rate	Train parameters step by step	Logistic regression, gradient boosting (Ch 37), neural networks (Ch 43, 53)
Scaling features	Make distances and gradients behave	Feature pipelines (Ch 36)
Bernoulli, binomial, Poisson, normal	Describe outcomes and counts	Classifiers (Ch 37), experiments (Ch 30), forecasting and anomalies (Ch 40)
Maximum likelihood	Choose the parameters that fit the data	Logistic regression, Naive Bayes (Ch 37), ARIMA (Ch 40)
Entropy and information gain	Pick the best split	Decision trees and random forests (Ch 37)
Cross-entropy / log loss	Train and score probability predictions	Logistic regression (Ch 37), calibration (Ch 39), neural networks (Ch 43), language models (Ch 54)
Covariance, eigenvectors, PCA	Compress correlated features	Dimensionality reduction (Ch 38), noise reduction before modeling

Interview extra point. When an interviewer asks "why log loss and not accuracy?" or "why not squared error for classification?", the answer that passes says log loss punishes confident mistakes. The answer that earns extra points adds the connection: *minimizing log loss is maximizing the likelihood of a Bernoulli model*, and then gives a benchmark, "a model that always predicts the base rate scores the entropy of the outcome, so I compare against that." The Machine Learning Question Bank (Chapter 74) has graded answers to this and other math-behind-the-model questions.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Measuring distance on features with different units	One feature (usually money) decides every distance; "nearest" customers are the ones with similar revenue	Standardize or scale features before any distance-based method
Using a raw dot product to compare direction	Big customers look similar to everyone	Divide by both lengths: use cosine similarity
Multiplying matrices with mismatched shapes	<code>ValueError: matmul: Input operand 1 has a mismatch in its core dimension</code>	Write the shapes down: (rows × columns) @ (columns) gives (rows)
Forgetting the column of ones	The fitted line is forced through zero	Add a column of ones for the intercept, or let the library fit it
Adding the gradient instead of subtracting it	The loss rises every step, smoothly	<code>new = old - learning_rate × slope</code>
Learning rate too high	Loss grows, jumps around, or becomes <code>nan</code> or <code>inf</code>	Divide the learning rate by 10 and plot the loss again

Mistake	Symptom	Fix
Learning rate too low	Loss falls, but very slowly; training "never finishes"	Multiply it by 3 to 10 while the loss still falls smoothly
Judging convergence by the loss alone	Loss looks flat while some parameters are still drifting	Scale features; check parameter changes as well as the loss
Assuming data is normal	Impossible predictions (negative order values), too few large values	Look at a histogram; use a distribution that fits the data's shape
Multiplying many small probabilities	Likelihood becomes 0.0 on a large dataset	Work with log-likelihood, a sum of logs
Judging a probability model by accuracy on imbalanced outcomes	An "80% accurate" model that never predicts a win	Compare log loss with the base-rate log loss; ask for the base rate
Predicting exactly 0 or 1	Log loss becomes infinite ($-\infty$), or a library warns and clips the value	Keep probabilities strictly between 0 and 1
Running PCA without standardizing	First component "explains" 99%+ and loads on one big-unit column	Standardize every feature to standard deviation 1 first
Reading a PCA component's sign as meaningful	Two runs or two libraries give "opposite" components	Fix the sign by a rule (for example, a chosen feature positive) before interpreting
Treating principal components as business features	A manager can't act on "PC2 is high"	Explain results through the original columns behind the component

In the real world: Meera and the 91% lead score

It's the first week of February 2026. Anita Rao, Riverstone's Sales Head, forwards Meera Iyer a slide from a software vendor. The vendor's lead-scoring add-on for CRMs "predicts which leads will convert with **91% accuracy**", and the offer is ₹40,000 a month. Anita's note says: *"The sales team likes the demo. Is this worth it? I need an answer by Friday's review."*

Meera starts where this chapter taught her to start: **what would a model that knows nothing score?**

The base rate. From the CRM export behind Chapter 13, Riverstone had 30 real enquiries in 2025 and won 6, a win rate of 20%. A "model" that says *not won* for every lead is right 24 times out of 30: 80% accurate. The vendor's 91% is an improvement on that, but not the improvement it sounds like. And the vendor's figure comes from its other clients. Meera asks the vendor's account manager what share of leads those clients win. The answer is about 8%. At an 8% win rate, answering *not won* for every lead is 92% accurate, **better than the vendor's model**. The headline number, on its own, says nothing about whether the model has learned anything.

A measure that can't be gamed that way. Meera asks for probabilities instead of yes-or-no labels, and for **log loss** on held-out leads. Then she works out Riverstone's benchmark herself: a model that predicts 20% for every lead scores 0.50 on Riverstone's 2025 leads (section 35.9). Any score worth ₹40,000 a month has to beat 0.50 clearly on Riverstone's leads, not the vendor's clients'.

How much can 30 leads tell anyone? Meera looks back at the likelihood curve for the win rate (Figure 35.4). With 30 leads, win rates from about 10% to 35% are all reasonably believable. If a whole year of leads can't pin down the win rate more tightly than that, a test on a few weeks of leads can't tell a good score from a lucky one. She notes it plainly: the evaluation needs more leads than Riverstone gets in a month.

What's inside it. Finally, she asks the three questions every model can answer: *What are the parameters? What loss was minimized? What does it learn from?* The vendor explains it's a gradient-boosted model (Chapter 37) trained with log loss on activity data: email opens, website visits, and how quickly a rep responds. That last feature makes Meera pause. At Riverstone, reps respond fastest to leads they *already* think are promising, so a model can learn "fast response means likely win" from the reps' own judgment, and then look clever by repeating it. Chapter 36 calls this leakage.

Her reply to Anita, on Thursday evening:

"The 91% accuracy figure doesn't tell us much. At our 20% win rate, predicting 'no' for every lead is already 80% accurate, and at the vendor's clients' 8% win rate it's 92%. What matters is whether the probabilities beat a simple benchmark on our own leads. My recommendation: ask for a three-month pilot where the tool scores new leads but reps don't see the scores, so they can't influence the result. At the end, we compare its log loss with our benchmark of 0.50. If it isn't clearly better, we don't buy. I've also asked them to confirm the model doesn't use rep response time, which would make it look better than it is."

Anita takes the pilot proposal to the vendor. It is accepted, at no charge.

Notice what Meera didn't do. She didn't build a competing model or write any code. She used four ideas from this chapter (base rates, log loss, likelihood, and "parameters, loss, optimizer") to turn a sales pitch into a test with a pass mark. That's what the math under the models is for.

Tools

- **Python 3 with NumPy and pandas.** Every example ran on Python 3.12.3, NumPy 2.4.4, and pandas 3.0.2. The features used (`arrays`, `@`, `np.linalg.norm`, `np.linalg.eigh`, `np.polyfit`, `read_csv`) have been stable for many years, so older versions give the same results. Chapter 17 covers installation.
- **scikit-learn** (1.8.0 here) for two cross-checks: `sklearn.metrics.log_loss` and `sklearn.decomposition.PCA`. Chapter 36 introduces it properly.
- **Python's standard library:** `math (exp, factorial, comb)` and `statistics.NormalDist`. Nothing to install.
- **Excel and Google Sheets** have every calculation in this chapter: `SUMPRODUCT` (dot product), `MMULT` and `TRANSPOSE` (matrices), `BINOM.DIST`, `POISSON.DIST`, and `NORM.DIST` (distributions; Google Sheets also lists them under the older names `BINOMDIST` and `NORMDIST`), `LN` and `LOG` (logs, with `LOG(x, 2)` for bits), and `COVARIANCE.S` (covariance). Doing section 35.6's three steps in a spreadsheet, one row per step, is a good way to check your hand work.

- **Pen and paper.** The hand calculations are the point. Do them before running the code.
- **Visual explainers.** 3Blue1Brown's free *Essence of Linear Algebra* and *Essence of Calculus* video series animate vectors, matrices, and derivatives, and pair well with this chapter.
- **Companion files** (`companion/ch35/`): `make_ch35_data.py` (rebuilds the three CSV files from the seeded one-year generator, `companion/generate_riverstone_2025.py`), `customers_2025.csv` , `orders_2025.csv` , and `leads_2025.csv` . Run the chapter's code from that folder.

The project: gradient descent and PCA from scratch

Goal: a short notebook or script, written by you without machine learning libraries, that trains a model with gradient descent and compresses a table with PCA, and checks both against a library. It's a small, honest portfolio piece that shows you know what `.fit()` does.

Option A: your own data. Use a table from your work or life with at least 20 rows and 4 numeric columns (for example, monthly figures for a set of products or branches). Remove names and anything confidential first, and check you're allowed to use it.

Option B: Riverstone. Use `customers_2025.csv` and `orders_2025.csv` from the companion files.

Steps:

1. **Pick a prediction.** Choose one numeric target and one feature (Option B: predict each customer's revenue from their units bought). Write the question in one sentence.
2. **Hand-check first.** Take four rows and do two steps of gradient descent on paper, exactly as in section 35.6.
3. **Write the loop.** Implement gradient descent for w and b in NumPy, recording the loss at every step. Your first two steps must match your paper. ✓
4. **Tune the learning rate.** Try at least four learning rates, plot the loss curves on one chart, and write one sentence on each.
5. **Scale and compare.** Center (or standardize) the feature and train again. Report how many steps each version needs to get within 1% of the best loss, and compare your final w and b with `np.polyfit` .
6. **PCA by hand, then in code.** Pick 4 to 6 numeric columns. Standardize them, build the covariance matrix with `X.T @ X / (n - 1)` , and find eigenvalues and eigenvectors with `np.linalg.eigh` . Check the variance shares against scikit-learn's `PCA` .
7. **Interpret.** Name the first two components from their weights, plot the rows on them, and write three sentences a manager could understand.
8. **Write it up.** A one-page summary: the question, the two loss charts, the PCA plot, what you learned, and one thing that went wrong and how you spotted it.

Stretch goals:

- Extend your gradient descent to many features with the matrix form of the gradient, `2 * X.T @ (X @ weights - y) / n` , and train it on three features at once.

- Add **stochastic gradient descent**: compute each step on a random batch of 5 rows (with a fixed seed) and compare the loss curve with full-batch descent.
 - Implement logistic regression for `leads_2025.csv` using log loss and gradient descent, with one feature (whether the lead came from the website). Check that your predicted probabilities for the two groups approach $1 \div 14$ and $5 \div 16$.
-

You've got it when...

- ☐ I can compute a distance, a dot product, and a cosine similarity by hand, and explain why scaling changes distances.
- ☐ I can say what shape `x @ weights` returns before running it.
- ☐ I can explain in one sentence what a derivative tells me about a loss, and check a derivative formula by nudging a parameter.
- ☐ I can do a step of gradient descent on paper and get the same numbers as my code.
- ☐ Looking at a loss curve, I can tell a learning rate that's too small from one that's too large.
- ☐ I know that a flat loss curve doesn't prove the parameters have settled, and that scaling features often fixes slow training.
- ☐ I can name a distribution for yes-or-no outcomes, for counts per period, and for symmetric measurements, and I check the shape of data before assuming normal.
- ☐ I can explain maximum likelihood, and why minimizing log loss is the same thing for a classifier.
- ☐ I can calculate entropy, information gain, and log loss, and compare a model against the base-rate benchmark.
- ☐ I can run PCA by hand on two features and in NumPy on many, standardizing first, and describe a component from its weights.

When someone says "the model learns", and you can picture the parameters, the loss, and the downhill step, this chapter has done its job.

Recap

- A **vector** is an ordered list of numbers; each row of a dataset is one. **Euclidean distance** compares positions, and it depends on units, so **scale** features first.
- The **dot product** multiplies matching components and adds them. Divided by both **lengths**, it gives **cosine similarity**, which compares direction regardless of size.
- A **matrix** holds rows of vectors. **Matrix** $\times$ **vector** takes the dot product of each row with the weights, predicting every row at once. **Transpose** swaps rows and columns.
- A **loss function** such as **mean squared error** scores how wrong a model is. Training means finding **parameters** that minimize it.

- A **derivative** is the slope of the loss for one parameter; the **gradient** is the vector of slopes for all of them and points uphill.
- **Gradient descent** steps each parameter against its slope, scaled by the **learning rate** (a **hyperparameter**). Too small crawls; too large diverges; unscaled features make narrow valleys and slow training.
- **Bernoulli** and **binomial** describe yes-or-no outcomes, **Poisson** describes counts per period, and the **normal** distribution describes symmetric measurements, but not skewed amounts like order values.
- **Likelihood** scores parameter values by how probable they make the observed data. The **maximum likelihood estimate** is the best one; work with **log-likelihood** in practice.
- **Entropy** measures uncertainty in bits; **information gain** is how much a split reduces it, the rule decision trees use.
- **Cross-entropy**, or **log loss**, scores predicted probabilities and punishes confident mistakes. Minimizing it is maximizing likelihood, and a base-rate model's log loss is the benchmark to beat.
- **PCA** finds the directions of greatest **variance** using the **eigenvectors** of the **covariance matrix**. **Standardize** first, and read components by their weights.
- Almost every model is **parameters + a loss + an optimizer**.

Practice exercises

Exercises marked (*hand*) need only a calculator. Code exercises use the companion files in `companion/ch35/` and the variables defined in the chapter (`x`, `y`, `customers`, `leads`, and so on); the answers repeat any setup they need. Predict each result before you calculate it.

Warm-up

1. (*hand*) Green Leaf Hotels' product mix is $[0.386, 0.614, 0, 0]$ and Tasty Tiffins' is $[0.096, 0.904, 0, 0]$. Calculate their dot product, both lengths, and their cosine similarity. Are they more or less similar than Sharma Hardware and Coastal Foods (0.614)?
2. (*hand*) A feature matrix **X** has shape 150×6 . Which of these work, and what shape do they return: (a) `x @ w` with `w` of length 6; (b) `x @ w` with `w` of length 150; (c) `x.T @ x`; (d) `x @ x`?
3. (*hand*) For the four customers in section 35.4 and the model *predicted revenue* = $w \times \text{orders}$, calculate the loss and the derivative at $w = 25$. Should w go up or down?
4. (*hand*) What is the entropy, in bits, of a yes-or-no outcome with $p = 0.5$? With $p = 0.9$? Which is more uncertain, and why does that make sense?

Core

5. (*hand, then code*) Continue section 35.6's gradient descent for one more step, starting from $w = 23.279$ and $b = 2.504$ with a learning rate of 0.005. Calculate the errors, the loss, both slopes, and the new w and b . Check with code.

6. Write gradient descent for the one-weight model $\text{predicted revenue} = w \times \text{orders}$ (no intercept) on the four customers, starting at $w = 0$ with a learning rate of 0.005. Print w and the loss after 1, 2, 3, and 20 steps. Compare with the exact answer 24.29.
7. (hand) Riverstone averages 3.33 orders a week. Using the Poisson distribution, what's the probability of **at least one** order in a week? A new customer places orders at an average of 1.5 a month: what's the probability they place none in a given month?
8. Of the 14 website leads, 1 was won. Calculate the log-likelihood of the website leads' outcomes at $p = 1/14$ and at $p = 0.2$. Which value is the maximum likelihood estimate for website leads, and by how much is its log-likelihood higher?
9. Two scoring tools rate four leads, of which the first two were won. Tool A predicts [0.6, 0.4, 0.4, 0.4]; tool B predicts [0.95, 0.05, 0.05, 0.05]. Using a threshold of 0.5, what is each tool's accuracy? What is each tool's log loss? Which would you trust, and why?
10. Repeat section 35.10's standardized PCA on the customers **without** the `revenue` column (six features). What share of variance do the first component and the first two components hold now?

Stretch

- l1. For the four-customer problem with w and b (uncentered), run 50 steps of gradient descent for each learning rate in [0.002, 0.004, 0.008, 0.012, 0.014, 0.016]. Print the final loss for each. Which is the largest learning rate that still ends below the starting loss, and what happens a little above it?
- l2. For every customer, find the **other** customer with the most similar product mix (highest cosine similarity). Print the pairs for Metro Mart, Coastal Foods, and Tasty Tiffins, with their similarities.
- l3. Calculate the information gain of splitting Riverstone's 30 leads into "Referral or Cold call" versus every other source. Is it a better first split than "Website or not" (0.071 bits)?

Think about it (no calculation needed)

- l4. A different vendor claims its churn model is "95% accurate". Riverstone loses about 5% of its customers a year. What two questions would you ask before being impressed, and why?
- l5. A colleague runs k-means clustering (Chapter 38) on customers using `orders` and `revenue` in rupees. Every cluster seems to differ only by revenue. Explain what happened to the manager in two sentences, without the words *variance* or *standardize*.
- l6. A training run's loss is fine for 11 steps, then shows `nan` from step 12 on. Give two likely causes and what you'd try for each.

Key terms

vector · component · dimension · feature · Euclidean distance · length (norm) · element-by-element arithmetic · scaling · dot product · cosine similarity · matrix · shape · feature matrix (X) · target (y) · parameters (weights) · slope and intercept · matrix-times-vector (@) · transpose · mean-centering · loss function · mean squared error (MSE) · derivative · partial derivative · gradient · gradient descent · learning rate · hyperparameter · convergence · stochastic gradient descent · Adam · probability distribution · Bernoulli distribution · binomial distribution · Poisson distribution · normal distribution ·

skewed data · likelihood · maximum likelihood estimate (MLE) · log-likelihood · entropy · bit · information gain · cross-entropy · log loss (binary cross-entropy) · nat · base rate · principal component analysis (PCA) · principal component · variance · covariance · covariance matrix · eigenvector · eigenvalue · standardization · loadings · explained variance

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 36, The Machine Learning Workflow & Feature Engineering**, turns this chapter's scaling lesson into a standard pipeline step, and builds the leakage-free lead-scoring features that Meera worried about.
- **Chapter 37, Supervised Learning Algorithms**, uses every idea here: least squares and gradient descent in linear regression, log loss in logistic regression, likelihood in Naive Bayes, entropy in decision trees, and distance in k-nearest neighbors.
- **Chapter 38, Unsupervised Learning**, applies distance and PCA to cluster Riverstone's customers, and adds t-SNE and UMAP.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, evaluates a real lead-scoring model with log loss, calibration, and cost-based thresholds, starting from the base-rate benchmark.
- **Chapter 43, A First Look at Deep Learning**, computes a neuron by hand and trains a network in PyTorch with the same loop as section 35.6, with gradients calculated automatically.
- **Chapter 54, Generative AI & Large Language Models**, shows dot products between vectors at work inside embeddings and attention.
- **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers gradient descent, learning rates, log loss, bias and variance, and PCA questions with graded answers; the Statistics, Probability & Experimentation Bank (Chapter 73) covers distributions and likelihood.

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Dot product = $0.386 \times 0.096 + 0.614 \times 0.904 = 0.037056 + 0.555056 = \mathbf{0.592112}$. Lengths: $\sqrt{(0.386^2 + 0.614^2)} = \sqrt{(0.148996 + 0.376996)} = \sqrt{0.525992} = 0.72525$; $\sqrt{(0.096^2 + 0.904^2)} = \sqrt{(0.009216 + 0.817216)} = \sqrt{0.826432} = 0.90908$. Cosine = $0.592112 \div (0.72525 \times 0.90908) = 0.592112 \div 0.659310 = \mathbf{0.898}$. They're **more** similar than Sharma and Coastal (0.614): both are kitchen-heavy, though Tasty Tiffins much more so. Check:

```
green_leaf = np.array([0.386, 0.614, 0, 0])
tasty = np.array([0.096, 0.904, 0, 0])
print(round(green_leaf @ tasty, 6),
      round(green_leaf @ tasty / (np.linalg.norm(green_leaf) * np.linalg.norm(tasty)), 3))
```

0.592112 0.898

2. (a) Works: $(150 \times 6) @ (6)$ returns a vector of **150** predictions, one per row. (b) Fails: the matrix has 6 columns, so the vector must have 6 numbers. (c) Works: $(6 \times 150) @ (150 \times 6)$ returns a **6 × 6** matrix, the shape of a covariance matrix. (d) Fails: 6 columns can't meet 150 rows. The rule: the inner numbers must match, and the outer numbers give the result's shape.

3. Predictions $25 \times [4, 7, 8, 12] = [100, 175, 200, 300]$; errors $[34, 1, 2, 3]$; squares $[1,156, 1, 4, 9]$.

$\text{loss}(25) = 1,170 \div 4 = \mathbf{292.5}$

$\text{derivative} = 2 \times (34 \times 4 + 1 \times 7 + 2 \times 8 + 3 \times 12) \div 4 = 2 \times (136 + 7 + 16 + 36) \div 4 = 2 \times 195 \div 4 = \mathbf{97.5}$

The derivative is positive, so increasing w would raise the loss: w should go **down**, toward the best value of 24.29. Notice that Om Sai's single large error (34) dominates both numbers.

4. At $p = 0.5$: $-(0.5 \times \log_2 0.5 + 0.5 \times \log_2 0.5) = -(-0.5 - 0.5) = \mathbf{1 \text{ bit}}$. At $p = 0.9$: $-(0.9 \times \log_2 0.9 + 0.1 \times \log_2 0.1) = -(0.9 \times (-0.1520) + 0.1 \times (-3.3219)) = 0.1368 + 0.3322 = \mathbf{0.469 \text{ bits}}$. The 50–50 outcome is more uncertain: you have no idea which way it will go. At 90%, you'd usually guess right, so there's less uncertainty to remove.

5. Predictions: $23.279 \times 4 + 2.504 = 95.62$; $\times 7 + 2.504 = 165.46$; $\times 8 + 2.504 = 188.74$; $\times 12 + 2.504 = 281.85$. Errors: 29.62, -8.54, -9.26, -15.15. Loss = $(877.3 + 72.9 + 85.7 + 229.5) \div 4 = \mathbf{316.4}$, matching the "after 3" row. ✓ Slope for $w = 2 \times (29.62 \times 4 - 8.54 \times 7 - 9.26 \times 8 - 15.15 \times 12) \div 4 = 2 \times (118.48 - 59.78 - 74.08 - 181.80) \div 4 = \mathbf{-98.6}$. Slope for $b = 2 \times (29.62 - 8.54 - 9.26 - 15.15) \div 4 = \mathbf{-1.67}$. New $w = 23.279 + 0.005 \times 98.6 = \mathbf{23.772}$; new $b = 2.504 + 0.005 \times 1.67 = \mathbf{2.512}$. The code carries full precision, so its last digits differ slightly from the rounded hand values (-15.14, -98.55, -1.66, 2.513):

```
w, b = 0.0, 0.0
for step in range(4):
    error = (w * x + b) - y
    slope_w, slope_b = 2 * np.mean(error * x), 2 * np.mean(error)
    if step == 3:
        print("errors:", error.round(2), " loss:", round(np.mean(error ** 2), 2))
        print(f"slope_w={slope_w:.2f} slope_b={slope_b:.2f}")
    w, b = w - 0.005 * slope_w, b - 0.005 * slope_b
print(f"after step 4: w={w:.3f} b={b:.3f}")
```

```
errors: [ 29.62 -8.54 -9.26 -15.14] loss: 316.36
slope_w=-98.55 slope_b=-1.66
after step 4: w=23.772 b=2.513
```

The slope for b is now tiny (-1.67), so b barely moves: the narrow valley from section 35.6, in numbers.

6.

```
w = 0.0
for step in range(1, 21):
    slope = 2 * np.mean((w * x - y) * x)
    w = w - 0.005 * slope
    if step in (1, 2, 3, 20):
        print(f"after step {step:>2}: w={w:.3f}  loss={np.mean((w * x - y) ** 2):.2f}")
print(f"exact: w={np.sum(x * y) / np.sum(x * x):.3f}")
```

```
after step  1: w=16.575  loss=4315.49
after step  2: w=21.838  loss=666.73
after step  3: w=23.508  loss=298.91
after step 20: w=24.286  loss=257.68
exact: w=24.286
```

With a single weight there's no narrow valley, so descent reaches the exact answer, 24.286, well within 20 steps. The loss at the best one-weight line (257.68) is higher than the two-parameter best (91.55): the intercept helps this model fit.

7. $P(\text{at least one}) = 1 - P(0) = 1 - e^{(-3.33)} = 1 - 0.0359 = \mathbf{0.964}$, or 96.4%. For the new customer, $P(0 \text{ in a month}) = e^{(-1.5)} = \mathbf{0.223}$: they'd go a whole month without ordering in about one month out of four and a half, **with no change in behavior**. A "quiet month" alert for such a customer would fire often by chance alone.

8.

```
website = leads.loc[leads["source"] == "Website", "won"].to_numpy()
wins, n = website.sum(), len(website)

def bernoulli_log_likelihood(p):
    return wins * np.log(p) + (n - wins) * np.log(1 - p)

print(f"{n} website leads, {wins} won")
print(f"log-likelihood at p=1/14: {bernoulli_log_likelihood(1 / 14):.4f}")
print(f"log-likelihood at p=0.2: {bernoulli_log_likelihood(0.2):.4f}")
print(f"difference: {bernoulli_log_likelihood(1 / 14) - bernoulli_log_likelihood(0.2):.4f}")
```

```
14 website leads, 1 won
log-likelihood at p=1/14: -3.6025
log-likelihood at p=0.2: -4.5103
difference: 0.9078
```

The MLE for website leads is their own win rate, $1/14 = \mathbf{0.071}$, and its log-likelihood is about 0.91 higher. That's a likelihood ratio of $e^{0.91}$, about 2.5: the website leads' outcomes are 2.5 times as likely under 7.1% as under 20%. With only 14 leads that's suggestive, not conclusive; Chapter 22 shows how to test it.

9.

```
outcome = np.array([1, 1, 0, 0])
tool_a = np.array([0.6, 0.4, 0.4, 0.4])
tool_b = np.array([0.95, 0.05, 0.05, 0.05])
for name, pred in [("A", tool_a), ("B", tool_b)]:
    accuracy = np.mean((pred >= 0.5) == outcome)
    print(f"tool {name}: accuracy {accuracy:.0%}, log loss {log_loss(outcome, pred):.3f}")
```

```
tool A: accuracy 75%, log loss 0.612
tool B: accuracy 75%, log loss 0.787
```

Both tools are **75%** accurate: each gets the second lead wrong. But tool A's log loss (0.612) is lower than tool B's (0.787). Tool B was 95% sure the second lead would be lost, and it was won; that one confident error costs $-\ln 0.05 = 3.00$, more than all of tool A's costs together. Trust **tool A**: its probabilities are more honest. Accuracy can't see the difference; log loss can. (With four leads, this is only an illustration: Chapter 39 evaluates on hundreds.)

10.

```
features_6 = [f for f in features if f != "revenue"]
F6 = customers[features_6].to_numpy(dtype=float)
share_6, _, _ = pca((F6 - F6.mean(axis=0)) / F6.std(axis=0))
print("share of variance:", share_6.round(3))
print(f"PC1: {share_6[0]:.1%}    PC1 + PC2: {share_6[:2].sum():.1%}")
```

```
share of variance: [0.542 0.199 0.144 0.092 0.022 0.    ]
PC1: 54.2%    PC1 + PC2: 74.1%
```

Almost nothing changes: the first component holds 54.2% instead of 53.5%, and the first two hold 74.1% instead of 73.9%. Revenue is largely orders $\times$ average order value, so the other columns already carried most of what it said. When dropping a column barely changes a PCA, that column was mostly repeating the others.

11.

```
for lr in [0.002, 0.004, 0.008, 0.012, 0.014, 0.016]:
    _, _, run = descend(x, lr, 50)
    print(f"learning rate {lr:<5}: loss after 50 steps = {run[-1]:>14,.2f}")
```

```
learning rate 0.002: loss after 50 steps =      274.23
learning rate 0.004: loss after 50 steps =      265.94
learning rate 0.008: loss after 50 steps =      250.46
learning rate 0.012: loss after 50 steps =      236.33
learning rate 0.014: loss after 50 steps =      289.34
learning rate 0.016: loss after 50 steps = 6,239,746,262,063.38
```

Up to **0.014**, the loss ends far below its starting value of 40,511. At 0.014 it converges only slowly, because each step overshoots and swings back a little less each time. At 0.016 each overshoot is bigger than the last, and the loss explodes. The boundary is about 0.01447 (section 35.6's Try it), and it's sharp: a learning rate a few percent too high doesn't train slowly, it fails completely. That's why the safe habit is to start low and increase.

12.


```
lengths = np.linalg.norm(mix, axis=1)
similarity = (mix @ mix.T) / np.outer(lengths, lengths)
np.fill_diagonal(similarity, -1) # ignore each customer's match with itself
for who in ["Metro Mart", "Coastal Foods", "Tasty Tiffins"]:
    i = list(names).index(who)
    j = np.argmax(similarity[i])
    print(f"{who:<14} -> {names[j]:<23} {similarity[i, j]:.4f}")
```

```
Metro Mart      -> Evergreen Mart      1.0000
Coastal Foods   -> Harbour Traders     0.9994
Tasty Tiffins   -> Sea Breeze Hotel     0.9939
```

How it works: `mix @ mix.T` takes the dot product of every customer with every other customer in one step, a 23×23 matrix. `np.outer(lengths, lengths)` builds the matching 23×23 table of length products, so dividing gives all cosine similarities at once. Setting the diagonal to -1 stops a customer matching itself. This all-pairs similarity matrix is exactly what Chapter 42's item-to-item recommender computes, with products instead of customers.

13.

```
def entropy(p):
    return 0.0 if p in (0, 1) else -(p * np.log2(p) + (1 - p) * np.log2(1 - p))

in_group = leads["source"].isin(["Referral", "Cold call"])
parent = entropy(leads["won"].mean())
children = sum(len(g) / len(leads) * entropy(g["won"].mean())
               for _, g in leads.groupby(in_group))
print(f"group sizes: {in_group.sum()} referral or cold call, {(~in_group).sum()} other")
print(f"won: {leads.loc[in_group, 'won'].sum()} and {leads.loc[~in_group, 'won'].sum()}")
print(f"information gain: {parent - children:.3f} bits")
```

```
group sizes: 8 referral or cold call, 22 other
won: 4 and 2
information gain: 0.133 bits
```

Referral and cold-call leads won 4 of 8 (50%); every other source won 2 of 22 (9.1%). The information gain is larger than the website split's 0.071 bits, so a decision tree would choose this split first. With 30 leads, though, one or two different outcomes would change the ranking; Chapter 37 shows how trees guard against splitting on noise.

14. First: *what does a model that predicts "no churn" for everyone score?* At a 5% churn rate, it's 95% accurate, the same as the vendor's model, while catching nobody. Second: *how well does it rank and predict probabilities on our data?* Ask for log loss (compared with the base-rate benchmark) and for how many of the customers who actually churned it flagged (recall, Chapter 39), measured on customers the model wasn't trained on. The common wrong reaction is to be impressed because 95% is close to 100%.

15. "The method groups customers by how far apart they are, and a difference of ₹1 counted exactly as much as a difference of one order. Since revenue differences run into lakhs and order differences into single digits, revenue decided everything, and the order counts were effectively ignored." (The fix is to put both on the same scale first, section 35.10.)

16. Most likely, **the learning rate is too high**: the loss grows each step until the numbers overflow to `inf` and then `nan`. Try dividing the learning rate by 10, and plot the loss for the first 12 steps to see whether it was already rising. Second, **a value that breaks a formula**: a missing value in the data, a division by zero, or a predicted probability of exactly 0 or 1 inside a log. Check the inputs for missing values, and check the batch or row being processed at step 12. (A third cause, features on very different scales, makes the first cause more likely; scaling often fixes both.)

Chapter 36. The Machine Learning Workflow & Feature Engineering

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: turn a business request into a machine learning problem with a clear target, prediction moment, and success measure · split data into training, validation, and test sets, and choose between a random and a time-based split · use cross-validation and read the spread of its scores · find data leakage with one question, and measure what each leak does to a score · engineer features from categories, numbers, dates, text, and activity logs · handle missing values so the model learns from them instead of choking on them · build a scikit-learn pipeline that makes leakage from preprocessing impossible · set simple baselines and judge a model against them.

Before you start: Chapter 13 (deduplicating leads and funnels in SQL), Chapter 17 (Python basics), Chapter 18 (pandas), and Chapter 35 (loss, log loss, scaling, and the base-rate benchmark). Chapter 21's idea of a sample helps.

Time needed: 8–12 hours over one to two weeks, including the project.

Tools: Python 3 with pandas, NumPy, and scikit-learn (all free). Everything runs on a laptop in seconds.

Practice data: Riverstone's full CRM export, built by `companion/generate_riverstone_crm.py` : `leads.csv` (12,294 rows), `activities.csv`, and `stage_history.csv`, covering every enquiry from January 2023 to December 2025. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Most machine learning failures at work aren't caused by choosing the wrong algorithm. They're caused by what happens before the algorithm:

- A model scores 98% in the notebook and is useless in production, because one column was filled in *after* the outcome was known.
- A model is tested on customers it has already seen, so the test says nothing about new ones.
- Everything is scaled using the whole dataset, and the test score quietly borrows information from the test set.
- Nobody checks whether the model beats "call referral leads first", a rule a sales manager could apply with no model at all.

This chapter is about the **workflow** that prevents those failures, and about **feature engineering**, the craft of turning raw columns into inputs a model can learn from. Algorithms change every few years; this workflow doesn't. It's also what interviewers probe hardest, because it separates people who have trained models from people who have shipped them.

You'll build it end to end on one real task: scoring Riverstone's incoming sales leads so reps know whom to call first.

In plain English

Think of a school exam.

A teacher wants to know whether students have *learned* the subject, not whether they've memorized last year's paper. So the teacher keeps the real exam questions locked away (the **test set**), lets students practice on past papers (the **training set**), and gives a mock exam halfway through to see who's ready (the **validation set**). If the real questions leak before the exam, every score goes up and means nothing: that's **data leakage**.

Students who revise with good notes do better than students who read the textbook raw. Summarizing a chapter into a few key points, turning dates into "which term", grouping similar topics together: that's **feature engineering**.

The teacher also runs the exam the same way every year: same rules, same timing, same marking. Writing that procedure down so nobody can accidentally hand out the answers is what a **pipeline** does.

And before praising a student for scoring 60%, the teacher checks what a student would score by guessing. That's a **baseline**.

36.1 Framing the problem

From a request to a question a model can answer

Anita Rao, Riverstone's Sales Head, has a problem. Enquiries have grown from about 290 a month in 2023 to about 400 in 2025, most of them from a B2B marketplace listing. Her three sales executives and the inside sales desk can't call everyone quickly. Her request: *"Can we tell which leads are worth calling first?"*

That's a business request, not yet a machine learning problem. Before touching data, pin down five things:

Decision	Riverstone lead scoring
Unit (one row = ?)	One real enquiry (duplicates removed)
Target (what we predict)	Won within 90 days of arriving: 1 or 0

Decision	Riverstone lead scoring
Prediction moment (when we predict)	24 hours after the lead arrives, when the morning call list is made
Allowed information	Only what is recorded in the CRM by that moment
Success measure	Better ranking than the current rule, and better probabilities than the base rate (log loss, Chapter 35)

This is a **supervised learning** problem: you have past examples with known answers (won or lost) and want to predict the answer for new ones. Because the target has two values, it's a **binary classification** problem. Predicting a number, such as order value, would be **regression**. Chapter 37 covers the algorithms for both.

The **prediction moment** is the decision people most often skip, and it's the one that prevents most leakage. Every feature must pass one test: *would this value be in the CRM 24 hours after the lead arrived?*

Watch out: a vague target makes a vague model. "Good lead" isn't a target. "Won" isn't quite one either, until you say *by when*. Without the 90-day window, a lead from last week that will be won next month counts as "lost" today, and the model learns that recent leads are bad.

A first look at the data

```
import numpy as np
import pandas as pd

leads = pd.read_csv("leads.csv", parse_dates=["created_at"])
print(leads.shape)
print(
    leads[
        [
            "lead_id",
            "created_at",
            "source",
            "segment",
            "company_size",
            "est_quantity",
            "status",
        ]
    ].head(4)
    .to_string(index=False)
)
print(leads["status"].value_counts().to_string())
```

```
(12294, 17)
lead_id      created_at      source      segment  company_size  est_quantity  status
100001 2023-01-01 08:39:23 Marketplace    Retail         11-50         78.0    Lost
100002 2023-01-01 08:58:54      Website    Retail         1-10         65.0    Lost
100003 2023-01-01 09:16:29 Marketplace      NaN         200+        1009.0    Won
100004 2023-01-01 09:38:54      Referral  Wholesale         11-50         103.0    Lost

status
Lost      10365
Open       1065
Won         864
```

This is Riverstone's full CRM, much bigger than the `leads` table in Chapter 13's one-year database, which holds only the enquiries reps logged themselves. The export also includes every marketplace and web-form lead handled by the inside sales desk, a small team that works through high-volume enquiries. It was taken at the end of 31 December 2025. It has 12,294 rows and 17 columns. Of these, 1,065 leads are still **Open**, so their outcome isn't known yet.

Only use leads whose outcome is settled

Leads that arrived in the last 90 days of 2025 haven't had their full 90 days. Some that will be won are still open, but the ones that were lost quickly are already marked Lost. Look at the win rate among closed leads by the quarter they arrived:

```
closed = leads[leads["status"] != "Open"]
quarter = closed["created_at"].dt.to_period("Q")
by_quarter = closed.groupby(quarter)["status"].apply(lambda s: (s == "Won").mean())
print((by_quarter.tail(6) * 100).round(1).to_string())
```

```
created_at
2024Q3      9.0
2024Q4      7.4
2025Q1      7.2
2025Q2      5.6
2025Q3      8.7
2025Q4     12.5
Freq: Q-DEC
```

The win rate jumps to 12.5% in the last quarter of 2025. That's not a sales miracle. Fewer than a quarter of that quarter's leads have closed by the export, and the ones that closed are the fast outcomes: a win often closes within weeks, while many lost leads stay open until they're closed by default at 90 days. Training on those rows would teach the model that October-to-December leads are unusually good. (With other closing habits, the bias runs the other way.) So the rule is: **only use leads created at least 90 days before the export**, which means on or before 2 October 2025.

This problem, where recent outcomes are incomplete, is called **censoring** (or *right-censoring*), and it appears in churn, credit default, and anything with a waiting period.

36.2 One row per real enquiry

Machine learning needs a clean table: one row per unit, one column per feature, and a target column. Chapter 13 removed duplicate web-form submissions in SQL; here's the same rule in pandas, plus the city spellings.

```
leads["email_norm"] = leads["email"].str.lower().str.strip()
leads = leads.sort_values("created_at")
gap = leads.groupby("email_norm")["created_at"].diff()
is_duplicate = gap.notna() & (gap <= pd.Timedelta(days=2))
print("duplicate submissions removed:", is_duplicate.sum())
leads = leads[~is_duplicate].copy()

export_time = pd.Timestamp("2025-12-31 23:59")
leads = leads[leads["created_at"] <= export_time - pd.Timedelta(days=90)].copy()
leads["won"] = (leads["status"] == "Won").astype(int)

city_fix = {
    "bombay": "Mumbai",
    "mumbai.": "Mumbai",
    "bangalore": "Bengaluru",
    "b'lore": "Bengaluru",
    "new delhi": "Delhi",
    "delhi ncr": "Delhi",
    "poona": "Pune",
    "madras": "Chennai",
    "calcutta": "Kolkata",
    "panaji": "Goa",
}

city_lower = leads["city"].str.strip().str.lower()
leads["city_clean"] = city_lower.map(city_fix).fillna(city_lower.str.title())

print(f"{len(leads):,} leads, {leads['won'].sum()} won ({leads['won'].mean():.1%})")
print(
    leads["city_clean"].nunique(),
    "cities after cleaning (was",
    leads["city"].nunique(),
    "spellings)",
    sep=" ",
)
```

```
duplicate submissions removed: 271
10,701 leads, 826 won (7.7%)
17 cities after cleaning (was 35 spellings)
```

How it works:

- Emails are lower-cased and trimmed first, so `METR012@...` and `metro12@...` count as the same person.
- `groupby(...).diff()` gives the time since the same email's previous lead. A repeat within two days is a resubmission of the same enquiry, not a new one.
- The city map sends each alternative spelling to one name; `.str.title()` fixes case for the rest ("pune" becomes "Pune").

Reconcile. 12,294 rows – 271 duplicates = 12,023 real enquiries, of which 10,701 arrived by 2 October 2025. They include 826 wins, a win rate of **7.7%**. That's the base rate: out of every 13 leads, about one is won.

Real-world note. The generator planted 268 duplicates; the two-day rule removed 271. Three genuine enquiries from the same email happened to fall within two days of each other. Every deduplication rule has edge cases; write the rule down, and count what it removes.

36.3 Train, validation, and test sets

Why three sets

A model's score on the data it trained on says little about how it will do on new data. A flexible enough model can memorize the training rows. So you hold data back:

- The **training set** is what the model learns from.
- The **validation set** is what you use to compare choices: which features, which settings, which model. You look at it many times.
- The **test set** is locked away and used **once**, at the very end, to report how the final model does on data nobody tuned against.

Every time you check the validation score and change something, a little information about the validation set leaks into your decisions. After fifty rounds of "try something, check validation", the validation score is optimistic. The untouched test set is what keeps you honest.

Split by time when the future is what you'll predict

The usual default is a **random split**: shuffle the rows and cut. That's fine when rows are independent and the world doesn't change. Riverstone's model will score *next month's* leads using a model trained on the past, and the mix of leads is changing. So split by **time**, the way the model will be used:


```

train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]

for name, part in [
    ("train (2023-2024)", train),
    ("valid (Jan-Jun 2025)", valid),
    ("test (Jul-2 Oct 2025)", test),
]:
    print(f"{name:<22} {len(part):>6,} leads    win rate {part['won'].mean():.2%}")

yearly = leads.groupby(leads["created_at"].dt.year)
print(
    "\nmarketplace share:  ",
    (yearly["source"].apply(lambda s: (s == "Marketplace").mean()) * 100)
    .round(1)
    .to_dict(),
)
print(
    "marketplace win rate:",
    (
        leads[leads["source"] == "Marketplace"]
        .groupby(leads["created_at"].dt.year)["won"]
        .mean()
        * 100
    )
    .round(2)
    .to_dict(),
)

```

```

train (2023-2024)      7,291 leads    win rate 7.90%
valid (Jan-Jun 2025)   2,225 leads    win rate 6.56%
test (Jul-2 Oct 2025)  1,185 leads    win rate 8.78%

marketplace share:     {2023: 30.1, 2024: 34.9, 2025: 44.5}
marketplace win rate:  {2023: 2.65, 2024: 3.23, 2025: 1.52}

```

The model learns from the past and is judged on later leads

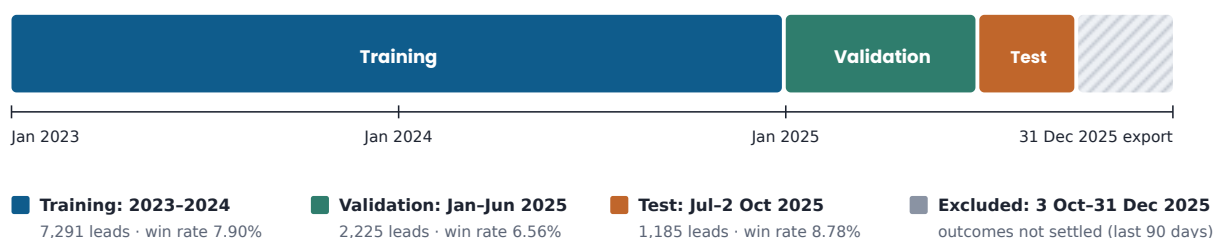


Figure 36.1 – Riverstone's time-based split. The model learns from the past and is judged on later leads, the way it will be used. The last 90 days are left out because their outcomes aren't settled.

Reading it. The world did change. The marketplace's share of leads rose from 30.1% in 2023 to 44.5% in 2025, and its win rate halved in 2025, to 1.52%, after Riverstone switched to a cheaper listing plan. A time split puts that change where it belongs: in the future the model must cope with. A random split would mix 2025 leads into training and hide it.

Watch out: scores from different splits can't be compared. You'll see in section 36.9 that on this data, a random split's score happens to be *lower* than the time split's. The 2025 validation set is full of marketplace leads that are clearly weak and rank low without difficulty, and a score depends on the mix of the rows it's measured on. The time split isn't chosen because it gives a better or worse number; it's chosen because it asks the right question. Compare models only on the **same** split.

The rows for the three sets are chosen now and don't change for the rest of the chapter. The test set isn't used again until section 36.9.

36.4 Cross-validation

A single validation set gives a single number, and with 2,225 leads and about 150 wins, that number is noisy. **Cross-validation** reuses the training data to get several estimates.

In **k-fold cross-validation**, the training rows are split into k equal parts (**folds**). The model is trained k times, each time holding out one fold for scoring and training on the rest. You get k scores; their average is the estimate and their spread shows how much it wobbles.

- **Stratified k-fold** keeps the win rate the same in every fold. With a 7.7% win rate, a plain random fold could get noticeably fewer wins than average; stratifying prevents that. Use it for classification.
- **Time series split** trains on earlier folds and tests on the next one, always forward in time. Use it when order matters.

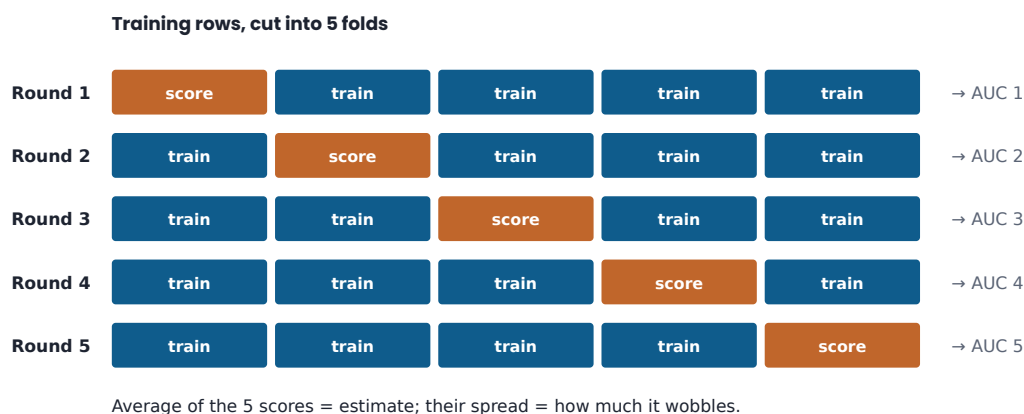


Figure 36.2 — Five-fold cross-validation. Every row is scored exactly once, by a model that didn't train on it.

Cross-validation needs a model to evaluate. Here's a first, deliberately simple model: logistic regression (Chapter 37) on a few columns, wrapped in a pipeline. Section 36.8 explains every piece of the pipeline; for now, treat it as "prepare the columns, then fit the model".

```

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, TimeSeriesSplit, cross_val_score

def make_model(categorical, numeric):
    prepare = ColumnTransformer(
        [
            (
                "cat",
                Pipeline(
                    [
                        (
                            "fill",
                            SimpleImputer(strategy="constant", fill_value="Missing"),
                        ),
                        ("onehot", OneHotEncoder(handle_unknown="ignore")),
                    ]
                ),
                categorical,
            ),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                        ("scale", StandardScaler()),
                    ]
                ),
                numeric,
            ),
        ]
    )
    return Pipeline(
        [("prepare", prepare), ("model", LogisticRegression(max_iter=1000))]
    )

first_cats = ["source", "segment", "company_size"]
first_nums = ["website_visits"]
X_first, y_train = train[first_cats + first_nums], train["won"]

folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=36)
scores = cross_val_score(
    make_model(first_cats, first_nums), X_first, y_train, cv=folds, scoring="roc_auc"
)
print(
    "stratified 5-fold AUC:",
    scores.round(3),
    " mean",
    scores.mean().round(3),
    " sd",
    scores.std().round(3),
)

forward = cross_val_score(
    make_model(first_cats, first_nums),
    X_first,
    y_train,
    cv=TimeSeriesSplit(n_splits=5),
    scoring="roc_auc",
)

```

```
)
print("time series split AUC:", forward.round(3), " mean", forward.mean().round(3))

stratified 5-fold AUC: [0.734 0.746 0.743 0.733 0.746] mean 0.741 sd 0.006
time series split AUC: [0.68 0.749 0.775 0.742 0.735] mean 0.736
```

How it works:

- `cross_val_score` runs the whole train-and-score loop for you: five fits, five scores. Because the full pipeline is passed in, the filling and scaling are re-learned inside each fold, from that fold's training rows only. That detail matters in section 36.5.
- `shuffle=True, random_state=36` makes the folds random but repeatable. Every random step in this book has a fixed seed, so you get the same numbers.
- `TimeSeriesSplit` needs rows in time order, which `train` already is (it was sorted by `created_at` in section 36.2). Its first fold trains on only the earliest sixth of the data.

AUC, briefly. The score here is **ROC-AUC** (area under the ROC curve). It has a plain meaning: *pick one won lead and one lost lead at random; AUC is the chance the model gives the won lead the higher score.* 0.5 is a coin toss; 1.0 is perfect ranking. It measures ranking, which is exactly what a call list needs. Chapter 39 builds the ROC curve and explains AUC's limits; log loss (Chapter 35) measures whether the probabilities themselves are right, and this chapter uses both.

Reading it. Even with four simple columns, the model ranks leads far better than chance, around 0.74. The five stratified folds agree closely here (a standard deviation of 0.006); the time-series folds vary much more, from 0.68 for the first fold, which trains on only the earliest sixth of the data, to 0.775. That spread is the honest size of your uncertainty: a change to the model that moves the mean score by 0.005 is noise, not progress.

36.5 Data leakage

The one question

Data leakage means information the model won't have when it's used gets into training or evaluation. Leakage makes scores look better than reality, sometimes spectacularly. Almost every leak is caught by asking, for each feature and each step: **would this be known at the prediction moment?**

Leakage comes in two kinds:

1. **Target leakage:** a feature is recorded after the outcome, or because of it.
2. **Train-test contamination:** information from the validation or test rows gets into training, usually through preprocessing.

The CRM export contains three columns that look like excellent features. Test each against the prediction moment, 24 hours after arrival:

Column	What it records	Known at 24 hours?
<code>quote_sent_date</code>	When a quote was sent	No. Quotes go to qualified leads, days or weeks later
<code>days_in_pipeline</code>	Days from arrival to closing	No. Only known once the lead is closed
<code>first_response_hours</code>	Hours until the first contact	Only if the answer is 24 or less. A value of 60 means "still not contacted at 24 hours", which is known; the exact 60 isn't

Now measure what each does to the score. The legitimate features are built in section 36.6; to keep this section self-contained, the code below adds those features first and then runs one model with each suspicious column added.

```

def add_features(df):
    df = df.copy()
    text = df["enquiry_text"].str.lower()
    df["text_strong"] = text.str.contains(
        "bulk|tender|urgent|monthly|new outlet"
    ).astype(int)
    df["text_weak"] = text.str.contains(
        "price list|sample|checking rates|catalogue|price please"
    ).astype(int)
    df["free_email"] = df["email_norm"].str.endswith("@gmail.com").astype(int)
    cap = 50_000 # quantities above this are typos
    df["log_quantity"] = np.log1p(df["est_quantity"].where(df["est_quantity"] <= cap))
    df["inside_desk"] = (df["owner_id"] == 9).astype(int)
    df["responded_24h"] = (df["first_response_hours"] <= 24).astype(int)
    df["has_quote"] = df["quote_sent_date"].notna().astype(int)
    return df

activities = pd.read_csv("activities.csv", parse_dates=["activity_at", "logged_at"])
deadline = leads.set_index("lead_id")["created_at"] + pd.Timedelta(hours=24)
visible = activities[activities["logged_at"] <= activities["lead_id"].map(deadline)]
leads["activities_24h"] = (
    leads["lead_id"].map(visible.groupby("lead_id").size()).fillna(0)
)

leads = add_features(leads)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]

cats = ["source", "segment", "city_clean", "company_size", "product_interest"]
nums = [
    "log_quantity",
    "website_visits",
    "free_email",
    "text_strong",
    "text_weak",
    "inside_desk",
    "activities_24h",
    "responded_24h",
]

from sklearn.metrics import roc_auc_score, log_loss

def score(categorical, numeric, fit_rows=None, eval_rows=None):
    fit_rows = train if fit_rows is None else fit_rows
    eval_rows = valid if eval_rows is None else eval_rows
    model = make_model(categorical, numeric).fit(
        fit_rows[categorical + numeric], fit_rows["won"]
    )
    p = model.predict_proba(eval_rows[categorical + numeric])[:, 1]
    return roc_auc_score(eval_rows["won"], p), log_loss(eval_rows["won"], p)

print("honest features:          AUC %.3f  log loss %.4f" % score(cats, nums))
for leak in ["has_quote", "days_in_pipeline", "first_response_hours"]:
    print(f"+ {leak}<27} AUC %.3f  log loss %.4f" % score(cats, nums + [leak]))

```

honest features:	AUC 0.823	log loss 0.1948
+ has_quote	AUC 0.978	log loss 0.0889
+ days_in_pipeline	AUC 0.875	log loss 0.1708
+ first_response_hours	AUC 0.831	log loss 0.1926

Adding one leaky column to the honest model

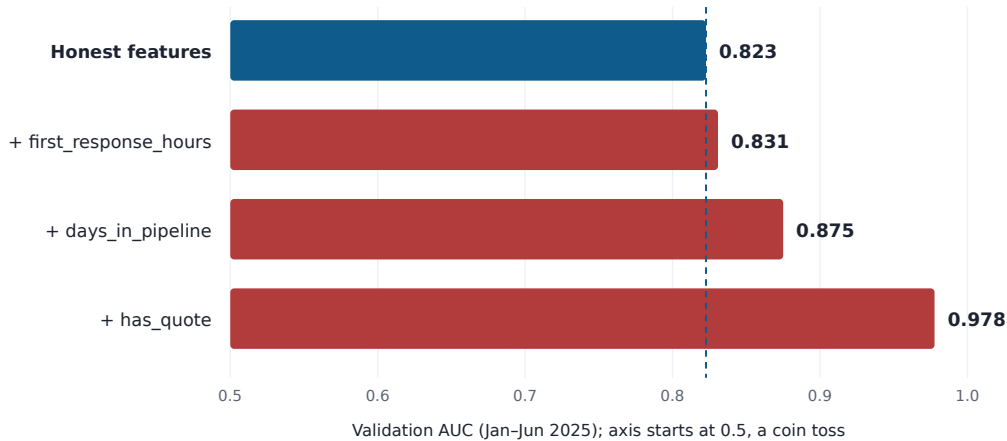


Figure 36.3 — What each leaky column does to the validation score. The bigger the jump, the easier the leak is to spot; the small one is the dangerous one.

Reading it.

- **has_quote** lifts AUC from 0.823 to **0.978**. A model that "knows" a quote was sent is nearly perfect, because Riverstone only quotes leads that are already going well. In production, at 24 hours, no lead has a quote yet: every value is 0, and the model collapses. A score that jumps this much should make you suspicious, not happy.
- **days_in_pipeline** lifts AUC to 0.875. Lost leads often close at exactly 90 days by default; won leads close whenever the order comes. The number is created by the outcome.
- **first_response_hours** lifts AUC only from 0.823 to **0.831**. That's the dangerous kind: plausible, modest, and readily explained away as "fast response wins deals". Part of it is real and already captured by `responded_24h`. The rest comes from two leaks. Reps respond faster to leads they already sense are good, so the model learns the reps' judgment (Chapter 35's warning). And the exact hours past 24 aren't known at the prediction moment, while leads never contacted at all, which are nearly always lost, are marked by a missing value that only exists once the 90 days are over.

Watch out: activity logs leak through timestamps. `activities.csv` has two times: when the call happened (`activity_at`) and when the rep logged it (`logged_at`). About 1 in 10 activities is logged days late, with the call's real time filled in. At 8 a.m. the next morning, a late-logged call is invisible. The code above counts only activities **logged** within 24 hours. Counting by `activity_at` would use calls the CRM didn't yet show. On this data the difference is small, but the habit matters: for every timestamp, ask which clock it is.

Contamination: target encoding on all the data

The second kind of leak happens in preprocessing. **Target encoding** replaces each category with the average target for that category, for example "the win rate of companies with this name". It's useful for columns with hundreds of values. Done carelessly, it's a leak:

```

wrong_rates = leads.groupby("company_name")["won"]
].mean() # uses ALL leads, including valid and test
leads["company_rate_wrong"] = leads["company_name"].map(wrong_rates)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]

print(leads["company_name"].nunique(), "distinct company names")
print(
    "with company win rate from ALL data: AUC %.3f  log loss %.4f"
    % score(cats, nums + ["company_rate_wrong"])
)

from sklearn.preprocessing import TargetEncoder

right = Pipeline(
    [
        (
            "prepare",
            ColumnTransformer(
                [
                    (
                        "cat",
                        Pipeline(
                            [
                                (
                                    "fill",
                                    SimpleImputer(
                                        strategy="constant", fill_value="Missing"
                                    ),
                                ),
                                ("onehot", OneHotEncoder(handle_unknown="ignore")),
                            ]
                        ),
                        cats,
                    ),
                    ("company", TargetEncoder(random_state=36), ["company_name"]),
                    (
                        "num",
                        Pipeline(
                            [
                                (
                                    "fill",
                                    SimpleImputer(
                                        strategy="median", add_indicator=True
                                    ),
                                ),
                                ("scale", StandardScaler()),
                            ]
                        ),
                        nums,
                    ),
                ],
            ),
        ),
        ("model", LogisticRegression(max_iter=1000)),
    ]
)
right.fit(train[cats + nums + ["company_name"]], train["won"])
p = right.predict_proba(valid[cats + nums + ["company_name"]])[:, 1]
print(
    "with TargetEncoder inside the pipeline: AUC %.3f  log loss %.4f"

```

```
% (roc_auc_score(valid["won"], p), log_loss(valid["won"], p))
)
```

```
300 distinct company names
with company win rate from ALL data: AUC 0.845 log loss 0.1849
with TargetEncoder inside the pipeline: AUC 0.823 log loss 0.1948
```

Reading it. Company names like "Lotus Kitchenware" carry no real signal in this data; they're generated at random. Yet encoding them with win rates computed on *all* leads lifts validation AUC from 0.823 to **0.845**, because each validation lead's own outcome went into its company's average. Done properly, with scikit-learn's `TargetEncoder` learning the rates from training rows only (and cross-fitting within them), the gain disappears: AUC is back at 0.823. The honest answer is that company name doesn't help.

How it works: `TargetEncoder` learns each category's target average (shrunk toward the overall average for rare categories) from the rows it's fitted on. When a pipeline is fitted, it calls the encoder's `fit_transform`, which uses internal **cross-fitting**: the training rows are split into folds, and each fold is encoded with averages learned from the other folds, so no training row's own outcome is used to encode that row. When the pipeline later transforms validation rows, it applies the averages learned from all the training rows. (Calling `fit` and then `transform` on the same rows skips the cross-fitting, which is why the encoder belongs inside the pipeline.)

A leakage checklist

Use this before trusting any model:

Check	Question to ask
Prediction moment	Is it written down, and is every feature available at that moment?
Timestamps	For every date, is it <i>when it happened</i> or <i>when it was recorded</i> ?
Outcome-driven fields	Was any field filled in by a process that only happens to some outcomes (quotes, approvals, cancellations)?
Missing values	Could "missing" mean "this hadn't happened yet"?
Duplicates	Does the same customer or enquiry appear in both training and test?
Preprocessing	Were scaling, filling, encoding, or feature selection fit on anything except training rows?
Target encodings and aggregates	Did any average include the row's own target, or future rows?
Too good to be true	Is the score far above what domain experts manage? Then assume a leak until proven otherwise

36.6 Feature engineering

A **feature** is a column the model learns from. **Feature engineering** turns raw columns into features that make patterns easier to learn. For the model families in this part of the book, especially linear models, it usually matters more than the choice of algorithm.

Numbers: transform, cap, and scale

`est_quantity` is the quantity the customer typed into the enquiry form. Look at it:

```
q = leads["est_quantity"]
print(q.describe(percentiles=[0.5, 0.9, 0.99]).round(0).to_string())
print("values above 50,000:", (q > 50_000).sum())
```

```
count      9161.0
mean       1120.0
std        18160.0
min         1.0
50%        130.0
90%        687.0
99%       3441.0
max      1214000.0
values above 50,000: 35
```

The median is 130 units and the 99th percentile about 3,400, but the maximum is over 1.2 million, and 35 values are above 50,000. Those extreme values are typing mistakes: someone entering "1200" in a field that already assumed "thousands", or adding zeros. Three steps handle this:

1. **Cap or remove impossible values.** Riverstone has never received an order anywhere near 50,000 units of one product, so values above that are set to missing, not kept.
2. **Transform skewed numbers.** The `log ( np.log1p`, which is $\ln(1 + x)$ and works for zeros) turns "each extra unit matters the same" into "each doubling matters the same". The difference between 10 and 100 units matters more than the difference between 2,010 and 2,100.
3. **Scale.** `StandardScaler` subtracts each column's mean and divides by its standard deviation, learned from the training rows. Chapter 35 showed why: gradient-based models train slowly, or badly, when features are on wildly different scales, and distance-based models are dominated by the biggest numbers.

When scaling doesn't matter. Decision trees and the tree ensembles built on them (random forests, gradient boosting, Chapter 37) split on one feature at a time at a threshold, so rescaling a feature doesn't change which rows fall on each side. Scaling them is harmless but unnecessary. Linear models, k-nearest neighbors, support vector machines, neural networks, k-means, and PCA all need it.

Categories: one-hot encoding

A model needs numbers, not the word "Referral". **One-hot encoding** creates one column per category, with a 1 in the column that matches and 0 elsewhere. `source` has 6 values, so it becomes 6 columns.

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(handle_unknown="ignore")
encoder.fit(train[["source"]])
sample = pd.DataFrame({"source": ["Referral", "Marketplace", "Newspaper ad"]})
print(encoder.get_feature_names_out())
print(encoder.transform(sample).toarray().astype(int))
```

```
[ 'source_Cold call' 'source_Marketplace' 'source_Partner'
  'source_Referral' 'source_Trade fair' 'source_Website']
[[0 0 0 1 0 0]
 [0 1 0 0 0 0]
 [0 0 0 0 0 0]]
```

How it works:

- The encoder learns the categories from the training rows. `get_feature_names_out` lists the new columns.
- A **Referral** lead becomes a 1 in the `source_Referral` column and 0 in the other five.
- **Newspaper ad** never appeared in training. With `handle_unknown="ignore"`, it becomes all zeros instead of crashing the model on the first unexpected value in production. New categories will appear; decide now how to treat them.
- The result is stored as a **sparse matrix**, which saves memory when most values are 0; `.toarray()` prints it as a normal grid.

Which encoding when:

Column type	Example	Encoding
Few categories, no order	source , segment	One-hot
Categories with a real order	company_size : 1-10 < 11-50 < 51-200 < 200+	One-hot, or ordinal numbers 1-4 if the order is meaningful to the model
Hundreds or thousands of categories	company name, postcode, product SKU	Target encoding inside the pipeline, grouping rare values into "Other", or leave it out
Free text	enquiry_text	Keyword flags (below), or TF-IDF (Chapter 41)

Dates and times

A timestamp is rarely useful raw; the model can't learn from "2024-10-14 11:32". Pull out the parts that could matter: **month** (festive season), **day of week** (weekend enquiries are often casual), **hour** (office-hours enquiries come from businesses), and **time since** something happened (days since the customer's last enquiry). Never use a date that's in the future relative to the prediction moment.

```

created = leads["created_at"]
date_parts = pd.DataFrame(
    {
        "month": created.dt.month,
        "weekday": created.dt.dayofweek,
        "hour": created.dt.hour,
        "won": leads["won"],
    }
)
by_month = (
    date_parts[date_parts["month"].isin([6, 7, 8, 9, 10, 11])]
    .groupby("month")["won"]
    .mean()
)
print("win rate by month (Jun-Nov):", (by_month * 100).round(1).to_dict())
hosp = leads[leads["segment"] == "Hospitality"]
festive = hosp[hosp["created_at"].dt.month.isin([8, 9, 10])]
print(
    f"hospitality win rate, Aug-Oct {hosp.loc[festive, 'won'].mean():.1%}"
    f" vs other months {hosp.loc[~festive, 'won'].mean():.1%}"
)

```

```

win rate by month (Jun-Nov): {6: 6.6, 7: 7.3, 8: 9.2, 9: 10.1, 10: 8.5, 11: 7.9}
hospitality win rate, Aug-Oct 8.9% vs other months 6.4%

```

Hospitality leads that arrive from August to October, as hotels and caterers prepare for the festive season, are won more often. A month feature alone gives a linear model an average monthly effect; the hospitality-in-festive-season effect is an **interaction** (two features together), which trees find on their own and linear models need built for them. Chapter 37 compares both.

Text: start with flags

`enquiry_text` is short free text. A good first step is **keyword flags**: 1 if the text mentions a strong buying signal ("bulk", "tender", "urgent"), 1 if it signals browsing ("price list", "sample only").

`add_features` built both:

```

flag_rates = leads.groupby(["text_strong", "text_weak"])["won"].agg(["count", "mean"])
flag_rates["mean"] = (flag_rates["mean"] * 100).round(1)
print(flag_rates.rename(columns={"count": "leads", "mean": "win rate %"}).to_string())
print(
    "free email (gmail) win rate: %.1f%% vs business email: %.1f%%"
    % tuple(leads.groupby("free_email")["won"].mean()[[1, 0]] * 100)
)

```

	text_strong	text_weak	leads	win rate %
0	0	0	5125	7.3
		1	3219	5.1
1	0		2357	12.1
free email (gmail) win rate: 4.8% vs business email: 10.0%				

Flags are crude but transparent, and a sales manager can check them. Chapter 41 turns text into features properly with TF-IDF and embeddings. The email domain is another text feature: a business domain (`@sharmahardware.in`) signals a real company.

Aggregates from activity logs

Many strong features are **aggregates**: counts, sums, averages, or recency computed from a related table. Here, `activities_24h` counts activities logged within 24 hours of arrival. Other useful aggregates for lead scoring would be the number of earlier enquiries from the same company, or days since the company's last order.

The rule for aggregates: compute them **as of the prediction moment**, for each row separately. "Total activities on this lead" (counting the whole 90 days) is target leakage; "activities logged in the first 24 hours" isn't.

36.7 Missing values in machine learning

Most scikit-learn models refuse to train on missing values (`ValueError: Input X contains NaN`). You have three choices: drop rows, drop columns, or **impute** (fill in) values. Before choosing, look at *which* values are missing and whether missingness means something:

```
missing = (
    leads[["segment", "company_size", "est_quantity", "log_quantity"]].isna().mean()
)
print("share missing:", (missing * 100).round(1).to_dict())
q_missing = leads["est_quantity"].isna()
print(
    f"win rate when quantity is missing {leads.loc[q_missing, 'won'].mean():.2%}, "
    f"when given {leads.loc[~q_missing, 'won'].mean():.2%}"
)
print(
    "quantity missing, by source:",
    (leads.groupby("source")["est_quantity"].apply(lambda s: s.isna().mean()) * 100)
    .round(1)
    .to_dict(),
)
```

```
share missing: {'segment': 5.2, 'company_size': 20.8, 'est_quantity': 14.4, 'log_quantity': 14.7}
win rate when quantity is missing 5.71%, when given 8.06%
quantity missing, by source: {'Cold call': 7.0, 'Marketplace': 28.4, 'Partner': 6.0, 'Referral':
5.1, 'Trade fair': 7.0, 'Website': 6.5}
```

Reading it. Quantity is missing for about one lead in seven, and those leads are won less often (5.71% against 8.06%). The reason shows in the last line: marketplace enquiry forms let buyers skip the quantity box, and marketplace leads convert poorly. **Missingness is information.** Filling the blanks with the median and moving on would throw that information away.

The standard approach has two parts:

1. **Impute** a reasonable value so the model can run: the median for numbers, a "Missing" category for categories.
2. **Add a missing indicator**, a 0/1 column saying "this was blank", so the model can learn whether blanks matter. `SimpleImputer(add_indicator=True)` does both at once.

Strategy	When it fits	Risk
Drop rows with any missing value	A handful of rows, missing at random	Loses data; biased if missingness is related to the target
Drop the column	Mostly empty, or not useful	Loses a feature
Median (numbers) or most frequent (categories)	Few blanks, no pattern	Hides informative missingness
"Missing" as its own category	Categorical columns	None serious; usually a good default
Impute + missing indicator	Missingness may carry signal	Adds columns
Model-based imputation (<code>KNNImputer</code> , <code>IterativeImputer</code>)	Many related columns	Slower; more complex; rarely worth it for lead scoring

Watch out: learn the fill values from training rows only. The median used to fill blanks in validation and test rows must be the **training** median. Computing it on the whole dataset is train-test contamination, the same mistake as the target-encoding leak, only smaller. Inside a pipeline, this happens automatically.

Gradient-boosting libraries such as XGBoost and LightGBM, and scikit-learn's `HistGradientBoostingClassifier` , handle missing values natively by learning which branch blanks should follow (Chapter 37). An indicator is still a good habit when blanks might mean something.

36.8 scikit-learn pipelines

The problem pipelines solve

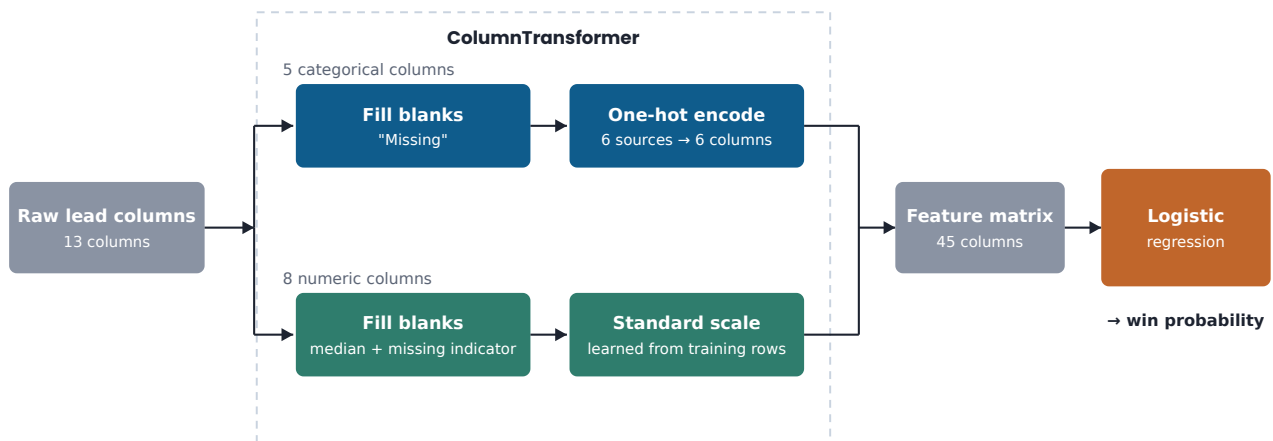
Every learned preprocessing step (the median, the scaler's mean and standard deviation, the categories, the target encoding) must be:

- **learned** from the training rows only,
- **applied** unchanged to validation, test, and new production rows,
- **re-learned** inside each fold during cross-validation.

Doing that by hand means writing the same steps three times and never making a mistake. A **pipeline** bundles the steps and the model into one object with one `fit` and one `predict` , so it can't go wrong.

The pieces

- **Pipeline** chains steps in order: each step's output feeds the next, and the last step is the model.
- **ColumnTransformer** applies different steps to different columns (one recipe for categories, another for numbers) and glues the results side by side.
- **Transformers** (`SimpleImputer` , `OneHotEncoder` , `StandardScaler` , `TargetEncoder`) have `fit` (learn from data) and `transform` (apply what was learned).



`fit(train)` learns every step from training rows · `predict_proba(new rows)` applies them unchanged

Figure 36.4 – The lead-scoring pipeline. `fit` learns every step from training rows; `predict_proba` applies them, unchanged, to new rows.

The `make_model` function from section 36.4 is exactly this. Look inside the fitted version:

```

pipeline = make_model(cats, nums).fit(train[cats + nums], train["won"])
prepare = pipeline.named_steps["prepare"]
names = prepare.get_feature_names_out()
print(len(cats) + len(nums), "raw columns ->", len(names), "model features")
print(names[:4], "...", names[-5:])
print(
    "median log quantity learned from training rows:",
    prepare.named_transformers_["num"].named_steps["fill"].statistics_[0].round(3),
)

one_new_lead = pd.DataFrame(
    [
        {
            "source": "Referral",
            "segment": "Hospitality",
            "city_clean": "Pune",
            "company_size": np.nan,
            "product_interest": "Kitchen",
            "log_quantity": np.loglp(400),
            "website_visits": 2,
            "free_email": 0,
            "text_strong": 1,
            "text_weak": 0,
            "inside_desk": 0,
            "activities_24h": 1,
            "responded_24h": 1,
        }
    ]
)
print(
    f"win probability for this lead: {pipeline.predict_proba(one_new_lead)[0, 1]:.1%}"
)

```

```

13 raw columns -> 45 model features
['cat_source_Cold call' 'cat_source_Marketplace' 'cat_source_Partner'
 'cat_source_Referral'] ... ['num_text_weak' 'num_inside_desk' 'num_activities_24h'
 'num_responded_24h' 'num_missingindicator_log_quantity']
median log quantity learned from training rows: 4.875
win probability for this lead: 44.5%

```

How it works:

- `named_steps` and `named_transformers_` let you reach inside a fitted pipeline to check what each step learned. The imputer's `statistics_` holds the training medians.
- The 13 raw columns become 45 features: the one-hot columns for the 5 categorical columns (including a `Missing` column where blanks exist), the 8 scaled numbers, and one missing indicator, for `log_quantity`, the only numeric column with blanks in training.
- Scoring a brand-new lead, even one with a blank company size, needs one line. The same object that was validated is the one that goes to production (Chapter 52 covers deploying it).

Proof that it prevents contamination

Section 36.4 passed the whole pipeline to `cross_val_score`, so every median and scaler was re-learned inside each fold. The common mistake is to prepare the data once, then cross-validate the model alone. For medians and scaling, the damage is usually tiny, as here; for target encoding and feature selection, it's large. The pipeline makes the question irrelevant.

```

from sklearn.base import clone

X_train = train[cats + nums + ["company_name"]]
folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=36)

# Wrong: target-encode company names on all training rows, then cross-validate
prepared_once = X_train.copy()
prepared_once["company_rate"] = train["company_name"].map(
    train.groupby("company_name")["won"].mean()
)
wrong = cross_val_score(
    make_model(cats, nums + ["company_rate"]),
    prepared_once[cats + nums + ["company_rate"]],
    train["won"],
    cv=folds,
    scoring="roc_auc",
)

# Right: the encoder is inside the pipeline, so each fold learns its own rates
honest = cross_val_score(
    clone(right), X_train, train["won"], cv=folds, scoring="roc_auc"
)
print("encode first, then cross-validate: mean AUC", wrong.mean().round(3))
print("encoder inside the pipeline:          mean AUC", honest.mean().round(3))

```

```

encode first, then cross-validate: mean AUC 0.82
encoder inside the pipeline:      mean AUC 0.78

```

The wrong version reports a cross-validation AUC several points higher than the honest one. Nothing about the model changed; only *where* the encoding was learned. `clone` makes a fresh, unfitted copy of the pipeline, so nothing from the earlier `fit` carries over.

36.9 Baselines, and the final test

Always beat something simple first

A model score means nothing on its own. Before celebrating an AUC of 0.8, compare it with things that need no machine learning:

1. **Base rate:** predict the training win rate for every lead. It can't rank anything (AUC 0.5), but its log loss is the benchmark for probabilities (Chapter 35).
2. **A rule of thumb:** what the sales team does today, "call referrals, trade-fair contacts, and partner leads first". Score it as 1 for those sources and 0 for the rest.
3. **The simplest real model:** logistic regression on a handful of clean features, the pipeline you've built.

Chapter 37's more complex models must beat number 3 by a margin worth their extra complexity.

```
def baselines(fit_rows, eval_rows):
    y = eval_rows["won"]
    base = np.full(len(eval_rows), fit_rows["won"].mean())
    rule = eval_rows["source"].isin(["Referral", "Trade fair", "Partner"]).astype(int)
    model = make_model(cats, nums).fit(fit_rows[cats + nums], fit_rows["won"])
    p = model.predict_proba(eval_rows[cats + nums])[:, 1]
    rate = fit_rows["won"].mean()
    print(f" base rate {rate:.2%}:   AUC 0.500   log loss {log_loss(y, base):.4f}")
    rule_auc = roc_auc_score(y, rule)
    print(f" rule (3 sources):      AUC {rule_auc:.3f}   log loss   n/a (not a probability)")
    model_auc, model_ll = roc_auc_score(y, p), log_loss(y, p)
    print(f" logistic regression: AUC {model_auc:.3f}   log loss {model_ll:.4f}")
    return p

print("VALIDATION (trained on 2023-2024):")
p_valid = baselines(train, valid)
print(
    f" average predicted win rate {p_valid.mean():.2%} vs actual {valid['won'].mean():.2%}"
)
```

```
VALIDATION (trained on 2023-2024):
base rate 7.90%:   AUC 0.500   log loss 0.2435
rule (3 sources):   AUC 0.680   log loss   n/a (not a probability)
logistic regression: AUC 0.823   log loss 0.1948
average predicted win rate 6.90% vs actual 6.56%
```

Reading it. On validation, the logistic regression ranks leads much better than the rule (AUC 0.823 against 0.680) and its probabilities beat the base rate clearly (log loss 0.1948 against 0.2435). It also passes a simple sanity check on the probabilities: it predicts an average win rate of 6.90% for the first

half of 2025, close to the actual 6.56%. That's slightly high, because the model learned from 2023–2024, when marketplace leads converted better. Chapter 39 calls this **calibration** and shows how to check it properly.

Back to the random split

Section 36.3 warned that scores from different splits can't be compared. Here's the evidence: the same pipeline, evaluated on a random slice of the same number of leads, drawn from all periods before July 2025:

```
from sklearn.model_selection import train_test_split

before_test = leads[leads["created_at"] < "2025-07-01"]
rand_train, rand_valid = train_test_split(
    before_test, test_size=len(valid), random_state=36, stratify=before_test["won"]
)
print("time split (valid = Jan-Jun 2025):  AUC %.3f  log loss %.4f" % score(cats, nums))
print(
    "random split (same number of rows): AUC %.3f  log loss %.4f"
    % score(cats, nums, rand_train, rand_valid)
)
```

```
time split (valid = Jan-Jun 2025):  AUC 0.823  log loss 0.1948
random split (same number of rows): AUC 0.791  log loss 0.2269
```

The random split scores lower on both measures. That isn't because it's more honest; its validation rows are a different mix (fewer of 2025's clearly weak marketplace leads, and a higher win rate). Neither number is "the" score. The time split's number answers the question Anita cares about, how the model does on later leads, so it's the one to report.

The one-time test

All the choices are made: features, encodings, split, model. Now, once, retrain on training plus validation (more recent data helps) and score the locked test set:

```
final_fit = pd.concat([train, valid])
print("TEST (trained on 2023 to June 2025, scored once on July to 2 October 2025):")
p_test = baselines(final_fit, test)
```

```
TEST (trained on 2023 to June 2025, scored once on July to 2 October 2025):
base rate 7.59%:  AUC 0.500  log loss 0.2983
rule (3 sources):  AUC 0.727  log loss  n/a (not a probability)
logistic regression: AUC 0.838  log loss 0.2369
```

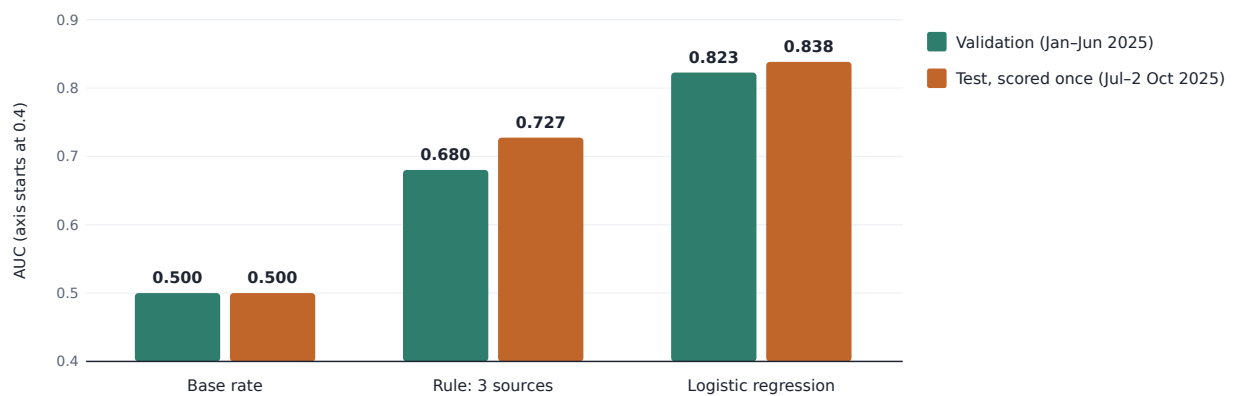


Figure 36.5 – Baselines against the model. The rule helps; the model helps more, on the validation set used for choices and the test set used once.

Reading it. On the untouched test set, the model scores an AUC of **0.838** and a log loss of **0.2369**, against the rule's 0.727 and the base rate's 0.2983. The test score is a little higher than validation, not lower: the test period's win rate (8.78%) and mix differ from the first half of 2025, and a few hundred leads with about a hundred wins give a noisy estimate. The honest summary isn't "0.838"; it's "around 0.8, clearly better than the current rule, on leads the model had never seen."

What to tell Anita. "Using only what's in the CRM the morning after a lead arrives, the model ranks leads clearly better than calling referral, trade-fair, and partner leads first. If a rep calls leads in the model's order, the leads they reach first are much more likely to be won. Before we rely on it, I want to run it quietly alongside the team for a month and compare, and Chapter 39's cost analysis will tell us how many leads to call." (That last sentence is a promise the next three chapters keep.)

Common mistakes and how to spot them

Mistake	Symptom	Fix
No prediction moment defined	Features from after the decision sneak in	Write down <i>when</i> the prediction is made; test every feature against it
Using leads whose outcome isn't settled	Recent periods look terrible	Keep only rows whose outcome window has fully passed
Random split for a problem that runs forward in time	Model looks fine in testing, disappoints in production	Split by time, in the direction it will be used
Tuning on the test set	Test score keeps improving as you try things	Use validation or cross-validation for choices; touch test once
Duplicates across train and test	Suspiciously good scores	Deduplicate first; for repeat customers, split by customer (group split)
Outcome-driven columns (quotes, closing dates, final status)	AUC near 0.95–1.0	Remove anything filled in after, or because of, the outcome

Mistake	Symptom	Fix
Preprocessing fit on all data	Validation score slightly optimistic	Put every learned step in a pipeline
Target encoding outside cross-validation	Big gains from high-cardinality columns	Use <code>TargetEncoder</code> inside the pipeline
Filling blanks and discarding the fact they were blank	Loses signal from informative missingness	<code>SimpleImputer(add_indicator=True)</code>
Keeping impossible values	Huge coefficients, convergence warnings, odd predictions	Cap or set to missing based on domain limits; log-transform skewed numbers
Failing on new categories in production	Error on the first unseen value	<code>OneHotEncoder(handle_unknown="ignore")</code>
No baseline	"0.8 AUC" with no idea whether that's good	Report base rate, the current rule, and a simple model together
Comparing scores from different splits	Arguments about which model is "better"	Compare models only on the same rows
Treating one validation score as exact	Chasing changes of 0.005	Check cross-validation spread; changes smaller than it are noise

In the real world: Vikram's 98% model

In March 2026, Vikram Singh, Riverstone's Sales Manager, shares a notebook from a freelance analyst he has hired for a month. The headline: *"Lead conversion model: 98% accuracy, AUC 0.99."* He wants to roll it out to the sales team the next Monday and asks Meera Iyer to "sanity check the numbers".

Meera doesn't start with the algorithm. She opens the list of features and asks the one question from section 36.5 of each: *would this be in the CRM the morning after the lead arrives?*

The feature list has 31 columns. Twenty-seven pass. Four don't:

- `stage_reached` , the furthest pipeline stage. It is the outcome, in slightly different words.
- `quote_value` , filled in only when a quote is sent.
- `total_activities` , counting every call and email over the lead's whole life. Won leads get many follow-up calls because they're being won.
- `lost_reason` , blank for every won lead. The model learned that "blank means won".

Next, the accuracy. At Riverstone's 7.7% win rate, predicting "lost" for every lead is 92.3% accurate. 98% is better, but not the miracle it sounds like.

Then the split. The analyst used a random 80/20 split across 2023–2025, and included the leads created in the last quarter of 2025, whose outcomes weren't settled. Many of those "lost" leads were still being worked.

Meera rebuilds the evaluation in an afternoon: the four columns removed, recent leads excluded, trained on 2023–2024, validated on the first half of 2025. The model's AUC falls from 0.99 to about 0.82, roughly the same as the simple pipeline from this chapter, and it still clearly beats the team's "referrals first" rule.

Her message to Vikram:

"The model is useful, but not for the reason the notebook says. Four of its columns are only filled in after a lead is won or lost, so on Monday morning they'd all be empty and the scores would be meaningless. With those removed and the model tested on later leads, it ranks leads at about 0.82 AUC, which is still clearly better than calling referrals first. I'd suggest we pilot the fixed version with Neha's leads for a month, and ask the analyst to add a written prediction-moment rule to the notebook."

Vikram's reply is one line: "Glad we didn't send the 98% slide to Anita."

The fixed model wasn't worse than the original. It was the same model, measured without the leaks.

Tools

- **scikit-learn:** every example was tested on 1.8.0; the current release at the time of writing is 1.9.1, so if a number differs slightly on your machine, check the version first. The pieces used are `Pipeline`, `ColumnTransformer`, `SimpleImputer`, `OneHotEncoder`, `StandardScaler`, `TargetEncoder` (available since version 1.3), `StratifiedKFold`, `TimeSeriesSplit`, `cross_val_score`, `LogisticRegression`, and the metrics `roc_auc_score` and `log_loss`. The official user guide's pages on cross-validation, pipelines, and "common pitfalls" (which covers leakage) are worth reading in full.
 - **pandas** (3.0.2) and **NumPy** (2.4.4) for cleaning and feature building.
 - **Python** 3.12.3. Every example was run on 17 September 2026; the APIs used have been stable since scikit-learn 1.3, when `TargetEncoder` was added.
 - **In SQL:** the deduplication, the 90-day maturity filter, and the "activities logged within 24 hours" aggregate can all be done in the database with Chapter 13's patterns, which is often where production feature pipelines start (Chapter 47 on data pipelines).
 - **Beyond this chapter:** `imbalanced-learn` works with pipelines for resampling (Chapter 39), and `feature-engine` and `category_encoders` offer more transformers. Use them once the basics here feel routine.
 - **Companion files:** `companion/generate_riverstone_crm.py` rebuilds the CRM export (seed 20236) into `companion/crm/`. Run the chapter's code from that folder. The data spec is `planning/data/riverstone-crm.md`.
-

The project: a leakage-free feature pipeline for Riverstone lead scoring

Goal: a tested, documented pipeline that turns raw CRM exports into model-ready features for lead scoring, with an honest evaluation against baselines. Chapters 37 and 39 build on it, so keep it tidy.

Option A: your own data. Any prediction problem at work with a clear outcome and a time dimension: invoices paid late, customers who don't reorder, support tickets that escalate. Remove personal and confidential fields first, and check you're allowed to use the data.

Option B: Riverstone. The CRM export in `companion/crm/`.

Steps:

1. **Frame it.** Write the five-row framing table from section 36.1: unit, target, prediction moment, allowed information, success measure.
2. **Clean to one row per unit.** Deduplicate, fix category spellings, and exclude rows whose outcome window hasn't passed. Record how many rows each step removes.
3. **Split.** Make a time-based train, validation, and test split. Print the size and target rate of each.
4. **Audit every column for leakage.** For each raw column, write one line: known at the prediction moment, yes or no, and why. Include at least one timestamp check.
5. **Engineer at least six features**, including one log-transformed number, one date part, one text flag, and one aggregate from `activities.csv` computed as of the prediction moment.
6. **Handle missing values** with imputation and missing indicators, and show the target rate for blank versus filled values of one column.
7. **Build the pipeline** with `ColumnTransformer`, and cross-validate it with stratified 5-fold. Report the mean and standard deviation.
8. **Demonstrate one leak.** Add one leaky feature, report the inflated score, remove it, and explain in two sentences why it leaks.
9. **Compare with baselines** on validation: base rate, a rule of thumb, and your pipeline. Then score the test set once.
10. **Write a one-page summary** for a non-technical manager: what the model does, how well, compared with what, and one risk.

Stretch goals:

- Add a `FunctionTransformer` or a small custom transformer class so that `add_features` itself runs inside the pipeline, and the pipeline takes raw CRM columns directly.
- Replace the three-source rule with the best single-feature rule you can find on training data, and check whether it still holds on validation.
- Add "number of earlier enquiries from the same email" as an aggregate, computed strictly from leads that arrived before each lead.
- Save the fitted pipeline with `joblib.dump` and load it in a fresh Python session to score five new leads (Chapter 52 covers this properly).

You've got it when...

- ☐ I can write the target, unit, prediction moment, and success measure for a business request before touching data.
 - ☐ I know why leads, loans, or subscriptions whose outcome window hasn't passed must be excluded.
 - ☐ I can explain the jobs of training, validation, and test sets, and why the test set is used once.
 - ☐ I choose a time-based split when the model will predict forward in time, and I compare models only on the same split.
 - ☐ I can run stratified and time-series cross-validation and interpret the spread of the scores.
 - ☐ For any feature, I ask whether it would be known at the prediction moment, and I check which clock each timestamp uses.
 - ☐ I can explain target leakage and train-test contamination, with an example of each.
 - ☐ I can engineer features from numbers (cap, log, scale), categories (one-hot, target encoding), dates, text flags, and aggregates.
 - ☐ I check whether missingness predicts the target, and I use imputation with missing indicators.
 - ☐ I can build a `Pipeline` with a `ColumnTransformer`, inspect what it learned, and score a new row with it.
 - ☐ I always report a model next to a base rate and a rule-of-thumb baseline.
-

Recap

- **Frame first:** unit, **target**, **prediction moment**, allowed information, and success measure. Most leakage is prevented here.
- Exclude rows whose outcome isn't settled (**censoring**).
- **Training** data teaches, **validation** data guides choices, the **test** set is used once. Split by **time** when the model predicts forward; scores from different splits can't be compared.
- **Cross-validation** gives several scores; their spread is your uncertainty. Use **stratified** folds for classification and **time-series splits** when order matters.
- **Data leakage** is information the model won't have at prediction time. **Target leakage** comes from columns created after or because of the outcome; **train-test contamination** comes from preprocessing learned on evaluation rows. Ask: *would this be known at the prediction moment?*
- Big leaks look spectacular; small leaks look plausible. Both are wrong.
- **Feature engineering:** cap impossible values, **log-transform** skewed numbers, **scale** for non-tree models, **one-hot encode** categories, **target encode** high-cardinality columns inside the pipeline, extract **date parts**, add **text flags**, and build **aggregates** as of the prediction moment.
- **Missingness can be information:** impute, and add **missing indicators**.
- A **pipeline** learns every step from training rows and applies it unchanged everywhere else, including inside cross-validation.

- Always compare with **baselines**: the base rate, a rule of thumb, and a simple model.

Practice exercises

Code exercises run from `companion/crm/` after the chapter's code (they use `leads`, `train`, `valid`, `test`, `cats`, `nums`, `make_model`, and `score`). Predict each answer before running it.

Warm-up

1. For each request, write the target and the prediction moment: (a) "Which customers will stop ordering?" (b) "Which invoices will be paid late?" (c) "How many Industrial Crates will we sell next month?" Say which is classification and which is regression.
2. Which of these columns could be used to score a lead 24 hours after it arrives? (a) `source`; (b) `status`; (c) the number of website visits before the enquiry; (d) the date the lead was marked Qualified; (e) whether the lead was assigned to the inside sales desk.
3. A colleague's lead model uses a random split and scores AUC 0.80 on its test set; yours uses a time split and scores 0.78. Can you conclude theirs is better? Explain in two sentences.
4. `company_size` has four bands: 1-10, 11-50, 51-200, 200+. Give one reason to one-hot encode it and one reason to encode it as the numbers 1 to 4.

Core

5. Count how many 2025 leads (before the 2 October cutoff) come from each `source`, and the win rate for each. Which source's win rate changed most compared with 2023–2024?
6. Add a `weekday` feature (0 = Monday) to the model as a categorical column. Report validation AUC and log loss before and after. Is the change bigger than the cross-validation spread you saw in section 36.4?
7. Count activities logged within 24 hours using `activity_at` instead of `logged_at`, rebuild the feature, and compare validation AUC with the honest version. How big is the difference, and why should you still use `logged_at`?
8. Train the model without `responded_24h` and `activities_24h`. How much validation AUC do the two process features add? Is there any reason to be cautious about them even though they pass the prediction-moment test?
9. Build a pipeline where the numeric imputer does **not** add missing indicators. Compare validation log loss with the original. Explain the result using what section 36.7 found about quantity.
10. Run stratified 5-fold cross-validation of the full feature set (`cats` and `nums`) on the training rows. Report each fold's AUC, the mean, and the standard deviation.

Stretch

11. Write a function `leak_check(column)` that adds one column to the honest features and reports the change in validation AUC. Run it on `days_in_pipeline`, `has_quote`, and `lead_id`. What change would make you investigate a column?

- l2. Some companies send several genuine enquiries over three years. Check how many distinct `email_norm` values appear in both `train` and `test`. Why could this matter, and what kind of split would prevent it?
- l3. Replace logistic regression in `make_model` with `HistGradientBoostingClassifier(random_state=36)`. Does it beat logistic regression on validation? (Chapter 37 explains why the answer can go either way.)

Think about it

- l4. Your model scores 0.97 AUC on validation for predicting which support tickets will escalate. Name three things you'd check before telling anyone.
- l5. The sales team starts calling leads in the model's order. Six months later you retrain on the new data. What has changed about the data that could make the new model misleading? (Hint: who got called, and when.)
- l6. A manager asks you to "just use all the columns; the model will figure out which ones matter". Write a two-sentence reply.

Key terms

supervised learning · binary classification · regression · target · unit of analysis · prediction moment · censoring · deduplication · training set · validation set · test set · random split · time-based split · data drift · cross-validation · fold · k-fold · stratified k-fold · time series split · ROC-AUC · data leakage · target leakage · train-test contamination · outcome-driven field · feature · feature engineering · log transform · capping · scaling (standardization) · one-hot encoding · sparse matrix · ordinal encoding · target encoding · cross-fitting · high cardinality · date parts · interaction · keyword flag · aggregate feature · missing value · imputation · missing indicator · informative missingness · pipeline · `ColumnTransformer` · `transformer (fit / transform)` · baseline · rule-of-thumb baseline · calibration

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 37, Supervised Learning Algorithms**, swaps different models into this pipeline, from regularized logistic regression to gradient boosting, and asks whether they beat the baseline by enough to matter.
- **Chapter 38, Unsupervised Learning**, reuses scaling and encoding for clustering, where there's no target to leak.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, takes this chapter's validation predictions and turns them into a call list with a cost-based threshold, checks calibration, and explains which features drive the scores.
- **Chapter 40, Time Series & Forecasting**, extends time-based splitting into backtesting.

- **Chapter 41, NLP Foundations**, replaces keyword flags with TF-IDF features for text.
- **Chapter 44, Capstone**, runs this whole workflow on a new question from start to presentation.
- **Chapters 47 and 52** move feature pipelines into scheduled data pipelines and deployed models.
- **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers leakage, splits, cross-validation, encoding, and missing values with graded answers. Leakage questions are among the most common in data science interviews.

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. (a) Target: the customer places no order in the next 90 days, 1 or 0. Prediction moment: the first day of each month, for every active customer, using order history up to that day. Classification. (b) Target: the invoice is paid more than N days after its due date, 1 or 0. Prediction moment: the day the invoice is issued. Classification. (c) Target: units of Industrial Crates sold next month, a number. Prediction moment: the last day of the current month. Regression (and a time series problem, Chapter 40). The common gap is the prediction moment: without it, "will stop ordering" can quietly use next month's orders.

2. (a) Yes, it's on the enquiry form. (b) No, `status` is the outcome. (c) Yes, if it counts visits *before* the enquiry. (d) No: qualification happens days after arrival and mostly to leads that go on to be won. (e) Yes: leads are assigned on arrival. Check how the assignment is made, though: if the desk is given leads a rep has already judged weak, the assignment carries that judgment, like the response-time leak.

3. No. Their score is measured on different rows, with a different mix of leads (a random slice of all years, not the most recent months), and section 36.3 showed the mix alone can move AUC by a few points. Score both models on the same time-based validation set before comparing.

4. One-hot encoding makes no assumption about the gaps between bands: the model can learn that 11-50 and 51-200 behave alike while 200+ is very different. Encoding as 1 to 4 uses one column instead of four and tells the model the bands are ordered, which helps when data is scarce and the effect really does rise steadily with size. A tree-based model can use the ordered numbers well; a linear model assumes each step up is worth the same.

5.

```

period = np.where(leads["created_at"] < "2025-01-01", "2023-24", "2025")
table = leads.pivot_table(
    index="source", columns=period, values="won", aggfunc=["count", "mean"]
)
table.columns = [f"{a}_{b}" for a, b in table.columns]
table["change_pts"] = (table["mean_2025"] - table["mean_2023-24"]) * 100
print(
    table.assign(**{c: table[c] * 100 for c in ["mean_2023-24", "mean_2025"]})
    .round(2)
    .to_string()
)

```

source	count_2023-24	count_2025	mean_2023-24	mean_2025	change_pts
Cold call	647	206	8.50	8.74	0.24
Marketplace	2381	1516	2.98	1.52	-1.46
Partner	559	170	10.91	12.94	2.03
Referral	838	290	19.33	23.79	4.46
Trade fair	845	411	13.37	16.06	2.69
Website	2021	817	5.64	6.36	0.72

The biggest change is the marketplace, whose win rate fell by about 1.4 percentage points while its lead count in 2025 grew to the largest of any source. A drop from about 3% to 1.5% is small in points but halves the rate. Referral, trade-fair, and partner leads rose by 2 to 4.5 points; with only a few hundred 2025 leads each, changes of that size can come from chance alone (Chapter 22 shows how to check).

6.

```

leads["weekday"] = leads["created_at"].dt.dayofweek.astype(str)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]
print("without weekday: AUC %.3f log loss %.4f" % score(cats, nums))
print("with weekday: AUC %.3f log loss %.4f" % score(cats + ["weekday"], nums))

```

```

without weekday: AUC 0.823 log loss 0.1948
with weekday: AUC 0.819 log loss 0.1965

```

Both scores get slightly worse (AUC 0.819, log loss 0.1965), a change within the fold-to-fold spread you saw in section 36.4. The generator planted no weekday effect, so this is the expected honest result: a feature that sounds reasonable and adds nothing. Keeping it would add seven columns of noise. (`astype(str)` makes the encoder treat days as categories, not as numbers 0 to 6.)

7.


```

by_happened = activities[
    activities["activity_at"] <= activities["lead_id"].map(deadline)
]
leads["activities_24h_wrong"] = (
    leads["lead_id"].map(by_happened.groupby("lead_id").size()).fillna(0)
)
train = leads[leads["created_at"] < "2025-01-01"]
valid = leads[
    (leads["created_at"] >= "2025-01-01") & (leads["created_at"] < "2025-07-01")
]
test = leads[leads["created_at"] >= "2025-07-01"]
wrong_nums = [c if c != "activities_24h" else "activities_24h_wrong" for c in nums]
print("logged_at (honest):      AUC %.3f  log loss %.4f" % score(cats, nums))
print("activity_at (leaky):     AUC %.3f  log loss %.4f" % score(cats, wrong_nums))
print(
    "leads whose count differs:",
    (leads["activities_24h"] != leads["activities_24h_wrong"]).sum(),
)

```

```

logged_at (honest):      AUC 0.823  log loss 0.1948
activity_at (leaky):     AUC 0.825  log loss 0.1929
leads whose count differs: 781

```

The leaky version scores slightly better (AUC 0.825 against 0.823), and 781 leads get a different count. On this data the gap is small. Use `logged_at` anyway: the honest feature is the one the CRM can actually show at 8 a.m., and in a team where logging late is common, the gap would be larger and would grow silently. Evaluation must mirror production exactly, not approximately.

8.

```

without_process = [c for c in nums if c not in ("responded_24h", "activities_24h")]
print("with process features:    AUC %.3f  log loss %.4f" % score(cats, nums))
print(
    "without process features: AUC %.3f  log loss %.4f" % score(cats, without_process)
)

```

```

with process features:    AUC 0.823  log loss 0.1948
without process features: AUC 0.814  log loss 0.1983

```

They add 0.009 AUC (0.823 against 0.814). Be cautious for the reason from section 36.5: a quick response partly reflects the rep's own judgment of the lead. They pass the prediction-moment test, but once the model's scores drive who gets called, reps will respond fastest to high-scoring leads, and the feature will start measuring the model's own past output (a feedback loop, exercise 15). Many teams leave process features out of the first model for that reason.

9.

```

def make_model_no_indicator(categorical, numeric):
    prepare = ColumnTransformer(
        [
            (
                "cat",
                Pipeline(
                    [
                        (
                            "fill",
                            SimpleImputer(strategy="constant", fill_value="Missing"),
                        ),
                        ("onehot", OneHotEncoder(handle_unknown="ignore")),
                    ]
                ),
                categorical,
            ),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler()),
                    ]
                ),
                numeric,
            ),
        ]
    )
    return Pipeline(
        [("prepare", prepare), ("model", LogisticRegression(max_iter=1000))]
    )

m = make_model_no_indicator(cats, nums).fit(train[cats + nums], train["won"])
p = m.predict_proba(valid[cats + nums])[:, 1]
print(
    "without indicators: AUC %.3f log loss %.4f"
    % (roc_auc_score(valid["won"], p), log_loss(valid["won"], p))
)
print("with indicators:    AUC %.3f log loss %.4f" % score(cats, nums))

```

```

without indicators: AUC 0.822 log loss 0.1952
with indicators:    AUC 0.823 log loss 0.1948

```

Without the indicator, a blank quantity becomes the median, so the model treats a marketplace lead that skipped the box like an ordinary lead with a typical order size. The indicator lets it learn that blanks win less often (section 36.7). Here the effect is small, because `source` already tells the model a lead came from the marketplace and so carries much of the same information; with fewer overlapping features, the indicator would matter more.

10.


```

folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=36)
cv_full = cross_val_score(
    make_model(cats, nums),
    train[cats + nums],
    train["won"],
    cv=folds,
    scoring="roc_auc",
)
print(
    "fold AUCs:",
    cv_full.round(3),
    " mean",
    cv_full.mean().round(3),
    " sd",
    cv_full.std().round(3),
)

```

```
fold AUCs: [0.779 0.78 0.801 0.761 0.784] mean 0.781 sd 0.013
```

With all features, the mean rises from 0.741 to 0.781, and the folds now spread more (a standard deviation of 0.013). Notice that the cross-validation mean on 2023–2024 is lower than the validation score on early 2025 (0.823): the 2025 mix, with more clearly weak marketplace leads, is easier to rank, the same effect as in section 36.3.

11.

```

honest_auc = score(cats, nums)[0]

def leak_check(column):
    change = score(cats, nums + [column])[0] - honest_auc
    print(f"{column:<18} AUC change {change:+.3f}")

for column in ["days_in_pipeline", "has_quote", "lead_id"]:
    leak_check(column)

```

```

days_in_pipeline    AUC change +0.052
has_quote            AUC change +0.155
lead_id              AUC change -0.000

```

`lead_id` changes nothing here. Don't take that as a rule: in many systems IDs rise over time or are issued in batches per source, so an ID column can smuggle in time or source information, and it never belongs in a model. A jump of more than about 0.02 from one column, when cross-validation folds vary by a few points, is worth investigating; a jump like `has_quote`'s is almost certainly a leak. But a small change doesn't prove a column is clean: `first_response_hours` only added 0.009. The prediction-moment question is the real test; the score change only tells you where to look first.

12.

```

shared = set(train["email_norm"]) & set(test["email_norm"])
print(len(shared), "emails appear in both train and test")
print(
    "test leads from those emails:",
    test["email_norm"].isin(shared).sum(),
    "of",
    len(test),
)

```

```

17 emails appear in both train and test
test leads from those emails: 17 of 1185

```

Only a small number of test leads share an email with a training lead, because this generator creates mostly one-off enquiries. In real CRMs, repeat enquirers are common, and they matter: if a customer who was won in training enquires again in test, the model can partly recognize the customer instead of learning general patterns. A **group split** (`GroupKFold` or `GroupShuffleSplit` , with the email or company as the group) keeps every lead from one customer on the same side. With a time split, the question becomes whether "has this customer been won before" should be a legitimate feature, computed as of the prediction moment, which it often should.

13.

```

from sklearn.ensemble import HistGradientBoostingClassifier

def make_boosted(categorical, numeric):
    base = make_model(categorical, numeric)
    return Pipeline(
        [
            ("prepare", base.named_steps["prepare"]),
            ("model", HistGradientBoostingClassifier(random_state=36)),
        ]
    )

boosted = make_boosted(cats, nums).fit(train[cats + nums], train["won"])
p = boosted.predict_proba(valid[cats + nums])[:, 1]
print(
    "gradient boosting:  AUC %.3f  log loss %.4f"
    % (roc_auc_score(valid["won"], p), log_loss(valid["won"], p))
)
print("logistic regression: AUC %.3f  log loss %.4f" % score(cats, nums))

```

```

gradient boosting:  AUC 0.785  log loss 0.2075
logistic regression: AUC 0.823  log loss 0.1948

```

With default settings, gradient boosting does not beat logistic regression here. The planted signal in this data is mostly additive (each feature pushes the odds up or down on its own), which is exactly what logistic regression models, and with about 580 wins in training there isn't much data for boosting to find subtle interactions without overfitting. That's a normal, honest result; Chapter 37 tunes boosting properly and shows when it wins. (The pipeline reuses the same preparation steps; `HistGradientBoostingClassifier` accepts the sparse one-hot output.)

14. First, leakage: list every feature and ask whether it exists at the moment of prediction, looking hardest at anything filled in during the ticket's life (priority changes, number of replies, assigned team). Second, the split: were tickets from the same customer or the same incident in both training and validation, and was the split by time? Third, the baseline and the base rate: what does "escalate if the ticket mentions 'refund'" or "escalate if the customer is enterprise" score? An AUC of 0.97 on a messy human process is far above what's normally achievable, so assume a leak until each check passes.

15. The outcomes are no longer independent of the model. High-scoring leads get called first and followed up more, so they're won more often partly *because* they scored high; low-scoring leads get less attention and are lost more often. A model retrained on that data learns the old model's opinions back, strengthened, and can't tell whether a low-scored lead would have been won with a call. This is a **feedback loop**. Keeping a small random sample of leads that are worked in normal order, not by score, gives unbiased data for retraining and for measuring the model's real effect (Chapter 30 on experiments).

16. "The model can't tell which columns were filled in after a lead was won, so 'all the columns' would include some that are the answer in disguise, and it would look excellent in testing and fail on real leads. I'll include every column that's known when we score a lead, and I'll show you the list and why the others were left out."

Chapter 37. Supervised Learning Algorithms

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: explain how each major supervised learning algorithm makes a prediction, and calculate one small step of each by hand · fit and read linear regression, ridge, lasso, and elastic net · fit and read logistic regression, including odds ratios · use k-nearest neighbors and Naive Bayes, and know their weak spots · build a decision tree split by hand with Gini and entropy, and see a tree overfit · explain why random forests and gradient boosting work, and use scikit-learn, XGBoost, LightGBM, and CatBoost · know when support vector machines are worth it · diagnose bias and variance with learning curves · tune hyperparameters with random search and Optuna without fooling yourself · pick an algorithm for a problem, and justify the choice.

Before you start: Chapter 35 (loss, gradients, likelihood, entropy) and Chapter 36 (splits, cross-validation, leakage, pipelines, baselines). This chapter reuses Chapter 36's pipeline and lead-scoring data.

Time needed: 14–18 hours over two to three weeks. It's the longest chapter in Part IV; take it one algorithm at a time.

Tools: Python 3 with scikit-learn, plus three free gradient-boosting libraries (XGBoost, LightGBM, CatBoost) and Optuna for tuning. Everything runs on a laptop CPU; the slowest block takes about a minute.

Practice data: two Riverstone datasets. **Customer accounts** (new in this chapter, built by `companion/generate_riverstone_accounts.py`): 5,000 B2B accounts described as of 31 December 2024, with what happened in 2025. **Leads** (from Chapter 36), loaded through `companion/ch37/lead_data.py`. Every number in this chapter was calculated, and every output shown is real.

Why this matters

There are hundreds of machine learning algorithms, and a handful of them do almost all the work in business data science. Interviewers expect you to know those few well: not only the function name, but how each one decides, what goes wrong, and which settings matter.

The practical questions come up every week:

- "Should we use XGBoost?" Maybe. On one of this chapter's datasets it clearly beats logistic regression; on the other it doesn't.

- "The model is 99% accurate on training data but poor on new data." That's variance, and you'll learn to see it on a chart.
- "Which features drive churn?" Some algorithms tell you directly; others need extra tools (Chapter 39).
- "How long should we spend tuning?" Less than most people think, and never on the test set.

This chapter teaches nine algorithm families, one at a time, on real Riverstone problems. For each one, you'll see how it works, calculate a small piece by hand, run it, and read the result. At the end, you'll compare them fairly and learn to choose.

In plain English

Think of different people guessing which customers will stop ordering.

- The **accountant** gives every warning sign a weight and adds them up: "late payments +3, no recent order +5, many product lines -2". That's **linear and logistic regression**.
- The **accountant with a strict boss** has to justify every weight, so they keep them small and drop the ones that barely matter. That's **regularization**.
- The **old-timer** says, "This customer reminds me of twenty others I've known; fifteen of them left." That's **k-nearest neighbors**.
- The **statistician** asks, "Among customers who left, how common was this sign? Among those who stayed?" and multiplies the evidence. That's **Naive Bayes**.
- The **sales manager** asks a series of yes/no questions: "No order in 90 days? Pays late? Small retailer?" That's a **decision tree**.
- A **committee** of 300 managers, each seeing slightly different customers and questions, votes. That's a **random forest**.
- A **relay team**, where each new member studies the mistakes of everyone before them and fixes a little of what they got wrong. That's **gradient boosting**.
- The **boundary drawer** looks for the widest possible gap between customers who left and customers who stayed. That's a **support vector machine**.

None of them is always best. The skill is knowing which to ask, and how to check whether they're right.

37.0 The data for this chapter

Riverstone customer accounts

Chapter 36 scored new leads. This chapter adds the other half of the customer life cycle: **accounts**, businesses that already buy from Riverstone. Each row describes one account as of 31 December 2024, the **prediction moment**, using only what was known then: segment, city tier, size, sales rep, tenure,

2024 orders and revenue, discounts, late payments, complaints, how many product categories it buys, and days since its last order. Two targets describe 2025:

- **churned_2025** : 1 if the account placed no order in 2025. A classification target.
- **revenue_2025** : the account's 2025 revenue. A regression target.

Because every account is described at the same moment and predicted over the same year, a stratified random split is appropriate here (Chapter 36's time split mattered because leads arrive over time, and the model scores future leads).

```
import warnings

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.metrics import log_loss, mean_absolute_error, r2_score, roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore", category=UserWarning)

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["rep_id"] = accounts["rep_id"].astype(str)
print(accounts.shape)
print(
    f"churned in 2025: {accounts['churned_2025'].sum():,} of {len(accounts):,} "
    f"({accounts['churned_2025'].mean():.1%})"
)

CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

rest, test_acc = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
train_acc, valid_acc = train_test_split(
    rest, test_size=0.25, random_state=37, stratify=rest["churned_2025"]
)
for name, part in [("train", train_acc), ("valid", valid_acc), ("test", test_acc)]:
    print(
        f"{name:<6} {len(part):>5,} accounts    churn {part['churned_2025'].mean():.1%}"
    )
```

```
(5000, 20)
churned in 2025: 484 of 5,000 (9.7%)
train 3,000 accounts churn 9.7%
valid 1,000 accounts churn 9.7%
test 1,000 accounts churn 9.7%
```

Reading it. 484 of 5,000 accounts, 9.7%, stopped ordering in 2025. The stratified split keeps that rate identical in the training (3,000), validation (1,000), and test (1,000) sets. The test set stays locked until section 37.11.

A fact you can't normally know. Because this dataset comes from a generator, we know the true churn probability of every account. A perfect model that knew those probabilities would score an AUC of **0.858** on these accounts. No real model can do better, because churn is partly chance. It's a useful ruler: in real work you never see this number, so you can't tell how close to the ceiling you are.

37.1 Linear regression

How it works

Linear regression predicts a number as a weighted sum of features plus an intercept:

$$\text{prediction} = b + w_1 \times \text{feature}_1 + w_2 \times \text{feature}_2 + \dots + w_n \times \text{feature}_n$$

It chooses the weights that minimize the mean squared error (Chapter 35). Unlike gradient descent, for linear regression there's an exact formula, the **least-squares solution**, so scikit-learn solves it directly in one step.

Predicting 2025 revenue. Money amounts are skewed (Chapter 35), so model the **log** of revenue. In a log-log model, a slope reads as a percentage relationship. Start with one feature: last year's revenue. Churned accounts have no 2025 revenue, so the regression uses the 4,516 accounts that stayed; predicting *whether* an account stays is the classification problem later in the chapter.


```

stayed = accounts[accounts["churned_2025"] == 0].copy()
for col in ["revenue_2024", "units_2024", "orders_2024", "revenue_2025"]:
    stayed["log_" + col.replace("_2025", "_next")] = np.log(stayed[col])
stayed = stayed.rename(columns={"log_revenue_next": "log_revenue_2025"})
reg_train, reg_test = train_test_split(stayed, test_size=0.25, random_state=37)
print(len(reg_train), "training accounts,", len(reg_test), "test accounts")

from sklearn.linear_model import LinearRegression

simple = LinearRegression().fit(
    reg_train[["log_revenue_2024"]], reg_train["log_revenue_2025"]
)
print(f"intercept {simple.intercept_:.3f}    slope {simple.coef_[0]:.3f}")
pred = simple.predict(reg_test[["log_revenue_2024"]])
print(f"R² on test: {r2_score(reg_test['log_revenue_2025'], pred):.4f}")
example = reg_test.iloc[0]
print(
    f"one account: 2024 ₹{example['revenue_2024']:.0f}"
    f" -> predicted 2025 ₹{np.exp(pred[0]):.0f}"
    f" (actual ₹{example['revenue_2025']:.0f})"
)

```

```

3387 training accounts, 1129 test accounts
intercept -0.140    slope 1.013
R² on test: 0.9546
one account: 2024 ₹65,500 -> predicted 2025 ₹65,555 (actual ₹71,100)

```

Reading it.

- **Slope 1.013:** an account with 1% more revenue in 2024 is predicted to have about 1.013% more in 2025. Revenue carries forward almost one for one.
- **R² 0.9546:** R² (the **coefficient of determination**) is the share of the variation in the target that the model explains, from 0 (no better than predicting the average) to 1 (perfect). Last year's revenue alone explains 95% of the variation in log revenue this year, which is typical: the biggest predictor of next year is this year.
- **The example account:** 2024 revenue of ₹65,500, predicted ₹65,555 for 2025, actual ₹71,100. Check the arithmetic: $\ln(65,500) = 11.0898$; $-0.140 + 1.013 \times 11.0898 = 11.0940$ (with the unrounded coefficients, 11.0906); $e^{11.0907} = ₹65,555$. ✓

Many features

```
REG_NUMS = [
    "log_revenue_2024",
    "log_units_2024",
    "log_orders_2024",
    "tenure_months",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

def reg_pipeline(model):
    prepare = ColumnTransformer(
        [
            ("cat", OneHotEncoder(handle_unknown="ignore", drop="first"), CATS),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median")),
                        ("scale", StandardScaler()),
                    ]
                ),
                REG_NUMS,
            ),
        ]
    )
    return Pipeline([("prepare", prepare), ("model", model)])

def reg_report(name, pipe):
    pipe.fit(reg_train[CATS + REG_NUMS], reg_train["log_revenue_2025"])
    pred = pipe.predict(reg_test[CATS + REG_NUMS])
    mae = mean_absolute_error(reg_test["revenue_2025"], np.exp(pred))
    print(
        f"{name:<18} R² {r2_score(reg_test['log_revenue_2025'], pred):.4f}    "
        f"MAE ₹{mae:,.0f}"
    )
    return pipe

ols = reg_report("linear regression", reg_pipeline(LinearRegression()))
names = ols.named_steps["prepare"].get_feature_names_out()
coefs = pd.Series(ols.named_steps["model"].coef_, index=names)
print(coefs.round(3).sort_values().to_string())
```

```

linear regression R² 0.9593 MAE ₹87,471
cat__segment_Retail -0.097
cat__segment_Wholesale -0.096
num__late_payment_days -0.048
cat__city_tier_Tier 2 -0.041
cat__city_tier_Tier 3 -0.036
num__complaints_2024 -0.033
num__website_logins_2024 -0.011
num__days_since_last_order -0.006
num__log_units_2024 -0.001
num__log_orders_2024 -0.001
cat__company_size_Small -0.001
num__tenure_months 0.000
cat__company_size_Medium 0.004
cat__rep_id_4 0.004
cat__rep_id_5 0.007
num__catalog_downloads_2024 0.008
num__avg_discount_pct 0.012
num__categories_bought 0.025
cat__rep_id_9 0.031
num__log_revenue_2024 1.099

```

How it works:

- `OneHotEncoder(drop="first")` drops one category per column, so `segment` becomes two columns (Retail, Wholesale) compared against the dropped one, **Hospitality**. Linear regression needs this: with all three columns plus an intercept, the columns always add up to 1, and the weights can't be pinned down (the **dummy variable trap**). Regularized models and trees don't need it.
- Numbers are standardized, so each numeric coefficient is "the change in log revenue for one standard deviation more of this feature". That makes coefficients comparable with each other.
- **MAE** (mean absolute error) converts predictions back to rupees and averages the size of the misses: ₹87,471. Chapter 39 covers regression metrics in full.

Reading the coefficients.

- `log_revenue_2024` dominates (1.099), as expected.
- `segment_Retail` -0.097: holding everything else equal, a retail account's 2025 revenue is predicted about 9% lower than a hospitality account's ($e^{-0.097} = 0.908$). The generator planted faster growth for hospitality, especially in metro cities.
- `late_payment_days` -0.048 and `complaints_2024` -0.033: accounts that pay late or complain grow less.
- `log_units_2024` and `log_orders_2024` are almost exactly 0. They carry nearly the same information as revenue, so once revenue is in the model, they add nothing. When features are strongly correlated (**multicollinearity**), their individual coefficients become unstable and hard to interpret, even when predictions are fine.
- `rep_id_9` +0.031: accounts handled by the inside sales desk are predicted to grow 3% faster. The generator planted **no** such effect on growth. Exercise 13 shows its confidence interval, and a likely reason it appears: this regression only includes accounts that *stayed*. A coefficient needs an interval and a second look before anyone acts on it; `statsmodels` gives the interval, and Chapter 22 explains how to read it.

Watch out: a coefficient is not a cause. "Late payments reduce growth by 5% per standard deviation" is a statement about this model, not about the world. Maybe late payers are struggling businesses that would shrink anyway. Chapter 31 covers methods for estimating causes from observational data.

Linear regression's assumptions, and what breaks when they fail:

Assumption	If it fails	Check
The relationship is linear (after any transforms)	Systematic errors in some ranges	Plot residuals against predictions
Errors are independent	Overconfident intervals	Think about repeated customers and time
Errors have constant spread	Intervals too narrow for big accounts	Residual plot fans out; use a log target
Features aren't nearly duplicates	Unstable, uninterpretable coefficients	Correlations between features; regularization
No extreme outliers dominate	One account drags the line	Look at the largest residuals

For prediction alone, the first and last matter most. For interpreting coefficients, all of them matter.

37.2 Regularization: ridge, lasso, and elastic net

The idea

With many features, especially correlated ones or few rows, least squares can give large, unstable weights that fit noise. **Regularization** adds a penalty for large weights to the loss, trading a little training fit for more stable predictions:

Ridge: $\text{loss} = \text{mean squared error} + \alpha \times (\text{sum of squared weights})$

Lasso: $\text{loss} = \text{mean squared error} + \alpha \times (\text{sum of absolute weights})$

Elastic net: a mix of both penalties, set by `l1_ratio`

α (alpha) controls the strength. At $\alpha = 0$ you get ordinary least squares; as α grows, weights shrink toward zero.

The two penalties behave differently. **Ridge** shrinks all weights a little and keeps every feature; with correlated features, it spreads the weight among them. **Lasso** pushes some weights to **exactly zero**, removing those features, so it doubles as **feature selection**. Elastic net sits between them, and handles groups of correlated features more gracefully than lasso, which tends to pick one at random.

Because the penalty treats every weight equally, **features must be standardized first**; otherwise a feature measured in rupees gets a tiny weight and escapes the penalty.

The `...CV` versions choose α by cross-validation inside the training data:

```
from sklearn.linear_model import ElasticNetCV, LassoCV, RidgeCV

ridge = reg_report(
    "ridge (RidgeCV)", reg_pipeline(RidgeCV(alphas=np.logspace(-3, 3, 25)))
)
lasso = reg_report("lasso (LassoCV)", reg_pipeline(LassoCV(cv=5, random_state=37)))
enet = reg_report(
    "elastic net",
    reg_pipeline(ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=5, random_state=37)),
)
print(
    "ridge alpha:",
    ridge.named_steps["model"].alpha_,
    " lasso alpha:",
    round(lasso.named_steps["model"].alpha_, 5),
    " elastic net l1_ratio:",
    enet.named_steps["model"].l1_ratio_,
)
lasso_coefs = pd.Series(lasso.named_steps["model"].coef_, index=names)
print("lasso set to exactly zero:", list(lasso_coefs[lasso_coefs == 0].index))
```

```
ridge (RidgeCV)    R² 0.9593    MAE ₹87,466
lasso (LassoCV)    R² 0.9594    MAE ₹87,463
elastic net        R² 0.9594    MAE ₹87,438
ridge alpha: 0.1   lasso alpha: 0.0011   elastic net l1_ratio: 0.8
lasso set to exactly zero: ['cat__company_size_Medium', 'cat__rep_id_4', 'cat__rep_id_5',
                             'num__log_units_2024', 'num__log_orders_2024', 'num__tenure_months']
```

Reading it. All four models score almost the same (R^2 0.9593–0.9594). That's the honest result for this dataset: with 3,387 training rows and 20 features, ordinary least squares isn't overfitting, so there's little for regularization to fix. What lasso does do is **simplify**: it removed six features, including `log_units_2024` and `log_orders_2024` (duplicates of revenue) and `tenure_months`, without losing any accuracy.

Increase α by hand to see the trade-off:

```
from sklearn.linear_model import Lasso

for alpha in [0.001, 0.01, 0.05, 0.2]:
    m = reg_pipeline(Lasso(alpha=alpha)).fit(
        reg_train[CATS + REG_NUMS], reg_train["log_revenue_2025"]
    )
    kept = (m.named_steps["model"].coef_ != 0).sum()
    r2 = r2_score(reg_test["log_revenue_2025"], m.predict(reg_test[CATS + REG_NUMS]))
    print(
        f"alpha {alpha:<5}  features kept {kept:>2} of {len(names)}  test R² {r2:.4f}"
    )
```

```
alpha 0.001  features kept 14 of 20  test R² 0.9594
alpha 0.01   features kept  5 of 20  test R² 0.9583
alpha 0.05   features kept  2 of 20  test R² 0.9537
alpha 0.2    features kept  1 of 20  test R² 0.9254
```

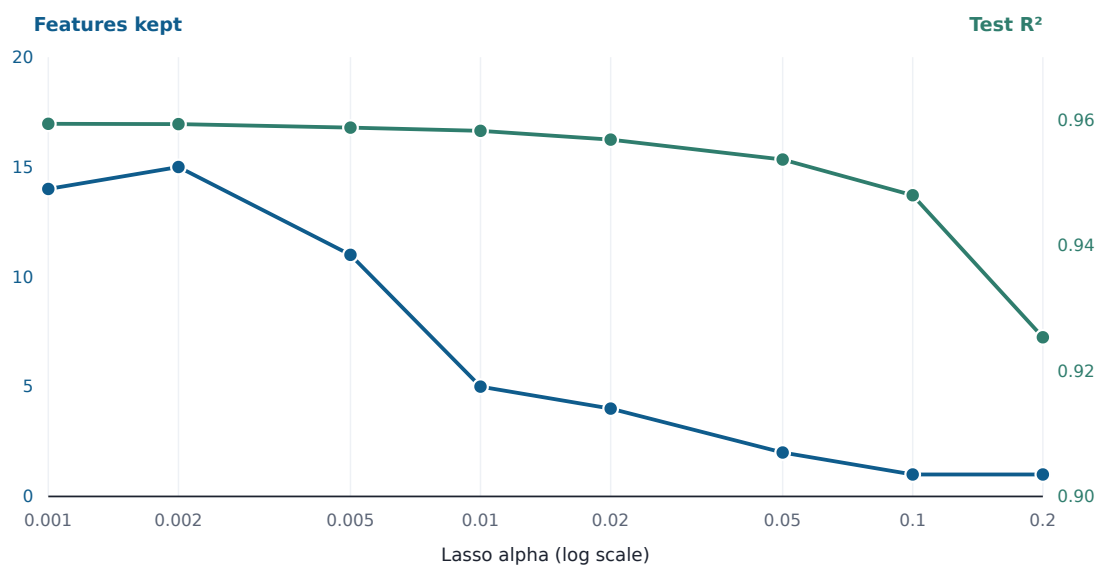


Figure 37.1 – Lasso as α grows. Five features give almost the same accuracy as fourteen; one feature (last year's revenue) still explains most of the variation.

What to tell Anita. "Next year's revenue for an account is mostly last year's revenue, adjusted up for hospitality accounts, accounts buying across several product categories, and down for accounts that pay late or complain." That sentence comes straight from the five features lasso keeps at $\alpha = 0.01$.

When regularization matters most: many features relative to rows (for example, 500 product-level features for 2,000 customers), correlated features, and any time you'd otherwise overfit. For logistic regression, scikit-learn regularizes by default (next section).

37.3 Logistic regression

How it works

Logistic regression predicts a **probability** for a yes-or-no outcome. It computes the same weighted sum as linear regression, called the **log-odds** or z , and passes it through the **sigmoid** function, which squeezes any number into the range 0 to 1:

$$z = b + w_1 \times \text{feature}_1 + \dots + w_n \times \text{feature}_n$$

$$\text{probability} = 1 \div (1 + e^{(-z)})$$

$z = 0$ gives 0.5; large positive z gives probabilities near 1; large negative z , near 0. The weights are chosen by **maximum likelihood**, which is the same as minimizing log loss (Chapter 35, section 35.9). There's no exact formula, so it's solved by an optimizer, a smarter relative of gradient descent.

Despite its name, it's a **classification** algorithm. Now the churn problem:

```

from sklearn.linear_model import LogisticRegression

def clf_pipeline(model, scale=True):
    num_steps = [("fill", SimpleImputer(strategy="median", add_indicator=True))]
    if scale:
        num_steps.append(("scale", StandardScaler()))
    prepare = ColumnTransformer(
        [
            ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
            ("num", Pipeline(num_steps), NUMS),
        ]
    )
    return Pipeline([("prepare", prepare), ("model", model)])

X_tr, y_tr = train_acc[CATS + NUMS], train_acc["churned_2025"]
X_va, y_va = valid_acc[CATS + NUMS], valid_acc["churned_2025"]
results = {}

def clf_report(name, pipe):
    pipe.fit(X_tr, y_tr)
    p = pipe.predict_proba(X_va)[:, 1]
    results[name] = roc_auc_score(y_va, p)
    print(f"{name:<30} AUC {results[name]:.3f}    log loss {log_loss(y_va, p):.4f}")
    return pipe

logit = clf_report(
    "logistic regression", clf_pipeline(LogisticRegression(max_iter=2000))
)
coef = pd.Series(
    logit.named_steps["model"].coef_[0],
    index=logit.named_steps["prepare"].get_feature_names_out(),
)
odds = np.exp(coef).round(2).sort_values()
print("odds ratios (per 1 standard deviation for numbers):")
print(pd.concat([odds.head(3), odds.tail(4)]).to_string())

```

```

logistic regression          AUC 0.764    log loss 0.2833
odds ratios (per 1 standard deviation for numbers):
num__categories_bought      0.57
num__units_2024             0.66
cat__segment_Wholesale      0.74
num__complaints_2024        1.30
num__revenue_2024           1.31
num__late_payment_days      1.53
num__days_since_last_order 2.10

```

How it works: `np.exp(coef)` turns each weight into an **odds ratio**: how the odds of churning are multiplied when the feature rises by one unit (here, one standard deviation, since numbers are scaled; for a one-hot column, when the category applies).

Reading it. On the validation set, logistic regression reaches an AUC of 0.764. The odds ratios read like a sales manager's instincts:

- **days_since_last_order 2.10:** one standard deviation longer without an order (about 80 days) doubles the odds of churning. The strongest signal.

- **late_payment_days 1.53** and **complaints_2024 1.30**: late payers and complainers are more likely to leave.
- **categories_bought 0.57**: each extra standard deviation of product breadth (about one more category) cuts the odds of churning by 43%. Customers who buy several product lines are stickier.

revenue_2024 1.31 and **units_2024 0.66** point in opposite directions. They're highly correlated, so the model balances one against the other; read them together, not separately (the same multicollinearity as in section 37.1).

The sigmoid by hand, and the setting C

Take the first validation account. The model's weighted sum for it is $z = -2.124$:

$$\text{probability} = 1 \div (1 + e^{2.124}) = 1 \div (1 + 8.365) = 1 \div 9.365 = \mathbf{0.1068}$$

```
one = X_va.iloc[[0]]
z = logit.decision_function(one)[0]
print(f"z = {z:.3f}    sigmoid(z) = 1 / (1 + e^-z) = {1 / (1 + np.exp(-z)):.4f}")
print(f"predict_proba: {logit.predict_proba(one)[0, 1]:.4f}")
for C in [0.01, 0.1, 1, 10]:
    clf_report(f"logistic, C={C}", clf_pipeline(LogisticRegression(C=C, max_iter=2000)))
```

```
z = -2.124    sigmoid(z) = 1 / (1 + e^-z) = 0.1068
predict_proba: 0.1068
logistic, C=0.01          AUC 0.762    log loss 0.2778
logistic, C=0.1          AUC 0.764    log loss 0.2821
logistic, C=1            AUC 0.764    log loss 0.2833
logistic, C=10           AUC 0.765    log loss 0.2831
```

The hand calculation matches `predict_proba` exactly. ✓

C is logistic regression's regularization setting, and it works backwards from α : **smaller C means stronger regularization**. `C=1` is the default, so scikit-learn's logistic regression is regularized (with the ridge-style L2 penalty) unless you change it. Here, C barely matters: AUC stays between 0.762 and 0.765. As with linear regression, 3,000 rows and 20 features leave little to regularize.

When to use logistic regression: almost always as your first real model. It trains in milliseconds, gives calibrated probabilities (usually), explains itself through odds ratios, and is hard to beat when effects are roughly additive, as Chapter 36's leads were. Its weakness is **interactions and thresholds**: it can't see "late payments matter only for small retailers" unless you build that feature. Section 37.10 shows how much that costs here, and how to fix it.

37.4 k-nearest neighbors

How it works

k-nearest neighbors (k-NN) doesn't learn weights at all. To predict for a new account, it finds the k most similar accounts in the training data, measured by Euclidean distance (Chapter 35), and predicts the share of those neighbors that churned. "Training" means storing the data.

That makes two things critical: **the choice of k** (few neighbors gives noisy predictions; many gives blurry ones) and **scaling** (distance is meaningless across unscaled units).

```
from sklearn.neighbors import KNeighborsClassifier

for k in [5, 25, 75]:
    clf_report(f"k-NN, k={k}", clf_pipeline(KNeighborsClassifier(n_neighbors=k)))
clf_report(
    "k-NN, k=25, NOT scaled",
    clf_pipeline(KNeighborsClassifier(n_neighbors=25), scale=False),
)
```

k-NN, k=5	AUC 0.684	log loss 1.5887
k-NN, k=25	AUC 0.765	log loss 0.5255
k-NN, k=75	AUC 0.775	log loss 0.3057
k-NN, k=25, NOT scaled	AUC 0.633	log loss 0.4735

Reading it.

- **$k = 5$** gives an AUC of 0.684 and a terrible log loss (1.59). With five neighbors, the only possible probabilities are 0, 0.2, 0.4, 0.6, 0.8, and 1, and a confident 0 for an account that churns is punished severely (Chapter 35).
- **$k = 25$ and 75** do much better (0.765 and 0.775), close to logistic regression.
- **Without scaling**, AUC collapses to 0.633. Revenue, measured in rupees, dominates every distance, so the "nearest" accounts are the ones with similar revenue, whatever their churn signals.

When to use k-NN: small datasets with few, meaningful, scaled features; as a quick baseline; and inside recommenders (Chapter 42 uses the same "nearest neighbors" idea on products). Avoid it with many features (in high dimensions, every point is roughly equally far from every other, the **curse of dimensionality**), with large datasets (prediction must search all training rows), and when you need to explain a decision.

37.5 Naive Bayes

How it works

Naive Bayes applies **Bayes' rule** to classification. For each class (churned, stayed), it multiplies:

the **prior**: how common the class is $\cdot$ the **likelihood** of each feature value, given the class

and compares the results. It's "naive" because it assumes features are **independent** given the class, so their likelihoods can be multiplied. That's almost never true, and the method often works anyway.

A worked example

Use two yes/no signals from the training accounts: *late-paying small retailer* (more than 25 days late on average, size Small, segment Retail) and *no order in the last 90 days*.

```
late_small_retail = (
    (train_acc["late_payment_days"] > 25)
    & (train_acc["company_size"] == "Small")
    & (train_acc["segment"] == "Retail")
)
recent = train_acc["days_since_last_order"] <= 90
table = pd.crosstab(
    [
        late_small_retail.rename("late small retail"),
        recent.rename("ordered in last 90 days"),
    ],
    train_acc["churned_2025"].rename("churned"),
)
print(table)
counts = train_acc["churned_2025"].value_counts()
print("class counts:", counts.to_dict())
for flag, name in [
    (late_small_retail, "late small retail"),
    (~recent, "no order in 90 days"),
]:
    print(
        f"P({name} | churned) = {flag[train_acc['churned_2025'] == 1].mean():.3f}  "
        f"P({name} | stayed) = {flag[train_acc['churned_2025'] == 0].mean():.3f}"
    )
```

```
churned
late small retail ordered in last 90 days
False      False      421  145
           True       2215  85
True       False       10  40
           True        64  20
class counts: {0: 2710, 1: 290}
P(late small retail | churned) = 0.207  P(late small retail | stayed) = 0.027
P(no order in 90 days | churned) = 0.638  P(no order in 90 days | stayed) = 0.159
```

For an account that is a late-paying small retailer **and** has had no order in 90 days:

churned: prior $290 \div 3,000 = 0.0967 \times P(\text{late small retail} \mid \text{churned}) 0.207 \times P(\text{no order in 90 days} \mid \text{churned}) 0.638 = \mathbf{0.01276}$

stayed: prior $2,710 \div 3,000 = 0.9033 \times 0.027 \times 0.159 = \mathbf{0.00388}$

$P(\text{churned}) = 0.01276 \div (0.01276 + 0.00388) = \mathbf{0.767}$

Naive Bayes says 76.7% (76.5% with the unrounded likelihoods). Check it against the table: training accounts with both signals churned 40 times out of 50, **80%**. The estimate is close, because these two signals really are roughly independent here. When features are strongly related (revenue, units, and orders, for example), multiplying their likelihoods counts the same evidence several times, and the probabilities become overconfident.

Now the full model. `GaussianNB` assumes each numeric feature follows a normal distribution within each class:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import FunctionTransformer

to_dense = FunctionTransformer(lambda X: X.toarray() if hasattr(X, "toarray") else X)
nb = clf_pipeline(GaussianNB())
nb.steps.insert(1, ("dense", to_dense))
clf_report("Gaussian Naive Bayes", nb)
```

Gaussian Naive Bayes	AUC 0.740	log loss 1.2866
----------------------	-----------	-----------------

How it works: `GaussianNB` can't read the sparse one-hot output, so a `FunctionTransformer` converts it to a normal (dense) array, inserted as an extra pipeline step. `nb.steps.insert(1, ...)` puts it between preparation and the model.

Reading it. AUC 0.740, a little below logistic regression, and a log loss of 1.29, far worse. Naive Bayes ranks reasonably but its probabilities are overconfident: the independence assumption, applied to 25 correlated columns, piles up evidence. Chapter 35 warned that skewed features like revenue aren't normal either.

When to use Naive Bayes: text classification (each word as a feature; Chapter 41 uses it for support tickets), very small datasets, and when you need something fast and simple. Don't use its probabilities without calibration (Chapter 39).

37.6 Decision trees

How it works

A decision tree asks a sequence of yes/no questions, each about one feature: "Is *days since last order more than 160*?" Every question splits the rows into two groups. Rows that end in the same final group, a **leaf**, get the same prediction: that leaf's churn rate.

To choose each question, the tree tries **every feature and every threshold** and picks the split that makes the groups purest. Purity is measured by **Gini impurity** or **entropy** (Chapter 35):

$$\text{Gini} = 1 - p^2 - (1 - p)^2 = 2 \times p \times (1 - p), \text{ where } p \text{ is the group's churn rate}$$

$$\text{entropy} = -(p \times \log_2 p + (1 - p) \times \log_2 (1 - p))$$

Both are 0 for a pure group and highest at $p = 0.5$.

A split by hand

Compare two candidate first questions on the 3,000 training accounts, 290 of which churned ($p = 0.0967$):

Gini before any split = $2 \times 0.0967 \times 0.9033 = \mathbf{0.1746}$

Split A: days since last order > 90. 616 accounts say yes, with a 30.0% churn rate; 2,384 say no, with 4.4%.

Gini(yes) = $2 \times 0.300 \times 0.700 = 0.4200$ Gini(no) = $2 \times 0.044 \times 0.956 = 0.0841$

weighted Gini after = $(616 \div 3,000) \times 0.4200 + (2,384 \div 3,000) \times 0.0841 = 0.0862 + 0.0669 = \mathbf{0.1531}$

Split B: late payment days > 25. Yes: 387 accounts, 23.0% churn. No: 2,613, 7.7%.

weighted Gini after = $(387 \div 3,000) \times (2 \times 0.230 \times 0.770) + (2,613 \div 3,000) \times (2 \times 0.077 \times 0.923) = 0.0457 + 0.1238 = \mathbf{0.1695}$

Split A reduces impurity from 0.1746 to 0.1531; split B only to 0.1695. The tree prefers A. Using entropy gives the same ranking (information gain, Chapter 35):

```
def gini(y):
    p = np.mean(y)
    return 1 - p**2 - (1 - p) ** 2

def entropy(y):
    p = np.mean(y)
    return 0.0 if p in (0, 1) else -(p * np.log2(p) + (1 - p) * np.log2(1 - p))

y_all = train_acc["churned_2025"]
for rule_name, rule in [
    ("days_since_last_order > 90", train_acc["days_since_last_order"] > 90),
    ("late_payment_days > 25", train_acc["late_payment_days"] > 25),
]:
    left, right = y_all[rule], y_all[~rule]
    w_l, w_r = len(left) / len(y_all), len(right) / len(y_all)
    print(
        f"{rule_name}: yes {len(left)} ({left.mean():.1%} churn),"
        f" no {len(right)} ({right.mean():.1%})"
    )
    print(
        f"    Gini {gini(y_all):.4f} -> {w_l * gini(left) + w_r * gini(right):.4f}    "
        f"entropy {entropy(y_all):.4f} -> {w_l * entropy(left) + w_r * entropy(right):.4f}"
    )
```

```
days_since_last_order > 90: yes 616 (30.0% churn), no 2384 (4.4%)
    Gini 0.1746 -> 0.1532    entropy 0.4583 -> 0.3881
late_payment_days > 25: yes 387 (23.0% churn), no 2613 (7.7%)
    Gini 0.1746 -> 0.1694    entropy 0.4583 -> 0.4411
```

(The code's weighted values, 0.1532 and 0.1694, differ from the hand values in the last digit because the churn rates were rounded to three decimals by hand.)

Depth, and overfitting

Keep splitting and every leaf eventually holds one account, and the tree memorizes the training data. Watch it happen:

```
from sklearn.tree import DecisionTreeClassifier, export_text

for depth in [2, 4, 6, 10, None]:
    tree = clf_pipeline(
        DecisionTreeClassifier(max_depth=depth, random_state=37), scale=False
    ).fit(X_tr, y_tr)
    tr_auc = roc_auc_score(y_tr, tree.predict_proba(X_tr)[: , 1])
    va_auc = roc_auc_score(y_va, tree.predict_proba(X_va)[: , 1])
    print(
        f"max_depth {str(depth):<5} train AUC {tr_auc:.3f}    valid AUC {va_auc:.3f}    "
        f"leaves {tree.named_steps['model'].get_n_leaves()}"
    )

small_tree = clf_pipeline(
    DecisionTreeClassifier(max_depth=2, random_state=37), scale=False
).fit(X_tr, y_tr)
print(
    export_text(
        small_tree.named_steps["model"],
        feature_names=list(small_tree.named_steps["prepare"].get_feature_names_out()),
        show_weights=True,
    )
)

clf_report(
    "decision tree, depth 4",
    clf_pipeline(
        DecisionTreeClassifier(max_depth=4, min_samples_leaf=20, random_state=37),
        scale=False,
    ),
)
```

```

max_depth 2      train AUC 0.759   valid AUC 0.708   leaves 4
max_depth 4      train AUC 0.829   valid AUC 0.748   leaves 16
max_depth 6      train AUC 0.886   valid AUC 0.733   leaves 48
max_depth 10     train AUC 0.969   valid AUC 0.609   leaves 161
max_depth None   train AUC 1.000   valid AUC 0.557   leaves 277
|--- num_days_since_last_order <= 160.50
| |--- num_days_since_last_order <= 94.50
| | |--- weights: [2305.00, 106.00] class: 0
| |--- num_days_since_last_order > 94.50
| | |--- weights: [232.00, 52.00] class: 0
|--- num_days_since_last_order > 160.50
| |--- num_late_payment_days <= 25.50
| | |--- weights: [160.00, 93.00] class: 0
| |--- num_late_payment_days > 25.50
| | |--- weights: [13.00, 39.00] class: 1

```

decision tree, depth 4 AUC 0.758 log loss 0.2810

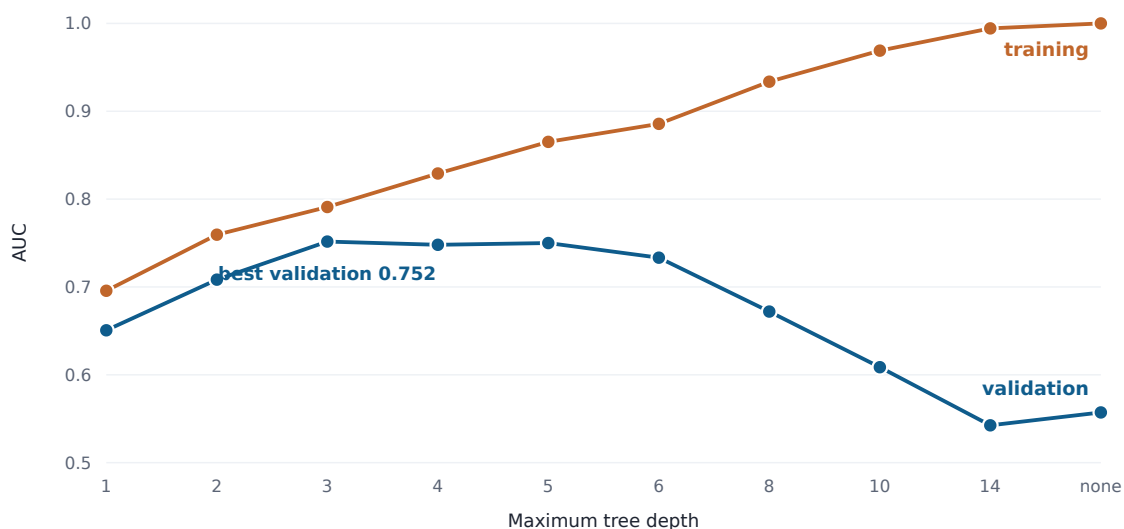


Figure 37.2 — A decision tree overfitting. Deeper trees fit the training data better and better, while their performance on new accounts peaks early and then collapses.

Reading it.

- A tree with no depth limit has **277 leaves** and a **perfect** training AUC of 1.000, and a validation AUC of 0.557, barely better than a coin. It memorized noise.
- Depth 4 is the best of the depths printed here (0.748); Figure 37.2, which tries more depths, finds depth 3 slightly better (0.752). The printed depth-2 tree is readable by anyone: accounts with no order for more than 160 days **and** more than 25 days of late payments churned 39 times out of 52. That's the planted interaction, found automatically.
- The tree found the interaction logistic regression couldn't see, but a single tree is **unstable**: small changes in the data change the first split and everything below it. Its best validation AUC is below logistic regression's.

Key settings: `max_depth`, `min_samples_leaf` (the fewest rows a leaf may hold; 20 to 100 is a sensible range for thousands of rows), and `criterion` ("gini" or "entropy"; they rarely differ much).

When to use a single tree: when a person must be able to follow the rules (a credit policy, a triage checklist), and as the building block for the next two methods. Trees need no scaling and handle interactions and thresholds naturally.

37.7 Random forests

How it works

A random forest trains **hundreds of trees** and averages their predictions. Averaging only helps if the trees make *different* mistakes, so each tree is made different in two ways:

1. **Bagging** (bootstrap aggregating): each tree trains on a random sample of the training rows, drawn with replacement.
2. **Random features:** at each split, each tree considers only a random subset of the features.

Each tree overfits in its own way; the average cancels much of the noise and keeps the signal. It's the committee from *In plain English*.

```
from sklearn.ensemble import RandomForestClassifier

forest = clf_report(
    "random forest",
    clf_pipeline(
        RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=37),
        scale=False,
    ),
)
imp = pd.Series(
    forest.named_steps["model"].feature_importances_,
    index=forest.named_steps["prepare"].get_feature_names_out(),
)
print(imp.sort_values(ascending=False).head(6).round(3).to_string())
```

random forest	AUC 0.785	log loss 0.2675
num_days_since_last_order	0.295	
num_late_payment_days	0.112	
num_revenue_2024	0.088	
num_units_2024	0.084	
num_tenure_months	0.070	
num_avg_discount_pct	0.066	

Reading it. AUC 0.785, the best so far and above logistic regression's 0.764. The forest can use thresholds and interactions without being told about them.

Feature importance. `feature_importances_` shows how much each feature reduced impurity across all trees. `days_since_last_order` leads by far, then `late_payment_days`. Two cautions: impurity-based importance is inflated for features with many distinct values (revenue has thousands; segment has three), and it says nothing about direction. Chapter 39 uses **permutation importance** and **SHAP**, which are more reliable.

Key settings: `n_estimators` (more trees never overfit, only cost time; 200–500 is usually plenty), `min_samples_leaf` or `max_depth` (limit the individual trees), and `max_features` (how many features each split considers; the default for classification is the square root of the number of features).

When to use a random forest: a strong, forgiving default for tabular data. It needs little tuning, rarely overfits badly, handles mixed feature types, and gives a quick importance ranking. It's usually a little behind well-tuned gradient boosting, and its models are large.

37.8 Gradient boosting

How it works

A forest builds trees **independently** and averages them. **Gradient boosting** builds them **one after another**, each new tree correcting the errors of the model so far:

1. Start with a simple prediction for everyone, such as the average.
2. Compute each row's **residual**: how far off the current prediction is. (More precisely, the negative gradient of the loss, Chapter 35. For squared error, that's the plain residual.)
3. Fit a **small tree** to the residuals.
4. Add that tree's predictions to the model, **scaled down** by a **learning rate**.
5. Repeat for hundreds of rounds.

Three rounds by hand

Take 8 training accounts, one feature (days since last order), and squared error, with a learning rate of 0.5 and one-split trees (**stumps**):

```
from sklearn.tree import DecisionTreeRegressor

toy = train_acc.head(8)
x_toy = toy[["days_since_last_order"]].to_numpy()
y_toy = toy["churned_2025"].to_numpy().astype(float)
pred = np.full(len(y_toy), y_toy.mean())
print("targets:      ", y_toy)
print(
    f"round 0: predict {y_toy.mean():.3f} for all"
    f"    squared error {np.mean((y_toy - pred) ** 2):.4f}"
)
for r in range(1, 4):
    residual = y_toy - pred
    stump = DecisionTreeRegressor(max_depth=1, random_state=37).fit(x_toy, residual)
    pred = pred + 0.5 * stump.predict(x_toy)
    print(
        f"round {r}: split at days <= {stump.tree_.threshold[0]:.1f}    "
        f"squared error {np.mean((y_toy - pred) ** 2):.4f}"
    )
print("final:      ", pred.round(3))
```

```

targets:      [0. 0. 1. 0. 0. 0. 0. 0.]
round 0: predict 0.125 for all    squared error 0.1094
round 1: split at days <= 77.5   squared error 0.0977
round 2: split at days <= 42.0   squared error 0.0818
round 3: split at days <= 77.5   squared error 0.0713
final:        [0.059 0.059 0.303 0.153 0.153 0.059 0.153 0.059]

```

Reading it. One of the eight accounts churned. Round 0 predicts the average, 0.125, for everyone. Each round fits a stump to what's still wrong, adds half of its correction, and the error falls: 0.1094, 0.0977, 0.0818, 0.0713. After three rounds, the churner's predicted value is 0.303, well above the rest. Real libraries do exactly this with log loss instead of squared error, deeper trees, and hundreds of rounds.

The libraries

Four implementations matter in practice:

Library	Strengths	Notes
scikit-learn HistGradientBoostingClassifier	Already installed; fast on medium data; handles missing values	Good default; fewer options
XGBoost	Mature, widely used, many options, GPU support	Industry standard in competitions and production
LightGBM	Very fast on large data; grows trees leaf-wise	num_leaves is its key setting; can overfit small data
CatBoost	Handles categorical columns natively and carefully; strong defaults	Slower to train; less tuning needed

The settings that matter most are shared across all four, under slightly different names: **number of trees** (max_iter, n_estimators, iterations), **learning rate** (smaller needs more trees but generalizes better; 0.01–0.1 is typical), **tree size** (max_depth, num_leaves), **minimum rows per leaf** (min_samples_leaf, min_child_samples), **row and column sampling** (subsample, colsample_bytree), and **L2 regularization**.

```

import catboost
import lightgbm
import xgboost
from sklearn.ensemble import HistGradientBoostingClassifier

print(
    "versions: xgboost",
    xgboost.__version__,
    "| lightgbm",
    lightgbm.__version__,
    "| catboost",
    catboost.__version__,
)
clf_report(
    "HistGradientBoosting default",
    clf_pipeline(HistGradientBoostingClassifier(random_state=37), scale=False),
)
clf_report(
    "HistGradientBoosting tuned",
    clf_pipeline(
        HistGradientBoostingClassifier(
            learning_rate=0.05,
            max_depth=3,
            min_samples_leaf=40,
            max_iter=300,
            random_state=37,
        ),
        scale=False,
    ),
)
clf_report(
    "XGBoost",
    clf_pipeline(
        xgboost.XGBClassifier(
            n_estimators=300,
            learning_rate=0.05,
            max_depth=3,
            subsample=0.8,
            colsample_bytree=0.8,
            random_state=37,
            n_jobs=1,
        ),
        scale=False,
    ),
)
clf_report(
    "LightGBM",
    clf_pipeline(
        lightgbm.LGBMClassifier(
            n_estimators=300,
            learning_rate=0.05,
            num_leaves=8,
            min_child_samples=40,
            subsample=0.8,
            subsample_freq=1,
            colsample_bytree=0.8,
            random_state=37,
            verbose=-1,
        ),
        scale=False,
    ),
)

cat_model = catboost.CatBoostClassifier(
    iterations=500,

```

```

learning_rate=0.05,
depth=4,
random_seed=37,
verbose=0,
thread_count=1,
)
cat_model.fit(X_tr, y_tr, cat_features=CATS)
p_cat = cat_model.predict_proba(X_va)[ :, 1]
results["CatBoost"] = roc_auc_score(y_va, p_cat)
print(
    f"'CatBoost (native categories)':<30} AUC {results['CatBoost']:.3f}"
    f"    log loss {log_loss(y_va, p_cat):.4f}"
)

```

```

versions: xgboost 3.4.1 | lightgbm 4.7.0 | catboost 1.2.10
HistGradientBoosting default    AUC 0.780    log loss 0.3151
HistGradientBoosting tuned      AUC 0.788    log loss 0.2731
XGBoost                          AUC 0.793    log loss 0.2700
LightGBM                        AUC 0.780    log loss 0.2794
CatBoost (native categories)    AUC 0.796    log loss 0.2710

```

How it works: XGBoost and LightGBM follow scikit-learn's `fit` and `predict_proba` interface, so they drop straight into the pipeline. CatBoost is given the raw columns and told which are categorical (`cat_features=CATS`); it encodes them itself, using a carefully ordered form of target encoding that avoids the leak from Chapter 36. On this run, CatBoost took about 1.7 seconds and the others 0.1–0.2 seconds each.

Reading it. All the boosting models beat logistic regression on validation (0.780–0.796 against 0.764). Default and tuned settings differ, and so do the libraries, by a couple of points. **With 1,000 validation accounts and 97 churners, differences of 0.01 are within noise.** Don't pick XGBoost over LightGBM on this evidence; section 37.10 uses cross-validation to compare properly.

When to use gradient boosting: the default choice for tabular business data when accuracy matters and you have at least a few thousand rows. It handles thresholds, interactions, missing values, and mixed features. Its costs: more settings to tune, slower training than linear models, overfitting on small data, and less transparency (Chapter 39's SHAP values help).

37.9 Support vector machines

How it works

A support vector machine (SVM) looks for the boundary between classes with the **widest margin**: the biggest gap to the nearest points on either side. Those nearest points, the **support vectors**, alone define the boundary. The setting **C** trades a wide margin against misclassified training points (as with logistic regression, smaller C means more regularization).

For boundaries that aren't straight lines, the **kernel trick** measures similarity between rows in a way that corresponds to a curved boundary, without computing new features explicitly. The **RBF** (radial basis function) kernel is the usual choice.

```

from sklearn.svm import SVC

clf_report(
    "SVM, linear kernel",
    clf_pipeline(SVC(kernel="linear", probability=True, random_state=37)),
)
clf_report(
    "SVM, linear, balanced",
    clf_pipeline(
        SVC(kernel="linear", class_weight="balanced", probability=True, random_state=37)
    ),
)
clf_report(
    "SVM, RBF, balanced",
    clf_pipeline(
        SVC(
            kernel="rbf",
            C=1.0,
            class_weight="balanced",
            probability=True,
            random_state=37,
        )
    ),
)

```

SVM, linear kernel	AUC 0.536	log loss 0.3337
SVM, linear, balanced	AUC 0.771	log loss 0.2787
SVM, RBF, balanced	AUC 0.780	log loss 0.2711

Reading it.

- The **linear SVM with default settings** scores an AUC of 0.536, almost a coin toss. With only 9.7% churners, the widest-margin solution is to push nearly all accounts to the "stayed" side, and the ranking within that side is nearly meaningless.
- **class_weight="balanced"** makes each churner count about nine times as much in the loss, so the boundary takes churners seriously. AUC jumps to 0.771. Chapter 39 covers class weights and other ways to handle imbalanced data.
- The **RBF kernel**, balanced, reaches 0.780, similar to the forest.

How it works: SVMs produce a distance from the boundary, not a probability. `probability=True` adds an extra calibration step (Platt scaling) with internal cross-validation, which slows training.

When to use SVMs: medium-sized datasets (up to tens of thousands of rows) with many features, such as text, and problems where the classes are well separated. Training time grows quickly with rows, they need scaling, and they don't give probabilities or explanations naturally. For typical business tables, gradient boosting has largely replaced them.

37.10 Bias, variance, and learning curves

The trade-off

A model's error on new data comes from two sources, beyond pure chance:

- **Bias:** the model is too simple to capture the pattern. A straight line can't follow a threshold. Symptom: training and validation scores are both mediocre and close together. **Underfitting.**
- **Variance:** the model is so flexible that it fits noise in the training sample. Symptom: training score much higher than validation. **Overfitting.** The unlimited decision tree was the extreme case.

The fix depends on which you have. For **high bias**, add features, use a more flexible model, or reduce regularization. For **high variance**, get more data, simplify the model, increase regularization, or average many models (a forest).

Compare properly first

Validation scores on 1,000 accounts are noisy. Before diagnosing, compare the three strongest candidates with 5-fold cross-validation on all 4,000 non-test accounts:

```
from sklearn.model_selection import StratifiedKFold, cross_val_score

folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=37)
candidates = {
    "logistic regression": clf_pipeline(LogisticRegression(max_iter=2000)),
    "random forest": clf_pipeline(
        RandomForestClassifier(n_estimators=300, min_samples_leaf=5, random_state=37),
        scale=False,
    ),
    "HistGradientBoosting tuned": clf_pipeline(
        HistGradientBoostingClassifier(
            learning_rate=0.05,
            max_depth=3,
            min_samples_leaf=40,
            max_iter=300,
            random_state=37,
        ),
        scale=False,
    ),
}
X_rest, y_rest = rest[CATS + NUMS], rest["churned_2025"]
for name, pipe in candidates.items():
    s = cross_val_score(pipe, X_rest, y_rest, cv=folds, scoring="roc_auc")
    print(f"{name:<28} mean AUC {s.mean():.3f}    sd {s.std():.3f}    folds {s.round(3)}")
```

logistic regression	mean AUC	0.807	sd	0.007	folds	[0.812 0.796 0.811 0.803 0.815]
random forest	mean AUC	0.826	sd	0.020	folds	[0.862 0.818 0.822 0.803 0.824]
HistGradientBoosting tuned	mean AUC	0.836	sd	0.018	folds	[0.868 0.828 0.838 0.817 0.829]

On 4,000 accounts, the picture is clearer: **tuned boosting 0.836, random forest 0.826, logistic regression 0.807**, and the fold spreads (0.007–0.020) are smaller than the gaps. Boosting's advantage over logistic regression is real on this data.

Learning curves

A **learning curve** trains a model on increasing amounts of data and plots training and cross-validation scores for each size:

```
from sklearn.model_selection import learning_curve

sizes = [0.1, 0.25, 0.5, 0.75, 1.0]
curves = {}
for name in ["logistic regression", "HistGradientBoosting tuned"]:
    n, tr_s, va_s = learning_curve(
        candidates[name],
        X_rest,
        y_rest,
        cv=folds,
        scoring="roc_auc",
        train_sizes=sizes,
        shuffle=True,
        random_state=37,
    )
    curves[name] = (n, tr_s.mean(axis=1), va_s.mean(axis=1))
for size, a, b in zip(n, tr_s.mean(axis=1), va_s.mean(axis=1)):
    print(
        f"{name:<27} {size:>5} rows   train AUC {a:.3f}"
        f"      CV AUC {b:.3f}   gap {a - b:.3f}"
    )
```

logistic regression	320 rows	train AUC 0.870	CV AUC 0.760	gap 0.110
logistic regression	800 rows	train AUC 0.827	CV AUC 0.777	gap 0.051
logistic regression	1600 rows	train AUC 0.822	CV AUC 0.802	gap 0.020
logistic regression	2400 rows	train AUC 0.822	CV AUC 0.808	gap 0.015
logistic regression	3200 rows	train AUC 0.820	CV AUC 0.807	gap 0.013
HistGradientBoosting tuned	320 rows	train AUC 1.000	CV AUC 0.747	gap 0.253
HistGradientBoosting tuned	800 rows	train AUC 0.995	CV AUC 0.792	gap 0.203
HistGradientBoosting tuned	1600 rows	train AUC 0.979	CV AUC 0.805	gap 0.174
HistGradientBoosting tuned	2400 rows	train AUC 0.959	CV AUC 0.815	gap 0.144
HistGradientBoosting tuned	3200 rows	train AUC 0.945	CV AUC 0.836	gap 0.109

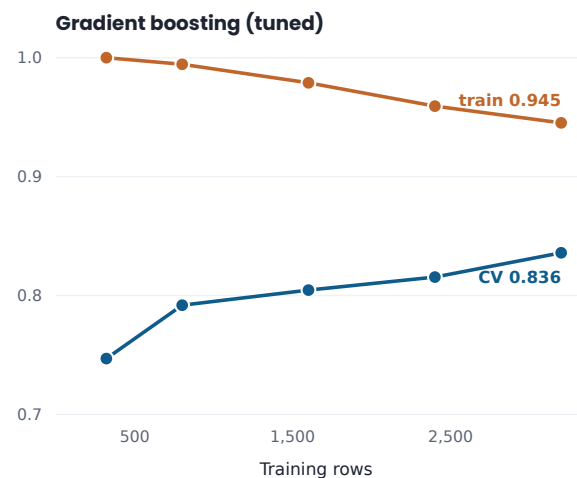


Figure 37.3 – Learning curves. Logistic regression has converged: more rows won't help, a sign of bias. Boosting still has a big train–CV gap and is still improving: a sign of variance that more data would reduce.

Reading it.

- **Logistic regression** has **high bias**. By 1,600 rows, its training and CV scores have nearly met (gap 0.013 at 3,200 rows), and the CV score has flattened at about 0.807. More data won't help. The model is missing something.
- **Boosting** has **higher variance**: a training AUC of 0.945 against 0.836 on CV. But its CV score is still climbing at 3,200 rows, so more accounts would likely help it further.

Fixing bias with features

Logistic regression's problem is structural: it can't represent "no order in 90 days" as a jump, or "late payments matter for small retailers" as an interaction. So build those as features, using what the tree in section 37.6 and the odds ratios suggested:

```
engineered = rest.copy()
test_eng = test_acc.copy()
for df in (engineered, test_eng):
    df["no_order_90"] = (df["days_since_last_order"] > 90).astype(int)
    df["no_order_180"] = (df["days_since_last_order"] > 180).astype(int)
    df["late_small_retail"] = (
        (df["late_payment_days"] > 25)
        & (df["company_size"] == "Small")
        & (df["segment"] == "Retail")
    ).astype(int)
    df["new_and_complaining"] = (
        (df["complaints_2024"] >= 2) & (df["tenure_months"] < 18)
    ).astype(int)
ENG = ["no_order_90", "no_order_180", "late_small_retail", "new_and_complaining"]
eng_pipe = Pipeline(
    [
        (
            "prepare",
            ColumnTransformer(
                [
                    ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
                    (
                        "num",
                        Pipeline(
                            [
                                (
                                    "fill",
                                    SimpleImputer(
                                        strategy="median", add_indicator=True
                                    ),
                                ),
                                ("scale", StandardScaler()),
                            ]
                        ),
                    ),
                ],
                NUMS + ENG,
            ),
        ),
        ("model", LogisticRegression(max_iter=2000)),
    ]
)
s = cross_val_score(
    eng_pipe, engineered[CATS + NUMS + ENG], y_rest, cv=folds, scoring="roc_auc"
)
print(f"logistic + 4 engineered features    mean AUC {s.mean():.3f}    sd {s.std():.3f}")
```

logistic + 4 engineered features mean AUC 0.849 sd 0.015

Reading it. With four engineered features, logistic regression's cross-validated AUC jumps from 0.807 to **0.849**, above tuned boosting's 0.836 and close to the 0.858 ceiling from section 37.0.

That's a useful lesson and an honest one, with a caveat. The features came from knowing where to look: thresholds and interactions that a tree and domain knowledge pointed to, and that match how this data was generated. On real data you won't know the true pattern, and boosting's advantage is that it finds such patterns without being told. The practical workflow is to use both: let boosting show what's possible, then ask whether a simpler model with good features gets close enough to be worth its transparency.

37.11 Hyperparameter tuning

Settings are chosen, not learned

Hyperparameters are the settings you choose before training: tree depth, learning rate, C , k . **Tuning** means trying different values and keeping the best, judged by cross-validation on the training data, never on the test set.

- **Grid search** (`GridSearchCV`) tries every combination of listed values. Five settings with four values each is $4^5 = 1,024$ combinations, times 5 folds: 5,120 fits. It grows too fast.
- **Random search** (`RandomizedSearchCV`) samples a fixed number of combinations from ranges you give. For the same budget, it explores more values of the settings that matter most. Use it instead of grid search by default.
- **Bayesian optimization** (Optuna, and others) chooses each next combination based on how earlier ones scored, spending more trials in promising regions.

Random search

```
from scipy.stats import loguniform, randint
from sklearn.model_selection import RandomizedSearchCV

search = RandomizedSearchCV(
    clf_pipeline(HistGradientBoostingClassifier(random_state=37), scale=False),
    param_distributions={
        "model__learning_rate": loguniform(0.01, 0.3),
        "model__max_depth": [2, 3, 4, 6, None],
        "model__min_samples_leaf": randint(10, 100),
        "model__max_iter": randint(100, 500),
        "model__l2_regularization": loguniform(1e-3, 10),
    },
    n_iter=25,
    cv=folds,
    scoring="roc_auc",
    random_state=37,
    n_jobs=1,
)
search.fit(X_rest, y_rest)
print(f"best CV AUC {search.best_score_:.3f}")
print(
    {
        k.replace("model__", ""): (round(float(v), 4) if isinstance(v, float) else v)
        for k, v in search.best_params_.items()
    }
)
top = pd.DataFrame(search.cv_results_).sort_values("rank_test_score")
print(
    top[["mean_test_score", "std_test_score"]].head(5).round(3).to_string(index=False)
)
```

```
best CV AUC 0.841
{'l2_regularization': 2.7349, 'learning_rate': 0.0199, 'max_depth': 3, 'max_iter': 334,
 'min_samples_leaf': 93}
mean_test_score  std_test_score
           0.841           0.017
           0.837           0.017
           0.837           0.016
           0.836           0.016
           0.835           0.015
```

How it works:

- `param_distributions` gives a range or list for each setting. `loguniform(0.01, 0.3)` samples learning rates evenly on a log scale, so 0.01–0.03 gets as many tries as 0.1–0.3. Use it for any setting that spans orders of magnitude.
- The names use the pipeline step: `model__learning_rate` means "the `learning_rate` of the step named `model`". That's how a search reaches inside a pipeline.
- Every one of the 25 combinations is scored with the same 5 folds, and the preparation steps are re-learned in each fold: no leakage (Chapter 36). On this run, the search took about 40 seconds on one CPU core.

Reading it. The best combination scores 0.841, only 0.005 above the hand-picked settings used earlier (0.836), and the top five are all within 0.006 of each other, well inside their fold-to-fold standard deviation of about 0.017. **Tuning helped a little, and the exact winner is partly luck.** That's typical:

the first sensible settings get you most of the way. The best settings share a pattern worth noticing: a small learning rate, shallow trees (depth 3), large leaves (93 rows), and strong L2 regularization. With only 290 churners in training, the model does best when it's kept simple.

Optuna

```
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)

def objective(trial):
    model = HistGradientBoostingClassifier(
        learning_rate=trial.suggest_float("learning_rate", 0.01, 0.3, log=True),
        max_depth=trial.suggest_int("max_depth", 2, 6),
        min_samples_leaf=trial.suggest_int("min_samples_leaf", 10, 100),
        max_iter=trial.suggest_int("max_iter", 100, 500),
        l2_regularization=trial.suggest_float("l2_regularization", 1e-3, 10, log=True),
        random_state=37,
    )
    return cross_val_score(
        clf_pipeline(model, scale=False), X_rest, y_rest, cv=folds, scoring="roc_auc"
    ).mean()

study = optuna.create_study(
    direction="maximize", sampler=optuna.samplers.TPESampler(seed=37)
)
study.optimize(objective, n_trials=25)
print(f"optuna {optuna.__version__}: best CV AUC {study.best_value:.3f}")
print(
    {
        k: round(v, 4) if isinstance(v, float) else v
        for k, v in study.best_params.items()
    }
)
```

```
optuna 5.0.0: best CV AUC 0.844
{'learning_rate': 0.0307, 'max_depth': 3, 'min_samples_leaf': 87, 'max_iter': 498,
 'l2_regularization': 5.3027}
```

How it works: Optuna calls `objective` repeatedly. Each `trial.suggest_...` call picks a value for one setting; the function returns the cross-validated score, and Optuna's default sampler (TPE, a Bayesian method) uses past results to choose the next values. `seed=37` makes the search repeatable.

Reading it. After the same 25 trials, Optuna finds 0.844, a little above random search. With larger search spaces or slower models, the advantage of choosing trials intelligently grows.

Watch out: tuning can overfit the validation data. Try enough combinations and one will score well by chance. Three habits keep you honest: tune with cross-validation, not one validation set; don't read much into differences smaller than the fold standard deviation; and score the **test set once**, at the end, for the final choice only.

The test set, once

Three candidates remain for churn: plain logistic regression, logistic regression with engineered features, and tuned boosting. Each is refitted on all 4,000 non-test accounts and scored once on the 1,000 test accounts:

```
final_models = {
    "logistic regression": clf_pipeline(LogisticRegression(max_iter=2000)),
    "logistic + engineered": eng_pipe,
    "tuned boosting (random search)": search.best_estimator_,
}
for name, pipe in final_models.items():
    cols = CATS + NUMS + (ENG if "engineered" in name else [])
    source = engineered if "engineered" in name else rest
    target = test_eng if "engineered" in name else test_acc
    pipe.fit(source[cols], source["churned_2025"])
    p = pipe.predict_proba(target[cols])[:, 1]
    print(
        f"{name:<32} test AUC {roc_auc_score(target['churned_2025'], p):.3f}    "
        f"log loss {log_loss(target['churned_2025'], p):.4f}"
    )
```

logistic regression	test AUC 0.797	log loss 0.2538
logistic + engineered	test AUC 0.850	log loss 0.2278
tuned boosting (random search)	test AUC 0.839	log loss 0.2375

Reading it. The test results agree with cross-validation: plain logistic regression **0.797**, tuned boosting **0.839**, logistic regression with engineered features **0.850**, also with the best log loss.

What to tell Anita. "We can rank accounts by their risk of stopping ordering well enough to act on. The strongest warning signs are no order in the last three months, late payments at small retail accounts, and complaints from newer accounts, and customers buying several product lines rarely leave. I recommend the simpler model built on those signs: it's slightly more accurate than the complex one on accounts it hadn't seen, and we can explain every score to the sales team."

37.12 Project result: lead scoring, baseline vs boosting

Now the question this chapter's project asks: does gradient boosting beat Chapter 36's logistic regression on Riverstone's leads? Tune boosting with a **time-series** cross-validation on the 2023–2024 leads (the leads arrive over time), then compare on validation and, once, on test:

```

import sys

sys.path.append(".") # so Python finds lead_data.py in this folder
from lead_data import LEAD_CATS, LEAD_NUMS, load_leads, make_model
from sklearn.model_selection import TimeSeriesSplit

leads_train, leads_valid, leads_test = load_leads()
LX = LEAD_CATS + LEAD_NUMS

lead_search = RandomizedSearchCV(
    make_model(LEAD_CATS, LEAD_NUMS, HistGradientBoostingClassifier(random_state=37)),
    param_distributions={
        "model__learning_rate": loguniform(0.01, 0.3),
        "model__max_depth": [2, 3, 4, 6, None],
        "model__min_samples_leaf": randint(20, 200),
        "model__max_iter": randint(100, 500),
        "model__l2_regularization": loguniform(1e-3, 10),
    },
    n_iter=20,
    cv=TimeSeriesSplit(n_splits=4),
    scoring="roc_auc",
    random_state=37,
    n_jobs=1,
)
lead_search.fit(leads_train[LX], leads_train["won"])
print(f"tuned boosting, time-series CV AUC {lead_search.best_score_:.3f}")

for name, fit_rows, eval_rows in [
    ("VALID", leads_train, leads_valid),
    ("TEST", pd.concat([leads_train, leads_valid]), leads_test),
]:
    base = make_model(LEAD_CATS, LEAD_NUMS).fit(fit_rows[LX], fit_rows["won"])
    boost = lead_search.best_estimator_.fit(fit_rows[LX], fit_rows["won"])
    for label, m in [("logistic (Ch 36)", base), ("tuned boosting", boost)]:
        p = m.predict_proba(eval_rows[LX])[:, 1]
        print(
            f"{name:<5} {label:<18} AUC {roc_auc_score(eval_rows['won'], p):.3f}  "
            f"log loss {log_loss(eval_rows['won'], p):.4f}"
        )

```

```

tuned boosting, time-series CV AUC 0.769
VALID logistic (Ch 36)    AUC 0.823    log loss 0.1948
VALID tuned boosting     AUC 0.829    log loss 0.1972
TEST  logistic (Ch 36)    AUC 0.838    log loss 0.2369
TEST  tuned boosting     AUC 0.832    log loss 0.2424

```

How it works: `lead_data.py` rebuilds Chapter 36's cleaned table and features in one call, so Chapter 36's steps aren't repeated here. `sys.path.append(".")` tells Python to look for it in the current folder (a notebook opened in `companion/ch37/` already does).

Reading it. On leads, **boosting doesn't win**. On validation it ranks marginally better (AUC 0.829 against 0.823) but its probabilities are worse (log loss 0.1972 against 0.1948). On the test set, logistic regression is ahead on both (0.838 against 0.832, and 0.2369 against 0.2424). The differences are small enough to be noise either way.

Why the opposite of the churn result? The leads data's signal is mostly **additive**: each feature (source, email type, text, quantity) pushes the odds up or down on its own, which is exactly the shape logistic regression assumes. There's little interaction for boosting to find, and with about 580 wins in training,

its extra flexibility mostly fits noise. The churn data has thresholds and interactions, and boosting's flexibility pays off.

The decision: keep logistic regression for lead scoring. It's as accurate, better calibrated, faster, and explainable. That's not a failure of the project; finding out that the simple model is enough is one of the most valuable results a data scientist can deliver.

Which algorithm when

Algorithm	Best for	Needs scaling?	Handles interactions?	Interpretable?	Watch out for
Linear regression	Numeric targets with roughly linear effects; explanation	Yes, for comparing or regularizing	No (build them)	Yes: coefficients	Outliers, multicollinearity, skewed targets
Ridge / lasso / elastic net	Many or correlated features; feature selection (lasso)	Yes	No	Yes	Choosing α ; lasso picking one of a correlated group
Logistic regression	First model for any classification; calibrated probabilities	Yes	No (build them)	Yes: odds ratios	Thresholds and interactions it can't see
k-nearest neighbors	Small data, few features; similarity lookups	Essential	Yes, implicitly	By example only	Many features, large data, choice of k
Naive Bayes	Text, very small data, speed	No (Gaussian: helps)	No	Partly	Overconfident probabilities
Decision tree	Rules people must follow; teaching	No	Yes	Yes, if shallow	Overfitting; instability
Random forest	Strong default with little tuning	No	Yes	Importance only	Large models; biased impurity importance
Gradient boosting	Best accuracy on tabular data with thousands of rows	No	Yes	With SHAP (Ch 39)	Tuning; overfitting small data
SVM	Medium data with many features (text); clear separation	Essential	Yes (kernels)	Little	Slow on large data; imbalance; no native probabilities

A default workflow for tabular problems: baselines (Chapter 36) → logistic or linear regression → a random forest or default gradient boosting → learning curves to diagnose → better features or tuned boosting, depending on what the curves say → cross-validated comparison → the simplest model that's close enough to the best → the test set, once.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reading coefficients of correlated features one by one	Signs that make no sense (revenue up, units down)	Interpret groups together; use lasso or drop duplicates
Treating a coefficient as a cause	"Late payments cause 5% less growth" in a slide	Say "associated with"; use Chapter 31's methods for causes
Acting on a coefficient without an interval	Decisions based on noise (the inside-desk +3%)	Use <code>statsmodels</code> for confidence intervals (Chapter 22)
Regularizing unscaled features	Rupee-scale features escape the penalty	Standardize inside the pipeline
Confusing C and α	Increasing C and expecting more regularization	Smaller C = stronger regularization
k-NN or SVM without scaling	AUC far below simpler models	Scale in the pipeline
Trusting Naive Bayes probabilities	Log loss far worse than AUC suggests	Use it for ranking, or calibrate (Chapter 39)
Unlimited tree depth	Training AUC 1.0, validation near 0.5	Limit <code>max_depth</code> or <code>min_samples_leaf</code>
Default SVM on imbalanced data	AUC near 0.5	<code>class_weight="balanced"</code>
Choosing a library on one validation set	Switching libraries for 0.01 AUC	Cross-validate; differences within fold spread are noise
Grid search over many settings	Hours of compute, little gain	Random search or Optuna with a fixed budget
Tuning on the test set	Test score improves with each try	Tune with CV on training data; score test once
Assuming boosting always wins	Complex model deployed for no gain	Compare with a baseline and a linear model with good features
Impurity importance as the final word	High-cardinality features look most important	Permutation importance or SHAP (Chapter 39)

In the real world: "Should we buy the AutoML tool?"

In April 2026, Riverstone's finance team is renewing software licences, and Vikram forwards a quote for an **AutoML** platform, a service that tries dozens of algorithms and settings automatically. The vendor's demo, run on Riverstone's own account data, showed a "stacked ensemble of 40 models" scoring a churn AUC of 0.85. The question for Meera: *is it worth ₹6 lakh a year?*

Meera doesn't argue about AutoML in general. She sets up a fair comparison on the same test accounts, using this chapter's workflow.

Step 1: what does simple get? Logistic regression: 0.80. That's the floor any tool must beat.

Step 2: what does standard get? Gradient boosting with 25 rounds of random search, about a minute of compute: 0.84.

Step 3: what does thinking get? Logistic regression with four features built from the tree's first splits and the sales team's own warning signs (no order for three months, late-paying small retailers, new accounts with complaints): 0.85.

Step 4: what does each option cost to live with? The AutoML ensemble can't be explained account by account without extra tooling, needs the vendor's platform to run, and must be retrained through it. The logistic model is a few lines in the existing pipeline, and each account's score can be broken down into the signs that drove it, which the account managers will want before they call a customer.

Her note to Vikram:

"On our own test accounts, the AutoML ensemble (0.85) matches what we already get from a simple model built on the sales team's warning signs (0.85), and beats our quickly tuned boosting model (0.84) by a margin that's within the noise for 1,000 accounts. The simple model is free, runs in our existing pipeline, and can explain every score. I'd skip the licence for now. If we later have problems with many more features, like predicting demand for 200 products, AutoML would be worth a trial, because that's where automatic search saves real time."

Vikram cancels the renewal request. Two months later, Meera does use an open-source AutoML library for a weekend experiment on product demand (Chapter 40), as a quick way to see what's possible before building by hand, which is where such tools shine.

Tools

- **scikit-learn** (tested on 1.8.0; the current release at the time of writing is 1.9.1): `LinearRegression`, `RidgeCV`, `LassoCV`, `ElasticNetCV`, `LogisticRegression`, `KNeighborsClassifier`, `GaussianNB`, `DecisionTreeClassifier` and `export_text`, `RandomForestClassifier`, `HistGradientBoostingClassifier`, `SVC`, `learning_curve`, `RandomizedSearchCV`.
 - **XGBoost** 3.4.1, **LightGBM** 4.7.0, **CatBoost** 1.2.10, installed with `pip install xgboost lightgbm catboost`. All are free and open source; versions move quickly, so check each project's documentation for the current parameter names.
 - **Optuna** 5.0.0 (`pip install optuna`) for Bayesian hyperparameter search.
 - **SciPy** for `loguniform` and `randint` distributions in random search.
 - **statsmodels** for regression with confidence intervals and p-values (Chapter 22).
 - Everything ran on one CPU core, Python 3.12.3, on 17 September 2026. The slowest step, 25 rounds of 5-fold random search, took under a minute.
 - **Companion files:** `companion/generate_riverstone_accounts.py` (seed 20237) builds `companion/accounts/accounts.csv`; `companion/ch37/lead_data.py` rebuilds Chapter 36's lead table. Run the chapter's code from `companion/ch37/`. Data spec: `planning/data/riverstone-accounts.md`.
-

The project: lead scoring, baseline vs boosting

Goal: a documented, fair comparison of algorithms on one problem, ending in a justified choice. Section 37.12 ran the core comparison for leads; the project extends it and makes it yours.

Option A: your own data. Any classification or regression problem you set up with Chapter 36's workflow.

Option B: Riverstone. Lead scoring (`lead_data.py`) or account churn (`accounts.csv`), or both.

Steps:

1. **Start from the Chapter 36 pipeline and baselines.** Record the base rate, the rule of thumb, and logistic regression on validation.
2. **Train one model from each family:** logistic regression, k-NN, Naive Bayes, a decision tree, a random forest, one gradient-boosting library, and an SVM. Use the same pipeline and validation set for all.
3. **Hand-check one thing per family:** one sigmoid probability, one Naive Bayes posterior, one Gini split, three rounds of boosting on eight rows.
4. **Cross-validate the top three** (use a time-series split for leads) and report means and standard deviations.
5. **Draw learning curves** for the simplest and the most flexible of the three, and write two sentences diagnosing bias or variance.
6. **Try to close the gap** with features or with tuning, depending on the diagnosis. Use random search or Optuna with a fixed budget.
7. **Score the test set once** for your final two candidates.
8. **Write a one-page decision memo:** which model you recommend and why, what it costs to run and explain, and what you'd try next.

Stretch goals:

- Add **early stopping** to the gradient-boosting model (`early_stopping=True` in `HistGradientBoostingClassifier` , or an evaluation set in XGBoost and LightGBM) and compare training time and score.
- Build a **stacking** model (`StackingClassifier`) that combines logistic regression and boosting, and check fairly whether it beats either alone.
- Fit the regression model in `statsmodels` and compare its confidence interval for the inside-sales-desk coefficient with the effect you see.
- Try a **monotonic constraint** in `HistGradientBoostingClassifier` (for example, churn risk must not fall as days since last order rise) and see whether it costs any accuracy.

You've got it when...

- ☐ I can explain, in two sentences each, how linear regression, logistic regression, k-NN, Naive Bayes, trees, random forests, gradient boosting, and SVMs make a prediction.
 - ☐ I can read a regression coefficient and an odds ratio in plain language, and I know why correlated features make them unreliable.
 - ☐ I know what ridge, lasso, and elastic net penalize, why features must be scaled for them, and that smaller C means stronger regularization.
 - ☐ I can compute a sigmoid probability, a Naive Bayes posterior, and a Gini split by hand.
 - ☐ I can recognize an overfitting tree from its training and validation scores.
 - ☐ I can explain why averaging trees (forest) and chaining trees (boosting) both beat a single tree.
 - ☐ I know the main settings of gradient boosting and what each controls.
 - ☐ I handle class imbalance in SVMs and know when an algorithm needs scaling.
 - ☐ I can read learning curves and say whether a model suffers from bias or variance, and what to do about each.
 - ☐ I tune with random search or Optuna inside cross-validation, and I treat differences smaller than the fold spread as noise.
 - ☐ I compare complex models with baselines and with simple models that have good features, and I can defend choosing the simpler one.
-

Recap

- **Linear regression** fits a weighted sum by least squares; interpret coefficients carefully (scaling, multicollinearity, noise, not causes).
- **Ridge** shrinks all weights, **lasso** zeroes some (feature selection), **elastic net** mixes them. Scale first. They help most with many or correlated features.
- **Logistic regression** passes a weighted sum through the **sigmoid** to get a probability, fitted by maximum likelihood. Read it with **odds ratios**. Smaller **C** means more regularization. It can't see thresholds or interactions unless you build them.
- **k-NN** predicts from the nearest training rows; choose *k*, always scale, avoid many features.
- **Naive Bayes** multiplies a prior by per-feature likelihoods, assuming independence; fast, good for text, overconfident probabilities.
- **Decision trees** split on the question that most reduces **Gini** or **entropy**; readable, find interactions, overfit when deep.
- **Random forests** average many decorrelated trees (**bagging** + random features): a robust default.
- **Gradient boosting** adds small trees one at a time, each fitting the current errors, scaled by a **learning rate**: usually the most accurate on tabular data. XGBoost, LightGBM, and CatBoost are the main libraries.

- **SVMs** find the widest-margin boundary; kernels bend it; they need scaling and class weights for imbalance.
- **Bias** is underfitting, **variance** is overfitting; **learning curves** tell them apart.
- **Tune** with random search or **Optuna** inside cross-validation; score the test set once.
- The best algorithm depends on the data: boosting won on churn, tied on leads, and a simple model with good features beat both on churn.

Practice exercises

Code exercises run from `companion/ch37/` after the chapter's code (they use `train_acc`, `valid_acc`, `rest`, `test_acc`, `CATS`, `NUMS`, `clf_pipeline`, `clf_report`, `reg_pipeline`, `reg_train`, `reg_test`, `folds`, and the other names defined above). Predict each result before running it.

Warm-up

1. (*hand*) A logistic regression gives an account $z = 0.8$. What churn probability does it predict? What z corresponds to a probability of exactly 0.5?
2. (*hand*) An odds ratio for `complaints_2024` (unscaled, per complaint) is 1.45. An account with a 10% churn probability files one more complaint. What are its new odds and new probability?
3. (*hand*) A group of 200 accounts has 50 churners. Calculate its Gini impurity and its entropy.
4. For each situation, name the algorithm you'd try first and one reason: (a) 300 rows, 4 features, the sales team must follow the rules by hand; (b) 50,000 accounts, 60 mixed features, maximum accuracy; (c) classifying 20,000 support tickets by their words; (d) predicting next month's revenue per customer for a finance report that must explain each driver.

Core

5. Fit `RidgeCV` and `LassoCV` on the regression data **without** `log_revenue_2024`. How do R^2 and the coefficients of `log_units_2024` and `log_orders_2024` change? What does that tell you about multicollinearity?
6. For k-NN, try `weights="distance"` with $k = 75$. Does weighting closer neighbors more help on validation?
7. Using the training accounts, compute by hand (in code, without a tree) the weighted Gini after splitting on `categories_bought >= 3`. Is it a better first split than `days_since_last_order > 90`?
8. Train a random forest with `n_estimators` of 10, 50, 300, and 1,000 (keep `min_samples_leaf=5`). Report validation AUC. Does adding trees ever make it worse?
9. Continue the eight-row boosting example for 20 rounds instead of 3, and print the squared error every 5 rounds. What happens to the error, and why is that a warning sign rather than good news?
10. Draw a validation curve for `HistGradientBoostingClassifier`'s `max_depth` (2, 3, 4, 6, 8) using `validation_curve` with the 5 folds on `rest`. Where does it peak?

Stretch

- l1. Build a `StackingClassifier` from logistic regression and tuned boosting (`search.best_estimator_`), with a logistic regression on top. Cross-validate it on `rest`. Does it beat both parts?
- l2. Add the four engineered features from section 37.10 to the tuned boosting model. Does boosting gain as much as logistic regression did? Explain.
- l3. Using `statsmodels`, fit an ordinary least squares regression of `log_revenue_2025` on the same features (one-hot with drop first, unscaled numbers) and read the 95% confidence interval for the inside-sales-desk dummy. Does it include zero?

Think about it

- l4. A colleague says: "Boosting got 0.841 in tuning and logistic regression got 0.807, so boosting is 4% better." Give two corrections.
- l5. Why can a random forest's feature importance rank `revenue_2024` above `complaints_2024` even if complaints matter more for churn?
- l6. The sales head asks why the model was accurate last year but is worse this year. Using bias and variance, and Chapter 36's ideas, give three possible explanations.

Key terms

supervised learning · regression · classification · linear regression · least squares · intercept · coefficient · R^2 (coefficient of determination) · mean absolute error (MAE) · log-log model · dummy variable trap · multicollinearity · residual · regularization · penalty · alpha (α) · ridge regression (L2) · lasso (L1) · elastic net · feature selection · logistic regression · log-odds · sigmoid · odds ratio · C (inverse regularization) · k-nearest neighbors · curse of dimensionality · Naive Bayes · Bayes' rule · prior · likelihood · posterior · conditional independence · decision tree · node · leaf · split · Gini impurity · entropy · `max_depth` · `min_samples_leaf` · random forest · bagging · bootstrap sample · feature importance · gradient boosting · residual fitting · stump · learning rate · XGBoost · LightGBM · CatBoost · support vector machine · margin · support vector · kernel · RBF kernel · class weight · bias · variance · underfitting · overfitting · learning curve · validation curve · hyperparameter · grid search · random search · Bayesian optimization · Optuna · AutoML · stacking · early stopping

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 38, Unsupervised Learning**, segments the same accounts without a target, using distance and PCA.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, turns churn and lead scores into decisions with cost-based thresholds, handles imbalance with class weights and resampling,

checks calibration, explains predictions with SHAP and partial dependence, and checks fairness.

- **Chapter 40, Time Series & Forecasting**, uses gradient boosting with lag features for demand forecasting.
- **Chapter 41, NLP Foundations**, uses Naive Bayes and logistic regression on text.
- **Chapter 43, A First Look at Deep Learning**, takes logistic regression's weighted sum and sigmoid, and stacks them into a neural network.
- **Chapter 22** (statistical inference) and **Chapter 31** (causal inference) cover confidence intervals for coefficients and estimating causes.
- **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers every algorithm here, bias and variance, regularization, and tuning with graded answers. "Explain gradient boosting to a non-technical person" and "random forest vs gradient boosting" are among the most common questions in data science interviews.

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. probability = $1 \div (1 + e^{-0.8}) = 1 \div (1 + 0.4493) = \mathbf{0.690}$. A probability of 0.5 needs $1 + e^{-z} = 2$, so $e^{-z} = 1$ and $\mathbf{z = 0}$. Any account whose weighted sum is exactly zero sits on the fence.

2. Odds = probability $\div$ (1 - probability) = $0.10 \div 0.90 = 0.1111$. One more complaint multiplies the odds by 1.45: $0.1111 \times 1.45 = \mathbf{0.1611}$. Back to a probability: odds $\div$ (1 + odds) = $0.1611 \div 1.1611 = \mathbf{0.139}$, or 13.9%. The common mistake is to multiply the probability itself ($10\% \times 1.45 = 14.5\%$); odds ratios multiply odds, not probabilities. The difference is small at low probabilities and large at high ones.

3. $p = 50 \div 200 = 0.25$. Gini = $2 \times 0.25 \times 0.75 = \mathbf{0.375}$. Entropy = $-(0.25 \times \log_2 0.25 + 0.75 \times \log_2 0.75) = -(0.25 \times (-2) + 0.75 \times (-0.4150)) = 0.5 + 0.3113 = \mathbf{0.811 \text{ bits}}$.

4. (a) A **shallow decision tree** (depth 2 or 3): with 300 rows, complex models overfit, and a tree's rules can be printed and followed by hand. (b) **Gradient boosting**: plenty of rows, mixed features, and accuracy as the goal, with logistic regression as the baseline to beat. (c) **Logistic regression or Naive Bayes on word features** (TF-IDF, Chapter 41), or a linear SVM: text gives thousands of sparse features, where linear models are fast and strong. (d) **Linear regression** (regularized if there are many features) on a log target: finance needs each driver's effect stated, which coefficients provide.

5.

```

REG_NUMS_FULL = REG_NUMS
REG_NUMS = [c for c in REG_NUMS_FULL if c != "log_revenue_2024"]
names_wo = (
    reg_pipeline(LinearRegression())
    .fit(reg_train[CATS + REG_NUMS], reg_train["log_revenue_2025"])
    .named_steps["prepare"]
    .get_feature_names_out()
)
for label, model in [
    ("ridge", RidgeCV(alphas=np.logspace(-3, 3, 25))),
    ("lasso", LassoCV(cv=5, random_state=37)),
]:
    fitted = reg_report(label + " without revenue", reg_pipeline(model))
    c = pd.Series(fitted.named_steps["model"].coef_, index=names_wo)
    print(
        f"    log_units_2024 {c['num__log_units_2024']:.3f}"
        f"    log_orders_2024 {c['num__log_orders_2024']:.3f}"
    )
REG_NUMS = REG_NUMS_FULL

```

```

ridge without revenue R² 0.9473    MAE ₹99,755
log_units_2024 0.987    log_orders_2024 0.074
lasso without revenue R² 0.9473    MAE ₹99,626
log_units_2024 0.991    log_orders_2024 0.074

```

Without revenue, the model leans on units instead: its coefficient jumps from nearly 0 to about 0.99, and R^2 drops only from 0.959 to 0.947, because units carry almost the same information. That's multicollinearity in action. The coefficient of a feature depends on which of its "twins" are also in the model, so a coefficient near zero doesn't mean a feature is useless; it may mean another feature is doing its job.

6.

```

clf_report("k-NN, k=75, uniform", clf_pipeline(KNeighborsClassifier(n_neighbors=75)))
clf_report(
    "k-NN, k=75, distance-weighted",
    clf_pipeline(KNeighborsClassifier(n_neighbors=75, weights="distance")),
)

```

```

k-NN, k=75, uniform          AUC 0.775    log loss 0.3057
k-NN, k=75, distance-weighted AUC 0.778    log loss 0.3050

```

Distance weighting changes the ranking only slightly here. With 75 neighbors in 25 dimensions, most neighbors are at similar distances (the curse of dimensionality again), so weighting them by distance makes little difference.

7.

```

y_all = train_acc["churned_2025"]
for label, rule in [
    ("categories_bought >= 3", train_acc["categories_bought"] >= 3),
    ("days_since_last_order > 90", train_acc["days_since_last_order"] > 90),
]:
    after = sum(len(g) / len(y_all) * gini(g) for g in (y_all[rule], y_all[~rule]))
    print(
        f"{label:<28} weighted Gini after split {after:.4f}    (before {gini(y_all):.4f})"
    )

```

```

categories_bought >= 3      weighted Gini after split 0.1710    (before 0.1746)
days_since_last_order > 90 weighted Gini after split 0.1532    (before 0.1746)

```

Splitting on three or more categories reduces Gini, but much less than the recency split, so a tree would still start with recency. The product-breadth signal is real (odds ratio 0.57) but spread across all accounts, whereas recency separates a small, high-risk group sharply. A Gini split rewards sharp separation.

8.

```

for n in [10, 50, 300, 1000]:
    clf_report(
        f"random forest, {n} trees",
        clf_pipeline(
            RandomForestClassifier(n_estimators=n, min_samples_leaf=5, random_state=37),
            scale=False,
        ),
    )

```

```

random forest, 10 trees      AUC 0.737    log loss 0.5069
random forest, 50 trees     AUC 0.774    log loss 0.2719
random forest, 300 trees    AUC 0.785    log loss 0.2675
random forest, 1000 trees   AUC 0.790    log loss 0.2659

```

Ten trees are clearly too few (AUC 0.737, with poor probabilities). The score rises with more trees and settles: going from 300 to 1,000 trees adds only 0.005. Adding trees doesn't cause overfitting in a random forest, because each tree is averaged in, not stacked on top; small ups and downs between sizes are noise from which random samples the trees drew. The only cost of more trees is time and memory.

9.

```

pred = np.full(len(y_toy), y_toy.mean())
for r in range(1, 21):
    residual = y_toy - pred
    stump = DecisionTreeRegressor(max_depth=1, random_state=37).fit(x_toy, residual)
    pred = pred + 0.5 * stump.predict(x_toy)
    if r % 5 == 0:
        print(f"round {r:>2}: squared error {np.mean((y_toy - pred) ** 2):.5f}")
print("predictions:", pred.round(3))

```



```
round 5: squared error 0.05239
round 10: squared error 0.02445
round 15: squared error 0.01143
round 20: squared error 0.00535
predictions: [0.032 0.032 0.807 0.022 0.022 0.032 0.022 0.032]
```

The error keeps falling toward zero: with enough rounds, boosting fits the eight training rows almost perfectly, including the one churner. But eight rows can't support a model that precise; it has memorized them. That's why real boosting needs a small learning rate, limits on tree size and leaf size, and a validation set or early stopping to decide when to stop adding trees. Training error going to zero is the symptom of variance from section 37.10.

10.

```
from sklearn.model_selection import validation_curve

depths = [2, 3, 4, 6, 8]
tr_scores, cv_scores = validation_curve(
    clf_pipeline(
        HistGradientBoostingClassifier(
            learning_rate=0.05, min_samples_leaf=40, max_iter=300, random_state=37
        ),
        scale=False,
    ),
    X_rest,
    y_rest,
    param_name="model__max_depth",
    param_range=depths,
    cv=folds,
    scoring="roc_auc",
)
for d, a, b in zip(depths, tr_scores.mean(axis=1), cv_scores.mean(axis=1)):
    print(f"max_depth {d}: train AUC {a:.3f}   CV AUC {b:.3f}")
```

```
max_depth 2: train AUC 0.901   CV AUC 0.836
max_depth 3: train AUC 0.945   CV AUC 0.836
max_depth 4: train AUC 0.977   CV AUC 0.829
max_depth 6: train AUC 0.999   CV AUC 0.820
max_depth 8: train AUC 1.000   CV AUC 0.817
```

The cross-validated score is best with shallow trees and declines as depth grows, while the training score keeps rising: deeper trees fit the training folds better and generalize worse. With about 390 churners in the 4,000 accounts, shallow trees (depth 2 or 3) are the right size, which matches what random search found.

11.

```

from sklearn.ensemble import StackingClassifier

stack = StackingClassifier(
    estimators=[
        ("logistic", clf_pipeline(LogisticRegression(max_iter=2000))),
        ("boosting", search.best_estimator_),
    ],
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5,
)
s = cross_val_score(stack, X_rest, y_rest, cv=folds, scoring="roc_auc")
print(f"stacking mean AUC {s.mean():.3f} sd {s.std():.3f}")
print(f"(tuned boosting alone {search.best_score_:.3f}; logistic alone 0.807)")

```

```

stacking mean AUC 0.840 sd 0.016
(tuned boosting alone 0.841; logistic alone 0.807)

```

The stack scores about the same as tuned boosting alone. Stacking helps when the models make different kinds of mistakes and each is strong; here, boosting already captures what logistic regression knows, so the combination adds little, at double the training time and complexity. That's the usual outcome in business problems, and why stacking is more common in competitions than in production.

12.

```

boost_eng = Pipeline(
    [
        (
            "prepare",
            ColumnTransformer(
                [
                    ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
                    (
                        "num",
                        SimpleImputer(strategy="median", add_indicator=True),
                        NUMS + ENG,
                    ),
                ],
            ),
        ),
        (
            "model",
            HistGradientBoostingClassifier(
                **{k.replace("model__", ""): v for k, v in search.best_params_.items()},
                random_state=37,
            ),
        ),
    ]
)
s = cross_val_score(
    boost_eng, engineered[CATS + NUMS + ENG], y_rest, cv=folds, scoring="roc_auc"
)
print(
    f"tuned boosting + engineered features mean AUC {s.mean():.3f} sd {s.std():.3f}"
)

```

```

tuned boosting + engineered features mean AUC 0.846 sd 0.013

```

Boosting gains only a little from the engineered features (0.846 against 0.841), far less than logistic regression's jump from 0.807 to 0.849. Trees can already represent thresholds ("days > 90") and interactions (late payment within small retail) by splitting; the engineered features mostly save them a few splits. Logistic regression couldn't represent them at all, so for it they were new information. Feature engineering matters most for the models that can't find structure on their own.

13.

```
import statsmodels.api as sm

design = pd.get_dummies(reg_train[CATS], drop_first=True, dtype=float)
design = pd.concat(
    [design, reg_train[REG_NUMS].fillna(reg_train[REG_NUMS].median())], axis=1
)
ols_sm = sm.OLS(reg_train["log_revenue_2025"], sm.add_constant(design)).fit()
low, high = ols_sm.conf_int().loc["rep_id_9"]
print(
    f"inside sales desk (rep_id_9): coefficient {ols_sm.params['rep_id_9']:.4f}    "
    f"95% CI [{low:.4f}, {high:.4f}]    p-value {ols_sm.pvalues['rep_id_9']:.3f}"
)
```

```
inside sales desk (rep_id_9): coefficient 0.0311    95% CI [0.0092, 0.0531]    p-value 0.006
```

The interval, 0.009 to 0.053, **excludes zero** ($p = 0.006$), even though the generator gave the inside sales desk no effect on growth. So where does it come from? The generator did make inside-desk accounts **more likely to churn**, and this regression only includes accounts that stayed. Inside-desk accounts that survived despite that extra risk tend to be healthier on other churn signals, such as the late-payment and complaint thresholds, which the model's straight-line terms don't fully capture. That's **selection bias**: analyzing only the survivors creates a relationship that doesn't exist among all accounts (Chapter 31). The lesson is broader than this dataset. A statistically significant coefficient can still be an artifact of how the rows were chosen, so before anyone reassigns accounts to the inside sales desk, ask how the data was filtered. (statsmodels uses unscaled features, so its coefficients are in original units, unlike the scaled ones in section 37.1; for a one-hot column like this one, the meaning is the same.)

14. First, the scores come from **different evaluations**: 0.841 is the best of 25 tuned settings on cross-validation, which is slightly optimistic because the winner was selected for scoring well, while 0.807 is logistic regression's untuned CV score. Compare like with like, ideally on the test set (0.839 against 0.797). Second, a difference in **AUC points isn't a percentage improvement** in anything a business cares about: 0.034 AUC doesn't mean 4% more churners saved. Translate it into decisions, such as "how many of the 100 highest-risk accounts actually churn", which is what Chapter 39 does.

15. Impurity-based importance adds up how much each feature reduces Gini across all splits. A continuous feature with thousands of distinct values, like `revenue_2024`, offers many possible thresholds, so trees use it for many small splits that fit noise, and it accumulates importance. `complaints_2024` has only a few values (0 to 6), and its effect is concentrated in an interaction (new accounts with two or more complaints) that occurs in few rows, so it's used rarely. Permutation importance on validation data, or SHAP values (Chapter 39), measure how much predictions actually depend on each feature and correct most of this bias.

16. (1) Drift (Chapter 36): customers' behavior changed, for example a new competitor makes even long-standing accounts leave, so patterns learned from 2024 don't hold. **(2) Variance:** last year's evaluation was on too few accounts and was lucky; the model was never as good as it looked. **(3) A new kind of bias:** the business changed (a new product line, a pricing change) in a way the features don't capture, so the model is now too simple for the new situation. A fourth possibility is a **pipeline problem:** a feature changed meaning or started arriving empty. Check that first; it's the most common and the easiest to fix.

Chapter 38. Unsupervised Learning

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: run k-means by hand and in scikit-learn, and choose k with the elbow, the silhouette, stability, and business sense · profile and name clusters so a sales team can act on them · use hierarchical clustering and read a dendrogram · use DBSCAN, and know when density-based clustering suits a problem · reduce dimensions with PCA, t-SNE, and UMAP, and read their pictures without over-reading them · find anomalies with an isolation forest · run market basket analysis with support, confidence, and lift, by hand and with Apriori · judge unsupervised results, which have no test set to fall back on.

Before you start: Chapter 35 (distance, variance, PCA), Chapter 36 (scaling and pipelines), and Chapter 37 (the customer accounts dataset). No target variable is needed anywhere in this chapter.

Time needed: 8–12 hours over one to two weeks.

Tools: Python 3 with scikit-learn and SciPy, plus `umap-learn` and `mlxtend` (both free).

Practice data: Riverstone's **customer accounts** (5,000 rows, from Chapter 37) and a new **order baskets** dataset: 33,931 orders and 92,359 order lines from 2025 across a 24-product catalog, built by `companion/generate_riverstone_baskets.py`. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Everything in Chapters 36 and 37 needed a target: won or lost, churned or stayed. Most business data has no target. Nobody labeled Riverstone's 5,000 accounts as "type A" or "type B", and nobody wrote down which products belong together. **Unsupervised learning** finds structure in data that has no answer key.

It shows up constantly:

- "Can we group our customers so each group gets a different sales approach?" Clustering.
- "Which products should we bundle or recommend together?" Market basket analysis.
- "Which accounts look unusual enough to check by hand?" Anomaly detection.
- "We have 40 columns. Can we see them on one chart?" Dimensionality reduction.

It also has a trap that supervised learning doesn't. With no answer key, **there's no score that proves you're right**. Any dataset can be split into four groups; whether those groups mean anything is a separate question. That question, how to judge an unsupervised result, is the thread running through this chapter.

In plain English

Think of a new sales manager walking into Riverstone's warehouse office with a list of 5,000 customers and no notes.

Nobody can tell them which customers are important, so they start sorting. They put the big frequent buyers in one pile, the small occasional ones in another, and the ones who haven't ordered in months in a third. That's **clustering**: grouping by similarity, with nobody saying what the groups should be.

How many piles? Two is too crude, twenty is unusable. They try a few and pick the number where each pile can be described in a sentence and treated differently. That's **choosing k**.

Some customers fit no pile: one placed a single enormous order and vanished. They set those aside to look at individually. That's **anomaly detection**.

Then they look at the order book and notice that crates and crate lids keep appearing together. That's **market basket analysis**.

Finally, to explain it all to the sales head, they draw one chart with customers as dots, arranged so similar customers sit close together. That's **dimensionality reduction**.

38.0 The accounts, prepared

Clustering measures distance, so scaling is not optional (Chapter 35). Revenue is also skewed, so it goes in as a log. Only descriptive features are used: no `churned_2025`, because that's a target, and unsupervised methods don't get one. Later the churn column becomes a way to **check** the result from the outside.

```

import warnings

import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", category=UserWarning)

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["log_revenue"] = np.log(accounts["revenue_2024"])
accounts["late_payment_days"] = accounts["late_payment_days"].fillna(
    accounts["late_payment_days"].median()
)

FEATURES = [
    "log_revenue",
    "orders_2024",
    "categories_bought",
    "avg_discount_pct",
    "late_payment_days",
    "days_since_last_order",
    "tenure_months",
]
X = StandardScaler().fit_transform(accounts[FEATURES])
print(X.shape, "accounts x features")
print(accounts[FEATURES].describe().loc[["mean", "std"]].round(2).to_string())
print(
    "after scaling, every column has mean 0 and standard deviation 1:",
    np.allclose(X.mean(axis=0), 0),
    np.allclose(X.std(axis=0), 1),
)

```

```

(5000, 7) accounts x features
   log_revenue  orders_2024  categories_bought  avg_discount_pct  late_payment_days
days_since_last_order  tenure_months
mean           12.34         16.34              1.93             5.47             14.53
59.86           30.71
std            1.09         17.32              1.08             2.70             10.48
79.28           20.54
after scaling, every column has mean 0 and standard deviation 1: True True

```

How it works: `StandardScaler` subtracts each column's mean and divides by its standard deviation. After scaling, one standard deviation of revenue counts exactly as much as one standard deviation of order count, which is what "similar accounts" should mean. Note the raw standard deviations: orders vary by about 17 and log revenue by about 1. Without scaling, order count and recency would drown out everything else.

38.1 k-means

How it works

k-means splits rows into k groups, each represented by a **centroid**, the average of its members. It repeats two steps until nothing changes:

1. **Assign:** put every row in the cluster whose centroid is nearest (Euclidean distance).

2. **Update:** move each centroid to the average of the rows now assigned to it.

It minimizes **inertia**: the total squared distance from each row to its own centroid. Inertia always falls as k rises, which matters for choosing k .

By hand, on six accounts

Take six accounts described by two scaled features, and start with two centroids at the first and last account:

Account	Feature 1	Feature 2
A	1.0	2.0
B	1.5	1.8
C	5.0	8.0
D	8.0	8.0
E	1.0	0.6
F	9.0	11.0

Step 1, assign. Account C is $\sqrt{((5-1)^2 + (8-2)^2)} = \sqrt{52} = 7.21$ from centroid 1 and $\sqrt{((5-9)^2 + (8-11)^2)} = \sqrt{25} = 5.00$ from centroid 2, so it joins cluster 2. Doing the same for the rest gives A, B, E in cluster 1 and C, D, F in cluster 2.

Step 1, update. Cluster 1's new centroid is the average of A, B, E: $((1.0 + 1.5 + 1.0) \div 3, (2.0 + 1.8 + 0.6) \div 3) = (1.17, 1.47)$. Cluster 2's is $((5 + 8 + 9) \div 3, (8 + 8 + 11) \div 3) = (7.33, 9.00)$.

Step 2. Reassign with the new centroids: nothing changes, so the centroids stay put and the algorithm stops.

```
small = np.array(
    [[1.0, 2.0], [1.5, 1.8], [5.0, 8.0], [8.0, 8.0], [1.0, 0.6], [9.0, 11.0]]
)
centres = np.array([[1.0, 2.0], [9.0, 11.0]]) # start: first and last account

for step in range(1, 4):
    distances = np.linalg.norm(small[:, None, :] - centres[None, :, :], axis=2)
    labels = distances.argmin(axis=1)
    inertia = (distances.min(axis=1) ** 2).sum()
    print(f"step {step}: labels {labels} inertia {inertia:.2f}")
    print(f"centres {centres.round(2).tolist()}")
    new_centres = np.array([small[labels == c].mean(axis=0) for c in range(2)])
    if np.allclose(new_centres, centres):
        print("centres stopped moving: done")
        break
    centres = new_centres
```

```
step 1: labels [0 0 1 1 0 1] inertia 37.25
centres [[1.0, 2.0], [9.0, 11.0]]
step 2: labels [0 0 1 1 0 1] inertia 15.98
centres [[1.17, 1.47], [7.33, 9.0]]
centres stopped moving: done
```

Reading it. Inertia fell from 37.25 to 15.98 in one move, and the second pass changed nothing. Check the final inertia by hand for cluster 1: A is $(1.0 - 1.17, 2.0 - 1.47)$ from its centroid, a squared distance of $0.0289 + 0.2809 = 0.3098$; B gives $0.1089 + 0.1089 = 0.2178$; E gives $0.0289 + 0.7569 = 0.7858$. Cluster 1 contributes 1.313, and cluster 2 the remaining 14.67. ✓

```
from sklearn.cluster import KMeans

check = KMeans(n_clusters=2, n_init=10, random_state=38).fit(small)
print(
    "scikit-learn centres:", np.sort(check.cluster_centers_, axis=0).round(2).tolist()
)
print("labels:", check.labels_, " inertia:", round(check.inertia_, 2))
```

```
scikit-learn centres: [[1.17, 1.47], [7.33, 9.0]]
labels: [1 1 0 0 1 0] inertia: 15.98
```

scikit-learn finds the same centroids. Its cluster **numbers** are different (0 and 1 are swapped), which is normal: cluster labels are arbitrary names, not values. Never compare labels between two runs directly; compare the groupings (section 38.2 shows how).

Key settings: `n_clusters` (k), and `n_init`, the number of random starts. k-means can land in a poor solution depending on where the centroids begin, so scikit-learn runs it several times and keeps the best. `random_state` fixes the starts so results are repeatable.

Watch out: k-means assumes round, similar-sized, similar-density clusters. It draws straight boundaries halfway between centroids. Long thin groups, rings, and groups of very different sizes defeat it; that's where DBSCAN (section 38.4) and hierarchical clustering (section 38.3) help.

38.2 Choosing k , and whether the clusters are real

Three ways to choose, none decisive

The elbow. Plot inertia against k . It always falls; look for the bend where extra clusters stop buying much. It's often ambiguous, and this dataset is a fair example.

The silhouette. For each row: a is its average distance to its own cluster's members, b is its average distance to the nearest other cluster's members, and its silhouette is $(b - a) \div \max(a, b)$. It runs from -1 (in the wrong cluster) through 0 (on the boundary) to 1 (snugly inside). The average over all rows scores the whole clustering.

```
from sklearn.metrics import silhouette_score

print(" k  inertia  silhouette")
for k in range(2, 9):
    model = KMeans(n_clusters=k, n_init=10, random_state=38).fit(X)
    score = silhouette_score(X, model.labels_, sample_size=2000, random_state=38)
    print(f"{k:>2} {model.inertia_:>7,.0f} {score:>10.3f}")
```

k	inertia	silhouette
2	25,503	0.277
3	22,142	0.232
4	19,744	0.203
5	17,825	0.204
6	16,086	0.210
7	14,887	0.195
8	14,175	0.176

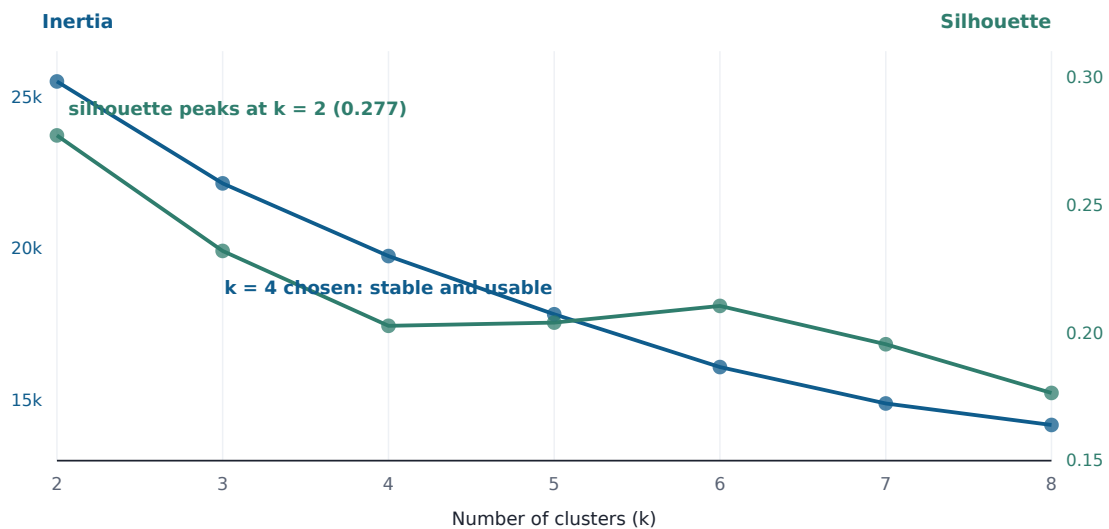


Figure 38.1 – Choosing k on Riverstone's accounts. Neither curve gives a clean answer: the elbow is a gentle curve, and the silhouette prefers the least useful answer, $k = 2$.

Reading it, plainly. There's no elbow worth the name: inertia slides down smoothly. The silhouette is highest at $k = 2$ (0.277) and never rises again. Taken literally, the maths says "two clusters", and even that score is modest; a silhouette below about 0.3 means the groups overlap heavily.

That's the truth about most business data. Customers don't fall into neat, separated balls; they spread out along a continuum of size and activity, and clustering **cuts** that continuum rather than discovering gaps in it. The cuts can still be useful. So the honest way to report this work is: "these groups are a convenient division of a continuous customer base, not a discovery of natural types."

Stability: would slightly different data give the same groups?

A more practical test than the silhouette: run the clustering again with different random starts, and on random subsets of the rows. If the groups survive, they're a stable description; if they scatter, they were an accident. **Adjusted Rand index (ARI)** compares two groupings of the same rows regardless of label names: 1 means identical, 0 means no more agreement than chance.

```

from sklearn.metrics import adjusted_rand_score

base = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(X)
for seed in [1, 2, 3]:
    other = KMeans(n_clusters=4, n_init=10, random_state=seed).fit_predict(X)
    print(
        f"seed {seed}: agreement with the first run"
        f" (adjusted Rand) {adjusted_rand_score(base, other):.3f}"
    )

rng = np.random.default_rng(38)
for trial in range(3):
    rows = rng.choice(len(X), size=int(0.8 * len(X)), replace=False)
    subset = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(X[rows])
    print(
        f"80% sample {trial + 1}: agreement with the same rows in the full run "
        f"{adjusted_rand_score(base[rows], subset):.3f}"
    )

```

```

seed 1: agreement with the first run (adjusted Rand) 0.996
seed 2: agreement with the first run (adjusted Rand) 0.997
seed 3: agreement with the first run (adjusted Rand) 0.994
80% sample 1: agreement with the same rows in the full run 0.991
80% sample 2: agreement with the same rows in the full run 0.984
80% sample 3: agreement with the same rows in the full run 0.969

```

Reading it. Agreement across random starts is 0.994–0.997, and across 80% samples 0.969–0.991. At $k = 4$, the division is highly reproducible even though the silhouette is unimpressive. Those two facts sit together comfortably: the boundaries land in the same places every time, they are drawn through a crowd rather than through empty space.

The business test

The last question is the one that decides: **can someone act differently for each group?** Four groups that map to four sales approaches beat two groups that score better and mean nothing. Riverstone's sales team can handle four; that, plus the stability above, is why the rest of this chapter uses $k = 4$.

38.3 Profiling and naming clusters

A cluster number means nothing until it's described. Profile every cluster on the features used, on features left out, and on size and value:

```

accounts["cluster"] = base
profile = accounts.groupby("cluster").agg(
    accounts=("account_id", "size"),
    revenue_2024=("revenue_2024", "median"),
    orders=("orders_2024", "median"),
    categories=("categories_bought", "mean"),
    discount_pct=("avg_discount_pct", "mean"),
    late_days=("late_payment_days", "mean"),
    days_since_order=("days_since_last_order", "median"),
    tenure=("tenure_months", "median"),
    churn_rate=("churned_2025", "mean"),
)
print(profile.round(2).to_string())
print()
print(
    (pd.crosstab(accounts["cluster"], accounts["segment"], normalize="index") * 100)
    .round(0)
    .to_string()
)
print()
print("share of 2024 revenue by cluster:")
print(
    (
        accounts.groupby("cluster")["revenue_2024"].sum()
        / accounts["revenue_2024"].sum()
        * 100
    )
    .round(1)
    .to_string()
)

```

	accounts	revenue_2024	orders	categories	discount_pct	late_days	days_since_order
tenure	churn_rate						
cluster							
0	1088	282400.0	13.0	3.21	5.63	11.40	23.0
26.0	0.03						
1	944	1083800.0	48.0	2.39	8.56	11.99	8.0
27.0	0.03						
2	2398	145600.0	7.0	1.28	4.50	16.34	35.0
26.0	0.08						
3	570	73450.0	2.0	1.48	4.16	17.10	228.5
24.0	0.41						

segment	Hospitality	Retail	Wholesale
cluster			
0	35.0	52.0	12.0
1	14.0	16.0	69.0
2	38.0	53.0	9.0
3	37.0	62.0	2.0

share of 2024 revenue by cluster:

cluster	
0	15.8
1	62.0
2	19.9
3	2.3

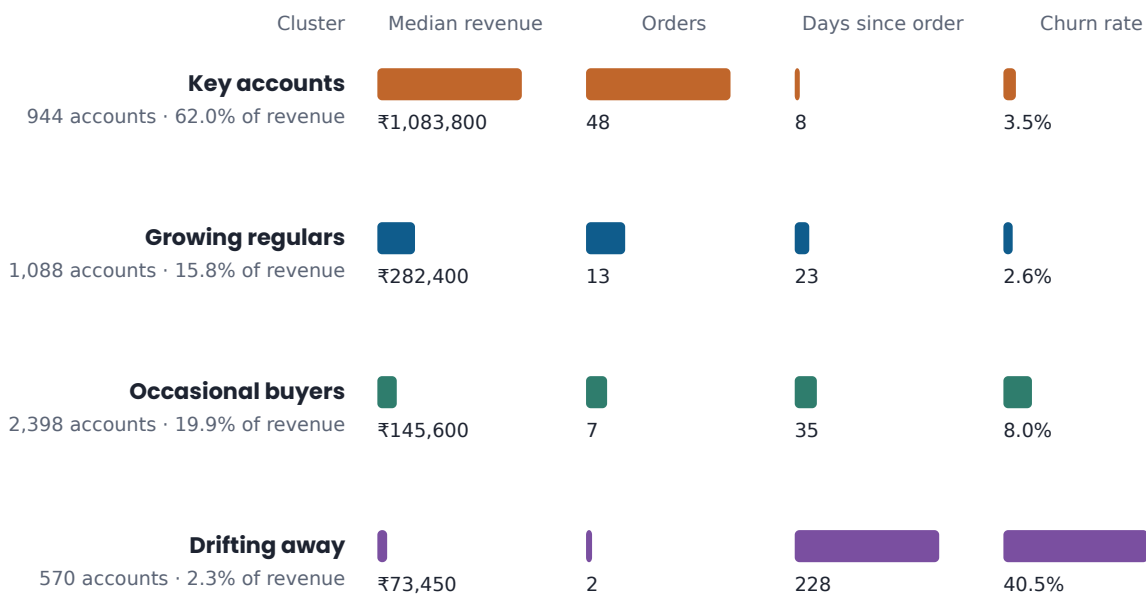


Figure 38.2 – The four clusters, described by the numbers that separate them. Cluster sizes and revenue shares are very different: the smallest cluster earns the most.

Naming them is the step that turns a model into something a sales team uses:

Cluster	Name	Size	Median revenue	What defines it	What to do
1	Key accounts	944	₹1,083,800	48 orders a year, deepest discounts, mostly wholesale (69%), ordered days ago	Named account managers; protect the relationship; review discounts
0	Growing regulars	1,088	₹282,400	13 orders, buys 3.2 of 4 categories, healthy recency	Cross-sell the fourth category; the most obvious upgrade path
2	Occasional buyers	2,398	₹145,600	7 orders, 1.3 categories, the largest group	Low-touch: catalog emails, self-service reordering
3	Drifting away	570	₹73,450	2 orders, last order 228 days ago, 41% churned in 2025	Win-back calls, or accept and stop spending on them

Three things worth noticing.

- **Value is concentrated.** The 944 key accounts are 19% of the customer base and **62%** of 2024 revenue. The 570 drifting accounts are 2.3%. If effort followed head count instead of value, it would be badly misplaced.
- **Segment labels and behavior differ.** Cluster 1 is 69% wholesale, but every cluster contains all three segments. "Wholesale" describes what a customer sells; the cluster describes how they buy.
- **Churn wasn't used** to build the clusters, yet the churn rate runs from 2.6% to 41%. That's an external check: the groups line up with something they were never shown. Section 38.8 measures it.

Watch out: clusters are a snapshot, not a category. Accounts move between clusters as they order more or go quiet. Re-run the clustering on a schedule, and expect the sales team to ask why a customer "changed type". Chapter 39's stability ideas and Chapter 52's monitoring apply here too.

38.4 Hierarchical clustering

How it works

Hierarchical clustering doesn't ask for k up front. **Agglomerative** clustering starts with every row as its own cluster and repeatedly merges the two closest, until everything is one cluster. The record of merges is a **dendrogram**, a tree you can cut at any height to get any number of clusters.

"Closest" needs a rule, the **linkage**:

- **Ward** (the usual default) merges the pair that increases total within-cluster variance least. It tends to give compact, similar-sized clusters.
- **Complete** linkage uses the distance between the two furthest members; **average** uses the mean distance; **single** uses the closest pair, which can chain long straggly clusters together.

The cost is memory and time: it compares every pair, so it grows with the square of the number of rows. Beyond a few thousand rows, cluster a sample.

```
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage

sample_rows = np.random.default_rng(38).choice(len(X), size=800, replace=False)
tree = linkage(X[sample_rows], method="ward")
for k in [2, 3, 4, 5]:
    labels = fcluster(tree, t=k, criterion="maxclust")
    print(
        f"cut into {k}: sizes {np.bincount(labels)[1:]}    "
        f"agreement with k-means {adjusted_rand_score(base[sample_rows], labels):.3f}"
    )
```

```
cut into 2: sizes [147 653]    agreement with k-means 0.256
cut into 3: sizes [147 287 366]    agreement with k-means 0.357
cut into 4: sizes [147 287 50 316]    agreement with k-means 0.413
cut into 5: sizes [147 119 168 50 316]    agreement with k-means 0.470
```

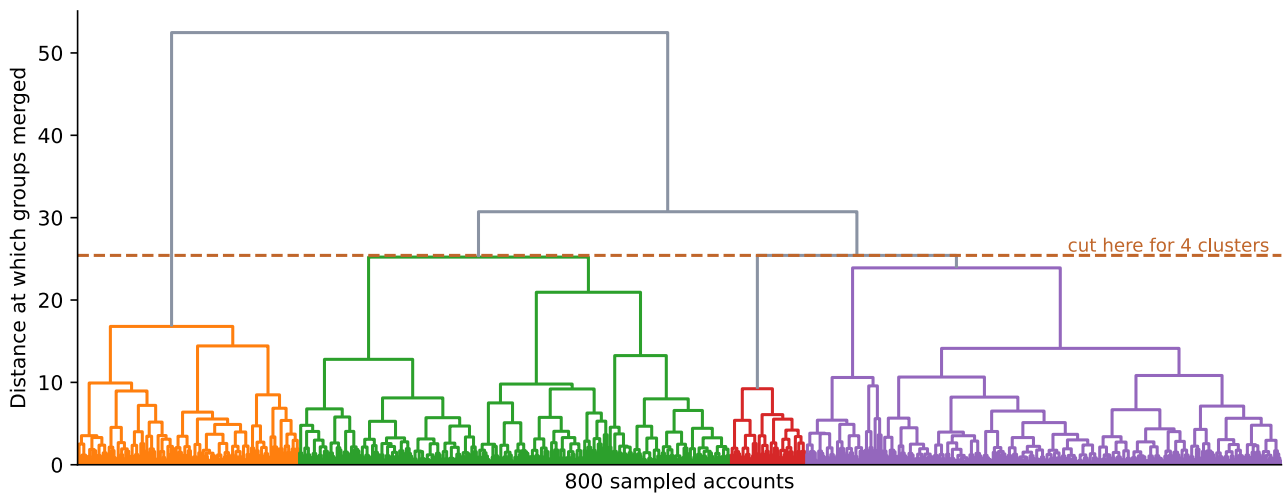



Figure 38.3 – A dendrogram of 800 accounts (Ward linkage). Each merge is drawn at the height that measures how different the merged groups were; cutting across the tree at a chosen height gives that many clusters.

Reading it. Cut into four, the tree gives clusters of 147, 287, 50, and 316 accounts, and it agrees with the k-means grouping to an ARI of 0.413: related, but far from the same. The two methods cut a continuous cloud in different places, another reminder that these groups are conveniences rather than discoveries. Hierarchical clustering earns its place when you want the tree itself: a picture of which groups sit inside which, which product ranges or store types nest together.

38.5 DBSCAN

How it works

DBSCAN (density-based spatial clustering of applications with noise) doesn't take k . It grows clusters from **dense** regions:

- A row is a **core point** if at least `min_samples` rows lie within a radius `eps` of it.
- Core points that are within `eps` of each other join the same cluster, and non-core rows within reach are attached at the edge.
- Everything else is labeled **noise** (`-1`), and isn't in any cluster.

Two properties follow. DBSCAN finds clusters of **any shape**, including long curved ones that defeat k-means, and it **refuses to classify** rows in sparse regions, which is useful when you want only the well-defined groups.

```
from sklearn.cluster import DBSCAN

print(" eps  clusters  noise points")
for eps in [0.5, 0.8, 1.0, 1.2, 1.6]:
    labels = DBSCAN(eps=eps, min_samples=10).fit_predict(X)
    n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
    print(f"{eps:>4}  {n_clusters:>8}  {(labels == -1).sum():>12}")
```

eps	clusters	noise points
0.5	2	4,678
0.8	7	1,971
1.0	1	854
1.2	1	313
1.6	1	47

Reading it. Every value of `eps` gives an unsatisfying answer. Small radii (0.5) leave 4,678 of 5,000 accounts as noise; large radii (1.2 and up) swallow almost everything into one cluster. There's no setting in between that gives several balanced clusters, and the reason is section 38.2's: this cloud has no genuine gaps for DBSCAN to find. **On this data, DBSCAN's honest answer is that there are no density-separated groups.**

That's worth taking seriously rather than tuning away. DBSCAN shines on spatial data (delivery points, sensor readings, GPS traces), where dense regions and empty space really exist, and it doubles as an anomaly detector: the noise points are the odd ones out.

Key settings: `eps` (the radius; sweep it as above) and `min_samples` (a common starting point is twice the number of features).

38.6 Seeing many dimensions: PCA, t-SNE, UMAP

Seven features can't be drawn. Three methods squeeze them into two dimensions for a picture:

- **PCA** (Chapter 35) finds the straight-line directions of greatest variance. It's fast, deterministic, reversible, and its axes have meaning through their loadings. It can't unfold curved structure.
- **t-SNE** places rows so that each row's **near neighbours** stay near, allowing the map to bend. It's excellent at showing local groupings, slow on large data, and its global layout (distances between blobs, blob sizes) is not meaningful.
- **UMAP** does something similar, usually faster, and preserves a little more of the global structure. Both have random starts and settings (`perplexity` , `n_neighbors`) that change the picture.

```
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
import umap

pca = PCA().fit(X)
print("PCA share of variance:", pca.explained_variance_ratio_[:4].round(3))
print(
    "first two components together:", f"{pca.explained_variance_ratio_[:2].sum():.1%}"
)
coords_pca = pca.transform(X)[: , :2]

plot_rows = np.random.default_rng(38).choice(len(X), size=1500, replace=False)
coords_tsne = TSNE(n_components=2, perplexity=30, random_state=38).fit_transform(
    X[plot_rows]
)
coords_umap = umap.UMAP(n_components=2, random_state=38).fit_transform(X[plot_rows])
print(f"t-SNE on {len(plot_rows):,} accounts, output shape {coords_tsne.shape}")
print("UMAP output shape:", coords_umap.shape)
```

```
PCA share of variance: [0.397 0.143 0.136 0.121]
first two components together: 54.1%
t-SNE on 1,500 accounts, output shape (1500, 2)
UMAP output shape: (1500, 2)
```

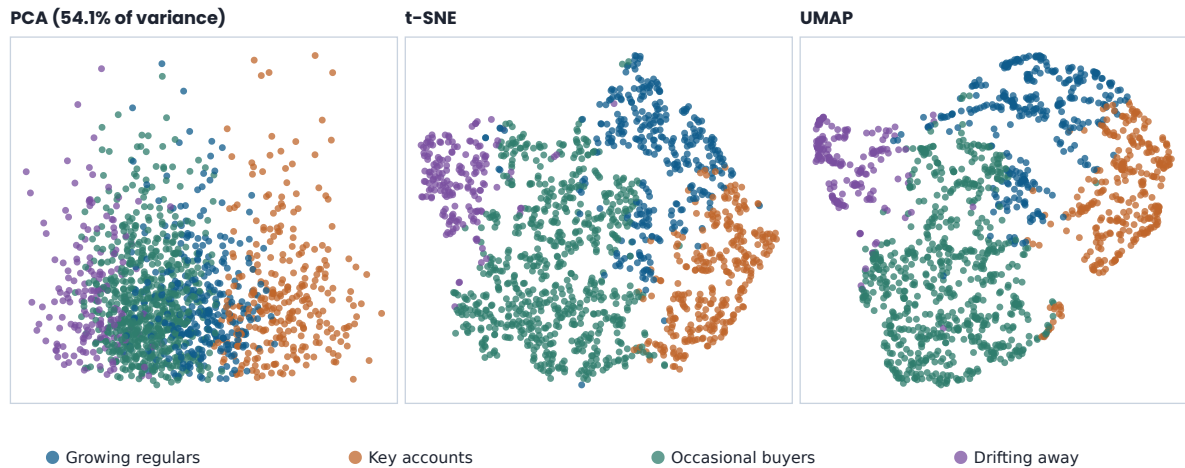


Figure 38.4 – The same accounts under PCA, t-SNE, and UMAP, colored by their k-means cluster. The clusters hold together in all three, and in all three they're touching, not separated: the same continuum the silhouette reported.

Reading it. PCA's first two components hold 54.1% of the variance, so the flat picture is missing nearly half the information. t-SNE and UMAP produce prettier, more separated pictures of the same 1,500 accounts, and that separation is partly a property of the methods, not of Riverstone's customers.

Watch out: don't over-read a t-SNE or UMAP picture. The distance between two blobs, the size of a blob, and the empty space between them carry little meaning; those methods are free to stretch and shrink regions. Never feed t-SNE coordinates into a predictive model as features, and never present one as evidence that groups are "clearly separated". Use them to explore and to illustrate, then verify with numbers.

38.7 Anomaly detection

Isolation forests

An **isolation forest** finds rows that are quick to isolate. It builds many random trees: at each step it picks a random feature and a random split point. Ordinary rows sit in crowded regions and need many splits to be separated; unusual rows get cut off after a few. The average number of splits needed becomes the **anomaly score**.

It's fast, it handles many features, and it needs no labels. The `contamination` setting says what share of rows to flag.

```

from sklearn.ensemble import IsolationForest

forest = IsolationForest(contamination=0.02, random_state=38).fit(X)
accounts["outlier_score"] = forest.decision_function(X) # lower = stranger
flagged = accounts.nsmallest(100, "outlier_score")
print(
    f"100 strangest accounts: churn rate {flagged['churned_2025'].mean():.1%} "
    f"vs {accounts['churned_2025'].mean():.1%} overall"
)
columns = [
    "segment",
    "revenue_2024",
    "orders_2024",
    "avg_discount_pct",
    "late_payment_days",
    "days_since_last_order",
    "churned_2025",
]
print(accounts.nsmallest(5, "outlier_score")[columns].to_string(index=False))

```

```

100 strangest accounts: churn rate 21.0% vs 9.7% overall
  segment  revenue_2024  orders_2024  avg_discount_pct  late_payment_days  days_since_last_order
churned_2025
Wholesale    7966300.0         77         11.7         40.0             7.0
0
Wholesale    1551400.0         82          9.0         45.0             3.0
0
  Retail      94600.0          2          2.6         63.0          365.0
1
Wholesale    4152200.0         62          6.9         32.0            10.0
0
Wholesale    3042000.0         68          7.0         55.0             3.0
1

```

Reading it. Among the 100 strangest accounts, 21% churned in 2025, against 9.7% overall. The flagged accounts fall into two kinds: enormous wholesale accounts that are unusual because nothing else is that large, and tiny accounts that ordered once long ago. Both are worth a human look, for opposite reasons.

That points at how anomaly detection is used in practice: **as a review queue, not a decision**. Nobody should be dropped because a model called them unusual. Typical uses are fraud triage (Chapter 39's cost thinking applies), data quality checks (an account with a 60% discount is probably a data-entry error), and sensor monitoring (Chapter 40 does this over time).

Other tools for the same job: `LocalOutlierFactor` (compares a row's local density with its neighbours'), `OneClassSVM` (learns a boundary around normal data), and DBSCAN's noise points. For anomalies over time, forecasting residuals work better (Chapter 40).

38.8 Judging unsupervised results

With no target, there are four kinds of evidence, in rising order of usefulness:

1. **Internal measures** (silhouette, Davies–Bouldin, inertia) score how compact and separated the clusters are, using only the features. They're comparable between runs on the same data, and they never prove that groups are real.
2. **Stability** (section 38.2) asks whether the same structure appears on different samples and seeds. Low stability is fatal; high stability is reassuring.
3. **External checks** compare the groups with something they were never given.
4. **Business usefulness**: can each group be described in a sentence and treated differently, and does acting on it change a number the company cares about? That's the one that matters, and it usually needs an experiment (Chapter 30).

The external check here: the clusters were built without any churn information. How much do they know about it?

```
from sklearn.metrics import roc_auc_score

rates = accounts.groupby("cluster")["churned_2025"].mean()
scored = accounts["cluster"].map(rates)
print("churn rate by cluster:", rates.round(3).to_dict())
auc = roc_auc_score(accounts["churned_2025"], scored)
print(f"using the cluster alone to rank churn risk: AUC {auc:.3f}")
print("(Chapter 37's supervised models scored 0.80 to 0.85 on this target.)")
```

```
churn rate by cluster: {0: 0.026, 1: 0.035, 2: 0.08, 3: 0.405}
using the cluster alone to rank churn risk: AUC 0.759
(Chapter 37's supervised models scored 0.80 to 0.85 on this target.)
```

Reading it. Ranking accounts by their cluster's churn rate gives an AUC of 0.759, against 0.80–0.85 for the supervised models in Chapter 37 that were trained on churn directly. Four unlabelled groups recover most of the ranking power of a model built for the job. That's a genuine external check, and also a caution: **when you have a target, use it.** Clustering isn't a substitute for a supervised model; it's what you reach for when there's no target at all, or when you need groups people can hold in their heads.

38.9 Market basket analysis

The question and the vocabulary

"Which products sell together?" The data is Riverstone's 2025 order lines: one row per product per order. A **basket** is one order, and the analysis counts how often sets of products appear in the same basket.

```

lines = pd.read_csv("../baskets/order_lines.csv")
products = pd.read_csv("../baskets/products.csv")
lines = lines.merge(
    products[["product_id", "product_name", "category"]], on="product_id"
)
lines = lines.merge(accounts[["account_id", "segment"]], on="account_id")
print(
    f"{lines['order_id'].nunique():,} orders, {len(lines):,} order lines, "
    f"{lines['product_name'].nunique():,} products, {lines['account_id'].nunique():,} accounts"
)
basket_sizes = lines.groupby("order_id").size()
print(
    "products per order: median",
    int(basket_sizes.median()),
    " mean",
    round(basket_sizes.mean(), 2),
    " max",
    int(basket_sizes.max()),
)
print(lines["product_name"].value_counts().head(5).to_string())

```

```

33,931 orders, 92,359 order lines, 24 products, 4,516 accounts
products per order: median 2 mean 2.72 max 10
product_name
Airtight Seal Pack      7286
Crate Lid (50L)         6500
Crate Lid (80L)         6313
Stacking Bin Large      5261
Drum Tap Fitting        5260

```

Three measures describe a rule such as *Storage Crate 50L* → *Crate Lid (50L)*:

support = share of all baskets containing the items (how common the pattern is)

confidence = share of baskets containing the "if" item that also contain the "then" item

lift = confidence ÷ the "then" item's own support (how much more likely than chance)

Lift is the key one. Lift of 1 means the two are independent; 3 means they appear together three times as often as chance would predict; below 1 means they repel each other.

Worked by hand

Of 33,931 orders, 4,170 contain a Storage Crate 50L, 6,500 contain a Crate Lid (50L), and 2,785 contain both:

$\text{support}(\text{crate}) = 4,170 \div 33,931 = 0.1229$

$\text{support}(\text{lid}) = 6,500 \div 33,931 = 0.1916$

$\text{support}(\text{both}) = 2,785 \div 33,931 = 0.0821$

$\text{confidence}(\text{crate} \rightarrow \text{lid}) = 0.0821 \div 0.1229 = \mathbf{0.668}$

$\text{lift} = 0.668 \div 0.1916 = \mathbf{3.49}$

Two thirds of crate orders include the matching lid, and that's 3.5 times the rate for orders in general.

```
baskets = lines.groupby("order_id")["product_name"].apply(set)
n_orders = len(baskets)
crate = baskets.apply(lambda s: "Storage Crate 50L" in s)
lid = baskets.apply(lambda s: "Crate Lid (50L)" in s)

support_a = crate.mean()
support_b = lid.mean()
support_ab = (crate & lid).mean()
confidence = support_ab / support_a
lift = confidence / support_b
print(f"orders: {n_orders:,}")
print(f"support(Storage Crate 50L)           {support_a:.4f}  ({crate.sum():,} orders)")
print(f"support(Crate Lid 50L)               {support_b:.4f}  ({lid.sum():,} orders)")
print(
    f"support(both)                          {support_ab:.4f}  ({(crate & lid).sum():,} orders)"
)
print(f"confidence(Crate 50L -> Lid 50L)      {confidence:.4f}")
print(f"lift                                    {lift:.2f}")
```

```
orders: 33,931
support(Storage Crate 50L)      0.1229  (4,170 orders)
support(Crate Lid 50L)         0.1916  (6,500 orders)
support(both)                  0.0821  (2,785 orders)
confidence(Crate 50L -> Lid 50L) 0.6679
lift                           3.49
```

Apriori: finding the rules automatically

Checking every possible rule is expensive: 24 products give 276 pairs and thousands of larger sets. The **Apriori** algorithm prunes the search with one observation: *if a set of items is rare, every larger set containing it is at least as rare*. So it finds items above a minimum support, then pairs, then triples, never looking at extensions of sets that already failed.

```
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder

encoder = TransactionEncoder()
matrix = pd.DataFrame(encoder.fit_transform(baskets.tolist()), columns=encoder.columns_)
frequent = apriori(matrix, min_support=0.01, use_colnames=True)
rules = association_rules(frequent, metric="lift", min_threshold=1.2)
rules["if"] = rules["antecedents"].apply(lambda s: ", ".join(sorted(s)))
rules["then"] = rules["consequents"].apply(lambda s: ", ".join(sorted(s)))
print(f"{len(frequent)} frequent item sets, {len(rules)} rules with lift above 1.2")
top = rules.sort_values("lift", ascending=False).head(8)
print(
    top[["if", "then", "support", "confidence", "lift"]].round(3).to_string(index=False)
)
```


confidence	lift	if	then	support
0.555	8.899	Garden Table	Garden Chair	0.032
0.513	8.899	Garden Chair	Garden Table	0.032
0.347	5.559	Chair Cushion Set	Garden Chair	0.022
0.356	5.559	Garden Chair	Chair Cushion Set	0.022
0.482	4.790	Drum Tap Fitting, Industrial Crate 200L	Drum 60L	0.012
0.116	4.790	Drum 60L	Drum Tap Fitting, Industrial Crate 200L	0.012
0.075	4.667	Drum Tap Fitting	Drum 60L, Industrial Crate 200L	0.012
0.723	4.667	Drum 60L, Industrial Crate 200L	Drum Tap Fitting	0.012

How it works: `TransactionEncoder` turns baskets into a table of true/false columns, one per product. `apriori` finds every item set above `min_support` (1% of baskets here, about 340 orders). `association_rules` turns those sets into if-then rules and scores them, keeping rules with lift above 1.2.

Reading it. The strongest rules are the planted ones and they're the ones a warehouse manager would recognize: garden chairs with garden tables (lift 8.9), chairs with cushions (5.6), drums with tap fittings, crates with lids. Rules go in both directions with the same lift but different confidence: 55.5% of table buyers add chairs, while 51.3% of chair buyers add a table, because chairs are more common overall.

The confidence trap

Confidence alone is misleading, because a very common product has high confidence with everything:

```
common = "Crate Lid (50L)"
has_common = baskets.apply(lambda s: common in s)
for item in ["Chopping Board", "Serving Tray", "Folding Stool"]:
    has_item = baskets.apply(lambda s, i=item: i in s)
    both = (has_item & has_common).mean()
    conf = both / has_item.mean()
    print(
        f"{item:<16} -> {common}: support {both:.4f}    confidence {conf:.3f}    "
        f"lift {conf / has_common.mean():.2f}"
    )
```

Chopping Board	-> Crate Lid (50L): support 0.0160	confidence 0.142	lift 0.74
Serving Tray	-> Crate Lid (50L): support 0.0176	confidence 0.152	lift 0.79
Folding Stool	-> Crate Lid (50L): support 0.0064	confidence 0.142	lift 0.74

Reading it. A Crate Lid (50L) appears in 14% of baskets that contain a chopping board, which sounds like a pattern. Its lift is **0.74**: a chopping-board basket is *less* likely than average to contain a crate lid. The lid is common enough (19% of all baskets) that 14% is below chance. Always read confidence next to lift and support.

Watch out: three more basket traps. (1) **Trivial rules:** "buys crate lid → buys crate" may be a packaging rule, not an insight. (2) **Support too low:** a rule holding in 12 of 33,931 baskets can have enormous lift and no business value; that's why `min_support` exists. (3) **Correlation isn't causation:** a rule says the two appear together, not that pushing one sells the other. Testing a bundle offer with an experiment (Chapter 30) is what proves it.

Turning rules into money

A rule earns its keep when it points at a specific action:

```
drum_orders = set(lines.loc[lines["product_name"] == "Drum 60L", "order_id"])
tap_orders = set(lines.loc[lines["product_name"] == "Drum Tap Fitting", "order_id"])
missing = drum_orders - tap_orders
missing_accounts = lines.loc[lines["order_id"].isin(missing), "account_id"].nunique()
print(f"orders with Drum 60L: {len(drum_orders):,}")
share = len(missing) / len(drum_orders)
print(f"of those, without a Drum Tap Fitting: {len(missing):,} ({share:.1%})")
print(f"accounts placing at least one such order: {missing_accounts:,}")
segment_mix = (
    lines[lines["order_id"].isin(missing)].groupby("segment")["order_id"].nunique()
)
print("those orders by segment:", segment_mix.to_dict())
```

```
orders with Drum 60L: 3,412
of those, without a Drum Tap Fitting: 1,001 (29.3%)
accounts placing at least one such order: 613
those orders by segment: {'Hospitality': 35, 'Retail': 69, 'Wholesale': 897}
```

What to tell the sales head. "Two thirds of orders containing a 50-litre crate already include the matching lid, and drums almost always come with a tap fitting. But 1,001 orders across 613 accounts, nearly all wholesale, included a 60-litre drum with no tap fitting. That's a checkout prompt, a pre-filled line on the order form, or a call from the account manager, and we can measure whether it works by trying it with half the accounts first."

Chapter 42 builds the next step: a recommender that scores products for each customer rather than working from global rules.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Clustering unscaled features	Clusters differ only by the biggest-unit column	Standardize inside a pipeline (Chapter 36)
Leaving skewed money columns raw	A handful of huge accounts dominate every distance	Log-transform first
Including the target in the features	"Clusters" that are the target in disguise	Cluster on descriptive features only; use the target as an external check

Mistake	Symptom	Fix
Choosing k by silhouette alone	$k = 2$ every time, and useless groups	Weigh elbow, silhouette, stability, and whether the business can act on the groups
Reporting clusters as discovered types	"Our customers fall into four natural groups"	Say "a convenient division of a continuum" when the silhouette is low
Comparing cluster numbers between runs	"Cluster 2 moved to cluster 0"	Compare groupings with the adjusted Rand index
No stability check	Groups change every time the data updates	Re-run with different seeds and on subsamples
Reading distances in a t-SNE or UMAP picture	"These two groups are far apart, so they're very different"	Treat those layouts as sketches; verify with numbers
Using t-SNE coordinates as model features	Unreproducible, unexplainable models	Use PCA if you need components as features
Hierarchical clustering on huge data	Memory errors or a very long wait	Cluster a sample, or use k-means
Tuning DBSCAN until it gives k clusters	Either everything is noise or everything is one cluster	Accept that there may be no density-separated groups
Treating anomalies as decisions	Customers cut off because a model called them odd	Use anomaly scores as a review queue
Judging rules by confidence	"Everything leads to our best-selling product"	Read lift and support alongside confidence
Acting on very rare rules	A bundle nobody buys	Set a sensible <code>min_support</code> for the business
Assuming a rule is causal	A bundle launch that changes nothing	Test it (Chapter 30)

In the real world: the four segments that already existed

In May 2026, Anita Rao asks Meera for "proper customer segments" before the annual sales planning meeting. The marketing agency Riverstone used two years ago delivered nine segments with names like *Urban Aspirants*; nobody ever used them.

Meera builds the clustering in this chapter and gets four groups. Before presenting anything, she does three checks.

Does it survive re-running? Different seeds and 80% samples give adjusted Rand scores above 0.96. The groups are stable.

Does it know anything it wasn't told? She never showed it churn. Cluster 3's churn rate is 41%, against 2.6% for cluster 1. It does.

Does it match how the team already thinks? This is the check that changes the meeting. She takes the four profiles to the three sales executives, without the cluster numbers, and asks how they'd describe their customers. Neha describes "my regulars", "the big wholesale accounts I babysit", "the ones who order once a quarter", and "the ones who stopped answering". Four groups, near enough the same four.

At the planning meeting, Meera doesn't present a model. She presents a table: four groups, what each is worth, how many accounts each contains, and which of the three executives owns most of each. The headline is the concentration: **19% of accounts produce 62% of revenue, and the 570 drifting accounts produce 2.3%.**

Anita's question is the right one: "So *what changes?*" Three things. Key accounts get quarterly reviews instead of ad-hoc contact. Growing regulars get the cross-sell campaign for their missing product category, which the basket rules in section 38.9 can fill in. And the drifting accounts get one scripted win-back call each, after which they stop receiving sales attention, which frees roughly a day a week across the team.

Vikram, skeptical, asks why the earlier agency's nine segments failed. Meera's answer: "*They were built from demographics we don't act on, and nobody could tell which segment a customer was in without asking the agency. Ours come from our own order data, we can re-run them every quarter, and any rep can work out a customer's group from three numbers.*"

Six months later, two of the four groups have been renamed by the sales team, which Meera counts as the clearest sign the work landed.

Tools

- **scikit-learn** (tested on 1.8.0; current release at the time of writing 1.9.1): `KMeans` , `AgglomerativeClustering` , `DBSCAN` , `silhouette_score` , `adjusted_rand_score` , `PCA` , `TSNE` , `IsolationForest` . `MiniBatchKMeans` handles millions of rows.
 - **SciPy** for `linkage` , `dendrogram` , and `fcluster` , which give the tree that scikit-learn's version doesn't draw.
 - **umap-learn** 0.5.12 (`pip install umap-learn`) for UMAP.
 - **mlxtend** 0.25.0 (`pip install mlxtend`) for `TransactionEncoder` , `apriori` , and `association_rules` . For very large baskets, the FP-Growth implementation in the same library is faster.
 - **In SQL:** basket counts are a self-join of order lines on `order_id` with a `product_a < product_b` filter, which is often how a first pass is done on data too large to bring into pandas (Chapter 13's patterns).
 - Everything ran on one CPU core, Python 3.12.3, on 18 September 2026. The slowest step is t-SNE on 1,500 accounts, a few seconds.
 - **Companion files:** `companion/generate_riverstone_accounts.py` (Chapter 37) and `companion/generate_riverstone_baskets.py` (seed 20238) build the two datasets. Run the chapter's code from `companion/ch38/` . Data spec: `planning/data/riverstone-baskets.md` .
-

The project: customer segmentation and a product-bundle analysis

Goal: a segmentation a sales team could use, and a short list of product bundles worth testing, both defended with evidence rather than a chart.

Option A: your own data. Any customer, product, or transaction table you can use. Remove personal details first.

Option B: Riverstone. `accounts.csv` and the basket files.

Steps:

1. **Choose features deliberately.** List every column and say why it's in or out. Exclude anything that is a target or an outcome.
2. **Prepare.** Log-transform skewed money columns, handle blanks, and scale.
3. **Cluster** with k-means for $k = 2$ to 8. Record inertia and silhouette, and plot both.
4. **Check stability** with at least three seeds and three 80% subsamples, reported as adjusted Rand scores.
5. **Choose k** and justify it in three sentences: what the numbers say, what the business can act on, and what you're giving up.
6. **Profile and name** each cluster, with size, value share, and two or three defining numbers. Every name must be a phrase a sales rep would use.
7. **Cross-check** with one method you didn't use (hierarchical or DBSCAN) and one external column the clustering never saw.
8. **Draw one picture** (PCA, and optionally t-SNE or UMAP), with a caption that says what the picture does *not* prove.
9. **Basket analysis:** run Apriori, keep rules above a support and lift you justify, and remove trivial ones.
10. **Write a one-page recommendation:** the groups, what changes for each, two bundles to test, and how you'd measure whether any of it worked.

Stretch goals:

- Cluster on **behavior over time** instead of totals: build monthly order counts per account and cluster the 12-number sequences.
- Try **RFM** (recency, frequency, monetary value), the classic retail segmentation, and compare it with your clusters using the adjusted Rand index.
- Use **Gaussian mixture models** (`GaussianMixture`), which give each account a probability of belonging to each group instead of a hard label, and compare with k-means.
- Run basket analysis **per segment** and see whether the rules differ between wholesale and hospitality baskets.

You've got it when...

- ☐ I can explain the two steps of k-means and run them by hand on a handful of rows.
 - ☐ I always scale (and log skewed money) before any distance-based method.
 - ☐ I choose k using the elbow, the silhouette, stability, and business usefulness together, and I say when the data has no natural groups.
 - ☐ I compare two clusterings with the adjusted Rand index rather than by cluster number.
 - ☐ I profile and name clusters so that someone who has never seen the model can use them.
 - ☐ I can read a dendrogram and say what cutting it at a given height does.
 - ☐ I know what DBSCAN's noise label means and when density-based clustering is the right tool.
 - ☐ I can explain why t-SNE and UMAP pictures shouldn't be read for distances, and why PCA components can be used as features but t-SNE coordinates shouldn't.
 - ☐ I can use an isolation forest to build a review queue, and I never let it make the decision.
 - ☐ I can calculate support, confidence, and lift by hand, and explain why a high-confidence rule can still be worthless.
 - ☐ I judge an unsupervised result by stability, an external check, and whether anything changes because of it.
-

Recap

- **Unsupervised learning** finds structure without a target, and has no test set to prove it right.
 - **k-means** alternates assigning rows to the nearest **centroid** and moving centroids to their members' average, minimizing **inertia**. Scale first; set `n_init`; labels are arbitrary.
 - Choose k with the **elbow**, the **silhouette**, **stability** (adjusted Rand across seeds and subsamples), and above all whether the business can act on the groups. On continuous data, expect low silhouettes and say so.
 - **Profile and name** clusters with size, value share, and defining numbers.
 - **Hierarchical clustering** builds a **dendrogram** you can cut at any height; Ward linkage is the usual default; it doesn't scale to large data.
 - **DBSCAN** grows clusters from dense regions and labels the rest **noise**; it finds any shape and refuses to invent groups where there are none.
 - **PCA** is fast and meaningful; **t-SNE** and **UMAP** show local neighborhoods but not global distances, and are for pictures, not features.
 - **Isolation forests** score how easily a row is separated: a review queue for odd accounts, fraud triage, and data errors.
 - **Market basket analysis** measures **support**, **confidence**, and **lift**; **Apriori** finds rules efficiently; read lift and support alongside confidence, and test a bundle before believing it.
 - Judge unsupervised work by **stability**, an **external check**, and whether a decision changes.
-

Practice exercises

Code exercises run from `companion/ch38/` after the chapter's code (they use `accounts`, `X`, `FEATURES`, `base`, `lines`, `baskets`, `matrix`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) Two centroids sit at (0, 0) and (4, 4). Which cluster does the point (3, 1) join, and what is its squared distance to that centroid?
2. (*hand*) A cluster has 3 members at (2, 2), (4, 6), and (6, 4). Where is its centroid, and what is the cluster's contribution to inertia?
3. (*hand*) Of 1,000 baskets, 200 contain tea, 300 contain sugar, and 90 contain both. Calculate support, confidence, and lift for *tea* → *sugar*. Is the pattern worth reporting?
4. Which method fits each job: (a) group delivery points into service zones of any shape; (b) split 5,000 customers into four groups for four sales approaches; (c) show 20 features on one chart; (d) find 50 accounts worth a manual data check?

Core

5. Re-run k-means on the same accounts **without scaling** the features. Profile the clusters and describe what happened.
6. Cluster with $k = 5$ and compare it with the $k = 4$ grouping using the adjusted Rand index. Which cluster split in two?
7. Add `complaints_2024` to the feature list and re-run k-means with $k = 4$. Does the grouping change (adjusted Rand against `base`), and do the profiles change?
8. Compute the silhouette score for each of the four clusters separately (use `silhouette_samples`). Which cluster is best defined, and which overlaps most?
9. Run Apriori with `min_support=0.005` and `min_threshold=1.5`, and count how many rules you get. How many involve three or more products?
10. For each of the four clusters, find the top three products by number of baskets (join the order lines to `accounts`). Do the clusters buy differently?

Stretch

11. Fit a `GaussianMixture` with four components on `X`, compare its labels with `base` (adjusted Rand), and report how many accounts have a top probability below 0.6.
12. Build monthly order counts per account from the basket data, cluster those 12-number profiles with $k = 4$, and describe the shapes you find.
13. Run market basket analysis separately for wholesale and hospitality baskets. Name one rule that appears in one and not the other, and explain it.

Think about it

14. A colleague clusters customers using revenue, orders, units, and order value. The clusters come out ordered purely by size. What happened, and what would you change?

- l5. The sales head asks: "Are these groups real?" Answer in three sentences, using this chapter's evidence.
- l6. A basket rule says *printer* → *printer paper* with lift 6. Marketing wants to promote paper to printer buyers. What's the flaw, and what would you propose instead?
-

Key terms

unsupervised learning · clustering · k-means · centroid · inertia · assignment step · update step · `n_init` · elbow method · silhouette score · adjusted Rand index (ARI) · stability · cluster profiling · hierarchical clustering · agglomerative · dendrogram · linkage (Ward, complete, average, single) · cutting the tree · DBSCAN · density · core point · `eps` · `min_samples` · noise points · dimensionality reduction · PCA · explained variance · t-SNE · perplexity · UMAP · local versus global structure · anomaly detection · isolation forest · contamination · local outlier factor · market basket analysis · basket (transaction) · item set · support · confidence · lift · Apriori · FP-Growth · trivial rule · external validation · Gaussian mixture model · RFM

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, takes the churn and lead models and turns their scores into decisions with costs, and covers the fairness checks that segmentation work also needs.
 - **Chapter 40, Time Series & Forecasting**, detects anomalies over time, where an isolation forest on static features can't help.
 - **Chapter 41, NLP Foundations**, clusters support tickets by topic, using the same k-means on text features.
 - **Chapter 42, Recommender Systems & Ranking**, goes beyond global basket rules to per-customer recommendations, reusing this chapter's basket data.
 - **Chapter 30, Experiments**, is how a bundle or a win-back campaign is proved to work.
 - **Chapters 55 and 58** cluster embeddings of documents, which is this chapter's k-means on a different kind of feature.
 - **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers k-means, choosing *k*, DBSCAN versus k-means, PCA versus t-SNE, and "how do you know your clusters are any good?", which is the question most candidates answer badly.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Squared distance to (0, 0) is $3^2 + 1^2 = 10$. To (4, 4) it is $(3 - 4)^2 + (1 - 4)^2 = 1 + 9 = 10$. It's an exact **tie**, which is the point of the question: the row sits on the boundary, and the algorithm has to break the tie by a rule (scikit-learn takes the lower cluster number). Rows near boundaries have silhouette scores near 0 and can flip between runs, which is why individual assignments in an overlapping cloud shouldn't be treated as facts about a customer.

2. Centroid = $((2 + 4 + 6) \div 3, (2 + 6 + 4) \div 3) = (4, 4)$. Squared distances: (2, 2) gives $4 + 4 = 8$; (4, 6) gives $0 + 4 = 4$; (6, 4) gives $4 + 0 = 4$. The cluster contributes **16** to inertia.

3. $\text{support}(\text{tea}) = 200 \div 1,000 = 0.20$; $\text{support}(\text{sugar}) = 0.30$; $\text{support}(\text{both}) = 90 \div 1,000 = 0.09$. $\text{confidence}(\text{tea} \rightarrow \text{sugar}) = 0.09 \div 0.20 = 0.45$. $\text{lift} = 0.45 \div 0.30 = 1.5$. Worth reporting: the support is high enough to matter (9% of all baskets) and tea buyers take sugar 1.5 times as often as basket buyers in general. Whether it's *actionable* is another question, since both are staples people may buy together out of habit.

4. (a) **DBSCAN**: service zones follow roads and rivers, so clusters have irregular shapes, and outlying addresses should be left unassigned. (b) **k-means**: a fixed number of similar-sized groups, which is exactly what it produces. (c) **PCA** first, because its axes mean something and it's reproducible; UMAP as a second picture if the structure looks curved. (d) **Isolation forest**: it ranks rows by how unusual they are, so you can take the 50 strangest.

5.

```
unscaled = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(
    accounts[FEATURES]
)
print(accounts.groupby(unscaled)[FEATURES].median().round(1).to_string())
print("\ncluster sizes:", np.bincount(unscaled).tolist())
print(
    "agreement with the scaled clustering:",
    round(adjusted_rand_score(base, unscaled), 3),
)
```

	log_revenue	orders_2024	categories_bought	avg_discount_pct	late_payment_days
0	12.7	15.0	2.0	5.9	11.0
13.0	27.0				
1	11.4	4.0	1.0	4.4	14.0
166.5	25.0				
2	11.2	2.0	1.0	4.1	14.5
332.0	25.0				
3	11.9	6.0	1.0	4.6	13.0
68.0	25.0				

cluster sizes: [3014, 468, 244, 1274]
agreement with the scaled clustering: 0.135

Without scaling, the clusters are almost entirely bands of `days_since_last_order`, with `orders_2024` a distant second: those two columns have the largest raw spread (about 79 and 17), while `log_revenue` varies by about 1 and `categories_bought` by about 1. The other five features barely influence the result, and the agreement with the scaled clustering is poor. This is Chapter 35's units lesson in its most expensive form.

6.

```
five = KMeans(n_clusters=5, n_init=10, random_state=38).fit_predict(X)
print("adjusted Rand, k=4 vs k=5:", round(adjusted_rand_score(base, five), 3))
print(pd.crosstab(base, five).to_string())
```

```
adjusted Rand, k=4 vs k=5: 0.723
col_0    0    1    2    3    4
row_0
0         0    0  107  944   37
1        892    0   26   22    4
2         0    0  521    3  1874
3         0  518   33    0   19
```

Read the table by rows: each $k = 4$ cluster should map mostly to one $k = 5$ cluster, except the one that split. The extra cluster comes mainly out of the largest group, the occasional buyers, which splits by recency and order count. That's the usual pattern when a continuum is cut more finely: the biggest slice divides rather than a new type appearing.

7.

```
with_complaints = FEATURES + ["complaints_2024"]
X2 = StandardScaler().fit_transform(accounts[with_complaints])
labels2 = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(X2)
print(
    "adjusted Rand vs the original clustering:",
    round(adjusted_rand_score(base, labels2), 3),
)
shown = ["revenue_2024", "orders_2024", "complaints_2024", "days_since_last_order"]
print(accounts.groupby(labels2)[shown].median().round(1).to_string())
```

```
adjusted Rand vs the original clustering: 0.897
revenue_2024  orders_2024  complaints_2024  days_since_last_order
0      1092000.0         48.0             1.0              8.0
1      298200.0         14.0             0.0             22.0
2       73300.0          2.0             0.0            227.0
3      144200.0          7.0             0.0             36.0
```

The grouping barely changes. Complaints rise with order volume, so the new feature mostly repeats information the clustering already had through `orders_2024`. Adding features that duplicate existing ones quietly doubles their weight in the distance calculation; adding new information is what changes a clustering.

8.

```

from sklearn.metrics import silhouette_samples

scores = silhouette_samples(X, base)
print(
    pd.DataFrame({"cluster": base, "silhouette": scores})
      .groupby("cluster")["silhouette"]
      .agg(["mean", "size"])
      .round(3)
      .to_string()
)
print("accounts with a negative silhouette:", int((scores < 0).sum()))

```

```

      mean  size
cluster
0      0.174 1088
1      0.205  944
2      0.206 2398
3      0.234  570
accounts with a negative silhouette: 126

```

The drifting cluster is the best defined (0.234), because "no order for months" separates it cleanly from everything else. The growing regulars score lowest (0.174): they sit between the key accounts and the occasional buyers, with neighbours on both sides. Every cluster's score is low, which is section 38.2's message again. The accounts with negative silhouettes sit closer to a neighboring cluster's members than to their own, and would move if the data shifted slightly.

9.

```

frequent_small = apriori(matrix, min_support=0.005, use_colnames=True)
rules_small = association_rules(frequent_small, metric="lift", min_threshold=1.5)
sizes = rules_small["antecedents"].apply(len) + rules_small["consequents"].apply(len)
print(f"{len(frequent_small)} item sets, {len(rules_small)} rules with lift above 1.5")
print("rules by number of products:", sizes.value_counts().sort_index().to_dict())
print(
    f"smallest support among those rules: {rules_small['support'].min():.4f} "
    f"({rules_small['support'].min() * len(baskets):.0f} orders)"
)

```

```

338 item sets, 464 rules with lift above 1.5
rules by number of products: {2: 32, 3: 394, 4: 38}
smallest support among those rules: 0.0050 (170 orders)

```

Halving the support threshold produces many more rules, most of them involving three or more products and most of them variations on the same planted pairs. More rules is not more insight: the useful output of a basket analysis is usually a handful of rules a person can act on, which is why you raise `min_support` until the list fits on one page.

10.

```

cluster_of = accounts.set_index("account_id")["cluster"]
lines["cluster"] = lines["account_id"].map(cluster_of)
top_products = (
    lines.dropna(subset=["cluster"])
    .groupby(["cluster", "product_name"])["order_id"]
    .nunique()
    .rename("orders")
    .reset_index()
)
for cluster, group in top_products.groupby("cluster"):
    best = group.nlargest(3, "orders")
    print(
        f"cluster {int(cluster)}: "
        + ", ".join(f"{r.product_name} ({r.orders:,})" for r in best.itertuples())
    )

```

```

cluster 0: Airtight Seal Pack (1,816), Crate Lid (50L) (1,186), Crate Lid (80L) (1,135)
cluster 1: Drum Tap Fitting (4,325), Dolly Wheels (set) (4,268), Crate Lid (50L) (3,814)
cluster 2: Airtight Seal Pack (2,159), Crate Lid (50L) (1,383), Crate Lid (80L) (1,279)
cluster 3: Airtight Seal Pack (192), Crate Lid (80L) (129), Serving Tray (119)

```

The lists overlap heavily, because the most common products are common everywhere. Differences show up in what follows the top three, and they match the segment mix of each cluster: the key-accounts cluster, mostly wholesale, leans on industrial items, while the occasional buyers' baskets are storage and kitchen. For sharper differences, compare each cluster's share of a product against the overall share rather than raw counts.

11.

```

from sklearn.mixture import GaussianMixture

mixture = GaussianMixture(n_components=4, random_state=38).fit(X)
soft = mixture.predict_proba(X)
labels_gmm = soft.argmax(axis=1)
print("adjusted Rand vs k-means:", round(adjusted_rand_score(base, labels_gmm), 3))
print(
    "accounts whose best group has probability below 0.6:",
    int((soft.max(axis=1) < 0.6).sum()),
)

```

```

adjusted Rand vs k-means: 0.394
accounts whose best group has probability below 0.6: 206

```

A Gaussian mixture allows stretched, tilted clusters instead of round ones, so its groups differ noticeably from k-means. The interesting output is the uncertainty: 206 accounts, about 4%, have no confident home, which is the same message as the low silhouette, stated more usefully. Soft memberships are worth the extra complexity when you want to treat borderline customers carefully rather than forcing them into a group.

12.

```

lines["month"] = pd.to_datetime(lines["order_date"]).dt.month
monthly = (
    lines.groupby(["account_id", "month"])["order_id"]
    .nunique()
    .unstack(fill_value=0)
    .reindex(columns=range(1, 13), fill_value=0)
)
monthly = monthly[monthly.sum(axis=1) >= 6]
shapes = StandardScaler().fit_transform(monthly.div(monthly.sum(axis=1), axis=0))
shape_labels = KMeans(n_clusters=4, n_init=10, random_state=38).fit_predict(shapes)
print(f"{len(monthly):,} accounts with at least 6 orders")
print(
    pd.DataFrame(
        monthly.div(monthly.sum(axis=1), axis=0).to_numpy(), index=shape_labels
    )
    .groupby(level=0)
    .mean()
    .round(3)
    .to_string()
)

```

```

1,727 accounts with at least 6 orders

```

	0	1	2	3	4	5	6	7	8	9	10	11
0	0.079	0.053	0.081	0.098	0.059	0.118	0.091	0.055	0.073	0.170	0.058	0.065
1	0.098	0.035	0.106	0.081	0.164	0.059	0.076	0.052	0.078	0.053	0.101	0.097
2	0.075	0.042	0.084	0.065	0.052	0.090	0.104	0.187	0.092	0.056	0.077	0.076
3	0.085	0.164	0.074	0.071	0.058	0.058	0.075	0.072	0.084	0.063	0.098	0.099

Each row here is a *shape*, the share of a customer's orders falling in each month, not a level, so big and small accounts can share a pattern. The four groups differ mainly in which months are busiest. Each group peaks in a different month, which looks like seasonality until you remember that this generator spreads orders evenly across the year: with 12 noisy numbers per account, k-means will always find groups whose averages peak somewhere. That is the honest answer: clustering finds groups whether or not the data contains any. Chapter 40 handles seasonality properly, with data that has it.

13.

```

for segment in ["Wholesale", "Hospitality"]:
    segment_baskets = (
        lines[lines["segment"] == segment]
        .groupby("order_id")["product_name"]
        .apply(set)
    )
    encoded = TransactionEncoder()
    grid = pd.DataFrame(
        encoded.fit_transform(segment_baskets.tolist()), columns=encoded.columns_
    )
    seg_rules = association_rules(
        apriori(grid, min_support=0.01, use_colnames=True),
        metric="lift",
        min_threshold=1.5,
    )
    seg_rules["if"] = seg_rules["antecedents"].apply(lambda s: ", ".join(sorted(s)))
    seg_rules["then"] = seg_rules["consequents"].apply(lambda s: ", ".join(sorted(s)))
    best = seg_rules.nlargest(3, "lift")[
        ["if", "then", "support", "confidence", "lift"]
    ]
    print(f"--- {segment} ({len(segment_baskets):,} baskets)")
    print(best.round(3).to_string(index=False))

```

```

--- Wholesale (16,574 baskets)
      if                                then support confidence lift
      Garden Chair                      Garden Table 0.014    0.476 18.531
      Garden Table                      Garden Chair 0.014    0.542 18.531
Drum Tap Fitting, Storage Crate 50L Crate Lid (50L), Drum 60L 0.010    0.313 9.493
--- Hospitality (7,905 baskets)
      if                                then support confidence lift
Airtight Seal Pack, Garden Chair                      Garden Table 0.015    0.540 7.370
Garden Table Airtight Seal Pack, Garden Chair 0.015    0.211 7.370
Garden Chair                      Garden Table 0.042    0.530 7.242

```

The pairs that are planted in the data (chairs with tables, drums with taps, crates with lids) appear in both segments, because the rules were built into the products, not the customers. What differs is which rules **clear the support threshold**, and how strong they look: the chair-and-table pair has a lift of 18.5 in wholesale baskets against 7.2 in hospitality (furniture is rarer in wholesale orders, so co-occurrence stands out more), and the three-product industrial rules appear only in wholesale. That's the practical lesson: run basket analysis per segment when the segments buy different catalogues, or the rules of the largest segment will be the only ones you see.

14. Those four features all measure the same thing, the size of the account, and three of them are strongly correlated, so distance is dominated by size and the clusters come out as "big, medium, small". A useful segmentation needs features that vary **independently** of each other: keep one size measure, and add behavior (recency, product breadth, discount level, payment habits). The quick check is a correlation matrix of the features before clustering.

15. "They're stable: re-running with different random starts and on 80% samples gives almost identical groups. They know something they were never told: the groups' churn rates run from 3% to 41%, and churn was not among the features. But they aren't natural types, because the silhouette score is low and the accounts form a continuum rather than separated balls, so the boundaries are our choice of where to cut."

16. The rule is **trivial**: people who buy a printer need paper, so the association tells you nothing you didn't know, and the buyer probably already adds paper without prompting. Lift measures association, not the effect of an intervention, so it can't say whether the promotion changes behavior. Better proposals: target printer buyers who **didn't** buy paper (section 38.9's approach), promote paper at the right *time* (weeks after purchase, when the first ream runs out), or find the non-obvious partner products that also have high lift with printers. And whichever is chosen, test it on half the customers first (Chapter 30).

Chapter 39. Evaluation, Tuning, Interpretation & Honesty

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: build a confusion matrix and compute precision, recall, F1, and specificity from it by hand · draw and read ROC and precision–recall curves, and know which one to trust on imbalanced data · judge a regression model with MAE, RMSE, MAPE, and R^2 , and know what each hides · check whether predicted probabilities are honest (calibration) and fix them when they aren't · choose a decision threshold from business costs and team capacity, not from a default of 0.5 · handle imbalanced classes with class weights and resampling, and know what they do and don't change · explain a model globally and one prediction at a time with SHAP and partial dependence · check whether a model treats groups of people fairly · write a model card that says what a model is for and where it fails.

Before you start: Chapter 35 (log loss and the base-rate benchmark), Chapter 36 (the lead-scoring pipeline and validation set), Chapter 37 (the churn models). This chapter evaluates the models you already built.

Time needed: 10–14 hours over two weeks.

Tools: Python 3 with scikit-learn, plus `shap` and `imbalanced-learn` (both free).

Practice data: the lead-scoring model and validation set from Chapter 36 (2,225 leads, 146 won), and the churn and revenue models from Chapter 37. Every number in this chapter was calculated, and every output shown is real.

Why this matters

A model's score is where most machine learning conversations start and stop: "It's 82% accurate." "AUC is 0.82." This chapter is about what those numbers hide and what a business actually needs to know:

- Chapter 36's lead model has an AUC of 0.823. **How many leads should the sales team call?** AUC can't say.
- The model predicts a 20% chance of winning a lead. **Does it win one in five of those?** That's calibration, and one of Chapter 37's models fails it badly.
- **Why did this lead score 65%?** A rep will ask, and "the model said so" is not an answer.
- **Does the model score inside-sales leads fairly?** Someone will ask that too, and the answer here is not simple.

- Predicting "no churn" for everyone is 90% accurate. **Accuracy is the wrong number** on any imbalanced problem, and most business problems are imbalanced.

The chapter's title includes *honesty* because that's the skill being taught: reporting what a model can and can't do, in units a manager understands, before anyone depends on it.

In plain English

Think of a smoke detector.

It can fail two ways: it goes off when there's no fire (a **false positive**: annoying, you open a window), or it stays silent during a fire (a **false negative**: catastrophic). Counting the four outcomes, alarm-fire, alarm-no fire, silent-fire, silent-no fire, is a **confusion matrix**.

How sensitive should it be? Turn it up and you catch every fire but it shrieks at toast. Turn it down and it's quiet but misses a smouldering sofa. That dial is the **threshold**, and the right setting depends on how much a false alarm costs against a missed fire. That's **threshold selection by cost**. A model that reports "70% chance of fire" should be right about 70% of the time when it says that; if it says 70% and there's a fire only 10% of the time, it's **poorly calibrated**. When the alarm goes off, you'd like to know why: smoke, heat, or a low battery. That's **interpretation**. And if the detector works in the kitchen but never in the bedroom, that's a **fairness** problem someone should have tested for.

39.1 The confusion matrix

Four numbers, then everything else

Start from Chapter 36's lead model on its validation set: 2,225 leads from the first half of 2025, of which 146 were won. The model gives each lead a probability; a **threshold** turns it into a yes-or-no decision, and comparing decisions with outcomes gives four counts:

	Actually won	Actually lost
Predicted won	True positive (TP)	False positive (FP)
Predicted lost	False negative (FN)	True negative (TN)

```

import sys
import warnings

import numpy as np
import pandas as pd

sys.path.append(".")
from lead_data import LEAD_CATS, LEAD_NUMS, load_leads, make_model

warnings.filterwarnings("ignore", category=UserWarning)

train, valid, test = load_leads()
X_cols = LEAD_CATS + LEAD_NUMS
model = make_model(LEAD_CATS, LEAD_NUMS).fit(train[X_cols], train["won"])
p_valid = model.predict_proba(valid[X_cols])[:, 1]
y_valid = valid["won"].to_numpy()
print(f"{len(valid):,} validation leads, {y_valid.sum()} won ({y_valid.mean():.2%}")
print(
    f"predicted probabilities: min {p_valid.min():.3f}, median {np.median(p_valid):.3f}, "
    f"max {p_valid.max():.3f}"
)

```

```

2,225 validation leads, 146 won (6.56%)
predicted probabilities: min 0.001, median 0.037, max 0.646

```

```

from sklearn.metrics import confusion_matrix

def matrix_at(threshold):
    predicted = (p_valid >= threshold).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_valid, predicted).ravel()
    return tp, fp, fn, tn

for threshold in [0.5, 0.2, 0.1]:
    tp, fp, fn, tn = matrix_at(threshold)
    print(
        f"threshold {threshold}: TP {tp:>4}  FP {fp:>5}  FN {fn:>4}  TN {tn:>5}    "
        f"accuracy {(tp + tn) / len(y_valid):.3f}"
    )

```

```

threshold 0.5: TP    4  FP    3  FN  142  TN  2076  accuracy 0.935
threshold 0.2: TP   55  FP   122  FN   91  TN  1957  accuracy 0.904
threshold 0.1: TP  100  FP   386  FN   46  TN  1693  accuracy 0.806

```

Reading it. At the default threshold of 0.5, the model predicts "won" for 7 of 2,225 leads. It's right about 4, and misses 142 of the 146 wins. Its accuracy is 93.5%, which is *worse* than predicting "lost" for everyone (93.4%, since $2,079 \div 2,225 = 0.934$). Accuracy tells you nothing here. The problem isn't the model: only 6.6% of leads are won, so a well-calibrated model rarely says a lead is *more likely than not* to be won. The threshold has to move.

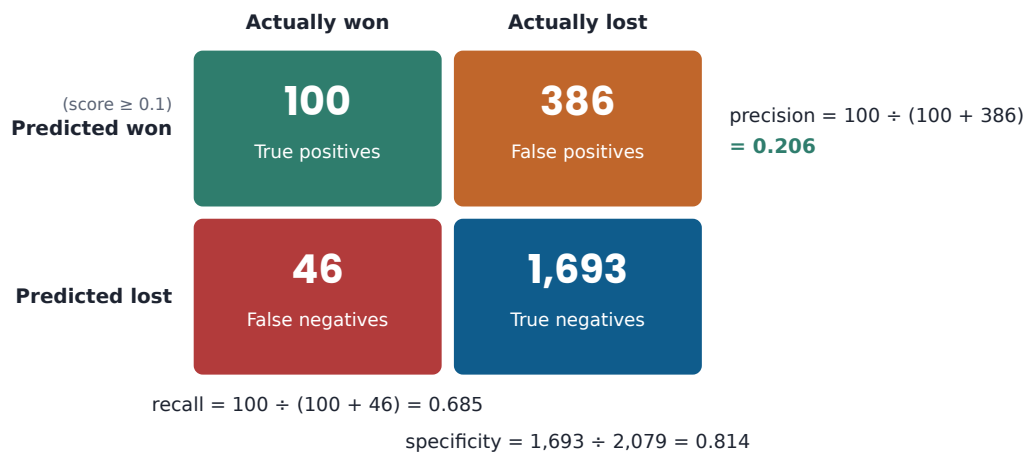


Figure 39.1 – The confusion matrix at a threshold of 0.1. Precision reads across the top row; recall reads down the first column.

Precision, recall, F1, and the rest

From the four counts at a threshold of 0.1 (TP 100, FP 386, FN 46, TN 1,693):

precision = $TP \div (TP + FP) = 100 \div 486 = 0.206$: of the leads the model flags, one in five is won

recall (sensitivity, true positive rate) = $TP \div (TP + FN) = 100 \div 146 = 0.685$: of the leads that are won, the model flagged two thirds

F1 = $2 \times \text{precision} \times \text{recall} \div (\text{precision} + \text{recall}) = 2 \times 0.206 \times 0.685 \div 0.891 = 0.317$ (0.316 from the unrounded precision and recall): the harmonic mean, which punishes lopsided pairs

specificity (true negative rate) = $TN \div (TN + FP) = 1,693 \div 2,079 = 0.814$; its complement, the **false positive rate**, is 0.186

```
from sklearn.metrics import f1_score, precision_score, recall_score

tp, fp, fn, tn = matrix_at(0.1)
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)
specificity = tn / (tn + fp)
print(
    f"at threshold 0.1: precision {precision:.3f}    recall {recall:.3f}    "
    f"F1 {f1:.3f}    specificity {specificity:.3f}"
)
predicted = (p_valid >= 0.1).astype(int)
print(
    f"scikit-learn:    precision {precision_score(y_valid, predicted):.3f}    "
    f"recall {recall_score(y_valid, predicted):.3f}    F1 {f1_score(y_valid, predicted):.3f}"
)
```

```
at threshold 0.1: precision 0.206    recall 0.685    F1 0.316    specificity 0.814
scikit-learn:    precision 0.206    recall 0.685    F1 0.316
```

Precision and recall pull against each other. Raise the threshold and you flag fewer leads, more of which are won (higher precision) but fewer of all the wins (lower recall). Which matters more depends entirely on the decision: a fraud alert that freezes a card wants precision; a cancer screen wants recall. F1 is a compromise for when you have no cost information; section 39.4 replaces it with something better.

39.2 Curves: ROC and precision–recall

A single threshold gives a single matrix. Sweeping the threshold from 1 down to 0 traces a curve, and the area under it summarizes ranking quality without choosing a threshold.

- The **ROC curve** plots recall (true positive rate) against the false positive rate. **ROC-AUC** is the area under it: the probability that a random won lead scores higher than a random lost one (Chapter 36).
- The **precision–recall (PR) curve** plots precision against recall. **PR-AUC**, usually reported as **average precision**, is the area under it.

```
from sklearn.metrics import (
    average_precision_score,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)

fpr, tpr, roc_thresholds = roc_curve(y_valid, p_valid)
prec, rec, pr_thresholds = precision_recall_curve(y_valid, p_valid)
print(
    f"ROC-AUC {roc_auc_score(y_valid, p_valid):.3f}    "
    f"PR-AUC (average precision) {average_precision_score(y_valid, p_valid):.3f}    "
    f"base rate {y_valid.mean():.3f}"
)
print(f"the ROC curve has {len(fpr)} points; the PR curve has {len(prec)}")
random_pr = y_valid.mean()
print(f"a random model would score PR-AUC {random_pr:.3f} and ROC-AUC 0.500")
```

```
ROC-AUC 0.823   PR-AUC (average precision) 0.302   base rate 0.066
the ROC curve has 240 points; the PR curve has 2226
a random model would score PR-AUC 0.066 and ROC-AUC 0.500
```

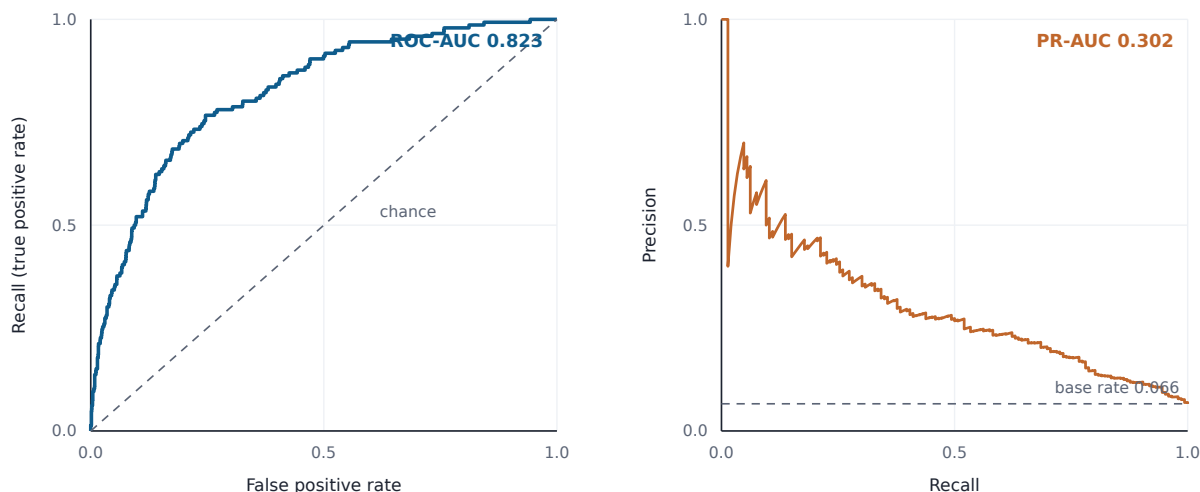


Figure 39.2 – ROC (left) and precision–recall (right) curves for the lead model. The ROC curve looks impressive; the PR curve shows what the sales team will experience: precision falls fast as they call further down the list.

Reading it. ROC-AUC is 0.823 and looks good. PR-AUC is 0.302, which looks poor, and both are correct. The difference is the base rate. A random model scores 0.5 on ROC-AUC regardless of imbalance, but only 0.066 on PR-AUC here (the base rate). The lead model's PR-AUC is 4.6 times a random model's, which is a real achievement, and it still means that a rep working the list will hear "no" most of the time.

Which to report. On imbalanced problems, **PR-AUC is the more honest summary**, because it's built from precision, the number the person acting on the model feels. ROC-AUC is fine for comparing models and for balanced problems. Report both, and say what the base rate is.

The list, not the curve

The most useful evaluation for a call list isn't a curve at all. It's "if we call the top N, how many do we win?"

```
order = np.argsort(-p_valid)
for k in [50, 100, 200, 400]:
    top = order[:k]
    print(
        f"top {k:>3} leads by score: {y_valid[top].sum():>3} won "
        f"({y_valid[top].mean():.1%} of them; "
        f"{y_valid[top].sum() / y_valid.sum():.1%} of all wins)"
    )
```

```
top  50 leads by score:  22 won (44.0% of them; 15.1% of all wins)
top 100 leads by score:  38 won (38.0% of them; 26.0% of all wins)
top 200 leads by score:  59 won (29.5% of them; 40.4% of all wins)
top 400 leads by score:  92 won (23.0% of them; 63.0% of all wins)
```

Reading it. The top 50 leads by score are won 44% of the time, nearly seven times the 6.6% base rate. The top 400 (18% of leads) contain 63% of all the wins. This table is what to show a sales head, because it answers the question they'll ask, and it leads straight into section 39.4.

39.3 Regression metrics

For models that predict a number, the confusion matrix has no equivalent; the errors are amounts. Chapter 37's model predicting each account's 2025 revenue from its 2024 revenue gives a worked example:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_absolute_percentage_error,
    r2_score,
    root_mean_squared_error,
)
from sklearn.model_selection import train_test_split

accounts = pd.read_csv("../accounts/accounts.csv")
stayed = accounts[accounts["churned_2025"] == 0].copy()
stayed["log_revenue_2024"] = np.log(stayed["revenue_2024"])
stayed["log_revenue_2025"] = np.log(stayed["revenue_2025"])
reg_train, reg_test = train_test_split(stayed, test_size=0.25, random_state=37)
reg = LinearRegression().fit(
    reg_train[["log_revenue_2024"]], reg_train["log_revenue_2025"]
)
pred_rupees = np.exp(reg.predict(reg_test[["log_revenue_2024"]]))
actual = reg_test["revenue_2025"]
print(f"MAE    ₹{mean_absolute_error(actual, pred_rupees):>12,.0f}")
print(f"RMSE    ₹{root_mean_squared_error(actual, pred_rupees):>12,.0f}")
print(f"MAPE     {mean_absolute_percentage_error(actual, pred_rupees):>12.1%}")
print(
    f"R²        {r2_score(actual, pred_rupees):>12.3f}    (R² on log scale "
    f"    {r2_score(reg_test['log_revenue_2025'], np.log(pred_rupees)):.3f})"
)
errors = (actual - pred_rupees).abs().sort_values(ascending=False)
print(
    f"largest single error ₹{errors.iloc[0]:,.0f}; the 10 largest errors are "
    f"    ₹{errors.iloc[:10].sum() / errors.sum():.1%} of the total absolute error"
)
```

```
MAE    ₹      92,933
RMSE    ₹     232,567
MAPE           19.2%
R²           0.927    (R² on log scale 0.955)
largest single error ₹3,697,812; the 10 largest errors are 15.2% of the total absolute error
```

The four common measures:

- **MAE** (mean absolute error): the average size of a miss, in rupees. ₹92,933 here. Simple, and the right number to quote to a manager: "our forecast is typically off by about ₹93,000 per account".
- **RMSE** (root mean squared error): the square root of the average squared miss. ₹232,567, two and a half times the MAE, because squaring makes big misses dominate: the ten largest errors are 15% of the total error, and one account is off by ₹3.7 million. RMSE is the right measure when large errors are disproportionately costly; otherwise it's dragged around by outliers.
- **MAPE** (mean absolute percentage error): the average miss as a percentage of the actual value. 19.2%. Managers like it because it's scale-free. It fails when actual values are near zero (a ₹100 miss on a ₹200 account is 50%), and it punishes over-prediction more than under-prediction, so it's biased when used to choose a model.

- **R²**: the share of variance explained (Chapter 37). 0.927 in rupees, 0.955 on the log scale the model was fitted on. The same model, two R² values: R² depends on what scale you measure it on, and comparing R² between models fitted to different targets is meaningless.

Watch out: the metric must match the loss. A model trained on log revenue minimizes percentage-type errors, so it does well on MAPE and worse on RMSE in rupees, where the largest accounts dominate. If the business cares about total rupees, train on rupees (or weight by size). Decide what error costs the business before you decide what to optimize.

For forecasting over time, Chapter 40 adds **WAPE** (weighted absolute percentage error), which fixes MAPE's problems and is the standard in demand planning.

39.4 Calibration: are the probabilities honest?

A model can rank well and still lie about probabilities. **Calibration** asks: among leads the model gave a 20% chance, were about 20% won? The check is a **reliability curve**: bin leads by predicted probability, and plot the actual win rate in each bin against the average prediction. A perfectly calibrated model lies on the diagonal.

Two summary numbers: **log loss** (Chapter 35) and the **Brier score**, the mean squared difference between predicted probability and outcome (0 is perfect; predicting the base rate for everyone gives about 0.061 here).


```

from sklearn.calibration import calibration_curve
from sklearn.metrics import brier_score_loss, log_loss
from sklearn.naive_bayes import GaussianNB
from sklearn.preprocessing import FunctionTransformer
from sklearn.ensemble import HistGradientBoostingClassifier

to_dense = FunctionTransformer(lambda X: X.toarray() if hasattr(X, "toarray") else X)
nb = make_model(LEAD_CATS, LEAD_NUMS, GaussianNB())
nb.steps.insert(1, ("dense", to_dense))
boost = make_model(
    LEAD_CATS, LEAD_NUMS, HistGradientBoostingClassifier(random_state=39)
)
probs = {
    "logistic regression": p_valid,
    "Naive Bayes": nb.fit(train[X_cols], train["won"]).predict_proba(valid[X_cols])[
        :, 1
    ],
    "gradient boosting": boost.fit(train[X_cols], train["won"]).predict_proba(
        valid[X_cols]
    )[:, 1],
}
curves = {}
for name, p in probs.items():
    frac_pos, mean_pred = calibration_curve(y_valid, p, n_bins=10, strategy="quantile")
    curves[name] = (mean_pred, frac_pos)
    print(
        f"{name:<20} log loss {log_loss(y_valid, p):.4f} "
        f" Brier {brier_score_loss(y_valid, p):.4f} "
        f"mean prediction {p.mean():.3f} vs actual {y_valid.mean():.3f}"
    )
print("\nNaive Bayes, by decile of predicted probability:")
print(
    pd.DataFrame(
        {
            "mean predicted": curves["Naive Bayes"][0],
            "actual win rate": curves["Naive Bayes"][1],
        }
    )
    .round(3)
    .to_string(index=False)
)

```

logistic regression	log loss 0.1948	Brier 0.0531	mean prediction 0.069 vs actual 0.066
Naive Bayes	log loss 0.4830	Brier 0.1243	mean prediction 0.180 vs actual 0.066
gradient boosting	log loss 0.2075	Brier 0.0556	mean prediction 0.065 vs actual 0.066

Naive Bayes, by decile of predicted probability:

mean predicted	actual win rate
0.000	0.009
0.000	0.014
0.000	0.013
0.000	0.027
0.002	0.018
0.010	0.032
0.045	0.050
0.192	0.076
0.610	0.149
0.939	0.269

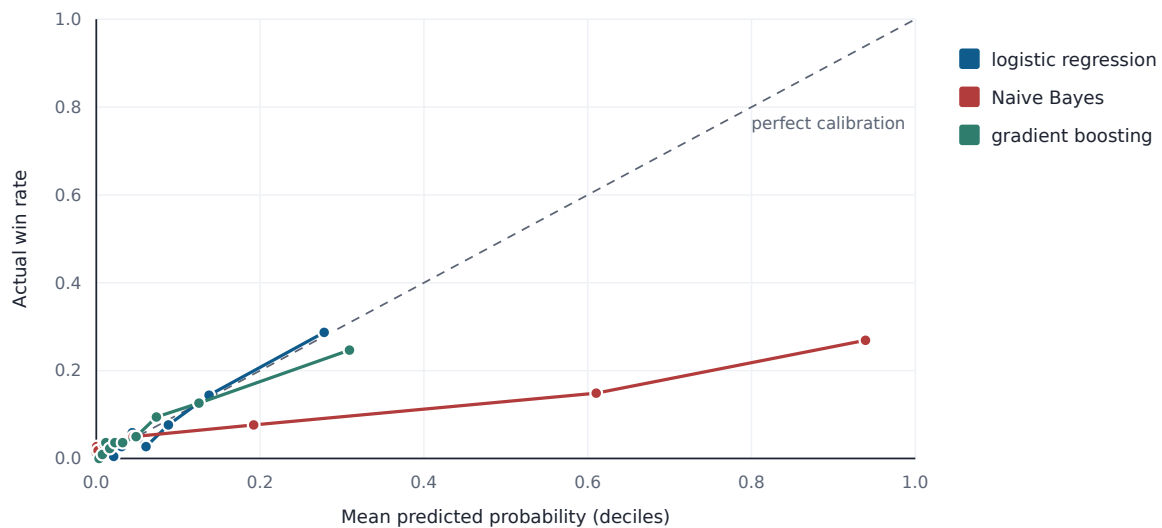


Figure 39.3 – Reliability curves for three models. Logistic regression and gradient boosting are honest; Naive Bayes is confidently wrong in both directions.

Reading it. Logistic regression and gradient boosting are well calibrated: their average prediction (6.9% and 6.5%) matches the actual win rate (6.6%), and their bins sit near the diagonal. Naive Bayes is a different story. Its average prediction is **18%**, nearly three times the real rate, and its top decile predicts a **94%** chance of winning for leads that are won **27%** of the time. It ranks reasonably (ROC-AUC 0.806) and its probabilities are fiction. This is the overconfidence Chapter 37 warned about, and it matters the moment someone uses the number: a rep told "94%" who wins one in four will stop trusting every model.

Fixing calibration

`CalibratedClassifierCV` wraps a model and learns a correction from predicted score to real probability, using cross-validation so it doesn't learn on the same rows twice. **Platt scaling** (`method="sigmoid"`) fits a logistic curve; **isotonic regression** (`method="isotonic"`) fits any rising step function and needs more data.

```
from sklearn.calibration import CalibratedClassifierCV

calibrated_nb = CalibratedClassifierCV(nb, method="isotonic", cv=5).fit(
    train[X_cols], train["won"]
)
p_nb_cal = calibrated_nb.predict_proba(valid[X_cols])[:, 1]
print(
    f"Naive Bayes, isotonic calibration: log loss {log_loss(y_valid, p_nb_cal):.4f}  "
    f"Brier {brier_score_loss(y_valid, p_nb_cal):.4f}  "
    f"ROC-AUC {roc_auc_score(y_valid, p_nb_cal):.3f}"
)
print(
    f"(before calibration: ROC-AUC {roc_auc_score(y_valid, probs['Naive Bayes']):.3f})"
```

```
Naive Bayes, isotonic calibration: log loss 0.2022  Brier 0.0544  ROC-AUC 0.802
(before calibration: ROC-AUC 0.806)
```

Reading it. After isotonic calibration, Naive Bayes's log loss drops from 0.483 to 0.202 and its Brier score from 0.124 to 0.054, close to logistic regression's. Its ranking barely changes (ROC-AUC 0.806 to 0.802), because calibration reshapes the scores without reordering them much. **Calibration fixes the numbers, not the ranking.**

When calibration matters: whenever the probability is used as a number: expected value calculations (the next section), thresholds by cost, combining models, showing a percentage to a person. When only the order matters, such as a call list, it doesn't.

39.5 Choosing the threshold by business cost

The two numbers you need

A threshold is a business decision disguised as a model setting. To set it, you need what a false positive and a true positive are worth. For lead scoring, working a lead properly costs time, and winning one earns margin:

```
WORK_COST = (
    1500 # ₹ to work a lead properly: calls, a quote, samples (about 2 rep-hours)
)
new_accounts = accounts[accounts["tenure_months"] <= 12]
FIRST_YEAR_REVENUE = new_accounts["revenue_2024"].median()
MARGIN = 0.15
WIN_VALUE = FIRST_YEAR_REVENUE * MARGIN
print(
    f"{len(new_accounts)} accounts in their first "
    f"year: median revenue ₹{FIRST_YEAR_REVENUE:,.0f}"
)
print(f"value of a won lead at {MARGIN:.0%} margin: ₹{WIN_VALUE:,.0f}")
print(f"break-even win probability: {WORK_COST / WIN_VALUE:.4f}")
```

```
878 accounts in their first year: median revenue ₹201,700
value of a won lead at 15% margin: ₹30,255
break-even win probability: 0.0496
```

How it works: the cost to work a lead, ₹1,500, covers calls, a quote, and samples, about two hours of a rep's time; change it to your own number. The value of a win is the median first-year revenue of Riverstone's newer accounts (₹201,700, from the accounts data) times a 15% gross margin: ₹30,255. Both are stated assumptions; the point of writing them as named variables is that the sales head can argue with them and re-run the analysis.

Break-even. Working a lead pays off when $p \times ₹30,255 > ₹1,500$, so $p > 1,500 \div 30,255 = \mathbf{0.0496}$. Any lead with a calibrated probability above about 5% is worth working. That number came from costs, not from the model, and it's nowhere near 0.5.

The profit curve

```
def profit_at(threshold, p=p_valid, y=y_valid):
    called = p >= threshold
    return (
        WIN_VALUE * y[called].sum() - WORK_COST * called.sum(),
        called.sum(),
        y[called].sum(),
    )

print("threshold   worked   wins   profit")
for threshold in [0.0, 0.02, 0.03, 0.05, 0.07, 0.10, 0.15, 0.20, 0.50]:
    profit, worked, wins = profit_at(threshold)
    print(f"{threshold:>9.2f}   {worked:>6}   {wins:>4}   ₹{profit:>10,.0f}")

grid = np.linspace(0.0, 0.5, 501)
profits = np.array([profit_at(t)[0] for t in grid])
best = grid[profits.argmax()]
print(f"\nbest threshold on the grid: {best:.3f}   profit ₹{profits.max():,.0f}")
print(f"working every lead: ₹{profit_at(0.0)[0]:,.0f}   working none: ₹0")
```

threshold	worked	wins	profit
0.00	2225	146	₹ 1,079,730
0.02	1476	138	₹ 1,961,190
0.03	1239	135	₹ 2,225,925
0.05	904	121	₹ 2,304,855
0.07	684	114	₹ 2,423,070
0.10	486	100	₹ 2,296,500
0.15	291	76	₹ 1,862,880
0.20	177	55	₹ 1,398,525
0.50	7	4	₹ 110,520

best threshold on the grid: 0.077 profit ₹2,448,060
working every lead: ₹1,079,730 working none: ₹0

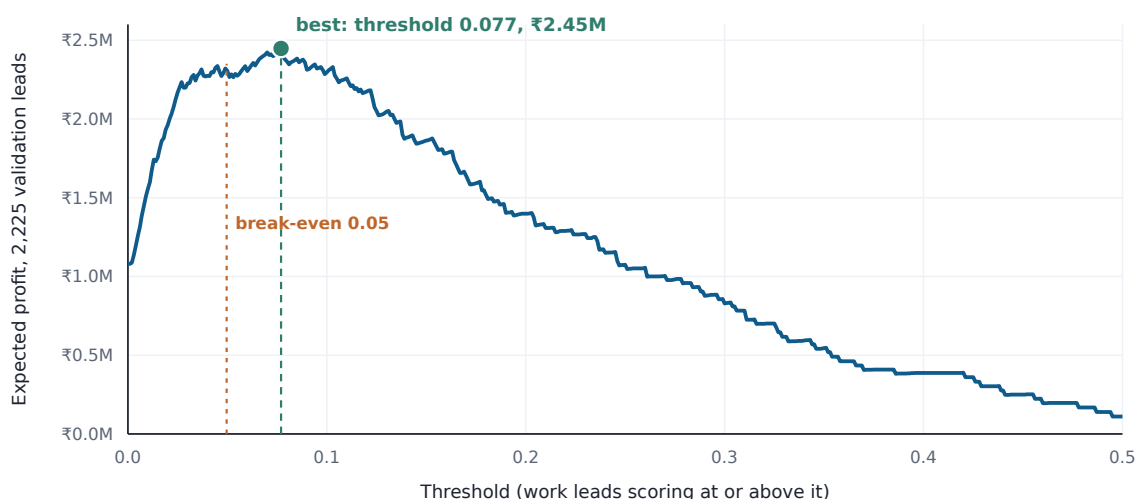


Figure 39.4 – Profit against threshold. Working every lead is profitable but wasteful; the peak is broad, so anything between 0.05 and 0.10 is close to best; above 0.15 the team is leaving money on the table.

Reading it. Working all 2,225 leads makes ₹1.08 million. Working only those above 0.077 makes **₹2.45 million**, from 627 leads instead of 2,225. The peak is broad: thresholds from 0.05 to 0.10 are all within 6% of the best, so precision about the exact number is false precision. Above 0.15 profit falls fast, because the wins the model is now skipping were worth more than the effort saved.

The break-even threshold (0.0496) and the profit-maximizing one (0.077) differ because the model's probabilities near 5% are slightly optimistic, and because the grid is on validation data with only 146 wins. In practice, quote the range.

Capacity, and the old rule

The team can't work 660 leads in six months; it can work about four a day. That's a different constraint, and it sets the threshold from the other direction:

```
DAYS = 126 # working days in the six-month validation period
LEADS_PER_DAY = 4 # what the team can work properly
budget = DAYS * LEADS_PER_DAY
top = np.argsort(-p_valid)[:budget]
threshold_at_budget = p_valid[top].min()
profit, worked, wins = profit_at(threshold_at_budget)
print(
    f"capacity: {budget} leads in the period -> work "
    f"leads scoring above {threshold_at_budget:.3f}"
)
print(f"{worked} worked, {wins} wins, profit ₹{profit:,.0f}")
rule = valid["source"].isin(["Referral", "Trade fair", "Partner"]).to_numpy()
rule_profit = WIN_VALUE * y_valid[rule].sum() - WORK_COST * rule.sum()
print(
    f"the old rule (3 sources): {rule.sum()} worked, "
    f"{y_valid[rule].sum()} wins, profit ₹{rule_profit:,.0f}"
)
best_profit, best_worked, best_wins = profit_at(0.05)
print(
    f"threshold 0.05 with no capacity limit: {best_worked} "
    f"worked, {best_wins} wins, ₹{best_profit:,.0f}"
)
```

```
capacity: 504 leads in the period -> work leads scoring above 0.097
504 worked, 102 wins, profit ₹2,330,010
the old rule (3 sources): 531 worked, 84 wins, profit ₹1,744,920
threshold 0.05 with no capacity limit: 904 worked, 121 wins, ₹2,304,855
```

Reading it. With capacity for 504 leads, the team works everything scoring above 0.097, wins 102, and makes ₹2.33 million. The old rule, "work referrals, trade fairs, and partners", works a similar number of leads (531), wins 84, and makes ₹1.74 million. **The model is worth about ₹590,000 over six months at the same effort**, or roughly ₹1.2 million a year. That's the sentence to put in front of Anita, with the two assumptions next to it.

Watch out: a threshold chosen on validation data is a validation-data threshold. Test it once on the test set, and expect the profit to be a little lower. And monitor it: as the mix of leads changes (Chapter 36 showed the marketplace share rising), the same threshold flags a different number of leads.

39.6 Imbalanced data

With 6.6% positives, the lead problem is **imbalanced**, and there's a large toolkit for that: class weights, undersampling the majority, oversampling the minority, and **SMOTE** (synthetic minority oversampling, which invents new minority rows between existing ones). Here's what each does to the lead model, with `imbalanced-learn`'s pipeline so that resampling happens **inside** training and never touches validation rows:

```
from imblearn.over_sampling import RandomOverSampler, SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.under_sampling import RandomUnderSampler
from sklearn.linear_model import LogisticRegression

def resampled(sampler):
    base = make_model(LEAD_CATS, LEAD_NUMS)
    return ImbPipeline(
        [
            ("prepare", base.named_steps["prepare"]),
            ("sample", sampler),
            ("model", LogisticRegression(max_iter=1000)),
        ]
    )

variants = {
    "plain logistic": make_model(LEAD_CATS, LEAD_NUMS),
    "class_weight='balanced'": make_model(
        LEAD_CATS, LEAD_NUMS, LogisticRegression(max_iter=1000, class_weight="balanced")
    ),
    "random undersampling": resampled(RandomUnderSampler(random_state=39)),
    "random oversampling": resampled(RandomOverSampler(random_state=39)),
    "SMOTE": resampled(SMOTE(random_state=39)),
}

print(f"{'variant':<26} ROC-AUC   PR-AUC   log loss   mean prob   wins in top 200")
for name, pipe in variants.items():
    p = pipe.fit(train[X_cols], train["won"]).predict_proba(valid[X_cols])[:, 1]
    top200 = y_valid[np.argsort(-p)[:200]].sum()
    print(
        f"{'name':<26} {roc_auc_score(y_valid, p):.3f}      "
        f"{'average_precision_score(y_valid, p):.3f}      "
        f"{'log_loss(y_valid, p):.4f}      {p.mean():.3f}      {top200}"
    )
```

variant	ROC-AUC	PR-AUC	log loss	mean prob	wins in top 200
plain logistic	0.823	0.302	0.1948	0.069	59
class_weight='balanced'	0.821	0.300	0.4899	0.353	61
random undersampling	0.814	0.286	0.5067	0.359	58
random oversampling	0.819	0.307	0.4927	0.355	61
SMOTE	0.820	0.297	0.4772	0.339	58

Reading it. Every variant ranks about the same: ROC-AUC 0.814–0.823, PR-AUC 0.286–0.307, and 58–61 wins in the top 200, all within noise. What changes is **the probabilities**: the average prediction jumps from 0.069 to about 0.35, and log loss gets much worse. Resampling teaches the model that wins are five times more common than they are, so it inflates every probability. That helps only if you were going to use a threshold of 0.5 and never think about it. If you choose the threshold by cost (section 39.5), rebalancing gives nothing, and it destroys calibration.

When rebalancing helps: with algorithms that struggle to learn the minority class at all (very deep trees, some neural networks), with extreme imbalance (fraud at 0.1%), and when the minority class is under-represented in *training* relative to production. For logistic regression and boosting on a 7% problem, the honest answer is: **fix the threshold, not the data**. Class weights are the cheapest option if you must, because they need no library and don't change the rows.

39.7 Interpretation: SHAP and partial dependence

Global: which features matter

SHAP values (SHapley Additive exPlanations) assign each feature a share of each prediction, so that base value + all shares = the prediction, in the model's own units (log-odds for a classifier). Averaging their absolute values over many rows gives a global importance that, unlike Chapter 37's impurity importance, is fair to features with few distinct values.

```
import shap

print("shap", shap.__version__)

def dense(matrix):
    return matrix.toarray() if hasattr(matrix, "toarray") else matrix

prepare = model.named_steps["prepare"]
feature_names = prepare.get_feature_names_out()
X_valid_prepared = dense(prepare.transform(valid[X_cols]))
explainer = shap.LinearExplainer(
    model.named_steps["model"], dense(prepare.transform(train[X_cols]))
)
shap_values = explainer.shap_values(X_valid_prepared)
importance = pd.Series(np.abs(shap_values).mean(axis=0), index=feature_names)
print("mean |SHAP| (log-odds units), top 8:")
print(importance.sort_values(ascending=False).head(8).round(3).to_string())
```

```
shap 0.52.0
mean |SHAP| (log-odds units), top 8:
cat__source_Marketplace    0.572
cat__company_size_1-10     0.317
cat__source_Website        0.270
cat__segment_Retail        0.265
num__log_quantity          0.243
num__free_email            0.241
num__activities_24h        0.235
num__text_strong           0.226
```

Reading it. For the lead model, the marketplace source matters most, then very small companies, website leads, and retail segment. The engineered features from Chapter 36 (quantity, free email, early activity, strong wording) follow. Nothing surprising, which is itself reassuring: a model whose top features nobody can explain is a model to distrust.

Local: why this lead scored 65%

```
row = np.argsort(-p_valid)[0] # the highest-scoring validation lead
contributions = pd.Series(shap_values[row], index=feature_names)
print(
    f"lead {valid.iloc[row]['lead_id']}: source {valid.iloc[row]['source']}, "
    f"segment {valid.iloc[row]['segment']}, predicted "
    f"{p_valid[row]:.1%}, actually won: {bool(y_valid[row])}"
)
print(
    f"base value (log-odds of an average lead) {explainer.expected_value:.3f} "
    f"= probability {1 / (1 + np.exp(-explainer.expected_value)):.3f}"
)
shown = contributions[contributions.abs() > 0.15].sort_values()
print(shown.round(3).to_string())
total = explainer.expected_value + contributions.sum()
print(
    f"base + all contributions = {total:.3f} -> probability {1 / (1 + np.exp(-total)):.3f}"
)
```

```
lead 108196: source Referral, segment Wholesale, predicted 64.6%, actually won: True
base value (log-odds of an average lead) -3.293 = probability 0.036
cat__product_interest_Industrial    -0.213
num__website_visits                  -0.197
num__responded_24h                   0.153
cat__source_Website                   0.170
cat__company_size_1-10               0.189
num__activities_24h                  0.201
cat__segment_Retail                  0.223
num__free_email                      0.237
num__log_quantity                    0.249
cat__company_size_200+               0.339
cat__source_Marketplace               0.467
cat__source_Referral                 0.647
num__text_strong                     0.693
base + all contributions = 0.603 -> probability 0.646
```

Reading it. An average lead sits at log-odds -3.293 , a 3.6% chance. This lead's largest pushes are strong wording in its enquiry (+0.69), coming through a referral (+0.65), not being a marketplace lead (+0.47; the one-hot column is 0, and the contribution is relative to the average lead, most of which are marketplace), and being a large company (+0.34). Two features pull it down slightly. Adding every contribution to the base gives 0.603, which is a probability of 64.6%, exactly what the model predicted. ✓ That's the sentence for the rep: "Referral, big company, and they asked for a bulk quote."

For a **linear model** these explanations are exact and cheap (`LinearExplainer`). For **tree models**, `TreeExplainer` is exact and fast. For anything else, `KernelExplainer` approximates and is slow; use it on a sample.

Partial dependence: what a feature does on average

```
sys.path.append("../ch37")
CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]
accounts["rep_id"] = accounts["rep_id"].astype(str)
acc_train, acc_test = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

churn_model = Pipeline(
    [
        # no scaling: trees don't need it, and raw units read better
        (
            "prepare",
            ColumnTransformer(
                [
                    (
                        "cat",
                        OneHotEncoder(handle_unknown="ignore", sparse_output=False),
                        CATS,
                    ),
                    ("num", SimpleImputer(strategy="median", add_indicator=True), NUMS),
                ]
            ),
        ),
        (
            "model",
            HistGradientBoostingClassifier(
                learning_rate=0.05,
                max_depth=3,
                min_samples_leaf=40,
                max_iter=300,
                random_state=37,
            ),
        ),
    ]
)
churn_model.fit(acc_train[CATS + NUMS], acc_train["churned_2025"])
prep_c = churn_model.named_steps["prepare"]
names_c = prep_c.get_feature_names_out()
Xc = prep_c.transform(acc_test[CATS + NUMS])
tree_explainer = shap.TreeExplainer(churn_model.named_steps["model"])
shap_c = tree_explainer.shap_values(Xc)
print("churn model, mean |SHAP|, top 6:")
print(
    pd.Series(np.abs(shap_c).mean(axis=0), index=names_c)
    .sort_values(ascending=False)
    .head(6)
    .round(3)
)
```

```
.to_string()
)
```

```
churn model, mean |SHAP|, top 6:
num__days_since_last_order    0.628
num__categories_bought         0.483
num__late_payment_days         0.289
num__tenure_months             0.282
num__complaints_2024           0.173
num__units_2024                0.151
```

```
from sklearn.inspection import partial_dependence

idx = list(names_c).index("num__days_since_last_order")
pd_result = partial_dependence(
    churn_model.named_steps["model"],
    Xc,
    features=[idx],
    grid_resolution=12,
    kind="average",
    method="brute",
    response_method="predict_proba",
)
print("days since last order -> average predicted churn probability")
for days, value in zip(pd_result["grid_values"][0], pd_result["average"][0]):
    print(f"{days:>6.0f} days    {value:.3f}")
```

```
days since last order -> average predicted churn probability
  2 days    0.065
 24 days    0.047
 45 days    0.039
 67 days    0.036
 88 days    0.066
110 days    0.152
132 days    0.141
153 days    0.142
175 days    0.176
196 days    0.385
218 days    0.385
240 days    0.349
```

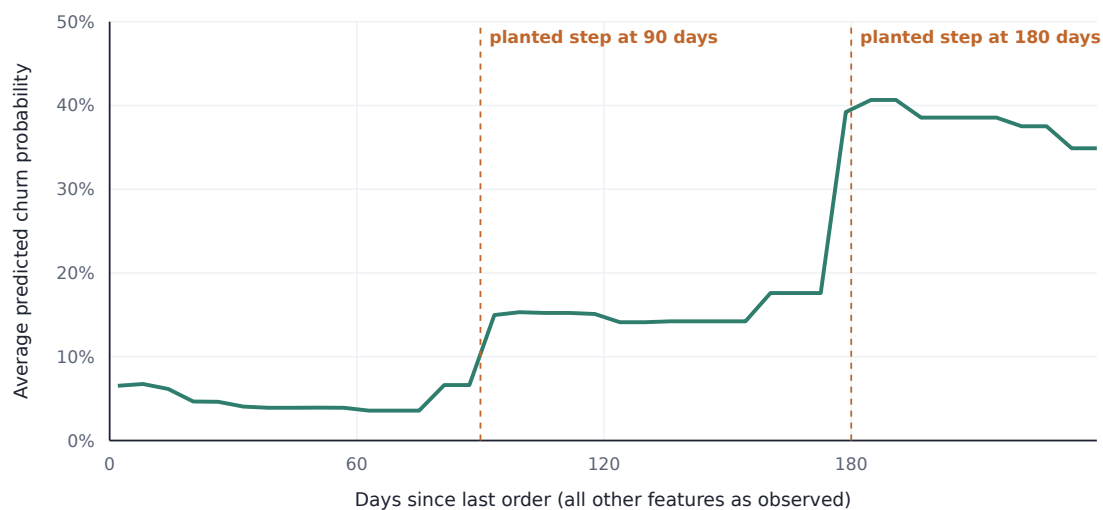


Figure 39.5 — Partial dependence of predicted churn on days since last order, from the gradient-boosting churn model. The two steps at about 90 and 180 days are the thresholds Chapter 37 built into the data, recovered by the model without being told.

Reading it. On the churn model, SHAP ranks days since last order first and product breadth second, as the odds ratios in Chapter 37 did. **Partial dependence** goes further: it fixes one feature at a series of values, averages the model's predictions over all accounts, and shows the feature's effect. Churn risk is flat at about 4–6% for accounts that ordered within 90 days, steps to about 15% between 100 and 180 days, and steps again to about 38% beyond 190 days. Those two steps are exactly what the generator planted. A partial dependence plot is how you check whether a black-box model has learned something sensible or something strange.

Watch out: interpretation is not causation. SHAP explains the *model*, not the customer. "Late payments push churn risk up" means the model uses late payments as a signal, not that reducing late payments reduces churn (Chapter 31). And partial dependence averages over unrealistic combinations (a wholesaler with a hospitality discount); `IndividualConditionalExpectation` plots and SHAP dependence plots show the spread.

39.8 Fairness checks

A model that scores one group of people systematically differently deserves a look, whether the group is defined by protected characteristics, by geography, or, as here, by who handles the lead.

Chapter 36 noted that leads assigned to the inside sales desk convert less often. Compare the two groups at the working threshold of 0.05:

```

valid_scored = valid.assign(p=p_valid, called=(p_valid >= 0.05))
groups = valid_scored.groupby("inside_desk")
report = pd.DataFrame(
    {
        "leads": groups.size(),
        "win rate": groups["won"].mean(),
        "mean score": groups["p"].mean(),
        "share called": groups["called"].mean(),
        "recall": groups.apply(lambda g: g.loc[g["won"] == 1, "called"].mean()),
        "precision": groups.apply(lambda g: g.loc[g["called"], "won"].mean()),
    }
)
report.index = ["sales reps", "inside sales desk"]
print(report.round(3).to_string())

```

	leads	win rate	mean score	share called	recall	precision
sales reps	1289	0.086	0.094	0.552	0.910	0.142
inside sales desk	936	0.037	0.035	0.205	0.571	0.104

Reading it. The model flags 55% of the reps' leads for follow-up and only 21% of the inside desk's. Recall differs sharply: it catches 91% of the reps' eventual wins but only 57% of the desk's. Precision is closer (14% against 10%).

Is that unfair? Three views, and they conflict:

- **The base rates differ.** Inside-desk leads are won 3.7% of the time, reps' leads 8.6%. A calibrated model *must* give lower scores to a group with a lower win rate; if it didn't, it would be miscalibrated for one of them. On this view the model is fair.
- **Equal opportunity** asks whether wins are equally likely to be *caught* in both groups. They aren't: 91% against 57%. Winnable inside-desk leads are being missed at twice the rate.
- **The feedback loop** (Chapter 36, exercise 15): if the desk's leads are flagged less, worked less, and therefore won less, the next model learns an even lower rate for them. The gap can be partly the model's own past doing.

There's no formula that settles this; it's a decision about what Riverstone owes each lead. Reasonable options include a separate threshold for the desk's leads, a floor on how many desk leads are worked, or an experiment (Chapter 30) to learn whether the desk's leads convert better when worked properly. What isn't reasonable is not checking.

```

by_segment = valid_scored.groupby("segment").apply(
    lambda g: pd.Series(
        {
            "leads": len(g),
            "win rate": g["won"].mean(),
            "share called": g["called"].mean(),
            "recall": g.loc[g["won"] == 1, "called"].mean(),
            "false positive rate": g.loc[g["won"] == 0, "called"].mean(),
        }
    )
)
print(by_segment.round(3).to_string())

```

segment	leads	win rate	share called	recall	false positive rate
Hospitality	728.0	0.049	0.419	0.861	0.396
Retail	941.0	0.051	0.317	0.812	0.290
Wholesale	448.0	0.121	0.571	0.815	0.538

By segment, the picture is what you'd expect from a calibrated model with different base rates: wholesale leads (12% win rate) are flagged more and have a higher false positive rate. Recall is similar across segments (81–86%). Report a table like this for every grouping that could matter, including ones you'd rather not look at.

Fairness vocabulary. *Demographic parity*: the same share flagged in each group. *Equal opportunity*: the same recall in each group. *Predictive parity*: the same precision in each group. *Calibration within groups*: the probabilities are honest for each group separately. When base rates differ, these cannot all hold at once; that's a mathematical fact, and choosing between them is a policy decision.

39.9 Model cards

A **model card** is a one-to-two-page document that travels with a model: what it's for, what it was trained on, how it performs, where it fails, and who to call. Here is the lead-scoring model's, condensed:

Section	Lead-scoring model, v1
Purpose	Rank incoming enquiries so reps work the most promising first. Not for pricing, not for deciding which leads to ignore permanently.
Owner	Meera Iyer (analytics); business owner Anita Rao (sales)
Training data	Riverstone CRM, enquiries created Jan 2023–Dec 2024 (7,291 leads, 7.9% won); features as of 24 hours after arrival; duplicates removed; outcome window 90 days
Evaluation	Validation Jan–Jun 2025 (2,225 leads): ROC-AUC 0.823, PR-AUC 0.302, log loss 0.195, well calibrated. Test Jul–Sep 2025 (1,185 leads): ROC-AUC 0.838. Top 50 leads by score: 44% won
Decision rule	Work leads scoring ≥ 0.05 (break-even at ₹1,500 per lead, ₹30,255 per win), capped by capacity of 4 leads per rep-day. Threshold reviewed quarterly
Known limitations	Lower recall on inside-sales-desk leads (57% vs 91%); trained before the 2025 marketplace change, which lowered marketplace conversion; assumes the outcome window and follow-up process stay the same
Fairness	Checked by segment, city, source, and owner (section 39.8); inside-desk gap flagged for policy decision
Monitoring	Weekly: share of leads flagged, predicted vs actual win rate for closed leads; retrain if the calibration drift exceeds 2 points (Chapter 52)
Do not use for	Individual performance reviews of reps; any decision without a human in the loop

The card is short on purpose. Its value is that the "known limitations" and "do not use for" rows exist at all; most model failures in business come from a model being used for something it was never evaluated for.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reporting accuracy on an imbalanced problem	"93.5% accurate" for a model that flags 7 leads	Report precision, recall, PR-AUC, and the base rate
Leaving the threshold at 0.5	Almost nothing flagged	Choose the threshold from costs or capacity
Reporting only ROC-AUC	Looks great, team disappointed	Add PR-AUC and the top-N table
Comparing R^2 across different targets	"Log model beats rupee model" from R^2 alone	Compare on one scale, in the business's units
Quoting RMSE without looking at outliers	One account drives the number	Report MAE and the largest errors too
Using probabilities from an uncalibrated model	"94% chance" for a lead won one time in four	Check a reliability curve; calibrate if needed
Rebalancing classes then trusting the probabilities	Average prediction of 35% on a 7% problem	Set the threshold by cost instead; if you rebalance, recalibrate
Choosing a threshold on the test set	Test profit is optimistic	Choose on validation or CV; test once
Reading SHAP as cause	"Late payments cause churn"	SHAP explains the model; causes need Chapter 31
Impurity importance as the explanation	Continuous features look most important	Use SHAP or permutation importance
No fairness check	Surprise when a group complains	Recall, precision, and flag rate by every relevant group
Demanding all fairness measures at once	"The model must be equal on everything"	With different base rates that's impossible; choose and document
No model card	The model gets used for something else	Write the card before deployment
Treating validation numbers as forever	Flag rate drifts as the lead mix changes	Monitor calibration and flag rate; retrain on a schedule

In the real world: the rep who scored lower

In June 2026, two months after the lead model goes live, Farah Khan asks for a meeting. Her leads, she says, are "always scored lower than Rahul's", and since the call list is now ordered by score, she's working fewer good leads and her numbers are down.

Meera checks. Farah's leads do score lower on average: 7.1% against Rahul's 9.4%. The model doesn't use the rep's name, so the difference has to come through the features. It does: Farah was given most of the marketplace leads this year, and marketplace leads convert at 1.5%. The model is calibrated within each rep's leads (Farah's leads scored 7–10% are won about 8% of the time). It isn't scoring *her*; it's scoring what she was handed.

That's the honest finding, and it's not the end of the meeting. Two things follow. First, the lead **assignment** was the unfair part, not the model, and the model made a hidden unfairness visible: the score gap is the first hard evidence of it. Vikram rebalances the marketplace leads across reps the same week. Second, Meera adds the check to the model card and the weekly monitoring: mean score and recall by rep, so the next such gap is seen in a dashboard before anyone has to raise it in a meeting.

Farah's numbers recover over the next quarter. She also becomes the model's most useful critic, because she's the one who reads the top-N table every Monday and says when the list feels wrong.

The lesson isn't that the model was right. It's that the evaluation work in this chapter, calibration within groups and recall by group, is what made a fair conversation possible.

Tools

- **scikit-learn** (tested on 1.8.0; current release 1.9.1): `confusion_matrix`, `precision_score`, `recall_score`, `f1_score`, `roc_curve`, `roc_auc_score`, `precision_recall_curve`, `average_precision_score`, `mean_absolute_error`, `root_mean_squared_error`, `mean_absolute_percentage_error`, `r2_score`, `calibration_curve`, `brier_score_loss`, `CalibratedClassifierCV`, `partial_dependence`, and `permutation_importance`.
- **shap** 0.52.0 (`pip install shap`): `LinearExplainer`, `TreeExplainer`, `KernelExplainer`, and its plotting functions (`summary_plot`, `waterfall`), which are the usual way to show SHAP values in a report.
- **imbalanced-learn** 0.14.2 (`pip install imbalanced-learn`): `RandomUnderSampler`, `RandomOverSampler`, `SMOTE`, and a `Pipeline` that resamples only during `fit`.
- **fairlearn** (not used here) provides group-metric tables and mitigation methods if fairness checks become routine.
- Everything ran on one CPU core, Python 3.12.3, on 18 September 2026.
- **Companion files:** `companion/ch39/lead_data.py` (a copy of Chapter 37's helper) rebuilds the lead table. Run the chapter's code from `companion/ch39/`; it also reads `../accounts/accounts.csv`.

The project: cost-based evaluation and explanation of the lead-scoring model

Goal: an evaluation report for a model that a sales head could act on: how many leads to work, what it's worth, where it's uncertain, whom it might treat unfairly, and a model card.

Option A: your own data. Any classifier you built with Chapter 36's workflow, plus two cost numbers you can defend.

Option B: Riverstone. The lead model and validation set.

Steps:

1. **Build the confusion matrix** at 0.5 and at two other thresholds, and compute precision, recall, F1, and specificity by hand for one of them.
2. **Draw ROC and PR curves** and report both areas with the base rate.
3. **Make the top-N table** for $N = 25, 50, 100, 200, 400$.
4. **Check calibration** with a reliability curve and the Brier score. If you have a second model, compare.
5. **Set costs:** write down the cost of a false positive and the value of a true positive, with your reasoning, as named variables.
6. **Find the threshold** by profit curve and by break-even, and state the range where profit is within 5% of the best.
7. **Add a capacity constraint** and compare the model with the current rule of thumb, in money.
8. **Explain** the model globally (mean |SHAP|) and for three individual leads: the highest-scoring, one at the threshold, and one the model got wrong.
9. **Check fairness** by at least two groupings: flag rate, recall, and precision per group, with a paragraph on what you'd recommend.
10. **Write the model card.**

Stretch goals:

- Test the chosen threshold **on the test set once** and compare the profit with validation.
- Build a **cost-sensitive** version by passing `sample_weight` during training and compare with threshold tuning.
- Use `permutation_importance` on validation data and compare its ranking with SHAP's.
- Draw **SHAP dependence plots** for the two most important features and look for interactions.

You've got it when...

- ☐ I can fill in a confusion matrix from a threshold and compute precision, recall, F1, and specificity by hand.
- ☐ I never report accuracy alone on an imbalanced problem, and I always state the base rate.
- ☐ I can explain why ROC-AUC flatters and PR-AUC doesn't on imbalanced data, and I make the top-N table.
- ☐ I know what MAE, RMSE, MAPE, and R^2 each hide, and I match the metric to what errors cost.
- ☐ I check calibration with a reliability curve before anyone uses a probability as a number, and I can recalibrate a model.

- ☐ I set thresholds from break-even costs, profit curves, or capacity, never from 0.5.
 - ☐ I know that class weights and resampling change probabilities more than rankings, and I recalibrate if I use them.
 - ☐ I can explain a model's top features with SHAP and a single prediction as base value plus contributions.
 - ☐ I can read a partial dependence plot and use it to sanity-check a black-box model.
 - ☐ I run fairness checks by group, and I can explain why calibration, equal opportunity, and predictive parity can't all hold when base rates differ.
 - ☐ I write a model card before a model is used.
-

Recap

- The **confusion matrix** (TP, FP, FN, TN) is the source of **precision**, **recall**, **F1**, and **specificity**. **Accuracy** is misleading whenever classes are imbalanced.
 - **ROC-AUC** measures ranking and ignores the base rate; **PR-AUC** (average precision) reflects what users experience. The **top-N table** is the most useful evaluation for a list.
 - **MAE** is the plain-language error; **RMSE** punishes outliers; **MAPE** breaks near zero and is biased; **R²** depends on the scale. Match the metric to the cost of errors.
 - **Calibration** asks whether predicted probabilities are honest. Check with a **reliability curve** and the **Brier score**; fix with `CalibratedClassifierCV` (Platt or isotonic). Calibration fixes numbers, not rankings.
 - Choose the **threshold** from **break-even** (cost ÷ value), a **profit curve**, or **capacity**. Quote the range, and re-test.
 - **Imbalance** tools (class weights, over- and undersampling, **SMOTE**) mostly inflate probabilities; on a cost-chosen threshold they add little. Fix the threshold first.
 - **SHAP** explains predictions as base value plus feature contributions; **partial dependence** shows a feature's average effect. Both explain the model, not the world.
 - **Fairness**: check flag rate, recall, and precision by group; with different base rates, the fairness definitions conflict, and choosing is a policy decision.
 - A **model card** records purpose, data, performance, limitations, and what the model must not be used for.
-

Practice exercises

Code exercises run from `companion/ch39/` after the chapter's code (they use `p_valid`, `y_valid`, `valid`, `model`, `profit_at`, `WIN_VALUE`, `WORK_COST`, `probs`, `shap_values`, `feature_names`, and the rest). Predict each answer before running it.

Warm-up

1. (hand) A fraud model on 10,000 transactions flags 300; 120 of the flagged are fraud, and 80 frauds were missed. Fill in the confusion matrix and compute precision, recall, and the false positive rate.
2. (hand) Model A has precision 0.9 and recall 0.1; model B has 0.4 and 0.4. Compute both F1 scores. Which would you prefer for (a) a cancer screening test and (b) blocking a credit card?
3. (hand) A win is worth ₹12,000 and following up costs ₹900. What's the break-even probability? Should a lead scored 6% be worked?
4. A forecast has MAE ₹10,000 and RMSE ₹50,000 on the same test set. What does the gap tell you, and what would you look at next?

Core

5. Compute the confusion matrix, precision, and recall at thresholds 0.05, 0.077, and 0.15, and explain the pattern in one sentence.
6. Make the top-N table for the **gradient boosting** probabilities in `probs` and compare it with logistic regression's. Which model would the sales team prefer, and why is the answer not obvious?
7. Change `WORK_COST` to ₹3,000 (a more expensive follow-up process) and recompute the break-even threshold and the profit-maximizing threshold. How much does the best profit fall?
8. Compute the Brier score for a model that predicts the base rate (0.066) for every lead, and for one that predicts 0.5 for every lead. Compare with the three models in section 39.4.
9. Find the highest-scoring **lost** lead in the validation set and explain its score with SHAP. What pushed it up, and would you call the model wrong?
10. Compute recall and precision by `source` at the 0.05 threshold. Which source has the lowest recall, and does the base-rate argument from section 39.8 explain it?

Stretch

- l1. Fit `CalibratedClassifierCV` with `method="sigmoid"` on Naive Bayes and compare its log loss and Brier score with the isotonic version. Which is better here, and why might sigmoid be safer with less data?
- l2. Use `permutation_importance` on the validation set (scoring `"neg_log_loss"`, 10 repeats) and compare its top five features with SHAP's. Where do they disagree, and why can they?
- l3. Apply the 0.077 threshold to the **test** set (train on training plus validation, as Chapter 36 did) and compute profit, worked leads, and wins. How does it compare with validation?

Think about it

- l4. A colleague says: "Our churn model is 91% accurate and the churn rate is 9%." What's your first question, and what two numbers do you ask for instead?
- l5. Marketing wants to show customers "you have a 70% chance of being approved". Which sections of this chapter must you complete before that's responsible, and what would you check monthly?
- l6. The inside sales desk's leads have a recall of 57% against 91% for reps' leads. Propose one policy that addresses it, and say what it costs.

Key terms

confusion matrix · true positive · false positive · false negative · true negative · threshold · accuracy · precision · recall (sensitivity, true positive rate) · specificity · false positive rate · F1 score · harmonic mean · ROC curve · ROC-AUC · precision–recall curve · PR-AUC (average precision) · base rate · top-N table · MAE · RMSE · MAPE · WAPE · R^2 · calibration · reliability curve · Brier score · Platt scaling · isotonic regression · `CalibratedClassifierCV` · break-even threshold · profit curve · capacity constraint · expected value · class imbalance · class weights · undersampling · oversampling · SMOTE · SHAP · Shapley value · base value · local explanation · global importance · permutation importance · partial dependence · ICE plot · fairness · demographic parity · equal opportunity · predictive parity · calibration within groups · feedback loop · model card · monitoring · drift

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 40, Time Series & Forecasting**, measures forecast accuracy with MAPE and WAPE, and backtests over time instead of a single split.
 - **Chapter 42, Recommender Systems & Ranking**, uses precision@k and NDCG, the ranking cousins of the top-N table.
 - **Chapter 44, Capstone**, ends with a model card and a presentation to non-technical leaders.
 - **Chapter 52, Deploying and Monitoring Models**, turns the monitoring row of the model card into dashboards and retraining rules.
 - **Chapter 30, Experiments**, is how threshold changes, fairness fixes, and win-back campaigns are proved to work.
 - **Chapter 31, Causal Inference**, is the answer when someone reads a SHAP plot as a cause.
 - **Chapter 61, Ethics and Responsible Data Practice**, covers fairness and model documentation in depth.
 - **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers precision versus recall, ROC versus PR, calibration, threshold selection, SMOTE, SHAP, and "how would you explain this model to a manager?", which is asked in nearly every data science interview.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Flagged 300, of which 120 fraud: TP = 120, FP = 180. Missed frauds: FN = 80. Total fraud = 200, so TN = 10,000 – 300 – 80 = **9,620**. Precision = $120 \div 300 = \mathbf{0.40}$; recall = $120 \div 200 = \mathbf{0.60}$; false positive rate = $180 \div (180 + 9,620) = \mathbf{0.018}$. The base rate is 2%, so a random flag would have 2% precision; 40% is twenty times better, and the team still investigates 180 innocent transactions for every 120 frauds.

2. $F1(A) = 2 \times 0.9 \times 0.1 \div 1.0 = \mathbf{0.18}$; $F1(B) = 2 \times 0.4 \times 0.4 \div 0.8 = \mathbf{0.40}$. (a) Screening wants **recall**: missing a cancer is far worse than a follow-up test, so B, and ideally a model with even higher recall. (b) Blocking a card wants **precision**, because every false block angers a customer and costs support time, so A's precision is attractive, but its 10% recall means it stops almost no fraud. Neither is good; the exercise shows why F1 alone can't choose, and why costs (section 39.5) are the better guide.

3. Break-even = $900 \div 12,000 = \mathbf{0.075}$. A lead at 6% has expected value $0.06 \times 12,000 - 900 = 720 - 900 = -\mathbf{₹180}$, so on average it loses money and shouldn't be worked, unless there's spare capacity that would otherwise be idle, in which case the cost is really lower than ₹900.

4. RMSE five times MAE means a few very large misses dominate; if errors were similar in size the two would be close (for normally distributed errors, RMSE is about $1.25 \times \text{MAE}$). Look at the ten largest errors: are they data problems (a wrong unit, a duplicated order), a segment the model doesn't understand, or unpredictable events? Then decide whether the business cares about those tails, which decides whether RMSE or MAE is the number to optimize.

5.

```
for threshold in [0.05, 0.077, 0.15]:
    tp, fp, fn, tn = matrix_at(threshold)
    print(
        f"threshold {threshold:<5} TP {tp:>3} FP {fp:>4} FN {fn:>3}  "
        f"precision {tp / (tp + fp):.3f}    recall {tp / (tp + fn):.3f}"
    )
```

threshold 0.05	TP 121	FP 783	FN 25	precision 0.134	recall 0.829
threshold 0.077	TP 112	FP 515	FN 34	precision 0.179	recall 0.767
threshold 0.15	TP 76	FP 215	FN 70	precision 0.261	recall 0.521

Raising the threshold trades recall for precision: fewer leads flagged, a higher share of them won, but more wins missed. Between 0.05 and 0.077 the model gives up 9 wins to save 277 follow-ups, which section 39.5 showed is worth it; between 0.077 and 0.15 it gives up another 36 wins to save 336 follow-ups, which isn't.

6.

```
p_boost = probs["gradient boosting"]
order_b = np.argsort(-p_boost)
print("      N    logistic wins    boosting wins")
for k in [50, 100, 200, 400]:
    print(
        f"{k:>6}    {y_valid[np.argsort(-p_valid)[:k]].sum():>13}  "
        f"    {y_valid[order_b[:k]].sum():>13}"
    )
```

N	logistic wins	boosting wins
50	22	23
100	38	33
200	59	50
400	92	80

The lists tie at the top 50 and logistic regression pulls ahead below that: 92 wins against 80 in the top 400, 12 wins out of 146. That's a real-looking gap, and still within what a different six months could reverse. The sales team would pick the list that looks better this quarter, which is a weak basis: choose on calibration, speed, and explainability (Chapter 37's reasons), and if it matters, run both for a period and compare (Chapter 30).

7.

```
WORK_COST = 3000
print(f"break-even probability: {WORK_COST / WIN_VALUE:.4f}")
profits_3k = np.array([profit_at(t)[0] for t in grid])
print(
    f"best threshold {grid[profits_3k.argmax():.3f]    profit ₹{profits_3k.max():.0f}    "
    f"(was ₹{profits.max():.0f} at ₹1,500 per lead)"
)
WORK_COST = 1500
```

```
break-even probability: 0.0992
best threshold 0.103    profit ₹1,633,500    (was ₹2,448,060 at ₹1,500 per lead)
```

Doubling the cost of working a lead doubles the break-even probability (to about 0.099) and moves the profit-maximizing threshold up to match, and the best profit falls by roughly ₹0.9 million. The threshold is a function of the costs, which is the point: when the process changes, re-run this, not the model.

8.

```
for name, p in [
    ("base rate for everyone", np.full(len(y_valid), y_valid.mean())),
    ("0.5 for everyone", np.full(len(y_valid), 0.5)),
]:
    print(f"{name:<24} Brier {brier_score_loss(y_valid, p):.4f}")
```

```
base rate for everyone    Brier 0.0613
0.5 for everyone          Brier 0.2500
```

Predicting the base rate scores 0.0613, and the three real models score 0.053 (logistic and boosting) to 0.124 (Naive Bayes). So logistic regression beats the know-nothing benchmark by a modest margin, and Naive Bayes is **worse than predicting the base rate**: a constant 6.6% would be a better probability than its confident numbers. Predicting 0.5 for everyone scores 0.25, the worst possible constant.

9.

```
lost = np.where(y_valid == 0)[0]
worst = lost[np.argmax(p_valid[lost])]
lead = valid.iloc[worst]
print(
    f"lead {lead['lead_id']}: {lead['source']}, "
    f"{lead['segment']}, size {lead['company_size']}, "
    f"predicted {p_valid[worst]:.1%}, lost"
)
pushes = pd.Series(shap_values[worst], index=feature_names)
print(pushes[pushes.abs() > 0.2].sort_values(ascending=False).round(3).to_string())
```



```

lead 107979: Trade fair, Wholesale, size 51-200, predicted 56.1%, lost
num__text_strong          0.693
cat__source_Marketplace   0.467
num__log_quantity         0.411
num__website_visits       0.329
num__free_email           0.237
cat__source_Trade fair    0.235
cat__segment_Retail       0.223
num__activities_24h       0.201
cat__product_interest_Industrial -0.213

```

The lead has every sign of a good one, which is why it scored high: the same features that mark most won leads. It was still lost. That isn't a model error in any useful sense: a well-calibrated 50–60% prediction means the lead is lost about half the time, and this is one of those. "Wrong" would be a pattern: a group of high-scoring leads that lose far more often than their scores say, which is what the reliability curve checks.

10.

```

by_source = valid_score.groupby("source").apply(
    lambda g: pd.Series(
        {
            "leads": len(g),
            "win rate": g["won"].mean(),
            "share called": g["called"].mean(),
            "recall": g.loc[g["won"] == 1, "called"].mean(),
            "precision": (
                g.loc[g["called"], "won"].mean() if g["called"].any() else np.nan
            ),
        }
    )
)
print(by_source.sort_values("recall").round(3).to_string())

```

	leads	win rate	share called	recall	precision
source					
Marketplace	1000.0	0.014	0.155	0.214	0.019
Website	554.0	0.065	0.401	0.722	0.117
Cold call	140.0	0.086	0.586	0.750	0.110
Trade fair	233.0	0.133	0.837	0.968	0.154
Referral	194.0	0.211	0.928	1.000	0.228
Partner	104.0	0.115	0.673	1.000	0.171

Marketplace leads have the lowest recall by far, and the lowest base rate: a calibrated model gives most of them scores below 5%, so the winnable ones are missed along with the rest. The base-rate argument explains the number, and the equal-opportunity view still objects to it, exactly as with the inside sales desk (which handles most marketplace leads). The two findings are the same finding seen through two groupings.

11.

```

sigmoid_nb = CalibratedClassifierCV(nb, method="sigmoid", cv=5).fit(
    train[X_cols], train["won"])
)
p_sig = sigmoid_nb.predict_proba(valid[X_cols])[:, 1]
print(
    f"sigmoid: log loss {log_loss(y_valid, p_sig):.4f} "
    f" Brier {brier_score_loss(y_valid, p_sig):.4f}"
)
print(
    f"isotonic: log loss {log_loss(y_valid, p_nb_cal):.4f} "
    f" Brier {brier_score_loss(y_valid, p_nb_cal):.4f}"
)

```

```

sigmoid: log loss 0.2063    Brier 0.0552
isotonic: log loss 0.2022    Brier 0.0544

```

Isotonic does slightly better here, because Naive Bayes's distortion isn't a simple S-shape and isotonic can bend to fit it. Sigmoid fits only three parameters, so it can't overfit the calibration data; with a few hundred positives or fewer, isotonic's step function can chase noise and make calibration *worse* on new data. Rule of thumb: sigmoid below about a thousand positives, isotonic above.

12.

```

from sklearn.inspection import permutation_importance

perm = permutation_importance(
    model, valid[X_cols], y_valid, scoring="neg_log_loss", n_repeats=10, random_state=39
)
perm_series = pd.Series(perm.importances_mean, index=X_cols).sort_values(
    ascending=False
)
print("permutation importance (increase in log loss when shuffled), top 5:")
print(perm_series.head(5).round(4).to_string())

```

```

permutation importance (increase in log loss when shuffled), top 5:
source          0.0334
company_size    0.0092
log_quantity    0.0073
segment         0.0034
activities_24h  0.0029

```

Permutation importance works on the **raw columns**, so `source` appears once, while SHAP scores each one-hot column (marketplace, website, referral) separately; that's the main reason the lists look different. They can also disagree in substance: permutation importance measures how much the score degrades without a feature, so a feature that's highly correlated with another can look unimportant (the other covers for it), while SHAP still assigns it a share. Neither is wrong; they answer different questions.

13.

```

final_model = make_model(LEAD_CATS, LEAD_NUMS).fit(
    pd.concat([train, valid])[X_cols], pd.concat([train, valid])["won"]
)
p_test = final_model.predict_proba(test[X_cols])[:, 1]
y_test = test["won"].to_numpy()
profit_t, worked_t, wins_t = profit_at(0.077, p_test, y_test)
print(
    f"test set ({len(test):,} leads, {y_test.sum():,} "
    f"won): {worked_t} worked, {wins_t} wins, "
    f"profit ₹{profit_t:,.0f}"
)
print(
    f"per lead: ₹{profit_t / len(test):,.0f} on test vs "
    f"₹{profits.max() / len(valid):,.0f} on validation"
)

```

```

test set (1,185 leads, 104 won): 332 worked, 80 wins, profit ₹1,922,400
per lead: ₹1,622 on test vs ₹1,100 on validation

```

Per lead, the test profit is higher than validation's, because the test period's win rate is higher (8.8% against 6.6%) and the model, retrained on more data, ranks it well. The reassurance is that it didn't fall: the threshold chosen on validation transferred. A large drop here would have meant the validation threshold was tuned to noise.

14. First question: "What does predicting 'no churn' for everyone score?" At a 9% churn rate, that's 91% accurate, the same as the model, so the headline says nothing. Ask for **recall at the threshold you'd act on** (what share of churners does it catch?) and **precision** (of the accounts flagged, how many churn?), or equivalently a top-N table: of the 100 highest-risk accounts, how many churned?

15. Sections 39.4 and 39.8, at minimum: the probability must be **calibrated** (a reliability curve, and a Brier score against the base rate), and it must be calibrated **within each group** the customer might belong to, or the number is a lie to some of them. Section 39.9's model card should say what the number means and what it's not for. Monthly: the reliability curve on recently decided applications, the mean predicted against the actual approval rate overall and by group, and the share of customers shown each band of probability, since a drift in the applicant mix will move all of these before anyone notices.

16. One option: **a separate threshold for inside-desk leads**, set so their recall matches the reps' (around 91%). Cost: the desk would work many more leads with lower precision, so the follow-up cost per win rises; at a 3.7% base rate that may mean working most desk leads, which is what happened before the model. A cheaper alternative is a **floor**: the desk always works its top 15% of leads by score, whatever the threshold says, which bounds the cost. The honest answer to "what does it cost" is a number from `profit_at` under each policy, which is exercise 7's method applied to one group.

Chapter 40. Time Series & Forecasting

Part IV – Machine Learning & Data Science

Chapter at a glance

You will learn to: break a time series into trend, seasonality, cycle, and noise · test for stationarity and know why it matters · build the baselines every forecast must beat, by hand · run exponential smoothing and Holt–Winters, and see the smoothing step on paper · read ACF and PACF plots and fit a SARIMA model · forecast with a machine learning model on lag features without leaking the future · backtest a forecast the way it will be used, and read the spread of errors · measure accuracy with MAPE, WAPE, and MASE, and know which to quote · turn a forecast into a production plan with safety stock · find anomalies in sensor data, including the kind that point-by-point checks miss.

Before you start: Chapter 21 (means, standard deviations, autocorrelation as an idea), Chapter 36 (splits and leakage), Chapter 37 (gradient boosting), and Chapter 39 (accuracy metrics). Chapter 18's pandas date handling is used throughout.

Time needed: 10–14 hours over two weeks.

Tools: Python 3 with pandas, statsmodels, scikit-learn, and Prophet (all free).

Practice data: two new Riverstone datasets built by seeded generators. **Demand:** units shipped per product category, weekly (365 weeks) and monthly (84 months), January 2019 to December 2025. **Sensor readings:** one week of one-minute readings from one injection-moulding machine (10,080 rows; the same generator builds the 4.8-million-row version used in Chapter 48). Every number in this chapter was calculated, and every output shown is real.

Why this matters

Every business runs on forecasts, most of them made in a spreadsheet by someone who copies last year's number and adds 10%. That method is better than it sounds, and one of the lessons of this chapter is that beating it is harder than most people expect.

- The production planner needs next quarter's demand per category to buy resin and schedule machines. Too low and orders ship late; too high and the warehouse fills with unsold stock.
- Finance needs a revenue forecast with an honest range around it, not a single number that will be wrong.
- The maintenance team wants to know that a machine's heater is failing before the shift supervisor notices scrap.
- An interviewer will ask what makes time series different from other data, and "the rows are in order" is a beginning, not an answer.

Time series break the assumption that most of Part IV relied on: that rows are independent. Yesterday predicts today. That's what makes forecasting possible, and it's what makes every shortcut from Chapter 36 (random splits, cross-validation, features that peek forward) fail quietly. This chapter is about doing it properly, and about measuring whether the effort beat the spreadsheet.

In plain English

Think of a shopkeeper who has kept a diary of daily sales for seven years.

They know the shop is busier every year (**trend**), that October and November are the peak because of the festive season (**seasonality**), that there are good years and bad years that last a while (**cycles**), and that any single day is unpredictable (**noise**). Reading the diary to separate those four is **decomposition**.

Asked what next October will bring, the shopkeeper's first answer is "about what last October brought, plus a bit for growth". That's the **seasonal naive** forecast, and it's surprisingly good. A more careful answer weighs recent months more than old ones (**exponential smoothing**), or notices that a good month tends to follow a good month (**autocorrelation**, the idea behind **ARIMA**).

To know whether a method works, the shopkeeper doesn't check it on last week; they ask, "if I'd used this method at the start of each of the last five years, how wrong would I have been?" That's **backtesting**. And "how wrong" is measured in units short or over across the whole year, not by the worst single day: that's **WAPE**.

Finally, if the shop's fridge thermometer reads 3 degrees higher every hour, the shopkeeper wants to know before the milk goes off, even though no single reading looks alarming. That's **anomaly detection in sensor data**, and the slow kind is the hard kind.

40.1 What a time series is made of

The data

```
import logging
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
logging.getLogger("cmdstanpy").setLevel(logging.ERROR)

weekly = pd.read_csv("../demand/weekly_demand.csv", parse_dates=["week_start"])
monthly = pd.read_csv("../demand/monthly_demand.csv", parse_dates=["month"])
print(
    f"weekly: {len(weekly):,} rows, {weekly['week_start'].min().date()} "
    f"to {weekly['week_start'].max().date()}"
)
print(
    "weekly problems: missing units",
    weekly["units"].isna().sum(),
    "| duplicated weeks",
    weekly.duplicated(["category", "week_start"]).sum(),
)
weekly = weekly.drop_duplicates(["category", "week_start"])
weekly["units"] = weekly.groupby("category")["units"].transform(
    lambda u: u.interpolate()
)

kitchen = (
    monthly[monthly["category"] == "Kitchen"]
    .set_index("month")["units"]
    .astype(float)
    .asfreq("MS")
)
print(f"\nKitchen, monthly: {len(kitchen)} months")
print(kitchen.groupby(kitchen.index.year).sum().astype(int).to_string())
```

```
weekly: 1,461 rows, 2019-01-07 to 2025-12-29
weekly problems: missing units 1 | duplicated weeks 1
```

```
Kitchen, monthly: 84 months
```

month	
2019	656605
2020	611652
2021	728878
2022	814600
2023	932607
2024	962052
2025	1051402

Kitchen products are Riverstone's largest category, and the monthly series is the one this chapter works with most. Two things in the weekly file were planted to be found: one missing week and one duplicated week. Real demand exports have both, and a forecasting model fed a duplicated week will learn a spike that never happened.

Four components

A time series is usually described as the combination of:

- **Trend:** the long-run direction. Kitchen units grew from 657,000 in 2019 to 1,051,000 in 2025.
- **Seasonality:** a pattern that repeats with a fixed period. Riverstone's peak is October–November, ahead of Diwali, when hotels and retailers stock up.
- **Cycles:** slower swings with no fixed period, such as the economy. Harder to model, often ignored.
- **Noise** (the residual): whatever is left after the others.

Decomposition estimates each part. The multiplicative version treats seasonality as a *factor* ("October is 1.43 times an average month"), which suits data that grows, because the seasonal swing grows with it:

```
from statsmodels.tsa.seasonal import seasonal_decompose

parts = seasonal_decompose(kitchen, model="multiplicative", period=12)
seasonal_index = parts.seasonal[:12]
seasonal_index.index = seasonal_index.index.strftime("%b")
print("seasonal index (1.0 = an average month):")
print(seasonal_index.round(3).to_string())
print(
    f"\ntrend, first and last available: {parts.trend.dropna().iloc[0]:,.0f} "
    f"-> {parts.trend.dropna().iloc[-1]:,.0f}"
)
print(
    f"residual: typical size {parts.resid.dropna().std():.3f} (as a multiplier), "
    f"largest {parts.resid.dropna().abs().max():.3f} "
    f"at {parts.resid.dropna().abs().idxmax():%b %Y}"
)
```

```
seasonal index (1.0 = an average month):
month
Jan    0.951
Feb    0.886
Mar    0.958
Apr    0.845
May    0.849
Jun    0.820
Jul    0.950
Aug    0.965
Sep    1.022
Oct    1.434
Nov    1.300
Dec    1.021

trend, first and last available: 55,477 -> 86,938
residual: typical size 0.105 (as a multiplier), largest 1.249 at Feb 2020
```

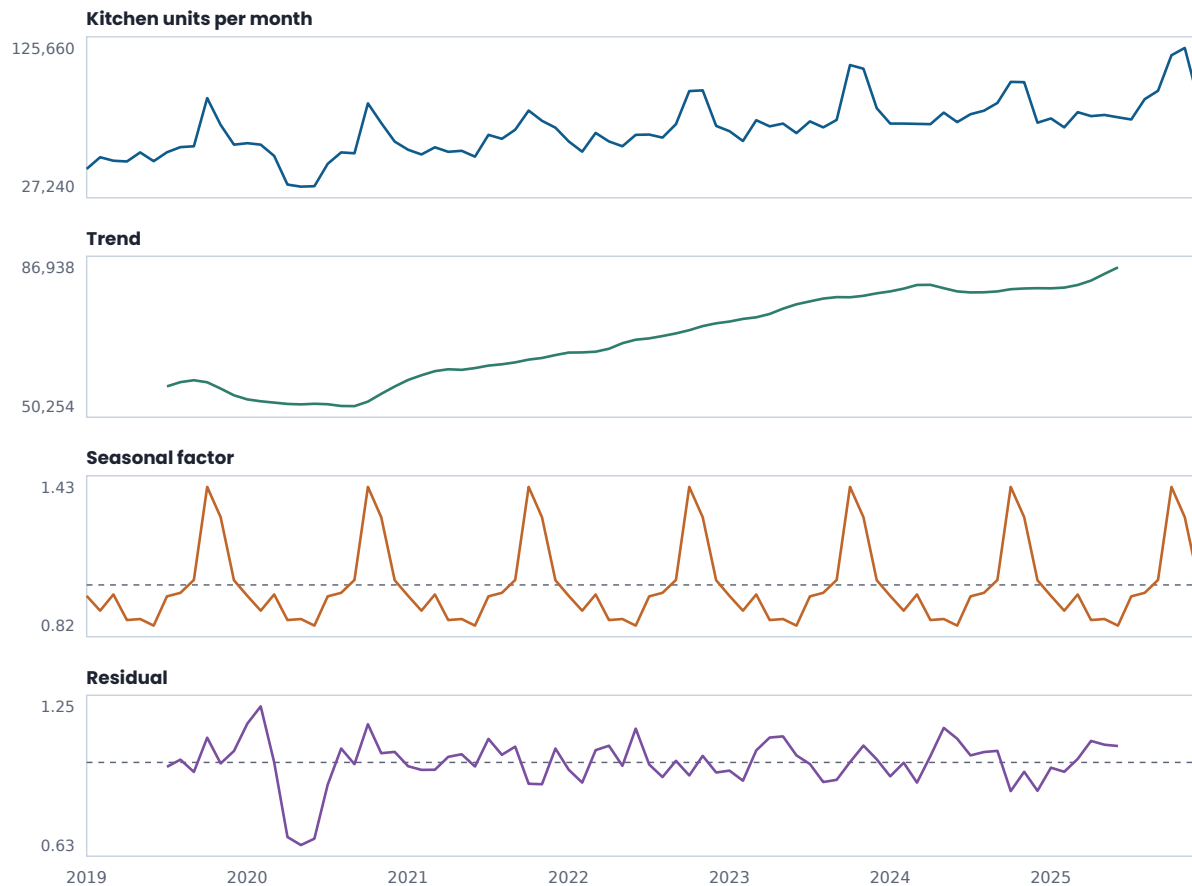



Figure 40.1 – Kitchen demand decomposed into trend, seasonality, and residual. The seasonal factor for October is 1.43; the residual's one large excursion is the spring of 2020.

Reading it. October runs at 143% of an average month and November at 130%, while April–June sit at about 82–85%. The trend roughly doubles over the period. The residual is typically about $\pm 10\%$, with one enormous exception: February to June 2020, when the factory shut and reopened. That event isn't seasonality or trend; it's a one-off that every model in this chapter has to live with.

Watch out: decomposition is a description, not a forecast. `seasonal_decompose` uses a centered moving average, which needs data on both sides of each point, so the trend has no value for the first and last six months. It's for understanding the series, not for predicting it.

40.2 Stationarity

Most classical forecasting methods assume the series is **stationary**: its mean, variance, and autocorrelation don't change over time. A series with a trend isn't stationary (the mean rises); one with growing seasonal swings isn't either (the variance rises).

The standard fix is **differencing**: model the *change* from one period to the next instead of the level. Seasonal differencing subtracts the value 12 months earlier. Taking logs first turns growing swings into steady ones. The **augmented Dickey–Fuller (ADF) test** checks the result; its null hypothesis is "not stationary", so a small p-value means stationary.

```
from statsmodels.tsa.stattools import adfuller

def adf_report(name, series):
    stat, p_value = adfuller(series.dropna())[:2]
    verdict = "stationary" if p_value < 0.05 else "NOT stationary"
    print(f"{name:<38} ADF statistic {stat:6.2f}   p-value {p_value:.4f}   {verdict}")

adf_report("units", kitchen)
adf_report("month-to-month change", kitchen.diff())
adf_report("log units, differenced", np.log(kitchen).diff())
adf_report("log units, differenced, then seasonal", np.log(kitchen).diff().diff(12))
```

units	ADF statistic	0.35	p-value 0.9797	NOT stationary
month-to-month change	ADF statistic	-3.95	p-value 0.0017	stationary
log units, differenced	ADF statistic	-3.52	p-value 0.0074	stationary
log units, differenced, then seasonal	ADF statistic	-3.55	p-value 0.0068	stationary

Reading it. The raw units fail ($p = 0.98$): of course, they trend upward. One difference is enough to pass, and the log-plus-seasonal-difference version, which section 40.5 uses, passes comfortably. The test is a check, not an oracle: it's weak on short series and says nothing about *which* model to fit. Its practical use is to tell you how many times to difference.

40.3 Baselines that are hard to beat

Chapter 36's rule applies with more force here: never report a forecast without the simple method it must beat. Three baselines, using 2019–2024 to forecast 2025:

- **Naive:** next month equals last month.
- **Seasonal naive:** next month equals the same month last year.
- **Moving average:** next month equals the average of the last 12.

```

train = kitchen[:"2024-12"]
test = kitchen["2025"]

def wape(actual, forecast):
    return (
        np.abs(np.asarray(actual) - np.asarray(forecast)).sum()
        / np.asarray(actual).sum()
    )

naive = np.repeat(train.iloc[-1], 12)
seasonal_naive = train["2024"].to_numpy()
moving_average = np.repeat(train.iloc[-12:].mean(), 12)
print(
    f"last value (Dec 2024): {train.iloc[-1]:,.0f}  "
    f" mean of 2024: {train.iloc[-12:].mean():,.0f}"
)
print(
    f"January 2025 actual {test.iloc[0]:,.0f}: naive says {naive[0]:,.0f}, "
    f"seasonal naive says {seasonal_naive[0]:,.0f} (Jan "
    f"2024), moving average says {moving_average[0]:,.0f}"
)
forecasts = {
    "naive": naive,
    "seasonal naive": seasonal_naive,
    "12-month moving average": moving_average,
}
for name, f in forecasts.items():
    print(f"{name:<26} WAPE over 2025 {wape(test, f):.1%}")

```

```

last value (Dec 2024): 72,627  mean of 2024: 80,171
January 2025 actual 75,641: naive says 72,627, seasonal naive says 71,958 (Jan 2024), moving
average says 80,171
naive                WAPE over 2025 17.7%
seasonal naive       WAPE over 2025 10.0%
12-month moving average WAPE over 2025 14.1%

```

How it works: WAPE, the accuracy measure used throughout, is the sum of absolute errors divided by the sum of actuals: "total units wrong as a share of total units". Section 40.8 works it by hand.

Reading it. The seasonal naive forecast, which needs no library and no skill, is within 10% over the year. Plain naive (17.7%) and the moving average (14.1%) do worse because they ignore the seasonal pattern entirely: the moving average predicts 80,171 for a January that runs well below average.

Seasonal naive is the number to beat for any strongly seasonal series, and several methods below don't beat it.

40.4 Exponential smoothing

Simple smoothing, by hand

Simple exponential smoothing keeps a running level that moves part of the way toward each new observation:

$$\text{new level} = \alpha \times \text{latest actual} + (1 - \alpha) \times \text{old level}$$

where α (alpha), between 0 and 1, sets how fast the level reacts. It's a weighted average of the whole history with weights that decay exponentially into the past. Five months of Kitchen data with $\alpha = 0.3$:

```
values = train["2024-08":"2024-12"]
alpha = 0.3
level = values.iloc[0]
print(f"start: level = first value = {level:,.0f}")
for month, y in values.iloc[1:].items():
    level = alpha * y + (1 - alpha) * level
    print(
        f"{month:%b %Y}: actual {y:>8,.0f}    level = 0.3 "
        f"x {y:,.0f} + 0.7 x previous = {level:>9,.0f}"
    )
```

```
start: level = first value = 81,128
Sep 2024: actual    86,644    level = 0.3 x 86,644 + 0.7 x previous =    82,783
Oct 2024: actual   101,587    level = 0.3 x 101,587 + 0.7 x previous =    88,424
Nov 2024: actual   101,361    level = 0.3 x 101,361 + 0.7 x previous =    92,305
Dec 2024: actual    72,627    level = 0.3 x 72,627 + 0.7 x previous =    86,402
```

Reading it. The level lags the actuals: it rises toward the October–November peak but never reaches it, and it comes down slowly in December. That's the point of smoothing (it ignores noise) and its weakness (it ignores real seasonal jumps too). Simple smoothing forecasts a flat line at the last level, so it can't follow seasonality.

Holt–Winters

Holt's method adds a second smoothed quantity for the trend (with its own parameter, β). **Holt–Winters** adds a third for the seasonal factors (γ). statsmodels chooses the parameters by minimizing the in-sample error:

```

from statsmodels.tsa.holtwinters import ExponentialSmoothing

ses = ExponentialSmoothing(train).fit()
holt_winters = ExponentialSmoothing(
    train, trend="add", seasonal="mul", seasonal_periods=12
).fit()
print(
    f"simple smoothing: alpha {ses.params['smoothing_level']:.3f}  "
    f"WAPE 2025 {wape(test, ses.forecast(12)):.1%}"
)
print(
    f"Holt-Winters:      alpha {holt_winters.params['smoothing_level']:.3f}  "
    f"beta {holt_winters.params['smoothing_trend']:.3f}  "
    f"gamma {holt_winters.params['smoothing_seasonal']:.3f}  "
    f"WAPE 2025 {wape(test, holt_winters.forecast(12)):.1%}"
)
forecasts["Holt-Winters"] = holt_winters.forecast(12).to_numpy()
hw_fc = holt_winters.forecast(12)
print("Holt-Winters vs actual, first four months of 2025:")
print(
    pd.DataFrame({"actual": test[:4], "forecast": hw_fc[:4].round()})
    .astype(int)
    .to_string()
)

```

```

simple smoothing: alpha 0.993  WAPE 2025 17.5%
Holt-Winters:      alpha 0.725  beta 0.011  gamma 0.000  WAPE 2025 17.5%
Holt-Winters vs actual, first four months of 2025:
      actual  forecast
2025-01-01  75641     67183
2025-02-01  69282     64400
2025-03-01  80091     68328
2025-04-01  77281     62052

```

Reading it, plainly. Holt–Winters scores **17.5%**, worse than seasonal naive's 10%. Look at why: the fitted α is 0.725, so the level chases the latest month, and γ is 0.000, so the seasonal factors were fixed from the early years and never updated. Starting from a low December 2024, the forecast for early 2025 runs 10–15% below actual. The optimizer minimized *in-sample* error and found a combination that fits the past and forecasts badly. This is a common outcome on short, noisy series; the fix is usually to constrain the parameters (`use_boxcox` , `damped_trend` , or fixing γ) and to judge by backtest, not by in-sample fit.

When smoothing works well: stable series with clear seasonality and lots of history, and as a fast, explainable default in planning tools (most commercial demand-planning software runs some form of Holt–Winters). It's also the method a planner can maintain in a spreadsheet.

40.5 ARIMA and SARIMA

The idea

ARIMA models a stationary series as a combination of:

- **AR** (autoregressive, order p): this month depends on the last p months' values.
- **I** (integrated, order d): the series was differenced d times to make it stationary.
- **MA** (moving average, order q): this month depends on the last q months' forecast errors.

SARIMA adds the same three at the seasonal lag (12 months here), written $(p, d, q)(P, D, Q)_s$. The most common seasonal model in practice, $(1, 1, 1)(0, 1, 1)_{12}$, says: difference once and seasonally once, then explain what's left with one autoregressive term, one error term, and one seasonal error term.

Reading ACF and PACF

The **autocorrelation function (ACF)** measures how a series correlates with itself at each lag; the **partial autocorrelation (PACF)** does the same after removing the effect of shorter lags. On a properly differenced series, spikes in the ACF suggest MA terms and spikes in the PACF suggest AR terms:

```
from statsmodels.tsa.stattools import acf, pacf

stationary = np.log(train).diff().diff(12).dropna()
lags = [1, 2, 3, 6, 11, 12, 13, 24]
acf_values = acf(stationary, nlags=24)
pacf_values = pacf(stationary, nlags=24)
print("lag    ACF    PACF    (differenced log series)")
for lag in lags:
    print(f"{lag:>3}    {acf_values[lag]:6.2f}    {pacf_values[lag]:6.2f}")
print(f"significance line: about +/-{2 / np.sqrt(len(stationary)):.2f}")
```

lag	ACF	PACF	(differenced log series)
1	0.06	0.06	
2	-0.07	-0.07	
3	-0.12	-0.11	
6	-0.06	-0.11	
11	-0.06	-0.02	
12	-0.34	-0.48	
13	-0.06	0.09	
24	-0.06	-0.05	

significance line: about +/-0.26

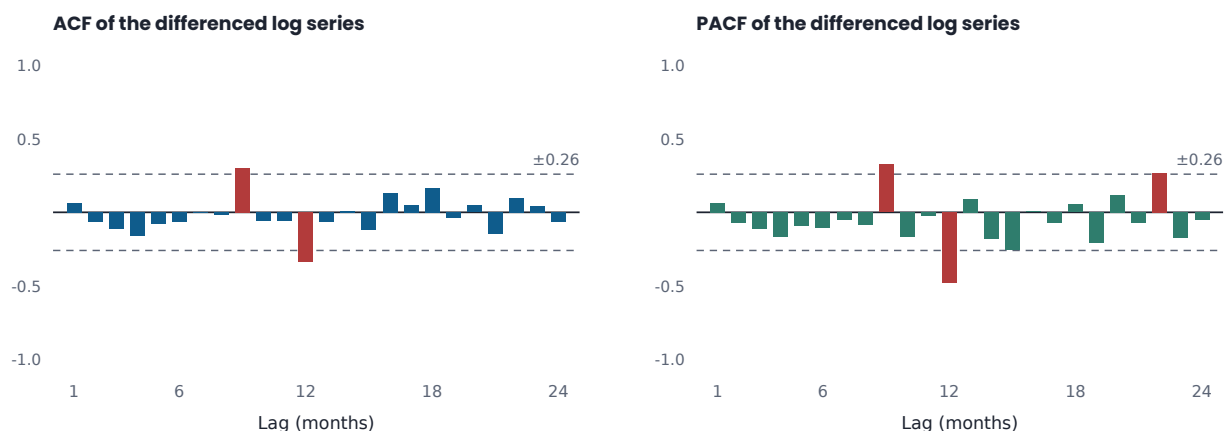


Figure 40.2 – ACF and PACF of the differenced log series. Almost everything sits inside the significance band except lag 12, the sign of a seasonal moving-average term.

Reading it. After differencing, almost nothing is left except a clear negative spike at lag 12 in both plots (-0.34 and -0.48 , well outside the ± 0.26 band). A negative ACF spike at the seasonal lag after seasonal differencing is the textbook sign of a seasonal MA(1) term, which is the $Q = 1$ in the model above. The short lags are all within the band, so p and q are small. In practice most people try the standard model, check the residuals, and then let `auto_arima` (from the `pmdarima` package) or a small grid search over orders confirm it.

Fitting SARIMA

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

sarima = SARIMAX(np.log(train), order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)).fit(
    disp=False
)
print(sarima.summary().tables[1])
sarima_fc = np.exp(sarima.forecast(12))
forecasts["SARIMA"] = sarima_fc.to_numpy()
print(
    f"\nSARIMA (1,1,1)(0,1,1)12 on log units: WAPE 2025 "
    f"{waape(test, sarima_fc):.1%}   AIC {sarima.aic:.1f}"
)
interval = np.exp(sarima.get_forecast(12).conf_int(alpha=0.2))
print(
    f"December 2025: forecast {sarima_fc.iloc[-1]:.0f}, 80% interval "
    f"{interval.iloc[-1, 0]:.0f} to "
    f"{interval.iloc[-1, 1]:.0f}, actual {test.iloc[-1]:.0f}"
)
```

```
=====
              coef      std err          z      P>|z|      [0.025      0.975]
-----
ar.L1          0.6969      0.175       3.972      0.000       0.353      1.041
ma.L1         -0.9974      4.852      -0.206      0.837     -10.507      8.512
ma.S.L12       -0.6357      0.236      -2.693      0.007      -1.098     -0.173
sigma2         0.0162      0.077       0.211      0.833      -0.135      0.167
=====

SARIMA (1,1,1)(0,1,1)12 on log units: WAPE 2025 4.0%   AIC -58.0
December 2025: forecast 90,680, 80% interval 70,932 to 115,926, actual 88,926
```

How it works: the model is fitted to **log** units so that the seasonal swing is proportional and the forecast can't go negative; `np.exp` converts back. `get_forecast(...).conf_int(alpha=0.2)` gives an 80% prediction interval.

Reading it. SARIMA scores **4.0%** on 2025, far better than seasonal naive. The coefficients say what the plots said: a seasonal MA term of -0.64 (significant) and an AR(1) of 0.70 . The MA(1) coefficient of -0.997 with a huge standard error looks like a redundant term; exercise 7 tests whether it can be dropped, and the answer is a clear no. A coefficient near the edge of its allowed range inflates its standard error without making the term useless. The December forecast of $90,680$ came with an 80% interval of $71,000$ to $116,000$; the actual, $88,926$, landed inside it. **Always show the interval.** A point forecast without one invites the reader to believe it.

Forecasting libraries

Beyond statsmodels, three tools are worth knowing:

- **Prophet** (Meta): fits trend, seasonality, and holiday effects as additive curves, handles missing data and outliers gracefully, and needs almost no tuning. Popular with analysts for exactly that reason.
- **pmdarima**: `auto_arima`, which searches SARIMA orders automatically.
- **statsforecast** and **sktime**: fast, scikit-learn-style libraries for forecasting many series at once, which is what a demand planner with 500 products needs.

```
from prophet import Prophet

frame = pd.DataFrame({"ds": train.index, "y": train.to_numpy()})
prophet = Prophet(
    yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False
)
prophet.fit(frame)
prophet_fc = prophet.predict(pd.DataFrame({"ds": test.index}))["yhat"].to_numpy()
forecasts["Prophet"] = prophet_fc
print(f"Prophet: WAPE 2025 {wape(test, prophet_fc):.1%}")
```

```
Prophet: WAPE 2025 5.9%
```

Reading it. Prophet scores **5.9%** on 2025 without any settings beyond "there's yearly seasonality". Not as good as the hand-chosen SARIMA here, and much better than Holt-Winters or seasonal naive, for one line of configuration. For someone forecasting dozens of series with no time to tune each, that trade is often the right one.

40.6 Machine learning for forecasting

Gradient boosting (Chapter 37) can forecast too, if the time series is turned into a table: each row is one period, and its features are **lags** (the values 1, 2, ..., 52 periods earlier), **rolling statistics** of past values, and **calendar** features (week of year). The target is this period's value. With 84 months that's too few rows; the weekly series (365 rows) is where this approach starts to make sense.

```

from sklearn.ensemble import HistGradientBoostingRegressor

wk = (
    weekly[weekly["category"] == "Kitchen"]
    .set_index("week_start")["units"]
    .astype(float)
)
features = pd.DataFrame({"units": wk})
for lag in [1, 2, 3, 4, 8, 13, 26, 52]:
    features[f"lag_{lag}"] = wk.shift(lag)
features["mean_4"] = wk.shift(1).rolling(4).mean()
features["mean_13"] = wk.shift(1).rolling(13).mean()
features["week_of_year"] = features.index.isocalendar().week.to_numpy().astype(int)
features["year"] = features.index.year
features = features.dropna()
wk_train, wk_test = features.loc[:"2024-12-31"], features.loc["2025"]
X_cols = [c for c in features.columns if c != "units"]
gbr = HistGradientBoostingRegressor(
    max_iter=300, learning_rate=0.05, max_depth=4, random_state=40
)
gbr.fit(wk_train[X_cols], wk_train["units"])
print(
    f"{len(wk_train)} training weeks, {len(wk_test)} test weeks, {len(X_cols)} features"
)
print(
    f"one-week-ahead WAPE on 2025 weeks: "
    f"{wape(wk_test['units'], gbr.predict(wk_test[X_cols])):.1%}"
)
wk_seasonal_naive = wk.shift(52).reindex(wk_test.index)
print(
    f"weekly seasonal naive (same week last year): "
    f"{wape(wk_test['units'], wk_seasonal_naive):.1%}"
)

```

```

261 training weeks, 52 test weeks, 12 features
one-week-ahead WAPE on 2025 weeks: 9.3%
weekly seasonal naive (same week last year): 11.3%

```

How it works:

- Every feature is built from `wk.shift(...)`: values strictly *before* the row's week.
`rolling(4).mean()` is applied to the **shifted** series, so the 4-week average never includes the current week. That's the time-series version of Chapter 36's prediction-moment rule, and getting it wrong (using `rolling(4)` on the unshifted series) is the most common leak in ML forecasting.
- The split is by time: train to December 2024, test on 2025.

Reading it. One week ahead, the model's WAPE is 9.3% against 11.3% for the weekly seasonal naive. But "one week ahead" means each prediction used the *actual* previous weeks, which a planner forecasting a year out won't have. The fair test is **recursive**: forecast week 1, feed the forecast in as the lag for week 2, and so on:

```

history = wk[:"2024-12-31"].copy()
recursive = []
for week in wk_test.index:
    row = {f"lag_{lag}": history.iloc[-lag] for lag in [1, 2, 3, 4, 8, 13, 26, 52]}
    row["mean_4"] = history.iloc[-4:].mean()
    row["mean_13"] = history.iloc[-13:].mean()
    row["week_of_year"] = int(week.isocalendar().week)
    row["year"] = week.year
    prediction = gbr.predict(pd.DataFrame([row])[X_cols])[0]
    recursive.append(prediction)
    history.loc[week] = prediction # feed the forecast back in, never the actual
recursive = pd.Series(recursive, index=wk_test.index)
print(
    f"52-week recursive forecast, WAPE on 2025: {wape(wk_test['units'], recursive):.1%}"
)
monthly_from_weekly = recursive.resample("MS").sum()
print(
    f"summed to months and compared with the monthly series: "
    f"{wape(test, monthly_from_weekly.reindex(test.index)):.1%}"
)

```

```

52-week recursive forecast, WAPE on 2025: 8.9%
summed to months and compared with the monthly series: 13.6%

```

Reading it. Recursively, the weekly model scores 8.9% over 2025, still ahead of the weekly baseline. Summed into months it scores 13.6% against the monthly series, worse than SARIMA's 4.0%, partly because weekly noise is harder to forecast than monthly totals and errors compound over 52 recursive steps. That's the honest position of ML forecasting on a single clean series: competitive, not dominant. Where it wins is with **many series and external features**: promotions, prices, weather, holidays, and the demand of related products, which SARIMA can't easily use. Chapter 48's sensor work and the capstone use it that way.

Watch out: the calendar can leak too. A feature such as "total units this year" or "average price this quarter" includes the future if computed over the whole period. Every feature must be computable on the forecast date from data available then.

40.7 Backtesting

One test year is one sample. 2025 happened to be an ordinary year; a method that does well on it might have failed in 2020. **Backtesting** (rolling-origin evaluation) repeats the forecast from several past starting points and looks at all the scores:

```
def backtest(series, make_forecast, horizon=12, folds=5):
    scores = []
    for fold in range(folds):
        cut = len(series) - horizon * (folds - fold)
        history, future = series.iloc[:cut], series.iloc[cut : cut + horizon]
        scores.append(wape(future, make_forecast(history, horizon)))
    return np.array(scores)

methods = {
    "seasonal naive": lambda h, n: h.iloc[-12:].to_numpy()[n],
    "Holt-Winters": lambda h, n: ExponentialSmoothing(
        h, trend="add", seasonal="mul", seasonal_periods=12
    )
    .fit()
    .forecast(n)
    .to_numpy(),
    "SARIMA": lambda h, n: np.exp(
        SARIMAX(np.log(h), order=(1, 1, 1), seasonal_order=(0, 1, 1, 12))
        .fit(dispatch=False)
        .forecast(n)
    ).to_numpy(),
}
print("backtest: 5 folds, each forecasting the next 12 months from 2021 to 2025")
print(
    f"{'method':<16} "
    + " ".join(f"fold {i + 1}" for i in range(5))
    + "    mean    worst"
)
for name, fn in methods.items():
    s = backtest(kitchen, fn)
    print(
        f"{'name':<16} "
        + " ".join(f"{v:6.1%}" for v in s)
        + f"    {s.mean():6.1%}    {s.max():6.1%}"
    )
```

```
backtest: 5 folds, each forecasting the next 12 months from 2021 to 2025
method      fold 1  fold 2  fold 3  fold 4  fold 5    mean    worst
seasonal naive  20.7%  10.5%  12.7%  10.2%  10.0%   12.8%   20.7%
Holt-Winters   14.4%  11.9%  14.0%   8.2%  17.5%   13.2%   17.5%
SARIMA         16.6%  15.0%  10.8%   9.9%   4.0%   11.3%   16.6%
```

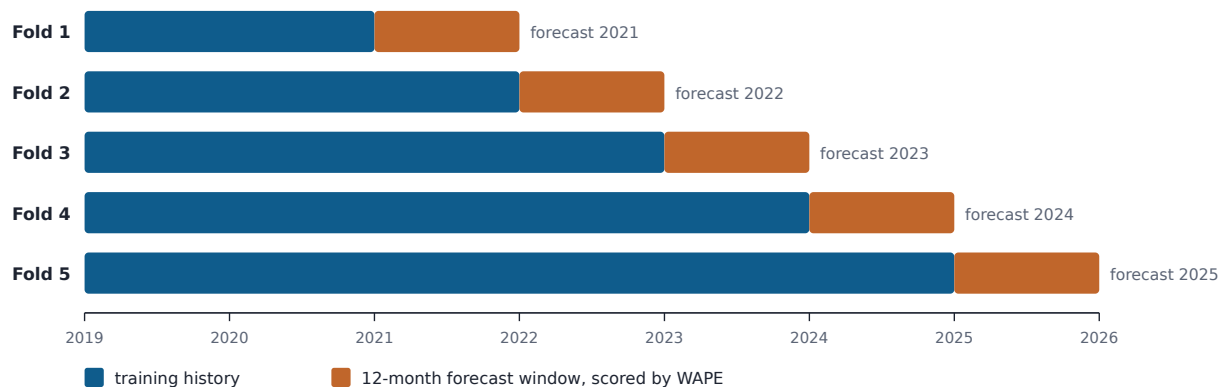


Figure 40.3 – Rolling-origin backtesting. Each fold trains on everything before its cut and forecasts the next 12 months, so every method is judged on five different years.

Reading it, plainly. On a single year (fold 5, which is 2025), SARIMA's 4.0% looks like a clear win. Over five years the picture is more modest: SARIMA averages **11.3%**, seasonal naive 12.8%, and Holt–Winters 13.2%. The worst years are 2021 for seasonal naive (20.7%, because it copied the shut-down months of 2020) and 2021 for SARIMA too (16.6%, for the related reason that it was trained through the shutdown). **The 2025 result overstated SARIMA's advantage by a factor of three.** This is why a single test period is never enough for a forecast: report the backtest mean and the worst fold, and expect the worst fold to happen again.

Which fold to trust? All of them, but not equally. The most recent folds are closest to how the model will be used; the 2020–2021 folds show how each method copes with a shock. A planner would rightly ask, "what happens next time something like 2020 happens?", and only the backtest can answer.

40.8 Forecast accuracy: MAPE, WAPE, MASE

Three measures, all on SARIMA's 2025 forecast, worked from the twelve monthly errors:

- **MAPE** (mean absolute percentage error): average of $|\text{error}| \div \text{actual}$ across months. Familiar, and flawed: it treats a 10% miss in a small month the same as in a big one, it can't handle zero actuals, and it penalizes over-forecasting more than under-forecasting.
- **WAPE** (weighted absolute percentage error): sum of $|\text{errors}| \div \text{sum of actuals}$. Big months count more, which is what a planner wants; it's stable with zeros; it equals "total units wrong $\div$ total units". **This is the number to quote.**
- **MASE** (mean absolute scaled error): mean $|\text{error}| \div$ the mean absolute error of the seasonal naive method on the training data. Below 1 means the model beats seasonal naive *on the scale of the training history*, and it's comparable across series of different sizes.

```
actual = test.to_numpy()
forecast = forecasts["SARIMA"]
errors = actual - forecast
mape = np.mean(np.abs(errors) / actual)
wape_value = np.abs(errors).sum() / actual.sum()
naive_in_sample = np.abs(train.to_numpy()[12:] - train.to_numpy()[:-12]).mean()
mase = np.abs(errors).mean() / naive_in_sample
print(
    f"MAPE {mape:.1%}    WAPE {wape_value:.1%}    MASE "
    f"{mase:.2f}    bias {errors.mean():+.0f} units/month"
)
print(
    f"largest month: {np.abs(errors).max():+.0f} units "
    f"({test.index[np.abs(errors).argmax()]:%b %Y})"
)
print(
    f"total 2025 actual {actual.sum():+.0f}, total forecast {forecast.sum():+.0f} "
    f"({(forecast.sum() - actual.sum()) / actual.sum():+.1%})"
)
```

```
MAPE 4.1%    WAPE 4.0%    MASE 0.37    bias +776 units/month
largest month: 10,464 units (Jul 2025)
total 2025 actual 1,051,402, total forecast 1,042,096 (-0.9%)
```

By hand for one month: in July 2025 the forecast missed the actual 74,926 by 10,464 units, 14%. Over the year the absolute errors add to about 42,000 against 1,051,402 actual units: $WAPE = 42,000 \div 1,051,402 = 4.0\%$. MAPE is slightly higher (4.1%) because it weights the small months' percentages equally. MASE of 0.37 says the model's average error is about a third of what seasonal naive achieved on the training data.

Bias matters too: the errors average +776 units a month, meaning the forecast ran slightly low, and the annual total was 0.9% under. A forecast that's 4% wrong month to month but 1% wrong for the year is fine for resin purchasing and less fine for weekly scheduling. Say which horizon the number refers to.

40.9 Demand forecasting for a manufacturer

All four categories

The project asks for a forecast by category. Here's the fair comparison, four methods against seasonal naive, on 2025:

```

rows = []
for cat in ["Storage", "Kitchen", "Industrial", "Furniture"]:
    series = (
        monthly[monthly["category"] == cat]
        .set_index("month")["units"]
        .astype(float)
        .asfreq("MS")
    )
    tr, te = series[:"2024-12"], series["2025"]
    hw_fc = (
        ExponentialSmoothing(tr, trend="add", seasonal="mul", seasonal_periods=12)
        .fit()
        .forecast(12)
    )
    sar_fc = np.exp(
        SARIMAX(np.log(tr), order=(1, 1, 1), seasonal_order=(0, 1, 1, 12))
        .fit(dispatch=False)
        .forecast(12)
    )
    pr = Prophet(
        yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False
    )
    pr.fit(pd.DataFrame({"ds": tr.index, "y": tr.to_numpy()}))
    pr_fc = pr.predict(pd.DataFrame({"ds": te.index}))["yhat"].to_numpy()
    rows.append(
        {
            "category": cat,
            "2025 units": int(te.sum()),
            "seasonal naive": wape(te, tr["2024"].to_numpy()),
            "Holt-Winters": wape(te, hw_fc),
            "SARIMA": wape(te, sar_fc),
            "Prophet": wape(te, pr_fc),
        }
    )
table = pd.DataFrame(rows).set_index("category")
print(
    (table.drop(columns="2025 units") * 100)
    .round(1)
    .astype(str)
    .add("%")
    .assign(units=table["2025 units"].map("{:,}".format))
    .to_string()
)

```

	seasonal naive	Holt-Winters	SARIMA	Prophet	units
category					
Storage	7.7%	7.2%	3.7%	6.7%	691,712
Kitchen	10.0%	17.5%	4.0%	5.9%	1,051,402
Industrial	5.7%	6.7%	6.7%	7.8%	400,694
Furniture	15.8%	9.3%	7.4%	9.6%	411,884

Reading it. No method wins everywhere. SARIMA is best on three categories; on Industrial, the least seasonal series, seasonal naive (5.7%) beats every model, and SARIMA and Holt-Winters both score worse than copying last year. Holt-Winters is fine on Storage and Furniture and poor on Kitchen. Prophet is never best and never bad on 2025 (its backtest, exercise 9, is less flattering). A planner choosing one method for all four would pick SARIMA; a planner choosing per category, judged by backtest rather than by 2025 alone, would run seasonal naive for Industrial and keep the model for the rest. **The comparison table is the deliverable**, more than any forecast.

From forecast to plan

A forecast becomes a production plan when it's combined with an allowance for being wrong. The simplest version: plan for the forecast plus **safety stock** sized from the backtest error:

```
sarima_scores = backtest(kitchen, methods["SARIMA"])
error_allowance = sarima_scores.mean() + sarima_scores.std()
plan = pd.DataFrame(
    {
        "forecast": sarima_fc.round(),
        "safety stock": (sarima_fc * error_allowance).round(),
    }
)
plan["plan"] = plan["forecast"] + plan["safety stock"]
plan["actual"] = test
plan.index = plan.index.strftime("%b %Y")
print(
    f"error allowance from the backtest: mean WAPE {sarima_scores.mean():.1%} "
    f"+ one sd {sarima_scores.std():.1%} = {error_allowance:.1%}"
)
print(plan.head(6).astype(int).to_string())
print(
    f"months where actual exceeded the plan: {(plan['actual'] > plan['plan']).sum()} of 12"
)
```

```
error allowance from the backtest: mean WAPE 11.3% + one sd 4.4% = 15.7%
   forecast  safety stock   plan  actual
Jan 2025    69547        10890  80437   75641
Feb 2025    69554        10891  80445   69282
Mar 2025    77752        12175  89927   80091
Apr 2025    73463        11503  84966   77281
May 2025    77806        12183  89989   78092
Jun 2025    74725        11701  86426   76488
months where actual exceeded the plan: 0 of 12
```

Reading it. The backtest says SARIMA is typically 11% off with a spread of 4%, so the plan carries 15.7% above forecast. In 2025 that covered every month; in a year like 2021 it wouldn't have. That's the trade the planner makes explicitly: more safety stock costs warehouse space and cash, less risks late orders, and the backtest is what makes the trade a calculation instead of a guess. Chapter 45's business metrics put rupee values on both sides.

What to tell the production planner. "For Kitchen, plan on the model's monthly forecast plus 16%, which would have covered every month this year. For Industrial, last year's numbers are as good as any model. October and November need 40% more capacity than an average month, and the model can't see a shock like 2020 coming, so keep the safety stock through the festive season."

40.10 Anomaly detection in sensor data

A week of readings

Riverstone's moulding machines log temperature, pressure, and cycle time every minute. The maintenance question isn't "forecast the temperature"; it's "tell me when something is wrong". Three kinds of wrong were planted in this week: a heater fault (temperature ramps up over 50 minutes), a

pressure spike, and a stoppage.

```
readings = pd.read_csv(
    "../sensors/machine_readings_week.csv", parse_dates=["timestamp"]
)
print(
    f"{len(readings):,} one-minute readings from {readings['machine_id'].iloc[0]}, "
    f"{readings['timestamp'].min()} to {readings['timestamp'].max()}"
)
print(
    readings[["temperature_c", "pressure_bar", "cycle_seconds"]]
    .describe()
    .loc[["mean", "std", "min", "max"]]
    .round(2)
    .to_string()
)
print("status counts:", readings["status"].value_counts().to_dict())
print("missing temperature readings:", readings["temperature_c"].isna().sum())
```

```
10,080 one-minute readings from M01, 2025-03-03 00:00:00 to 2025-03-09 23:59:00
   temperature_c  pressure_bar  cycle_seconds
mean           215.35         117.66          18.50
std              2.53           6.84           0.26
min            209.33           0.00          17.56
max            225.25         150.98          19.61
status counts: {'running': 10048, 'stopped': 32}
missing temperature readings: 18
```

Point anomalies: the rolling z-score

The obvious first tool: compare each reading with the trailing two hours, and flag anything more than four standard deviations away.

```
temp = readings.set_index("timestamp")["temperature_c"].interpolate()
rolling_mean = (
    temp.rolling("120min").mean().shift(1)
) # the past two hours, not including now
rolling_std = temp.rolling("120min").std().shift(1)
z = (temp - rolling_mean) / rolling_std
alerts = z[z.abs() > 4]
print(
    f"readings more than 4 standard deviations from the trailing 2-hour mean: {len(alerts)}"
)
for when, value in alerts.items():
    print(
        f"   {when:%a %H:%M}: {temp[when]:.1f} C against a trailing "
        f"mean of {rolling_mean[when]:.1f} (z = {value:+.1f})"
    )
hottest = temp.idxmax()
print(
    f"\nhottest minute of the week: {hottest:%a %H:%M} at {temp[hottest]:.1f} C, "
    f"z-score there only {z[hottest]:+.1f}"
)
```

```
readings more than 4 standard deviations from the trailing 2-hour mean: 3
Sat 18:49: 214.2 C against a trailing mean of 217.8 (z = -4.3)
Sun 07:29: 216.8 C against a trailing mean of 213.0 (z = +4.9)
Sun 11:29: 220.0 C against a trailing mean of 216.7 (z = +4.1)

hottest minute of the week: Wed 22:53 at 225.2 C, z-score there only +2.8
```

How it works: `rolling("120min")` uses a time-based window; `.shift(1)` excludes the current reading from its own baseline, so a spike can't hide itself. The missing readings are interpolated first.

Reading it. The z-score flags three single readings, each a one-minute blip of 3–4 degrees that reverts immediately: noise, not faults. And it **missed the heater fault**. The hottest minute of the week, 225.2 °C on Wednesday night, had a z-score of only +2.8. A ramp that rises a fraction of a degree a minute drags the trailing mean and standard deviation up with it, so no single reading ever looks far from "normal". **Point-by-point tests can't see slow drift.** That's the most important thing to know about sensor monitoring.

Slow drift: a slow baseline and persistence

Two changes fix it: measure deviation from a **slow, robust baseline** (the trailing 24-hour median, which a 50-minute ramp can't drag), and require the deviation to **persist** for several readings before alerting, which silences the blips:

```

baseline = (
    temp.rolling("24h").median().shift(1)
) # a slow, robust "normal" for the machine
excess = temp - baseline
sustained = (excess > 6).rolling(
    10
).sum() == 10 # more than 6 C above normal for 10 minutes running
first = sustained[sustained].index.min()
episode = excess[(excess > 6) & (excess.index >= first - pd.Timedelta("15min"))]
print(
    f"heater fault: temperature first exceeded normal by "
    f"6 C for 10 straight minutes at {first:%a %H:%M}"
)
print(
    f" episode from {episode.index.min():%H:%M} to "
    f"{episode.index.max():%H:%M}, peak excess {excess.max():.1f} C"
)

pressure = readings.set_index("timestamp")["pressure_bar"]
running = readings.set_index("timestamp")["status"] == "running"
pressure = pressure.where(running) # ignore minutes when the machine was stopped
smooth = (
    pressure.ewm(halflife="30min", times=pressure.index, ignore_na=True).mean().shift(1)
)
residual = (pressure - smooth).dropna()
threshold = 5 * residual[:"2025-03-03 23:59"].std()
spikes = residual[residual.abs() > threshold]
print(
    f"pressure spikes above {threshold:.1f} bar (5 sd of day-one residuals): "
    f"{len(spikes)} readings at {[t.strftime('%a %H:%M') for t in spikes.index]}"
)
stops = readings[readings["status"] == "stopped"]
print(
    f"stoppage: {stops['timestamp'].min():%a %H:%M} to "
    f"{stops['timestamp'].max():%H:%M} ({len(stops)} minutes)"
)

```

```

heater fault: temperature first exceeded normal by 6 C for 10 straight minutes at Wed 22:51
 episode from 22:37 to 23:04, peak excess 9.7 C
pressure spikes above 7.5 bar (5 sd of day-one residuals): 3 readings at ['Tue 12:00', 'Tue 12:01',
'Tue 12:02']
stoppage: Wed 22:02 to 22:33 (32 minutes)

```

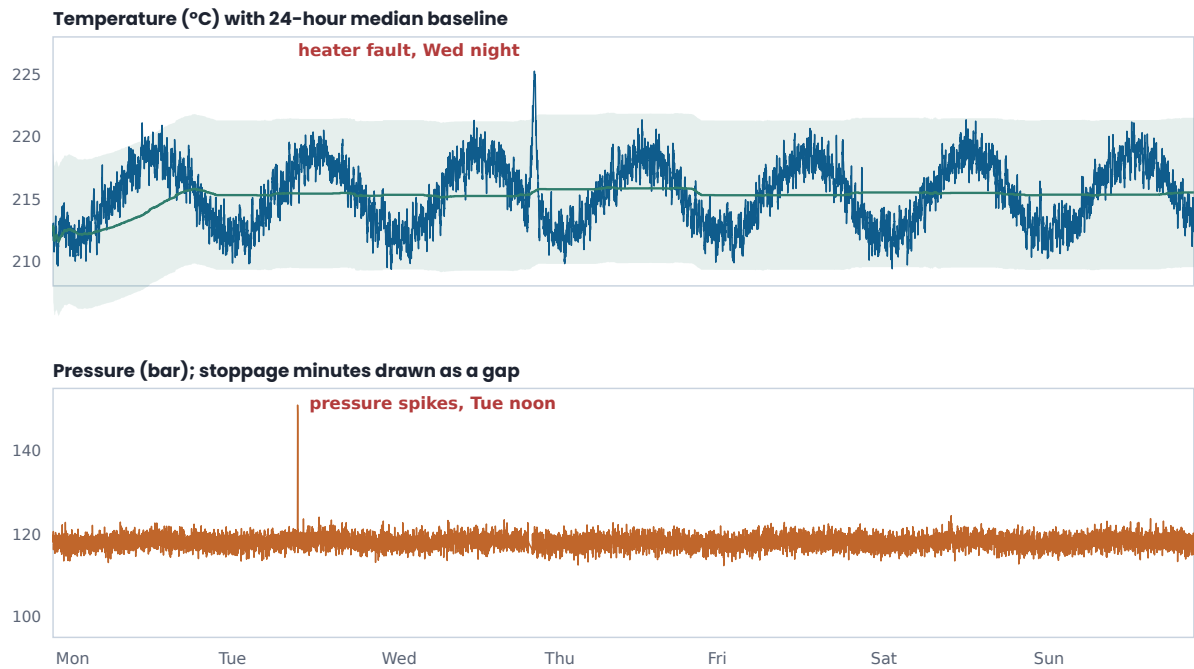


Figure 40.4 — Sensor anomalies. Top: the heater fault stands out against a 24-hour median baseline, though no single minute was extreme. Bottom: pressure spikes caught by the forecast-residual rule, and the stoppage, which is a status change rather than a sensor anomaly.

Reading it. The persistence rule fires at 22:51 on Wednesday. The temperature had crossed the 6 °C line at 22:37, minutes after the machine restarted from its 22:02–22:33 stoppage (a link an engineer would want pointed out), and stayed high until 23:04. The alert comes late in the ramp; exercise 13 shows how a lower threshold trades earlier warning for false alerts. The z-score, by contrast, never fired at all. The pressure rule uses a **forecast residual**: an exponentially weighted forecast of "what pressure should be now", with anything more than 5 standard deviations off flagged. It finds exactly the three planted readings at Tuesday noon and nothing else, once the stoppage minutes (pressure zero, and not an anomaly) are excluded by status.

Three general rules for sensor anomalies: **use the status column** (a stopped machine isn't an anomaly), **choose the baseline for the fault you're looking for** (fast for spikes, slow for drift), and **require persistence** before paging anyone. Chapter 48 runs this on 12 machines and 40 weeks, where the question becomes one of engineering rather than statistics.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Random train/test split on a time series	Wonderful validation scores, bad forecasts	Split by time; backtest with rolling origins
No seasonal naive baseline	"Our model is 12% accurate" with no comparison	Always report seasonal naive alongside
Judging on one test period	Method chosen on a lucky year	Backtest over several years; report mean and worst fold

Mistake	Symptom	Fix
Rolling features that include the current period	ML forecast looks too good one step ahead	<code>shift(1)</code> before <code>rolling(...)</code>
One-step-ahead scores for a multi-step problem	Model can't deliver in practice	Evaluate recursively or with direct multi-horizon models
Trusting in-sample fit	Holt-Winters with $\gamma = 0$ and a bad forecast	Choose parameters by backtest
Forecasting raw units with growing seasonality	Intervals too narrow at peaks; negative forecasts	Log-transform or use a multiplicative model
Quoting MAPE on lumpy or small series	Huge percentages from tiny months	Use WAPE; add MASE for comparisons
A point forecast with no interval	Readers believe the number	Report an 80% or 95% prediction interval
Duplicated or missing periods in the input	Phantom spikes; models that refuse to fit	Check for duplicates and gaps before anything else
Forecasting through a known one-off	The 2020 dip repeats in the model's mind	Dummy variable, or exclude the period, and say so
Ignoring the status column in sensor data	Stoppages flagged as anomalies	Mask by status first
Point z-scores for slow drift	Heater fault missed entirely	Slow robust baseline plus a persistence rule
Alerting on single readings	Maintenance ignores the alerts	Require n consecutive readings

In the real world: the planner who already had a model

In July 2026, Riverstone's production planner, Deepak Nair, shows Meera his demand sheet. It has a column called "model": last year's month times a growth factor he adjusts by hand. He's been using it for four years, and he asks, slightly defensively, whether "the data science" is going to replace it.

Meera doesn't answer until she's backtested his sheet against everything in this chapter. His method is seasonal naive with a hand-tuned growth factor. Over five years, it scores a mean WAPE of 11.9% on Kitchen, better than Holt-Winters and within a point of SARIMA. On Industrial it beats every model.

She shows him the table. Then she shows him two things his sheet can't do: an **interval** (his single number gives the resin buyer no idea how much to hedge), and the **2021 fold**, where his sheet, like seasonal naive, copied the shutdown and under-forecast the recovery by 20%.

The result isn't a replacement. Deepak keeps his sheet for Industrial and as the sanity check for everything else. The SARIMA forecasts go into a second column, with an 80% interval in a third, and safety stock is now sized from the backtest error instead of "15% because it's always been 15%". Once a quarter, Meera re-runs the backtest and they look together at whether the model is still earning its column.

The thing Deepak says afterwards is the thing to remember: *"So the model's job is to tell me how wrong I'm likely to be."* That's most of forecasting.

Tools

- **pandas** (3.0.2): `date indexes`, `asfreq`, `shift`, `rolling` with time-based windows, `resample`, `ewm` with `times`.
 - **statsmodels** 0.15.0: `seasonal_decompose`, `adfuller`, `acf`, `pacf`, `ExponentialSmoothing`, `SARIMAX`. Its documentation on state-space models is the reference for prediction intervals.
 - **Prophet** 1.4.0 (`pip install prophet`; installs CmdStan on first use, which can take a few minutes).
 - **scikit-learn** 1.8.0 for `HistGradientBoostingRegressor`.
 - Not used here but worth knowing: **pmdarima** (`auto_arima`), **statsforecast** (very fast classical models over thousands of series), **sktime** (forecasting with a scikit-learn interface), **darts** (deep learning forecasters). Deep learning models such as N-BEATS and the Temporal Fusion Transformer exist and mostly matter with many related series and rich features; on a single monthly series they rarely beat SARIMA.
 - Everything ran on one CPU core, Python 3.12.3, on 18 September 2026. The slowest step is the backtest with SARIMA, about 20 seconds.
 - **Companion files:** `companion/generate_riverstone_demand.py` (seed 20240) and `companion/generate_riverstone_sensors.py` (seed 20241; `--full` builds the 4.8-million-row version for Chapter 48). Run the chapter's code from `companion/ch40/`. Data specs: `planning/data/riverstone-demand.md` and `riverstone-sensors.md`.
-

The project: forecast Riverstone's monthly demand by product category

Goal: a forecast for 2026 by category, with intervals, a backtested comparison against seasonal naive, and a one-page note the production planner could use.

Option A: your own data. Any monthly or weekly series with at least three years of history: sales, tickets, web traffic, energy use.

Option B: Riverstone. The monthly and weekly demand files.

Steps:

1. **Check the data.** Find and fix duplicates and gaps; plot every series.
2. **Decompose** each category and describe its trend and seasonal factors in one sentence each.
3. **Baselines.** Compute naive, seasonal naive, and moving average WAPE for 2025.
4. **Fit** Holt–Winters, SARIMA, and Prophet for each category. Record the parameters.
5. **Backtest** every method over at least four folds and tabulate mean and worst WAPE. Choose a method per category and justify it.
6. **Forecast 2026** with the chosen method, with 80% intervals.
7. **Size safety stock** from the backtest error and show the plan for the first six months.

8. **Weekly ML model** for one category with lag features, evaluated recursively, and compare with the monthly result.
9. **Write the planner's note:** the forecast, the interval, what it would have got wrong in 2020–2021, and what to watch.

Stretch goals:

- Add a **dummy variable** for the 2020 shutdown months to SARIMA (`exog=`) and see whether the 2021 fold improves.
- Forecast **all four categories at once** with `statsforecast` and compare its `AutoARIMA` choices with yours.
- Build a **direct multi-horizon** ML model (one model per horizon, 1 to 12 months) and compare with the recursive approach.
- Run the sensor anomaly rules on the `--full` dataset for one more machine and tune the persistence window.

You've got it when...

- ☐ I can name trend, seasonality, cycle, and noise in a series and describe what a decomposition shows.
- ☐ I check for duplicates and gaps before modeling.
- ☐ I know what stationarity is, what differencing does, and how to read an ADF p-value.
- ☐ I always compute the seasonal naive baseline first.
- ☐ I can do one step of exponential smoothing by hand and explain α , β , and γ .
- ☐ I can read an ACF and PACF plot well enough to suggest SARIMA orders.
- ☐ I fit SARIMA on a log scale when seasonality grows, and I always show the prediction interval.
- ☐ I build lag features with `shift` before `rolling`, split by time, and evaluate recursively for multi-step forecasts.
- ☐ I backtest with rolling origins and report the mean and worst fold.
- ☐ I quote WAPE, explain why not MAPE, and use MASE to compare series.
- ☐ I can size safety stock from backtest error and explain the trade-off.
- ☐ I know why point z-scores miss slow drift, and I use a slow baseline with a persistence rule.

Recap

- A time series has **trend**, **seasonality**, **cycles**, and **noise**; **decomposition** separates them for understanding, not forecasting.
- **Stationarity** is what classical models need; **differencing** (plain and seasonal) and **logs** get you there; the **ADF test** checks.

- **Seasonal naive** is the baseline to beat, and it often isn't beaten.
- **Exponential smoothing** weights recent values more; **Holt-Winters** adds trend and seasonality, and can fail when parameters are chosen by in-sample fit.
- **SARIMA** combines autoregression, differencing, and moving-average terms at ordinary and seasonal lags; read **ACF/PACF**, fit on logs, show **intervals**.
- **Prophet** is a strong low-effort option; **ML with lag features** competes on weekly data and wins with external features, if you avoid leaks and evaluate **recursively**.
- **Backtest** with rolling origins; one test year lies.
- **WAPE** is the planner's metric; **MAPE** misleads on small months; **MASE** compares across series.
- A forecast becomes a **plan** with safety stock sized from backtest error.
- Sensor anomalies: **status first**, **fast baselines for spikes**, **slow robust baselines for drift**, and **persistence** before alerting.

Practice exercises

Code exercises run from `companion/ch40/` after the chapter's code (they use `kitchen`, `monthly`, `weekly`, `train`, `test`, `waape`, `backtest`, `methods`, `forecasts`, `readings`, `temp`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) A series has these last four months: 100, 120, 110, 130. With $\alpha = 0.5$ and a starting level of 100, compute the smoothed level after each month.
2. (*hand*) A forecast for three months was 90, 100, 110 against actuals 100, 100, 100. Compute MAPE and WAPE. Now swap: forecast 100, 100, 100 against actuals 90, 100, 110. Compute both again, and explain why one measure changed and the other didn't.
3. (*hand*) Monthly series, 12-month seasonality, trending upward. Which SARIMA orders would you try first, and what does each of the six numbers mean?
4. Which baseline suits each series: (a) daily website visits with a strong weekly pattern; (b) a slowly rising count of active customers with no seasonality; (c) a stable product with occasional promotions?

Core

5. Decompose the **Industrial** series. What are its October and April seasonal factors, and how do they compare with Kitchen's? What does that imply for which method to use?
6. Fit Holt-Winters to Kitchen with `damped_trend=True` and with the seasonal parameter fixed at `smoothing_seasonal=0.2`. Does either beat the default's 17.5% on 2025?
7. Fit SARIMA orders $(1, 1, 0)(0, 1, 1)_{12}$ and $(0, 1, 1)(0, 1, 1)_{12}$. Compare AIC and 2025 WAPE with the chapter's model. Is the simpler model as good?
8. Compute an 80% prediction interval from SARIMA for every month of 2025 and count how many actuals fell inside it. Is that what an 80% interval should give?

9. Backtest Prophet with the same five folds. Where does it land against SARIMA and seasonal naive on the mean and the worst fold?
10. Rebuild the weekly ML features **without** `shift(1)` before the rolling means (the leaky version) and report the one-week-ahead WAPE. How much better does the leak make it look?

Stretch

11. Add a 0/1 `exog` variable for March–September 2020 to the SARIMA model and re-run the backtest. Which folds change, and by how much?
12. Fit one `HistGradientBoostingRegressor` per horizon (1, 4, 13, and 26 weeks ahead) on the weekly data, each with lags that respect its horizon, and compare their WAPE with the recursive model at the same horizons.
13. Apply the temperature persistence rule with thresholds of 4 °C and 8 °C, and windows of 5 and 20 minutes. For each combination report the alert time and the number of false alerts.

Think about it

14. The sales head asks for "the forecast" as one number per month. What two things do you add, and how do you explain them in a sentence each?
15. Your SARIMA model scored 4% on last year and 16% on the backtest fold that included the 2020 shutdown. A colleague says the 16% "doesn't count because that was a pandemic". Respond.
16. The maintenance team turned off the temperature alerts because they fired too often. What three changes would you make before turning them back on?

Key terms

time series · trend · seasonality · cycle · noise (residual) · decomposition · multiplicative model · stationarity · differencing · seasonal differencing · log transform · augmented Dickey–Fuller test · naive forecast · seasonal naive · moving average · exponential smoothing · alpha · Holt's method · Holt–Winters · damped trend · autocorrelation (ACF) · partial autocorrelation (PACF) · ARIMA · autoregressive (AR) · moving average (MA) · SARIMA · seasonal order · prediction interval · Prophet · `auto_arma` · lag feature · rolling feature · recursive forecast · direct multi-horizon · backtesting · rolling origin · fold · MAPE · WAPE · MASE · bias · safety stock · horizon · point anomaly · drift · rolling z-score · robust baseline · persistence rule · forecast residual · status masking

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 44, Capstone**, can take demand forecasting as its end-to-end project, from question to planner's note.
- **Chapter 45, Business Metrics**, puts rupee values on forecast error, safety stock, and stock-outs.

- **Chapter 48, Big Data and Distributed Computing**, runs this chapter's anomaly rules on 12 machines and 40 weeks with PySpark, DuckDB, and Polars.
- **Chapter 50, Streaming and Real-Time Data**, turns the sensor rules into alerts that fire as readings arrive.
- **Chapter 52, Deploying and Monitoring Models**, schedules the forecast, monitors its WAPE, and retrains it.
- **Chapter 30, Experiments**, and **Chapter 31, Causal Inference**, cover what forecasting can't: what happens if we change the price.
- **Interview preparation**: the Machine Learning Question Bank (Chapter 74) covers stationarity, ARIMA versus ML, backtesting, MAPE versus WAPE, and "how would you forecast demand for a new product with no history?"

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Month 1: $0.5 \times 100 + 0.5 \times 100 = 100$. Month 2: $0.5 \times 120 + 0.5 \times 100 = 110$. Month 3: $0.5 \times 110 + 0.5 \times 110 = 110$. Month 4: $0.5 \times 130 + 0.5 \times 110 = 120$. The level ends at 120 while the last actual is 130: smoothing always lags a rising series, by more when α is small.

2. First case: errors 10, 0, 10 against actuals of 100 each. $\text{MAPE} = (10\% + 0\% + 10\%) \div 3 = 6.7\%$; $\text{WAPE} = 20 \div 300 = 6.7\%$. Second case: the same errors of 10, 0, 10, but the actuals are 90, 100, 110. $\text{WAPE} = 20 \div 300 = 6.7\%$ again, unchanged. $\text{MAPE} = (10 \div 90 + 0 + 10 \div 110) \div 3 = (11.1\% + 0 + 9.1\%) \div 3 = 6.7\%$ as well here, because the two misses happen to balance; try actuals 50, 100, 150 with the same errors and MAPE becomes $(20\% + 0\% + 6.7\%) \div 3 = 8.9\%$ while WAPE stays 6.7%. MAPE moves whenever the *same* error lands on a smaller actual; WAPE only depends on total error against total volume.

3. Start with $(1, 1, 1)(0, 1, 1)_{12}$ or its simpler cousin $(0, 1, 1)(0, 1, 1)_{12}$. The six numbers: $p = 1$ autoregressive term (this month depends on last month), $d = 1$ ordinary difference (to remove the trend), $q = 1$ moving-average term (this month depends on last month's error); then at the seasonal lag of 12: $P = 0$ seasonal AR terms, $D = 1$ seasonal difference (to remove the yearly pattern), $Q = 1$ seasonal MA term. Fit on the log scale if the seasonal swing grows with the level.

4. (a) **Seasonal naive with a 7-day period**: the same weekday last week. (b) **Naive with drift** (last value plus the average change), since there's no season to copy and a plain naive would ignore the rise. (c) **Seasonal naive or a moving average on non-promotion weeks**, with promotions handled separately as an adjustment, because no baseline should learn from a spike it can't predict.

5.

```

industrial = (
    monthly[monthly["category"] == "Industrial"]
    .set_index("month")["units"]
    .astype(float)
    .asfreq("MS")
)
ind_parts = seasonal_decompose(industrial, model="multiplicative", period=12)
ind_index = ind_parts.seasonal[:12]
ind_index.index = ind_index.index.strftime("%b")
print("Industrial seasonal index:", ind_index[["Apr", "Oct", "Nov"]].round(3).to_dict())
print(
    "Kitchen, for comparison: ",
    seasonal_index[["Apr", "Oct", "Nov"]].round(3).to_dict(),
)
print(f"Industrial seasonal range: {ind_index.min():.3f} to {ind_index.max():.3f}")

```

```

Industrial seasonal index: {'Apr': 0.949, 'Oct': 1.097, 'Nov': 1.023}
Kitchen, for comparison:  {'Apr': 0.845, 'Oct': 1.434, 'Nov': 1.3}
Industrial seasonal range: 0.928 to 1.097

```

Industrial's seasonal factors barely move (a few percent either side of 1), against Kitchen's 0.85 to 1.43. With so little seasonality, a seasonal model has nothing to model and mostly fits noise, which is why seasonal naive beat SARIMA on this category in section 40.9. For a series like this, a simple smoothing or drift method, judged by backtest, is the sensible choice.

6.

```

damped = ExponentialSmoothing(
    train, trend="add", damped_trend=True, seasonal="mul", seasonal_periods=12
).fit()
fixed_gamma = ExponentialSmoothing(
    train, trend="add", seasonal="mul", seasonal_periods=12
).fit(smoothing_seasonal=0.2)
print(
    f"damped trend:      WAPE 2025 {wape(test, damped.forecast(12)):.1%}    "
    f"alpha {damped.params['smoothing_level']:.2f}    "
    f"gamma {damped.params['smoothing_seasonal']:.3f}"
)
print(
    f"gamma fixed at 0.2: WAPE 2025 {wape(test, fixed_gamma.forecast(12)):.1%}    "
    f"alpha {fixed_gamma.params['smoothing_level']:.2f}"
)

```

```

damped trend:      WAPE 2025 16.4%    alpha 0.69    gamma 0.000
gamma fixed at 0.2: WAPE 2025 18.7%    alpha 0.70

```

Damping the trend helps a little (16.4%); fixing γ at 0.2 makes it worse (18.7%), because the optimizer then pushes α higher still. Neither comes close to seasonal naive's 10%. On this series Holt–Winters has a structural problem, not a tuning problem: the level term chases the festive spike and overshoots. The lesson is that in-sample optimizers need supervision on short series, and that the backtest (section 40.7), not one year, decides.

7.

```

for order in [(1, 1, 1), (1, 1, 0), (0, 1, 1)]:
    fit = SARIMAX(np.log(train), order=order, seasonal_order=(0, 1, 1, 12)).fit(
        disp=False
    )
    fc = np.exp(fit.forecast(12))
    print(
        f"order {order}(0,1,1)12: AIC {fit.aic:7.1f} WAPE 2025 {wape(test, fc):.1%}"
    )

```

```

order (1, 1, 1)(0,1,1)12: AIC   -58.0   WAPE 2025  4.0%
order (1, 1, 0)(0,1,1)12: AIC   -54.1   WAPE 2025 15.8%
order (0, 1, 1)(0,1,1)12: AIC   -54.1   WAPE 2025 15.7%

```

Dropping the MA(1) term is a disaster: WAPE jumps from 4.0% to about 16%, and the AIC gets worse (-54 against -58). The term with the alarming standard error was doing real work; its coefficient sits at the edge of the allowed range (-1), which is what inflated the standard error. The lesson runs against the usual advice to drop insignificant terms: for forecasting, judge a term by AIC and by out-of-sample error, not by its p-value alone. Simpler is better only when it forecasts as well.

8.

```

interval_80 = np.exp(sarima.get_forecast(12).conf_int(alpha=0.2))
inside = (test.to_numpy() >= interval_80.iloc[:, 0].to_numpy()) & (
    test.to_numpy() <= interval_80.iloc[:, 1].to_numpy()
)
print(f"months inside the 80% interval: {inside.sum()} of 12")
width = (interval_80.iloc[:, 1] - interval_80.iloc[:, 0]) / test.to_numpy()
print(
    f"interval width as a share of actual: {width.iloc[0]:.0%} "
    f"in January, {width.iloc[-1]:.0%} in December"
)

```

```

months inside the 80% interval: 12 of 12
interval width as a share of actual: 31% in January, 51% in December

```

An 80% interval should contain about 10 of 12 months. More than that means the intervals are wider than they need to be (the model's own error estimate includes the 2020 shock); fewer would mean overconfidence. Note that the interval widens with the horizon: December's is far wider than January's, which is honest, since a twelve-month-ahead forecast knows less.

9.


```
def prophet_forecast(history, n):
    model = Prophet(
        yearly_seasonality=True, weekly_seasonality=False, daily_seasonality=False
    )
    model.fit(pd.DataFrame({"ds": history.index, "y": history.to_numpy()}))
    future = pd.date_range(
        history.index[-1] + pd.DateOffset(months=1), periods=n, freq="MS"
    )
    return model.predict(pd.DataFrame({"ds": future}))["yhat"].to_numpy()

prophet_scores = backtest(kitchen, prophet_forecast)
print(
    "Prophet folds:",
    [f"{v:.1%}" for v in prophet_scores],
    f" mean {prophet_scores.mean():.1%}   worst {prophet_scores.max():.1%}",
)
```

```
Prophet folds: ['33.4%', '10.8%', '7.7%', '8.5%', '5.9%']   mean 13.3%   worst 33.4%
```

Prophet's mean, 13.3%, is the worst of the four methods, and its 2021 fold is a 33% miss, far worse than anyone else's. Prophet fits a piecewise-linear trend, and a trend fitted through the 2020 collapse extrapolates the collapse into 2021. Its good 2025 score hid this. That's the backtest doing its job, and a reminder that the convenience of a method that "handles outliers" doesn't mean it handles shocks.

10.

```
leaky = pd.DataFrame({"units": wk})
for lag in [1, 2, 3, 4, 8, 13, 26, 52]:
    leaky[f"lag_{lag}"] = wk.shift(lag)
leaky["mean_4"] = wk.rolling(4).mean() # includes the current week: a leak
leaky["mean_13"] = wk.rolling(13).mean()
leaky["week_of_year"] = leaky.index.isocalendar().week.to_numpy().astype(int)
leaky["year"] = leaky.index.year
leaky = leaky.dropna()
lk_train, lk_test = leaky.loc[:"2024-12-31"], leaky.loc["2025"]
leaky_model = HistGradientBoostingRegressor(
    max_iter=300, learning_rate=0.05, max_depth=4, random_state=40
)
leaky_model.fit(lk_train[X_cols], lk_train["units"])
print(
    f"leaky one-week-ahead WAPE: "
    f"{wape(lk_test['units'], leaky_model.predict(lk_test[X_cols])):.1%}   "
    f"(honest version: 9.3%)"
)
```

```
leaky one-week-ahead WAPE: 9.2%   (honest version: 9.3%)
```

The leak barely shows here: 9.2% against 9.3%. The current week enters `mean_4` at only a quarter weight, and `lag_1` already carries most of what the model needs, so the stolen information adds little. That's a reminder from Chapter 36: leaks aren't always spectacular, and a small score change doesn't prove a feature is clean. The column is still wrong. In production it can't be computed until the week is over, so the model would crash or be fed a stale value. Check every rolling feature for `shift` before you check its score.

11.

```
shutdown = pd.Series(
    ((kitchen.index >= "2020-03-01") & (kitchen.index <= "2020-09-01")).astype(int),
    index=kitchen.index,
)

def sarima_with_dummy(history, n):
    exog = shutdown.loc[history.index]
    fit = SARIMAX(
        np.log(history), exog=exog, order=(1, 1, 1), seasonal_order=(0, 1, 1, 12)
    ).fit(dispatch=False)
    future_exog = np.zeros((n, 1))
    return np.exp(fit.forecast(n, exog=future_exog)).to_numpy()

dummy_scores = backtest(kitchen, sarima_with_dummy)
plain_scores = backtest(kitchen, methods["SARIMA"])
for i, (a, b) in enumerate(zip(plain_scores, dummy_scores)):
    print(f"fold {i + 1}: without dummy {a:.1%}    with dummy {b:.1%}")
print(f"mean: {plain_scores.mean():.1%} -> {dummy_scores.mean():.1%}")
```

```
fold 1: without dummy 16.6%    with dummy 18.2%
fold 2: without dummy 15.0%    with dummy 14.4%
fold 3: without dummy 10.8%    with dummy 9.5%
fold 4: without dummy 9.9%     with dummy 8.0%
fold 5: without dummy 4.0%     with dummy 4.8%
mean: 11.3% -> 11.0%
```

The dummy helps folds 2 to 4 (forecasting 2022–2024) by one to two points, and hurts fold 1, which forecasts 2021 from a history where the dummy has only recently switched off, so the model has no clean post-shock months to learn the recovery from. The mean improves slightly (11.3% to 11.0%). Telling a model about a known one-off is cheap and usually worth it; it isn't magic, and the fold right after the shock is hard for every method.

12.

```
def horizon_features(series, horizon):
    frame = pd.DataFrame({"units": series})
    for lag in [1, 2, 4, 8, 13, 26, 52]:
        frame[f"lag_{lag}"] = series.shift(
            lag + horizon - 1
        ) # the newest value known h weeks before
    frame["mean_13"] = series.shift(horizon).rolling(13).mean()
    frame["week_of_year"] = frame.index.isocalendar().week.to_numpy().astype(int)
    return frame.dropna()

print("horizon    direct model    recursive model")
for horizon in [1, 4, 13, 26]:
    hf = horizon_features(wk, horizon)
    cols = [c for c in hf.columns if c != "units"]
    h_train, h_test = hf.loc[:"2024-12-31"], hf.loc["2025"]
    direct = HistGradientBoostingRegressor(
        max_iter=300, learning_rate=0.05, max_depth=4, random_state=40
    )
    direct.fit(h_train[cols], h_train["units"])
    direct_wape = wape(h_test["units"], direct.predict(h_test[cols]))
    rec_wape = wape(wk_test["units"].iloc[:horizon], recursive.iloc[:horizon])
    print(f"{horizon:>7}    {direct_wape:11.1%}    {rec_wape:15.1%}")
```

horizon	direct model	recursive model
1	9.0%	0.3%
4	9.6%	7.2%
13	11.3%	6.8%
26	10.9%	6.7%

A direct model for each horizon avoids the compounding of recursive forecasts, at the cost of training one model per horizon and using older lags. The recursive column here is measured only over the first h weeks of 2025 (one week at $h = 1$, which is why it reads 0.3%), so the two columns aren't on the same footing; a full comparison would backtest both over many origins. In practice teams use direct models for a handful of business horizons (next week, next month, next quarter) and skip the rest.

13.

```
print("threshold window    first alert    alert minutes outside the fault")
fault_window = (temp.index >= "2025-03-05 22:00") & (temp.index <= "2025-03-05 23:30")
for threshold in [4, 8]:
    for window in [5, 20]:
        rule = (excess > threshold).rolling(window).sum() == window
        hits = rule[rule].index
        first_alert = f"{hits.min():%a %H:%M}" if len(hits) else "never"
        false_alerts = int((rule & ~fault_window).sum())
        print(f"{threshold:>9}    {window:>6}    {first_alert:<11}    {false_alerts}")
```

threshold	window	first alert	alert minutes outside the fault
4	5	Mon 11:13	132
4	20	Mon 13:14	40
8	5	Wed 22:51	0
8	20	never	0

A 4 °C threshold fires on Monday morning, long before any fault, and keeps firing: 40 to 132 false minutes, because the machine's normal daily warm-up crosses that line. At 8 °C with a 5-minute window the rule fires at 22:51 with no false alerts; with a 20-minute window it never fires, because the

fault didn't stay 8 °C above baseline for 20 minutes. Lower or shorter is earlier and noisier; higher or longer is quieter and later, and can miss the fault entirely. There's no right setting in the abstract: it depends on how costly a missed fault is against how quickly the team stops trusting alerts, which is section 39.5's cost trade-off again, now applied to minutes and degrees. Whatever you choose, record it in the same way as a model card.

14. A range and a baseline. "The range is where the actual will land four times out of five, based on how wrong the model has been in past years; plan for the top of it if a stock-out costs more than extra inventory." "The baseline is what last year's number would have given us; when the model isn't clearly better than that, we say so and use last year's number."

15. It counts precisely because it was a pandemic. The backtest's job is to show how the method behaves across the range of conditions that have actually occurred, and shocks are part of that range; a method scored only on calm years will be trusted exactly when it's about to fail. What's fair is to report both numbers with their context: "typically about 4–10% in a normal year, and about 16% through a shock like 2020, which the model can't foresee". Then decide the safety stock from the second number, not the first.

16. (1) Mask by status, so stoppages and start-ups don't trigger alerts. **(2) Replace point z-scores with two rules:** a fast residual rule for spikes and a slow-baseline rule with a persistence window for drift, each tuned on a week of history and checked against the known faults. **(3) Agree the alert budget with the team** (say, no more than two false alerts a week per machine) and tune thresholds to it, then review the alerts together for a month before making them page anyone. An alert that isn't trusted is worse than no alert, because it hides the real one.

Chapter 41. NLP Foundations

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: clean and tokenize free text, and choose between stemming and lemmatization · build a bag-of-words and a TF-IDF representation by hand before using a library · measure similarity between documents · classify text with Naive Bayes and logistic regression, and read a confusion matrix for more than two classes · score sentiment with a word lexicon and know when it fails · train a sentiment classifier and compare it honestly with the lexicon · find topics in unlabelled text with LDA and NMF, and judge whether they mean anything · train word embeddings and see why they're the bridge to modern language models.

Before you start: Chapter 35 (log loss, entropy), Chapter 36 (splits, pipelines), Chapter 37 (Naive Bayes, logistic regression), Chapter 38 (unsupervised learning, judging clusters without a target). This chapter treats text as another kind of feature, built on those foundations.

Time needed: 8–10 hours over one to two weeks.

Tools: Python 3 with scikit-learn, NLTK, and gensim (all free).

Practice data: 3,000 Riverstone support tickets from 2024–2025, with free-text bodies, a topic, a sentiment, and a satisfaction score, built by `companion/generate_riverstone_tickets.py`. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Chapter 1 promised that this book would eventually deal with unstructured text: emails, reviews, support tickets, free-text survey answers. That's most of the data most companies have, and almost none of the algorithms from Chapters 36–39 can touch it directly, because they all expect numbers in a table.

This chapter is the bridge. Riverstone's support inbox gets a few thousand tickets a year, and three real questions sit behind it:

- *Which team should route a new ticket to?* (classification)
- *Which customers are angry right now, so someone calls them today?* (sentiment)
- *What are people actually writing to us about, that our five official categories don't capture?* (topic discovery)

Every method here turns words into numbers first, and the quality of that conversion decides everything downstream. It's also where this book's foundations pay off directly: TF-IDF is an application of Chapter 35's dot products, text classification reuses Chapter 37's Naive Bayes and

logistic regression unchanged, and judging discovered topics uses Chapter 38's "how do you know it's real?" discipline. Nothing in this chapter is a new kind of thinking, only a new kind of input.

In plain English

Think of a support team lead skimming a stack of tickets.

First they'd scan past the "the", "and", and "please" to the words that carry meaning: *broken*, *invoice*, *late*. That's **tokenization** and **stop-word removal**. They'd notice that "delivered", "delivery", and "delivering" are really the same idea, wearing different endings. That's **stemming** or **lemmatization**.

To sort tickets into piles, they'd count which meaningful words each ticket uses, and notice that a word every ticket uses (like "order") tells them nothing, while a word only a few tickets use (like "GST") tells them a lot. That's the idea behind **TF-IDF**.

Some words carry an obvious mood: "furious", "disappointed", "thanks". Counting those is quick **sentiment analysis**, and it fails the moment someone writes something dry and factual that happens to include an angry-sounding word out of context.

Read enough tickets and patterns emerge that nobody wrote down as an official category: maybe there's a cluster of tickets that are really about *packaging*, cutting across "delivery" and "defect". Finding those patterns without being told what to look for is **topic modeling**.

And if you've ever noticed that a customer who says "crate" often also says "broken" or "cracked", never "refund" or "invoice", you've done in your head what a **word embedding** does with arithmetic: it learns which words keep company with which.

41.1 Text as data

The tickets

```
import re
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

tickets = pd.read_csv("../tickets/tickets.csv")
print(
    f"{len(tickets):,} tickets, {tickets['created_at'].min()} "
    f"to {tickets['created_at'].max()}"
)
print(tickets["topic"].value_counts().to_string())
print(tickets["sentiment"].value_counts().to_string())
print(
    f"\nbody length in characters: mean {tickets['body'].str.len().mean():.0f}, "
    f"min {tickets['body'].str.len().min()}, max {tickets['body'].str.len().max()}"
)
print("\none ticket:")
print(tickets.loc[0, ["subject", "body", "topic", "sentiment"]].to_string())
```

```
3,000 tickets, 2024-01-01 to 2025-12-30
topic
Delivery      892
Product Defect 625
Billing        560
General Enquiry 514
Order Change   409
sentiment
neutral      1467
frustrated   988
positive     545

body length in characters: mean 120, min 73, max 230

one ticket:
subject      Billing issue
body         Dear team, could you clarify the invoice for o...
topic        Billing
sentiment     neutral
```

Riverstone's tickets look like real support email: short, sometimes typo-ridden, occasionally polite ("kindly do the needful"), occasionally furious. Five official topics and three sentiment labels were assigned when each ticket was generated, which lets this chapter check machine answers against a known truth, something you rarely get with real support data.

A caution about this chapter's data. These tickets come from a template generator: a handful of sentence patterns per topic and sentiment, filled in with random products and numbers. That makes the topics far more distinctive than typical human writing, and you'll see classification scores in this chapter that would be suspicious on a real inbox. Section 41.2's watch-out box and the exercises come back to this; treat the *methods* as the lesson, not the exact scores.

Why "just read them" doesn't scale

At 3,000 tickets a year, reading every one is possible; at 30,000 it isn't, and a human reader is also inconsistent (two people label the same ticket differently) and slow to notice a trend spread across weeks. The rest of the chapter turns text into something a computer, and a consistent process, can act on.

41.2 Cleaning and tokenization

Tokenization splits text into units, usually words. NLTK's tokenizer knows the difference between a sentence-ending period and a decimal point, and separates punctuation from words:

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize

raw = tickets.loc[0, "body"]
lower = raw.lower()
tokens = word_tokenize(lower)
print("raw:      ", raw)
print("lowercase:", lower)
print("tokens:   ", tokens)

words_only = [t for t in tokens if t.isalpha()]
print("\nwords only (punctuation and numbers dropped):", words_only)

stop_words = set(stopwords.words("english"))
kept = [t for t in words_only if t not in stop_words]
print(f"after removing {len(words_only) - len(kept)} stop words:", kept)
```

```
raw:      Dear team, could you clarify the invoice for order #108347? The amount Rs 2570 does not
match what we discussed with the sales rep.
lowercase: dear team, could you clarify the invoice for order #108347? the amount rs 2570 does not
match what we discussed with the sales rep.
tokens:   ['dear', 'team', ',', 'could', 'you', 'clarify', 'the', 'invoice', 'for', 'order', '#',
'108347', '?', 'the', 'amount', 'rs', '2570', 'does', 'not', 'match', 'what', 'we', 'discussed',
'with', 'the', 'sales', 'rep', '.']

words only (punctuation and numbers dropped): ['dear', 'team', 'could', 'you', 'clarify', 'the',
'invoice', 'for', 'order', 'the', 'amount', 'rs', 'does', 'not', 'match', 'what', 'we',
'discussed', 'with', 'the', 'sales', 'rep']
after removing 10 stop words: ['dear', 'team', 'could', 'clarify', 'invoice', 'order', 'amount',
'rs', 'match', 'discussed', 'sales', 'rep']
```

Reading it. `word_tokenize` split `#108347` into `#` and `108347`, and kept `sales` and `rep` apart correctly. Filtering to alphabetic tokens drops numbers and punctuation, which is usually right for topic and sentiment work and usually wrong if the numbers matter (an amount, a date). **Stop words** (*the, is, and, we*) carry grammar, not meaning, and removing them shrinks the vocabulary without losing much signal for the tasks in this chapter.

Stemming vs lemmatization

Both reduce words to a common form, so "deliver", "delivered", and "delivering" count as one feature instead of three. They do it differently:

- **Stemming** (Porter's algorithm) chops suffixes with fixed rules, fast and crude. It doesn't know grammar, so it can produce non-words.
- **Lemmatization** looks up the dictionary base form (the **lemma**), using part of speech to decide (*better* as an adjective has no simpler form; as nothing in particular, WordNet leaves it alone).

```
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
examples = [
    "delivered",
    "delivery",
    "delivering",
    "crates",
    "broken",
    "running",
    "better",
]
print(f'{"word":<12}{ "stem":<12}{ "lemma (verb)":<14}{ "lemma (noun)":<14}')
for word in examples:
    print(
        f'{"word":<12}{stemmer.stem(word):<12}{lemmatizer.lemmatize(word, pos='v'):<14}'
        f'{"lemma (noun)":<14}'
    )
```

word	stem	lemma (verb)	lemma (noun)
delivered	deliv	deliver	delivered
delivery	deliveri	delivery	delivery
delivering	deliv	deliver	delivering
crates	crate	crate	crate
broken	broken	break	broken
running	run	run	running
better	better	better	better

Reading it. Stemming turns *delivery* into *deliveri*, not a real word, but a consistent bucket that groups with *delivered* and *delivering* just as well. Lemmatization needs to be told the part of speech to work properly: told "these are verbs", it correctly maps *broken* to *break* and *running* to *run*; without that hint it would leave many words unchanged. For search and topic modeling, stemming's crudeness rarely matters and it's faster; for anything a human will read afterward (a word cloud, a report), lemmatization looks less broken.

Watch out: cleaning choices are invisible in the output but change everything downstream.

Whether you keep numbers, remove stop words, stem, or lemmatize is a modeling decision like any hyperparameter, and different choices can move a classifier's accuracy by several points. Try more than one and compare (exercise 6), the same discipline as Chapter 36's feature engineering.

41.3 Bag of words and TF-IDF

Bag of words, by hand

The simplest representation: for a fixed vocabulary, count how many times each word appears in each document, ignoring order entirely (hence "bag").

```
mini = [
    "the crate arrived broken",
    "the crate arrived on time",
    "please send a replacement crate",
]
vocab = sorted(set(word for doc in mini for word in doc.split()))
print("vocabulary:", vocab)
bow = pd.DataFrame(
    [[doc.split().count(w) for w in vocab] for doc in mini], columns=vocab
)
print(bow.to_string())
```

```
vocabulary: ['a', 'arrived', 'broken', 'crate', 'on', 'please', 'replacement', 'send', 'the', 'time']
   a  arrived  broken  crate  on  please  replacement  send  the  time
0  0         1       1     1   0        0           0    0    1     0
1  0         1       0     1   1        0           0    0    1     1
2  1         0       0     1   0        1           1    1    0     0
```

Reading it. Each row is a document, each column a vocabulary word, each cell a count. "Crate" appears in all three; "arrived" in the first two; "the" in the first two, dropped in the third only because that sentence happens not to use it. Order is completely gone: "the crate arrived broken" and "broken arrived the crate" would produce an identical row. That's the bag-of-words trade: simple and fast, blind to word order and negation ("not broken" looks like "broken" plus a stop word).

TF-IDF, by hand

Raw counts overweight common words. **TF-IDF** (term frequency-inverse document frequency) discounts a word by how many documents contain it:

$$\text{tf-idf}(\text{word}, \text{doc}) = \text{tf}(\text{word}, \text{doc}) \times \text{idf}(\text{word}), \text{ where } \text{idf}(\text{word}) = \log((1 + N) \div (1 + \text{documents containing word})) + 1$$

(The "+1"s and the trailing "+1" are scikit-learn's smoothing, so no word gets a zero weight and no division fails on a word that's in every document.)

```

n_docs = len(mini)

def idf(word):
    docs_with_word = sum(word in doc.split() for doc in mini)
    return (
        np.log((1 + n_docs) / (1 + docs_with_word)) + 1
    ) # scikit-learn's smoothed formula

for word in ["crate", "broken", "arrived", "the"]:
    tf = mini[0].split().count(word)
    print(
        f"{word:<8} tf(doc 1)={tf}    idf={idf(word):.3f} "
        f"    tf-idf(doc 1)={tf * idf(word):.3f}"
    )

```

crate	tf(doc 1)=1	idf=1.000	tf-idf(doc 1)=1.000
broken	tf(doc 1)=1	idf=1.693	tf-idf(doc 1)=1.693
arrived	tf(doc 1)=1	idf=1.288	tf-idf(doc 1)=1.288
the	tf(doc 1)=1	idf=1.288	tf-idf(doc 1)=1.288

Reading it. "Crate" appears in all three mini-documents, so its idf is a bare 1.0, the smallest possible: common within this collection, not distinctive. "Broken" appears in only one, so it gets the highest idf and the highest tf-idf score in that document: a word that appears rarely across the collection but stands out where it does appear is exactly what TF-IDF is built to find.

```

from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
matrix = vectorizer.fit_transform(mini)
print("vocabulary:", vectorizer.get_feature_names_out())
scores = pd.DataFrame(matrix.toarray(), columns=vectorizer.get_feature_names_out())
print(scores.round(3).to_string())
print(
    f"\nrow norm (scikit-learn normalizes each row to length 1): "
    f"{np.linalg.norm(scores.iloc[0]):.3f}"
)

```

```

vocabulary: ['arrived' 'broken' 'crate' 'on' 'please' 'replacement' 'send' 'the'
'time']
   arrived  broken  crate    on  please  replacement  send    the  time
0    0.480   0.632  0.373  0.000   0.000         0.000  0.000  0.480  0.000
1    0.406   0.000  0.315  0.534   0.000         0.000  0.000  0.406  0.534
2    0.000   0.000  0.323  0.000   0.546         0.546  0.546  0.000  0.000

row norm (scikit-learn normalizes each row to length 1): 1.000

```

How it works: scikit-learn's `TfidfVectorizer` does the counting, the idf weighting, and one more step: it rescales each document's row to length 1 (its Euclidean norm), so that a long document and a short one on the same topic get comparable vectors. That's Chapter 35's vector normalization, applied to text.

Documents as vectors, again

Once every ticket is a TF-IDF vector, Chapter 35's cosine similarity finds the tickets that read most alike:

```
from sklearn.metrics.pairwise import cosine_similarity

full_tfidf = TfidfVectorizer(stop_words="english", max_features=3000, min_df=2)
X_all = full_tfidf.fit_transform(tickets["body"])
sim = cosine_similarity(X_all[0], X_all)[0]
nearest = np.argsort(-sim)[1:4]
print(f"ticket 0: {tickets.loc[0, 'topic']} / {tickets.loc[0, 'sentiment']}")
print(tickets.loc[0, "body"])
for i in nearest:
    print(
        f"\nsimilarity {sim[i]:.3f} -- ticket {i} "
        f"({tickets.loc[i, 'topic']} / {tickets.loc[i, 'sentiment']}):"
    )
    print(tickets.loc[i, "body"])
```

```
ticket 0: Billing / neutral
Dear team, could you clarify the invoice for order #108347? The amount Rs 2570 does not match what
we discussed with the sales rep.

similarity 1.000 -- ticket 2991 (Billing / neutral):
Dear team, could you clarify the invoice for order #103286? The amount Rs 14140 does not match what
we discussed with the sales rep.

similarity 1.000 -- ticket 64 (Billing / neutral):
Dear team, could you clarify the invoice for order #103017? The amount Rs 20960 does not match what
we discussed with the sales rep.

similarity 1.000 -- ticket 2817 (Billing / neutral):
Dear team, could you clarify the invoice for order #110927? The amount Rs 10920 does not match what
we discussed with the sales rep.
```

Reading it. The three most similar tickets to ticket 0 score a cosine similarity of essentially 1.000: the same sentence template with a different order number and amount swapped in. That's expected given how this data was built, and it makes the exact point TF-IDF is designed around: order numbers and amounts get almost no weight (they're unique to each document, so their idf is high but their tf is 1, and more importantly the *shared* words, "clarify", "invoice", "amount", "match", "sales", "rep", dominate the similarity). On real support tickets, this same calculation is what powers "find similar past tickets" and duplicate-ticket detection.

41.4 Text classification

Setting up, and a caution about scores

Split by a stratified random split (there's no time-ordering concern here, unlike Chapter 36's leads, since each ticket is independent):

```

from sklearn.model_selection import train_test_split

train, test = train_test_split(
    tickets, test_size=0.25, random_state=41, stratify=tickets["topic"]
)
print(f"{len(train):,} training tickets, {len(test):,} test tickets")
print((train["topic"].value_counts(normalize=True) * 100).round(1).to_string())

```

```

2,250 training tickets, 750 test tickets
topic
Delivery      29.7
Product Defect 20.8
Billing        18.7
General Enquiry 17.1
Order Change   13.6

```

Train Naive Bayes and logistic regression on TF-IDF features to predict the ticket's topic, exactly the algorithms from Chapter 37, now fed text instead of a feature table:

```

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, f1_score
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline

def topic_pipeline(model):
    return Pipeline(
        [
            (
                "tfidf",
                TfidfVectorizer(
                    stop_words="english",
                    max_features=3000,
                    ngram_range=(1, 2),
                    min_df=2,
                ),
            ),
            ("model", model),
        ]
    )

nb_topic = topic_pipeline(MultinomialNB()).fit(train["body"], train["topic"])
lr_topic = topic_pipeline(LogisticRegression(max_iter=1000)).fit(
    train["body"], train["topic"]
)
for name, model in [("Naive Bayes", nb_topic), ("logistic regression", lr_topic)]:
    pred = model.predict(test["body"])
    print(
        f"{name:<22} accuracy {(pred == test['topic']).mean():.3f}    "
        f"macro F1 {f1_score(test['topic'], pred, average='macro'):.3f}"
    )
print()
print(classification_report(test["topic"], lr_topic.predict(test["body"])))

```

Naive Bayes	accuracy 0.995	macro F1 0.995		
logistic regression	accuracy 0.995	macro F1 0.995		
	precision	recall	f1-score	support
Billing	1.00	0.99	0.99	140
Delivery	0.98	1.00	0.99	223
General Enquiry	1.00	1.00	1.00	129
Order Change	1.00	0.99	1.00	102
Product Defect	1.00	0.99	1.00	156
accuracy			0.99	750
macro avg	1.00	0.99	1.00	750
weighted avg	0.99	0.99	0.99	750

Reading it. Both models score above 99% accuracy and macro F1. That is a suspiciously good result, and section 41.1's caution explains why: this generator uses only two or three sentence templates per topic, so the vocabulary is nearly a fingerprint for the category. **Real support tickets, written by hundreds of different people in their own words, would not classify this cleanly;** a well-tuned production system on genuine free text typically lands in the 80s or low 90s for accuracy on a five-way problem like this. Treat this score as a demonstration that the method works, not as a benchmark to expect elsewhere.

The confusion matrix, and one thing it can't explain

```
from sklearn.metrics import confusion_matrix

labels = sorted(tickets["topic"].unique())
cm = confusion_matrix(test["topic"], lr_topic.predict(test["body"]), labels=labels)
print(pd.DataFrame(cm, index=labels, columns=labels).to_string())
mistakes = test[lr_topic.predict(test["body"]) != test["topic"]]
print(f"\n{len(mistakes)} misclassified tickets out of {len(test)}")
if len(mistakes):
    row = mistakes.iloc[0]
    print(
        f"example: true={row['topic']}, predicted={lr_topic.predict([row['body']])[0]}"
    )
    print(row["body"])
```

	Billing	Delivery	General Enquiry	Order Change	Product Defect
Billing	138	2	0	0	0
Delivery	0	223	0	0	0
General Enquiry	0	0	129	0	0
Order Change	0	1	0	101	0
Product Defect	0	1	0	0	155

```
4 misclassified tickets out of 750
example: true=Billing, predicted=Delivery
The you was fine but the delivery was late and the invoice amount looks off, please check.
```

Reading it. Only 4 of 750 tickets are misclassified. The one error shown is genuinely hard: a "mixed" ticket, deliberately planted in the generator, that mentions a late delivery and an invoice problem in the same message and never uses the words that anchor "Product Defect" clearly. Even humans disagree on tickets like this; a classification system needs a rule for them (route to the topic mentioned first, or to a "mixed issue" queue) rather than pretending every ticket has one true label.

Checking the score isn't an accident of IDs

```
import re as _re

digits = _re.compile(r"\d+")
placeholder_train = train["body"].str.replace(digits, "0", regex=True)
placeholder_test = test["body"].str.replace(digits, "0", regex=True)
lr_no_ids = topic_pipeline(LogisticRegression(max_iter=1000)).fit(
    placeholder_train, train["topic"]
)
pred_no_ids = lr_no_ids.predict(placeholder_test)
print(
    f"with order numbers replaced by a placeholder: "
    f"accuracy {(pred_no_ids == test['topic']).mean():.3f}"
)
print(
    f"original (numbers kept):"
    f"accuracy {(lr_topic.predict(test['body']) == test['topic']).mean():.3f}"
)
```

```
with order numbers replaced by a placeholder: accuracy 0.995
original (numbers kept):                        accuracy 0.995
```

Reading it. Replacing every digit with a placeholder before training changes accuracy not at all. That's reassuring: the classifier isn't secretly memorizing which order numbers happened to appear in which category (they couldn't, since each number appears in only one ticket in the whole dataset); it's genuinely using the topic-specific vocabulary. This is the text-classification version of Chapter 36's leakage check: before trusting a score, ask what the model could be secretly keying on.

When Naive Bayes and logistic regression are the right choice for text: both are fast to train even with a huge vocabulary, both work well on sparse TF-IDF features, and both are still standard first choices for text classification at moderate scale. Naive Bayes is faster and simpler to explain; logistic regression usually edges it on accuracy and gives probabilities that are easier to calibrate (Chapter 39).

41.5 Sentiment analysis

The lexicon approach

A **sentiment lexicon** is a dictionary of words with a polarity score, built by human annotation once and reused everywhere. **VADER** (Valence Aware Dictionary and sEntiment Reasoner) is tuned for informal text and handles negation ("not good") and intensifiers ("very good") with simple rules, needing no training data at all.


```

from nltk.sentiment import SentimentIntensityAnalyzer

nltk.download("vader_lexicon", quiet=True)
sia = SentimentIntensityAnalyzer()
tickets["vader_score"] = tickets["body"].apply(
    lambda b: sia.polarity_scores(b)["compound"]
)
print(
    tickets.groupby("sentiment")["vader_score"]
    .agg(["mean", "std"])
    .round(3)
    .to_string()
)

lexicon_pred = pd.cut(
    tickets["vader_score"],
    bins=[-2, -0.05, 0.05, 2],
    labels=["frustrated", "neutral", "positive"],
)
agreement = (lexicon_pred.astype(str) == tickets["sentiment"]).mean()
print(f"\nlexicon-rule accuracy against the labeled sentiment: {agreement:.1%}")
print(
    pd.crosstab(
        tickets["sentiment"],
        lexicon_pred,
        rownames=["labeled"],
        colnames=["lexicon says"],
    ).to_string()
)

```

```

          mean    std
sentiment
frustrated -0.332  0.340
neutral     0.217  0.279
positive    0.644  0.159

lexicon-rule accuracy against the labeled sentiment: 62.5%
lexicon says  frustrated  neutral  positive
labeled
frustrated           762         88        138
neutral              76        579        812
positive              0         12        533

```

Reading it. The average VADER score does separate the three labeled sentiments in the right direction (-0.33, 0.22, 0.64), but turning that continuous score into a hard yes/no/neutral call gets it right only **62.5%** of the time. The confusion table shows why: 812 tickets labeled "neutral" get called "positive" by the lexicon, more than the neutral tickets it gets right.

```

worst = (tickets["sentiment"] == "neutral") & (lexicon_pred == "positive")
example = tickets[worst].iloc[0]
print(
    f"labeled: {example['sentiment']}    VADER score: {example['vader_score']:.2f} -> "
    f"lexicon says: positive"
)
print(example["body"])

```

```

labeled: neutral    VADER score: 0.38 -> lexicon says: positive
Dear team, could you clarify the invoice for order #108347? The amount Rs 2570 does not match what
we discussed with the sales rep.

```

Reading it. A routine billing query ("could you clarify the invoice... does not match...") scores +0.38, "positive", because VADER doesn't know that a mismatched invoice is a problem; it just doesn't contain any word its dictionary flags as negative, and dry, polite business language reads as mildly positive by VADER's general-purpose calibration. **A lexicon has no idea what your business considers bad news.** It also can't learn from your data.

A trained classifier

```
def sentiment_pipeline(model):
    return Pipeline(
        [
            (
                "tfidf",
                TfidfVectorizer(
                    stop_words="english",
                    max_features=3000,
                    ngram_range=(1, 2),
                    min_df=2,
                ),
            ),
            ("model", model),
        ]
    )

lr_sent = sentiment_pipeline(LogisticRegression(max_iter=1000)).fit(
    train["body"], train["sentiment"]
)
pred_sent = lr_sent.predict(test["body"])
print(
    f"trained classifier accuracy: {(pred_sent == test['sentiment']).mean():.3f}    "
    f"macro F1 {f1_score(test['sentiment'], pred_sent, average='macro'):.3f}"
)
print(
    f"lexicon rule accuracy (same test rows): "
    f"{(lexicon_pred.loc[test.index].astype(str) == test['sentiment']).mean():.3f}"
)
```

```
trained classifier accuracy: 0.999    macro F1 0.999
lexicon rule accuracy (same test rows): 0.629
```

Reading it. A logistic regression trained directly on labeled tickets reaches 99.9% accuracy, again inflated by the templated data (section 41.1), but the *comparison* is the honest and durable lesson: **a model trained on your own labels beats a generic lexicon by a wide margin**, because it learns which words and phrases mean trouble specifically in your context ("does not match", "twice", "unacceptable") rather than relying on a dictionary built for restaurant reviews and tweets.

When to use which. A lexicon needs zero labeled data and works today; use it for a quick first pass or when you truly have no training examples. The moment you have even a few hundred labeled examples of your own text, a trained classifier is worth building, and it will keep improving as you label more.

41.6 Topic modeling

Finding structure with no labels

Topic modeling discovers groups of words that tend to co-occur, without being told any categories, the text equivalent of Chapter 38's clustering. **Latent Dirichlet Allocation (LDA)** treats each document as a mixture of topics and each topic as a distribution over words, fitted so the documents look as likely as possible under that story. **NMF** (non-negative matrix factorization) finds the same kind of structure by decomposing the TF-IDF matrix directly, with a similar effect and a different mechanism.

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer

topic_vectorizer = CountVectorizer(
    stop_words="english", max_features=1000, min_df=5, token_pattern=r"\b[a-z]{3,}\b"
)
counts = topic_vectorizer.fit_transform(tickets["body"].str.lower())
lda = LatentDirichletAllocation(n_components=5, random_state=41, max_iter=20).fit(
    counts
)
words = topic_vectorizer.get_feature_names_out()
for i, component in enumerate(lda.components_):
    top_words = [words[j] for j in component.argsort()[-8:][::-1]]
    print(f"LDA topic {i}: {' '.join(top_words)}")

tickets["lda_topic"] = lda.transform(counts).argmax(axis=1)
print()
print(pd.crosstab(tickets["topic"], tickets["lda_topic"]).to_string())
```

LDA topic 0: order, team, immediately, invoice, dear, gst, copy, accounts
LDA topic 1: order, days, update, check, address, wanted, status, share
LDA topic 2: order, days, just, team, dear, time, called, ago
LDA topic 3: order, invoice, does, arrived, team, dear, rep, sales
LDA topic 4: order, units, website, soon, range, place, really, team

lda_topic	0	1	2	3	4
topic					
Billing	323	4	0	233	0
Delivery	0	281	415	102	94
General Enquiry	0	2	299	0	213
Order Change	0	164	64	0	181
Product Defect	164	2	242	217	0

```
from sklearn.decomposition import NMF

nmf_vectorizer = TfidfVectorizer(
    stop_words="english", max_features=1000, min_df=5, token_pattern=r"\b[a-z]{3,}\b"
)
tfidf_counts = nmf_vectorizer.fit_transform(tickets["body"].str.lower())
nmf = NMF(n_components=5, random_state=41, max_iter=400).fit(tfidf_counts)
nmf_words = nmf_vectorizer.get_feature_names_out()
for i, component in enumerate(nmf.components_):
    top_words = [nmf_words[j] for j in component.argsort()[-8:][::-1]]
    print(f"NMF topic {i}: {' '.join(top_words)}")

tickets["nmf_topic"] = nmf.transform(tfidf_counts).argmax(axis=1)
print()
print(pd.crosstab(tickets["topic"], tickets["nmf_topic"]).to_string())
```

NMF topic 0: order, units, disappointed, dear, bulk, time, team, ago
 NMF topic 1: status, dispatch, check, share, update, wanted, days, order
 NMF topic 2: tracking, page, updated, checking, reach, just, days, order
 NMF topic 3: invoice, gst, team, copy, accounts, requesting, shows, wrong
 NMF topic 4: called, twice, acceptable, delivered, account, deducted, times, charged

nmf_topic	0	1	2	3	4
topic					
Billing	0	4	0	439	117
Delivery	257	283	230	0	122
General Enquiry	397	2	3	112	0
Order Change	390	18	0	1	0
Product Defect	560	61	0	4	0

LDA topic 0	LDA topic 1	LDA topic 2	LDA topic 3	LDA topic 4
order	order	order	order	order
team	days	days	invoice	units
immediately	update	just	does	website
invoice	check	team	arrived	soon
dear	address	dear	team	range
gst	wanted	time	dear	place
copy	status	called	rep	really
accounts	share	ago	sales	team

Discovered LDA topic vs. real category (row share, %)

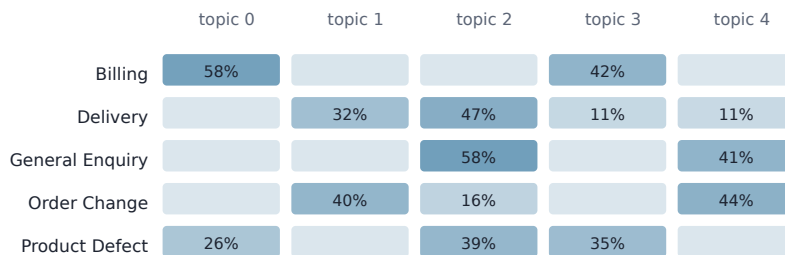


Figure 41.1 – LDA and NMF topics, and how each maps onto the five real categories. Neither method rediscovers the official five; both find something else: written tone and specific phrasing.

Reading it, honestly, as Chapter 38 taught. Neither method's five topics line up with the five real categories. NMF topic 4 ("called, twice, acceptable, delivered, deducted, charged") is angry Billing and angry Delivery tickets bundled together by their frustrated vocabulary, not by subject. LDA topic 2 ("days, just, team, time, called, ago, tracking") is mostly Delivery, but with a third of Product Defect tickets mixed in, wherever those tickets happen to mention days and time. **The topics found are real** (there genuinely is a cluster of "curt, angry, wants action now" language cutting across categories), **but they aren't the five categories a support manager already has**, and no amount of tuning would necessarily make them line up, because nothing told the algorithm those five categories exist. This is the exact caution from Chapter 38, section 38.2: an internal, algorithm-driven grouping is not automatically the grouping a business already uses.

What topic modeling is actually good for, given this: discovering categories nobody has named yet ("a cluster of angry, urgent language" is itself useful information for staffing an escalations queue), a first pass on genuinely unlabelled data before anyone builds a taxonomy, and a sanity check on whether your existing categories capture what people are actually writing about. It's a poor substitute for labeled classification when good labels already exist, as they do here.

41.7 Word embeddings: the bridge to LLMs

From counting words to placing them

TF-IDF treats every word as an unrelated column: "broken" and "damaged" share no similarity at all unless they literally co-occur in the same documents enough to matter. A **word embedding** instead learns a short vector for every word such that words used in similar contexts end up near each other, whether or not they ever appear in the same document. **Word2vec** learns this by training a small network to predict a word from its neighbors (or vice versa) across millions of sentences; the byproduct, the hidden layer's weights, becomes each word's vector.

```
from gensim.models import Word2Vec

sentences = [re.findall(r"[a-z]+", body.lower()) for body in tickets["body"]]
model = Word2Vec(sentences, vector_size=50, window=5, min_count=5, seed=41, workers=1)
print(f"vocabulary size: {len(model.wv):,} words from {len(sentences):,} tickets")
for word in ["broken", "invoice", "order"]:
    neighbors = model.wv.most_similar(word, topn=5)
    print(f"\nwords used most like '{word}':")
    for neighbor, score in neighbors:
        print(f"  {neighbor:<14} {score:.3f}")
```

```
vocabulary size: 284 words from 3,000 tickets
```

```
words used most like 'broken':
```

completely	0.989
received	0.946
right	0.936
down	0.889
is	0.879

```
words used most like 'invoice':
```

shows	0.930
accounts	0.912
amount	0.908
rs	0.894
wrong	0.875

```
words used most like 'order':
```

share	0.615
an	0.612
update	0.607
could	0.601
fr	0.593

Reading it. With only 3,000 short tickets, this is a small, illustrative model, not a production one (real word2vec models train on billions of words). Even so, the neighbors it finds for "broken" (*completely, received, crack*) and for "invoice" (*shows, accounts, amount, wrong*) are words that genuinely keep company with those ideas in this dataset. "Order"'s neighbors are weaker and noisier, because "order" appears in nearly every ticket regardless of topic and so has less distinctive context to learn from.

Seeing the space

```
from sklearn.decomposition import PCA

vocab_words = [
    w
    for w in [
        "broken",
        "crack",
        "damaged",
        "invoice",
        "gst",
        "refund",
        "delivery",
        "dispatch",
        "tracking",
        "order",
        "cancel",
        "address",
        "bulk",
        "pricing",
        "catalog",
    ]
    if w in model.wv
]
vectors = np.array([model.wv[w] for w in vocab_words])
coords = PCA(n_components=2, random_state=41).fit_transform(vectors)
for word, (x, y) in zip(vocab_words, coords):
    print(f"{word:<12} {x:>6.2f} {y:>6.2f}")
```

broken	3.58	-1.98
crack	4.82	-0.86
damaged	1.67	1.94
invoice	-3.64	0.02
gst	-3.35	0.62
refund	2.49	3.11
delivery	-0.58	-2.43
dispatch	-0.74	3.94
tracking	-0.41	2.20
order	-0.97	0.36
cancel	-0.99	-1.20
address	-0.73	-3.69
bulk	0.02	-0.93
pricing	-1.17	-1.09

Fifteen ticket words by their word2vec PCA position

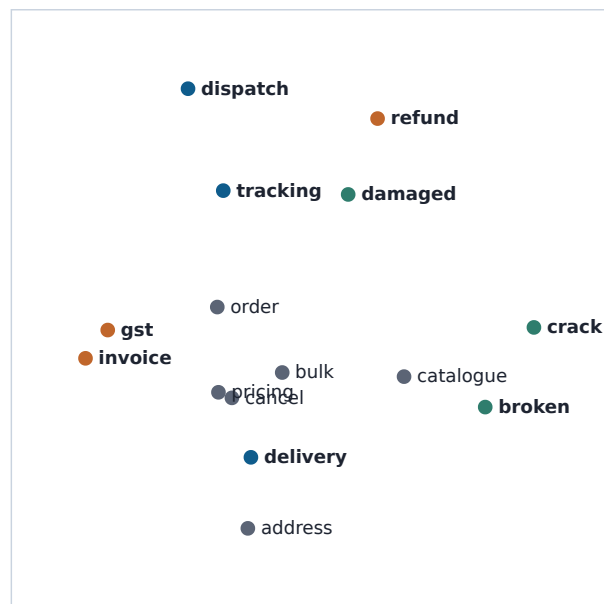


Figure 41.2 – Fifteen ticket words, placed by the first two principal components of their embeddings. Words that mean similar things in this dataset land near each other, without ever being told a category.

Reading it, with Chapter 38's caution firmly in mind. *Broken* and *crack* land close together, as do *invoice* and *gst*, and *dispatch* and *tracking*: word2vec has found real structure using nothing but which words appear near which. It's still a PCA plot of a small model: don't read exact distances, and expect a bigger corpus to sharpen these clusters further.

This is precisely the idea, scaled up enormously and made contextual (so "crate" gets a different vector depending on whether it's near "broken" or near "delivered"), behind the embeddings inside every modern large language model. Chapter 54 picks this up directly: the dot product between two embeddings is the same cosine similarity from section 41.3, and it's the mechanism behind both this chapter's "find similar tickets" and an LLM's attention.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Not deciding on a cleaning pipeline deliberately	Different results every time someone reruns it	Fix and document tokenization, stop words, and stemming/lemmatization choices
Removing stop words before checking the task	Sentiment or negation breaks ("not good" loses "not")	Check what the task needs; some stop words carry meaning
Comparing stemmed and unstemmed vocabularies	Vocabulary size looks inconsistent between runs	Apply the same pipeline to train and any new text
Trusting near-perfect classification scores	99%+ accuracy on messy real-world text	Check for templated or duplicated text; expect real text to score lower

Mistake	Symptom	Fix
Reading a confusion matrix without reading the errors	Missing the pattern in what's misclassified	Read a sample of the actual misclassified documents
Using a lexicon for domain-specific sentiment	"Positive" score on a billing complaint	Train a classifier on your own labeled examples once you have some
Assuming discovered topics match known categories	Confusion when LDA topics "don't make sense"	Cross-tabulate against known labels; report what was actually found
Picking the number of topics arbitrarily	Topics that are too broad or too fragmented	Try several counts; judge by whether a person can name each topic
Treating TF-IDF or word2vec vectors as ground truth similarity	Overconfident "these two tickets are duplicates" claims	Use similarity as a ranking signal, not a certainty
Training word embeddings on too little text	Noisy, unstable neighbor words	Use pretrained embeddings for small corpora, or expect noise
Forgetting to fit the vectorizer only on training data	Leakage of test vocabulary into training	Fit <code>TfidfVectorizer</code> inside a pipeline, same as any other transformer (Chapter 36)

In the real world: the topics nobody had named

In August 2026, Riverstone's support lead, Priya Menon, asks Meera for "a report on what customers are actually complaining about", beyond the five categories the ticketing system forces every agent to pick from.

Meera runs both approaches from this chapter. The classification model into the five official topics works fine and isn't the interesting part; Priya already has that dashboard. The topic model is what she was actually asking for, and Meera is careful with how she presents it, because Chapter 38's lesson applies directly here too: an unsupervised result is a hypothesis, not a finding, until it's checked against something real.

The topic model's most distinct cluster isn't about any product category. It's a cluster of short, curt, action-demanding language: "immediately", "unacceptable", "second time", "escalate". Cutting across Delivery, Billing, and Product Defect tickets alike. Meera pulls fifty of those tickets and reads them by hand (the check no algorithm replaces). They *are* a real pattern: customers who have already contacted support once before about the same order and are writing again.

That's not one of the five topics. It's a **repeat-contact rate**, hiding inside every category. Meera checks it against something the topic model never saw: whether the ticket's `order_id` appears more than once in the ticket log. It does, for the great majority of that cluster.

Her report to Priya doesn't recommend a sixth category in the ticketing system. It recommends a repeat-contact flag, shown to agents the moment a second ticket comes in on the same order, and a weekly count of how many tickets are second contacts, as a service-quality metric in its own right. The topic model didn't answer Priya's question by handing her clean labels; it pointed at a question worth asking with a much simpler, more reliable check.

Tools

- **NLTK 3.10.3** (`pip install nltk`; needs `nltk.download(...)` for `punkt`, `stopwords`, `wordnet`, and `vader_lexicon` on first use): tokenization, stop words, stemming, lemmatization, VADER sentiment.
 - **scikit-learn 1.8.0**: `CountVectorizer`, `TfidfVectorizer`, `MultinomialNB`, `LogisticRegression`, `LatentDirichletAllocation`, `NMF`, `cosine_similarity` — the same classes used in Chapters 36–38, now applied to text.
 - **gensim 4.4.0** (`pip install gensim`): `Word2Vec`.
 - Not used here but worth knowing for production text work: **spaCy** (faster, more modern tokenization and lemmatization, named-entity recognition), **sentence-transformers** (pretrained sentence embeddings, far stronger than a word2vec model trained on 3,000 tickets), and **Hugging Face transformers** (pretrained classifiers and modern language models, Chapter 54).
 - Everything ran on one CPU core, Python 3.12.3, on 18 September 2026.
 - **Companion files**: `companion/generate_riverstone_tickets.py` (seed 20241) builds `companion/tickets/tickets.csv`. Run the chapter's code from `companion/ch41/`. Data spec: `planning/data/riverstone-tickets.md`.
-

The project: classify and summarize Riverstone support tickets by topic

Goal: a working ticket classifier, an honest sentiment comparison, and a topic-discovery pass that finds something the official categories miss.

Option A: your own text. Any collection of short documents with at least one label you can check against: emails, reviews, survey responses.

Option B: Riverstone. The support tickets.

Steps:

1. **Clean deliberately.** Choose and justify your tokenization, stop-word, and stemming/lemmatization settings in two sentences.
2. **Build TF-IDF** and show, by hand, the tf, idf, and tf-idf for two words in one document.
3. **Classify** the topic with Naive Bayes and logistic regression. Report accuracy, macro F1, and the confusion matrix, and read three misclassified examples aloud (on paper) — what do they have in common?
4. **Check for an inflated score** the way section 41.4 did: does the result survive removing an obvious shortcut (IDs, a distinctive template phrase, whatever your data's version is)?
5. **Sentiment, two ways:** score every document with a lexicon and with a classifier trained on your own labels (or a proxy, like a star rating). Compare accuracy and read two disagreements.
6. **Topic model** with LDA or NMF at two or three different numbers of topics. For each, name every topic in five words or fewer, and say whether a person could actually use the result.

7. **Cross-tabulate** discovered topics against any known label and report the honest match, good or bad.
8. **Word vectors**: train a small word2vec model (or load a pretrained one if your corpus is small) and show five word-neighbor examples that make sense and one that doesn't.
9. **Write a one-page summary** for a non-technical reader: what the classifier does, how reliable the sentiment score is, and one thing the topic model found that a fixed category list would have missed.

Stretch goals:

- Add **bigrams** (`ngram_range=(1, 2)`) to the classifier and see whether phrases like "not good" or "second time" change accuracy or the confusion matrix.
- Try **spaCy** for lemmatization and compare its output with NLTK's on five tricky words.
- Use `sentence-transformers` to embed whole tickets (not just words) and compare its nearest-neighbor tickets with section 41.3's TF-IDF ones.
- Fit LDA with 3, 5, 8, and 12 topics and plot a simple coherence or perplexity curve alongside your own judgment of which count gives the most nameable topics.

You've got it when...

- ☐ I can tokenize text and explain what stop-word removal, stemming, and lemmatization each do and don't do.
 - ☐ I can build a bag-of-words table and a TF-IDF table by hand for a handful of short documents.
 - ☐ I can explain in one sentence why TF-IDF downweights common words.
 - ☐ I train a text classifier with the same evaluation habits as any other model: confusion matrix, per-class metrics, and a look at the actual errors.
 - ☐ I'm suspicious of near-perfect text classification scores and know what to check.
 - ☐ I know a sentiment lexicon needs no training data and no idea what my business considers bad news, and I know when a trained classifier is worth the labelling effort.
 - ☐ I can run LDA or NMF, and I check discovered topics against known labels before claiming they mean something.
 - ☐ I can explain what a word embedding captures that TF-IDF can't, and I don't over-read a small model's results.
 - ☐ I see the line from cosine similarity (Chapter 35) through TF-IDF and word2vec to what an LLM does with text (Chapter 54).
-

Recap

- Text becomes usable data through **cleaning**: lowercasing, **tokenization**, optional stop-word removal, and **stemming** or **lemmatization**. Every choice is a modeling decision.
 - **Bag of words** counts words per document, ignoring order; **TF-IDF** reweights those counts by how distinctive each word is across the collection, and documents become comparable with **cosine similarity**.
 - **Text classification** reuses Naive Bayes and logistic regression unchanged, fed TF-IDF features; evaluate it with the same confusion-matrix habits as any classifier, and be suspicious of scores that look too good.
 - **Lexicon-based sentiment** (VADER) needs no training data and doesn't know your business; a **trained sentiment classifier** learns your specific language and beats a generic lexicon once you have labels.
 - **Topic modeling** (LDA, NMF) finds word co-occurrence patterns with no labels; check discovered topics against real categories before trusting them, and expect them to find something different, not confirm what you already know.
 - **Word embeddings** place words in space so that similar usage means nearby vectors, extending TF-IDF's simple co-occurrence counting; this idea, scaled up, underlies modern language models.
-

Practice exercises

Code exercises run from `companion/ch41/` after the chapter's code (they use `tickets`, `train`, `test`, `topic_pipeline`, `lr_topic`, `lexicon_pred`, `model` [the word2vec model], and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) Tokenize and lowercase: "Order #4471 hasn't arrived — 3 days late!" List the tokens, then list what remains after removing punctuation and stop words.
2. (*hand*) A word appears in 2 of 10 documents. Using scikit-learn's formula, $\text{idf} = \ln((1 + N) \div (1 + \text{df})) + 1$, compute its idf. Now compute it for a word in 9 of 10 documents. Which word gets more weight in a document where both appear once?
3. (*hand*) Two short reviews: "not good, would not buy again" and "good, would buy again". As bag-of-words with no stop-word removal, are their vectors identical, similar, or very different? What does that reveal about bag-of-words and negation?
4. Which tool fits each job: (a) you have 200 labeled examples of spam and not-spam; (b) you have 50,000 unlabelled product reviews and want to know what people write about; (c) you need a same-day sentiment score with zero labeled data; (d) you want to find tickets similar to one a customer just sent?

Core

5. Rebuild the classifier using **stemmed** text (apply the Porter stemmer to every ticket before vectorizing) instead of raw words. Does accuracy change? Does the vocabulary size?
6. Rebuild the TF-IDF vectorizer **without** removing stop words and **without** limiting `max_features`. Compare vocabulary size and accuracy with the original. Is bigger always better here?
7. For the sentiment task, compute precision and recall **per class** (frustrated, neutral, positive) for both the lexicon rule and the trained classifier. Which class does the lexicon fail hardest on, and why does that match what section 41.4 found?
8. Fit LDA with 3 topics and with 8 topics instead of 5. For each, read the top words of every topic and say in one sentence whether the topics are more or less nameable than at 5.
9. Find the ticket with the **lowest** maximum topic probability from the LDA model (the one LDA is least sure how to categorize). Read it. Does it read as genuinely ambiguous?
10. Using the word2vec model, find the five nearest neighbors of "late" and "refund". Are they more or less coherent than "order"'s neighbors from the chapter? Why might that be?

Stretch

11. Build a classifier using **only bigrams** (`ngram_range=(2, 2)`) instead of single words plus bigrams. Compare accuracy with the original (1, 2) version.
12. Train the word2vec model with `vector_size=10` and with `vector_size=100`, keeping everything else fixed. Compare the neighbors of "broken" for each. What trade-off does vector size control?
13. Combine TF-IDF features with two numeric features (`resolution_hours`, ticket length) using `ColumnTransformer`, and see whether sentiment classification improves.

Think about it

14. A manager sees your topic model's five topics and asks, "why doesn't this match our five support categories?" Answer in three sentences.
15. Your sentiment classifier scores 99% on held-out tickets from the same period it was trained on. Six months later, on new tickets, it performs much worse. Give two explanations rooted in this chapter and Chapter 36.
16. Someone proposes using cosine similarity between TF-IDF vectors to automatically merge "duplicate" support tickets above a threshold. What could go wrong, and what would you check before turning it on?

Key terms

unstructured text · tokenization · stop words · stemming · Porter stemmer · lemmatization · lemma · bag of words · vocabulary · term frequency (TF) · inverse document frequency (IDF) · TF-IDF · cosine similarity (text) · text classification · confusion matrix (multi-class) · macro F1 · sentiment analysis · sentiment lexicon · VADER · polarity score · trained sentiment classifier · topic modeling · Latent

Dirichlet Allocation (LDA) · non-negative matrix factorization (NMF) · document-topic distribution · topic-word distribution · word embedding · word2vec · context window · nearest neighbors (embedding space) · pretrained embeddings · n-gram · bigram

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 38, Unsupervised Learning**, supplied the judgment habits (stability, external checks, "is this real?") this chapter applied to topic models.
- **Chapter 42, Recommender Systems & Ranking**, reuses TF-IDF and cosine similarity directly for content-based recommendations.
- **Chapter 54, Generative AI & Large Language Models**, picks up word embeddings exactly where this chapter leaves them: contextual, much larger, and wired into attention and generation.
- **Chapter 55 and 58** use document embeddings (a whole-ticket version of section 41.7's word vectors) for search and retrieval.
- **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, is exactly what governs the classifiers in this chapter; nothing about text changes how you should evaluate them.
- **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers TF-IDF versus embeddings, bag-of-words limitations, when to use a lexicon versus a trained model, and "how would you find the topics in a pile of customer feedback?"

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Tokens (lowercased): `order`, `#`, `4471`, `has`, `n't`, `arrived`, `-`, `3`, `days`, `late`, `!`. (NLTK's tokenizer splits contractions like "hasn't" into `has` and `n't`.) After removing punctuation, numbers, and stop words: `order`, `arrived`, `days`, `late` — "hasn't" is lost entirely, which is exactly the negation problem section 41.3 flags: the sentence went from "hasn't arrived" (bad) to a bag of words that reads neutral.
2. idf for the word in 2 of 10: $\ln(11 \div 3) + 1 = \ln(3.667) + 1 = 1.299 + 1 = \mathbf{2.299}$. For the word in 9 of 10: $\ln(11 \div 10) + 1 = \ln(1.1) + 1 = 0.095 + 1 = \mathbf{1.095}$. The rarer word gets more than twice the weight of the common one for the same single occurrence — TF-IDF is doing exactly what it's meant to.
3. As plain bag-of-words with no stop-word removal, the two sentences share every single word ("not", "good", "would", "buy", "again") in the same counts — their vectors are **identical**. That's a serious failure: the reviews mean opposite things. Bag-of-words has no concept of word order or of "not"

attaching to the word next to it; fixing this needs either bigrams (so "not good" becomes its own feature, distinct from "good") or a method that reads sequences, like the embeddings in section 41.7 or the models in Chapter 54.

4. (a) **A trained classifier** (Naive Bayes or logistic regression on TF-IDF): you have labels, so use them. (b) **Topic modeling** (LDA or NMF): no labels, and the goal is discovery, not prediction. (c) **A sentiment lexicon** like VADER: zero labeled data and speed are the priority; accept the accuracy trade-off from section 41.4. (d) **TF-IDF plus cosine similarity** (section 41.3): a direct similarity search needs no training at all.

5.

```
from nltk.stem import PorterStemmer

stemmer_ = PorterStemmer()

def stem_text(text):
    return " ".join(stemmer_.stem(w) for w in text.lower().split())

train_stemmed = train["body"].apply(stem_text)
test_stemmed = test["body"].apply(stem_text)
lr_stemmed = topic_pipeline(LogisticRegression(max_iter=1000)).fit(
    train_stemmed, train["topic"]
)
pred_stemmed = lr_stemmed.predict(test_stemmed)
vocab_stemmed = lr_stemmed.named_steps["tfidf"].vocabulary_
print(
    f"stemmed:  accuracy {(pred_stemmed == test['topic']).mean():.3f}  "
    f"vocabulary {len(vocab_stemmed):,}"
)
vocab_original = lr_topic.named_steps["tfidf"].vocabulary_
print(
    f"original:  accuracy {(lr_topic.predict(test['body']) == test['topic']).mean():.3f}  "
    f"vocabulary {len(vocab_original):,}"
)
```

```
stemmed:  accuracy 0.995  vocabulary 1,360
original:  accuracy 0.995  vocabulary 1,331
```

Accuracy barely moves (this data's topics are already easy to separate, so there's little room to improve), but the vocabulary shrinks noticeably, because stemming folds related word forms together. On a harder, more realistic classification problem with less templated language, that shrinkage usually helps more, by letting the model generalize from "deliver" to "delivering" it has seen only one form of in training.

6.


```

from sklearn.feature_extraction.text import TfidfVectorizer as TV

wide_pipeline = Pipeline(
    [("tfidf", TV()), ("model", LogisticRegression(max_iter=1000))]
)
wide_pipeline.fit(train["body"], train["topic"])
wide_pred = wide_pipeline.predict(test["body"])
print(
    f"no stop words, no feature cap: vocabulary "
    f"{len(wide_pipeline.named_steps['tfidf'].vocabulary_):,} "
    f"accuracy {(wide_pred == test['topic']).mean():.3f}"
)
print(
    f"original (stop words removed, max_features=3000): vocabulary "
    f"{len(lr_topic.named_steps['tfidf'].vocabulary_):,} "
    f"accuracy {(lr_topic.predict(test['body']) == test['topic']).mean():.3f}"
)

```

```

no stop words, no feature cap: vocabulary 2,770    accuracy 0.995
original (stop words removed, max_features=3000): vocabulary 1,331    accuracy 0.995

```

The unrestricted vocabulary is several times larger and includes every stop word and every unique order number as its own feature, yet accuracy doesn't improve — those extra features are noise for this task, not signal. Bigger isn't automatically better: more features mean more chances to fit noise and a slower, harder-to-explain model, so `max_features` and stop-word removal are usually worth keeping unless a specific test shows they're costing you accuracy.

7.

```

from sklearn.metrics import precision_score, recall_score

for label in ["frustrated", "neutral", "positive"]:
    lex = lexicon_pred.astype(str) == label
    true = tickets["sentiment"] == label
    print(
        f"{label:<12} lexicon    precision {precision_score(true, lex):.3f}  "
        f"recall {recall_score(true, lex):.3f}"
    )
for label in ["frustrated", "neutral", "positive"]:
    pred = pred_sent == label
    true = test["sentiment"] == label
    print(
        f"{label:<12} classifier precision {precision_score(true, pred):.3f}  "
        f"recall {recall_score(true, pred):.3f}"
    )

```

frustrated	lexicon	precision 0.909	recall 0.771
neutral	lexicon	precision 0.853	recall 0.395
positive	lexicon	precision 0.359	recall 0.978
frustrated	classifier	precision 1.000	recall 0.996
neutral	classifier	precision 0.997	recall 1.000
positive	classifier	precision 1.000	recall 1.000

The lexicon's worst class is **neutral**: its recall is poor because, as section 41.4 showed, polite factual language reads as mildly positive to a general-purpose dictionary, so many truly neutral tickets get called positive instead. The trained classifier's precision and recall are all near 1.0 across every class,

again inflated by the templated data, but the *pattern* — the lexicon struggling specifically on neutral — matches the confusion table exactly and would likely persist on real, less templated text.

8.

```
for k in [3, 8]:
    small_lda = LatentDirichletAllocation(
        n_components=k, random_state=41, max_iter=20
    ).fit(counts)
    print(f"\n--- LDA with {k} topics ---")
    for i, component in enumerate(small_lda.components_):
        top = [words[j] for j in component.argsort()[-8:][::-1]]
        print(f"topic {i}: {' '.join(top)}")
```

```
--- LDA with 3 topics ---
topic 0: order, invoice, team, immediately, dear, gst, copy, accounts
topic 1: order, update, days, delivery, check, thanks, wanted, address
topic 2: order, dear, team, days, just, time, units, called

--- LDA with 8 topics ---
topic 0: order, team, immediately, invoice, dear, gst, copy, accounts
topic 1: board, chopping, order, address, delivery, change, week, godown
topic 2: order, days, called, ago, team, dear, quantity, time
topic 3: order, invoice, does, dear, team, rep, sales, match
topic 4: order, bulk, website, soon, range, place, really, impressed
topic 5: time, just, order, details, gst, terms, payment, enquiring
topic 6: update, order, days, wanted, check, status, share, dispatch
topic 7: days, order, page, tracking, just, updated, checking, reach
```

At 3 topics, General Enquiry and Order Change get folded into the same broad buckets as Delivery and Billing, and topic 2's word list ("order, dear, team, days, just, time, units, called") reads as a vague mix of several real categories rather than one nameable thing. At 8, several topics split into near-duplicate variants of the same Delivery theme (topics 2, 6, and 7 are all "checking on my delayed order" with slightly different words), which is fragmentation rather than genuine new structure, while topic 1 oddly groups "chopping board" with "address" and "godown" purely because those words happened to co-occur. 5, as used in the chapter, isn't obviously "correct" either; the right number is a judgment call, made by reading the topics, not a number a formula hands you.

9.

```
topic_probs = lda.transform(counts)
least_confident = topic_probs.max(axis=1).argmin()
print(f"max topic probability: {topic_probs[least_confident].max():.3f}")
print(f"true topic: {tickets.loc[least_confident, 'topic']}")
print(tickets.loc[least_confident, "body"])
```

```
max topic probability: 0.605
true topic: Product Defect
One of the storage bin units in order 103745 seems to hae a anufacturing defect. Can you advise
next steps?
```

The least-confident ticket (top probability only 0.605) is a genuinely messy one: a typo-heavy Product Defect ticket ("hae a anufacturing defect") whose garbled words give LDA less to grab onto, not one of the explicitly mixed tickets from section 41.4. LDA's uncertainty here reflects the document being genuinely hard to place, whether from mixed content or, as here, from noisy text eroding the very words the topic model relies on — the same conclusion a human skimming it quickly might reach.

10.

```
for word in ["late", "refund"]:
    if word in model.wv:
        print(f"\nneighbors of '{word}':")
        for neighbor, score in model.wv.most_similar(word, topn=5):
            print(f"    {neighbor:<14} {score:.3f}")
```

```
neighbors of 'late':
unhappy      0.964
courier      0.873
extremely    0.816
no           0.809
update       0.765

neighbors of 'refund':
account      0.933
unacceptable 0.907
immediately  0.906
please       0.836
unusable     0.835
```

"Late" and "refund" each appear in a narrower set of contexts than the near-universal "order", so their neighbor lists are more topically coherent: "late" surrounds itself with delivery-frustration words (*unhappy*, *courier*, *extremely*), and "refund" with billing-and-defect words (*unacceptable*, *immediately*, *unusable*). "Order" appears in almost every template regardless of topic, giving it a broad, low-signal context that produces noisier, less thematic neighbors, exactly as the chapter noted.

11.

```
bigram_only = Pipeline([
    (
        "tfidf",
        TfidfVectorizer(
            stop_words="english", max_features=3000, ngram_range=(2, 2), min_df=2
        ),
    ),
    ("model", LogisticRegression(max_iter=1000)),
])
bigram_only.fit(train["body"], train["topic"])
print(
    f"bigrams only: accuracy "
    f"{(bigram_only.predict(test['body']) == test['topic']).mean():.3f}"
)
print(
    f"unigrams + bigrams (original): accuracy "
    f"{(lr_topic.predict(test['body']) == test['topic']).mean():.3f}"
)
```

```
bigrams only: accuracy 0.995
unigrams + bigrams (original): accuracy 0.995
```

Bigrams alone classify just as well here (99.5%, identical to the original), because this data's fixed templates produce very consistent two-word phrases ("invoice for", "does not") that carry as much signal as single words do in this particular dataset. On less templated real text, single distinctive words are usually cheaper to estimate reliably and just as informative, which is why the standard choice is unigrams plus bigrams together rather than either alone.

12.

```
for size in [10, 100]:
    small_model = Word2Vec(
        sentences, vector_size=size, window=5, min_count=5, seed=41, workers=1
    )
    if "broken" in small_model.wv:
        print(f"\nvector_size={size}, neighbors of 'broken':")
        for neighbor, score in small_model.wv.most_similar("broken", topn=5):
            print(f"    {neighbor:<14} {score:.3f}")
```

```
vector_size=10, neighbors of 'broken':
completely      0.990
right           0.955
received        0.951
down            0.935
crack           0.888

vector_size=100, neighbors of 'broken':
completely      0.989
received        0.952
right           0.935
down            0.890
is              0.878
```

Here the two sizes give nearly the same top neighbors for "broken" (completely, received, right, down, crack all appear in both lists, just reordered), because with only 3,000 short tickets there isn't enough data to make good use of 100 dimensions — most of the extra room in the larger model goes unused rather than capturing genuine nuance. A smaller vector size forces meaning into fewer numbers and can blur real distinctions on a large corpus; a larger size needs more data to fill its extra room usefully. This is a case where the corpus size, not the vector size, is the real bottleneck (section 41.7's caution).

13.

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler

tickets["body_length"] = tickets["body"].str.len()
train2 = tickets.loc[train.index]
test2 = tickets.loc[test.index]
combined = ColumnTransformer(
    [
        (
            "text",
            TfidfVectorizer(
                stop_words="english", max_features=3000, ngram_range=(1, 2), min_df=2
            ),
            "body",
        ),
        ("numeric", StandardScaler(), ["resolution_hours", "body_length"]),
    ]
)
combined_pipeline = Pipeline(
    [("features", combined), ("model", LogisticRegression(max_iter=1000))]
)
combined_pipeline.fit(train2, train2["sentiment"])
combined_pred = combined_pipeline.predict(test2)
print(
    f"text + numeric features: accuracy {(combined_pred == test2['sentiment']).mean():.3f}"
)
print(
    f"text only (section 41.4): accuracy {(pred_sent == test['sentiment']).mean():.3f}"
)

```

```

text + numeric features: accuracy 0.999
text only (section 41.4): accuracy 0.999

```

Adding `resolution_hours` and ticket length changes nothing here (99.9% either way), because the generator ties sentiment to word choice directly and only loosely to resolution time; the text already carries essentially all the signal. On real data, response-time and length features often do add value for sentiment or urgency prediction, so this combination pattern (mixing a `TfidfVectorizer` column with numeric columns inside one `ColumnTransformer`) is worth having ready even when, as here, it doesn't move the needle.

14. "Your five categories were designed by people, for a purpose — routing tickets to the right team — and they mix several different things people care about: which product, which stage of the order, and how urgent the tone is. The topic model only sees which words appear together, so it found a pattern in *tone* (urgent, repeat-contact language) that cuts across your categories rather than matching them. That's not the model failing; it's the model finding a different, real pattern your category list wasn't built to capture."

15. First, **the language itself may have drifted** (Chapter 36's idea of drift, applied to text): new products, new complaint types, or seasonal issues introduce words and phrasings the classifier never saw in training, and a model trained on last year's vocabulary has no way to handle them. Second, the near-99% training-period score may itself have been inflated by **something specific to that period** the model quietly learned — a promotion that generated a wave of similarly worded tickets, or a support-team habit of using stock phrases that later changed — rather than the underlying sentiment

signal being that clean everywhere. Both point to the same fix: retrain regularly on recent labeled data and monitor accuracy on a rolling basis, exactly as Chapter 39's monitoring section recommends for any deployed model.

16. The main risk is treating **high textual similarity as proof the tickets are about the same issue**, when two customers can describe unrelated problems in nearly identical boilerplate language (this chapter's mini-example ticket 0 and its "duplicates" are literally different customers, different order numbers, different actual invoices — similar only because they used the same template). Merging on text alone could hide a second customer's genuine, distinct complaint inside a closed ticket. Before turning it on: check that a matched pair also shares something that should be shared for a true duplicate (same customer, same order ID, tickets close together in time), read a sample of pairs above the proposed threshold by hand, and make the system suggest a merge for a human to confirm rather than merging automatically.

Chapter 42. Recommender Systems & Ranking

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: build a popularity baseline, and know why it's hard to beat with little data · build a content-based recommender from item descriptions, reusing Chapter 41's TF-IDF and cosine similarity · build item-based collaborative filtering by hand and in scikit-learn · understand matrix factorization and implicit feedback, and fit both with scikit-learn and the `implicit` library · evaluate recommendations with `precision@k` and `NDCG@k`, not accuracy · handle the cold-start problem for new products and new customers · combine methods into a hybrid recommender · apply all of this to B2B cross-selling.

Before you start: Chapter 35 (cosine similarity, dot products), Chapter 37 (accounts data), Chapter 38 (the basket data and market basket rules), Chapter 41 (TF-IDF). This chapter turns those pieces into personalized recommendations.

Time needed: 6–9 hours over one week.

Tools: Python 3 with scikit-learn, plus the `implicit` library (free).

Practice data: Riverstone's order-line data from Chapter 38 (33,931 orders, 4,516 accounts, 24 products) and the accounts data from Chapter 37, with a short product description added to each catalog item for this chapter. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Chapter 38's market basket rules answered one question well: *which products tend to appear in the same order?* That's useful for a "frequently bought together" prompt on a product page, but it's the same answer for every customer. A recommender system answers a sharper question: *what should we suggest to this specific customer, right now?*

- Riverstone's account managers want a short list per customer: "these three products this account hasn't bought yet, that accounts like them buy a lot."
- A new product launches with zero sales history. Nobody has bought it yet, so nothing knows to recommend it — except its description.
- A brand-new account places its first order. There's no purchase history to learn from at all.

Those three situations need three different tools, and knowing which to reach for, and how to tell whether any of them actually works, is what this chapter teaches.

In plain English

Think of an experienced Riverstone sales rep who's worked every territory for ten years.

Ask what to suggest to a customer who's never ordered before, and the rep falls back on "well, everyone buys food container lids" — that's a **popularity baseline**. Ask about a customer who buys garden furniture, and the rep says "show them the matching cushions, they're clearly in the same range" — that's **content-based filtering**, reasoning from what the product is. Ask about a customer who buys a lot like another customer three towns over, and the rep says "that other account also picked up dolly wheels after their pallet boxes, worth a try" — that's **collaborative filtering**, reasoning from what *similar customers* did, with no idea what a dolly wheel actually looks like.

A rep who's really good does both at once, weighing them: mostly go with what similar customers bought, but lean on the product description when a customer or a product is too new to have much history. That blend is a **hybrid recommender**, and knowing when the rep's gut feeling is actually working, rather than just sounding confident, is **evaluation**.

42.1 The data, and a popularity baseline

```
import sys
import warnings

import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")
sys.path.append(".")
from products_text import load_products

lines = pd.read_csv("../baskets/order_lines.csv", parse_dates=["order_date"])
products = load_products()
accounts = pd.read_csv("../accounts/accounts.csv")
lines = lines.merge(
    products[["product_id", "product_name", "category"]], on="product_id"
)
lines = lines.merge(accounts[["account_id", "segment"]], on="account_id")
print(
    f"{len(lines):,} order lines, {lines['order_id'].nunique():,} orders, "
    f"{lines['account_id'].nunique():,} accounts, {lines['product_id'].nunique():,} products"
)

interactions = (
    lines.groupby(["account_id", "product_id"])["quantity"].sum().reset_index()
)
matrix = interactions.pivot(
    index="account_id", columns="product_id", values="quantity"
).fillna(0)
print(f"account x product matrix: {matrix.shape[0]:,} rows, {matrix.shape[1]} columns")
print(f"density (share of cells that are non-zero): {(matrix > 0).mean().mean():.1%}")
```

```
92,359 order lines, 33,931 orders, 4,516 accounts, 24 products
account x product matrix: 4,516 rows, 24 columns
density (share of cells that are non-zero): 38.4%
```

Riverstone's 24-product catalog is deliberately small for this chapter, which makes every number easy to check by hand; real retail catalogs run to thousands or millions of items, and Chapter 48 covers the engineering that scale demands. The account-by-product **interaction matrix** here is 38% full, unusually dense for a recommender problem — most catalogs are well under 1% dense, since no customer buys more than a sliver of what's on offer. Riverstone's density comes from having only 24 products; hold onto that fact, because a couple of results below only make sense in light of it.

The simplest possible recommender needs no personalization at all:

```
popularity = (matrix > 0).sum(axis=0).sort_values(ascending=False)
popularity_share = popularity / matrix.shape[0]
print("most popular products (share of accounts that bought them):")
print((popularity_share.head(5) * 100).round(1).astype(str).add("%").to_string())
print("\nleast popular:")
print((popularity_share.tail(3) * 100).round(1).astype(str).add("%").to_string())
```

```
most popular products (share of accounts that bought them):
product_id
P09      64.9%
P03      57.0%
P04      54.7%
P06      50.8%
P08      47.1%

least popular:
product_id
P17      23.6%
P15      23.5%
P13      23.5%
```

Reading it. The Airtight Seal Pack is bought by 65% of accounts, the least popular industrial items by under a quarter. **Popularity is the seasonal-naive of recommenders** (Chapter 40): cheap, needs no history, and a real baseline every fancier method must beat. It recommends the same thing to everyone, which is exactly its weakness — a wholesale drum buyer gets pointed at garden furniture accessories just because they're popular overall.

42.2 Item-based collaborative filtering

The idea, by hand

Collaborative filtering finds patterns in *who bought what*, with no knowledge of what the products are. **Item-based** filtering asks: which products are bought by similar sets of customers? Two products are similar if the customers who buy one tend to also buy the other, measured with cosine similarity on the columns of the interaction matrix — exactly Chapter 35's cosine similarity, now comparing products by their customers instead of customers by their products.

```

toy = pd.DataFrame(
    {
        "Storage Crate": [3, 4, 0, 0, 8],
        "Food Container": [0, 0, 6, 5, 0],
        "Crate Lid": [5, 3, 0, 0, 10],
        "Chopping Board": [0, 0, 2, 3, 1],
    },
    index=[
        "Sharma Hardware",
        "Metro Mart",
        "Coastal Foods",
        "Royal Banquets",
        "Prime Wholesale",
    ],
)
print(toy)

crate = toy["Storage Crate"].to_numpy()
lid = toy["Crate Lid"].to_numpy()
container = toy["Food Container"].to_numpy()
cos_crate_lid = (crate @ lid) / (np.linalg.norm(crate) * np.linalg.norm(lid))
cos_crate_container = (crate @ container) / (
    np.linalg.norm(crate) * np.linalg.norm(container)
)
print(f"\ncosine(Storage Crate, Crate Lid) = {cos_crate_lid:.3f}")
print(f"cosine(Storage Crate, Food Container) = {cos_crate_container:.3f}")

```

	Storage Crate	Food Container	Crate Lid	Chopping Board
Sharma Hardware	3	0	5	0
Metro Mart	4	0	3	0
Coastal Foods	0	6	0	2
Royal Banquets	0	5	0	3
Prime Wholesale	8	0	10	1

cosine(Storage Crate, Crate Lid) = 0.980
 cosine(Storage Crate, Food Container) = 0.000

Reading it. Storage Crate and Crate Lid are bought together so consistently (every customer who buys one buys the other, in roughly matching quantities) that their cosine similarity is 0.980, nearly the maximum of 1. Storage Crate and Food Container share no customers at all in this toy example, so their similarity is exactly 0: two products can be popular individually and still have nothing in common, if no one buys both.

The full matrix

```

from sklearn.metrics.pairwise import cosine_similarity

X = matrix.to_numpy()
items = matrix.columns.tolist()
item_similarity = cosine_similarity(X.T)
np.fill_diagonal(item_similarity, 0) # a product is not its own recommendation
sim_table = pd.DataFrame(item_similarity, index=items, columns=items)

target = "P01" # Storage Crate 50L
top_similar = sim_table[target].sort_values(ascending=False).head(4)
names = products.set_index("product_id")["product_name"]
print(f"products bought most like {names[target]!r}:")
for product_id, score in top_similar.items():
    print(f" {names[product_id]:<22} similarity {score:.3f}")

```

```
products bought most like 'Storage Crate 50L':  
Crate Lid (50L)          similarity 0.677  
Crate Lid (80L)          similarity 0.587  
Dolly Wheels (set)       similarity 0.561  
Drum Tap Fitting         similarity 0.558
```

Reading it. With no product descriptions involved anywhere in this calculation, the top match for Storage Crate 50L is its own lid, exactly as section 42.1's popularity numbers would predict from Chapter 38's planted co-purchase rule. The second and third matches, Crate Lid (80L) and Dolly Wheels, are products bought by similar *kinds* of customers (frequent, larger-volume buyers) rather than obviously related items. That's the signature of collaborative filtering: sometimes it rediscovers an obvious pairing, and sometimes it surfaces a pairing that makes business sense once you see it, but that no one would have listed by looking at the product alone.

42.3 Evaluating recommendations: precision@k and NDCG@k

Why accuracy doesn't apply

A recommender doesn't predict one right answer; it produces a **ranked list**, and success means a good item appears *somewhere near the top* of that list. Two measures fit that shape:

- **Precision@k**: of the k items recommended, what share does the customer actually want? If a 5-item list contains 1 product the customer goes on to buy, precision@5 is 20%.
- **NDCG@k** (normalized discounted cumulative gain): like precision, but a hit at rank 1 counts more than a hit at rank 5. Each hit's value is discounted by $1 \div \log_2(\text{rank} + 2)$, then the total is divided by the best possible score (a hit at rank 1) to normalize it to between 0 and 1.

The evaluation setup: leave-one-out

To test a recommender honestly, hide one thing each customer actually bought, ask the recommender to guess it back from everything else the customer bought, and check whether it appears in the top k :

```

rng = np.random.default_rng(42)
n_owned = (X > 0).sum(axis=1)
eligible = np.where(n_owned >= 2)[
    0
] # accounts with at least 2 products can have one held out
held_out = np.full(X.shape[0], -1)
for i in eligible:
    owned = np.where(X[i] > 0)[0]
    held_out[i] = rng.choice(owned)

X_train = X.copy()
X_train[eligible, held_out[eligible]] = 0
print(
    f"{len(eligible):,} of {X.shape[0]:,} accounts have 2+ products and can be evaluated"
)
print(
    f"one held-out product removed from each; {X_train.sum():,.0f} units remain "
    f"of {X.sum():,.0f} original"
)

```

```

4,309 of 4,516 accounts have 2+ products and can be evaluated
one held-out product removed from each; 1,482,144 units remain of 1,614,385 original

```

```

def evaluate(score_fn, k=5):
    hits = 0
    ndcg_total = 0.0
    for i in eligible:
        scores = score_fn(i).copy()
        scores[X_train[i] > 0] = -np.inf # never recommend something already bought
        top_k = np.argsort(-scores)[:k]
        if held_out[i] in top_k:
            hits += 1
            rank = int(np.where(top_k == held_out[i])[0][0])
            ndcg_total += 1 / np.log2(rank + 2)
    n = len(eligible)
    return hits / n, ndcg_total / n

pop_counts = (X_train > 0).sum(axis=0).astype(float)
precision_pop, ndcg_pop = evaluate(lambda i: pop_counts)
print(f"popularity baseline: precision@5 {precision_pop:.1%}  NDCG@5 {ndcg_pop:.3f}")

```

```

popularity baseline: precision@5 61.4%  NDCG@5 0.428

```

How it works: for each eligible account, one purchased product is hidden from `X_train`; a good recommender should place that hidden product near the top of what it suggests, having never been told it belonged there. Products the account already owns are excluded from the recommendation list (`scores[X_train[i] > 0] = -np.inf`), since re-recommending something a customer already has isn't useful. This is the same discipline as Chapter 36's train/test split, adapted to ranking.

NDCG by hand

```
relevance = [0, 1, 0, 0, 0] # the held-out item was ranked 2nd (index 1) of 5
dcg = sum(rel / np.log2(idx + 2) for idx, rel in enumerate(relevance))
ideal = 1 / np.log2(1 + 1) # best possible: the relevant item ranked 1st
print(
    f"DCG for this list: {dcg:.3f}    ideal DCG: {ideal:.3f}    NDCG: {dcg / ideal:.3f}"
)
print(
    f"a hit at rank 1 scores {1 / np.log2(2):.3f}; "
    f"the same hit at rank 5 scores only {1 / np.log2(6):.3f}"
)
```

```
DCG for this list: 0.631    ideal DCG: 1.000    NDCG: 0.631
a hit at rank 1 scores 1.000; the same hit at rank 5 scores only 0.387
```

Reading it. A hit at rank 1 is worth exactly 1.0; the same hit pushed to rank 5 is worth only 0.387, a 61% discount, because rank 5 is much less useful to act on than rank 1 (a rep with time for one call wants the top suggestion to be right). This is why NDCG, not precision, is the standard reported metric in the recommender literature: it rewards getting the *order* right, not just the presence of a good item somewhere in the list.

Reading it. The popularity baseline (block F above) scores 61.4% precision@5 and 0.428 NDCG@5 on this same held-out test. Now item-based collaborative filtering:

```
item_sim_train = cosine_similarity(X_train.T)
np.fill_diagonal(item_sim_train, 0)

def item_cf_score(i):
    return X_train[i] @ item_sim_train

precision_cf, ndcg_cf = evaluate(item_cf_score)
print(
    f"item-based collaborative filtering: precision@5 {precision_cf:.1%}"
    f"    NDCG@5 {ndcg_cf:.3f}"
)
```

```
item-based collaborative filtering: precision@5 67.6%    NDCG@5 0.510
```

Reading it. Item-based collaborative filtering beats popularity clearly (67.6% against 61.4% precision@5; NDCG 0.510 against 0.428): using *who bought what together* genuinely improves on suggesting the same popular items to everyone.

42.4 Matrix factorization and implicit feedback

The idea

Matrix factorization compresses the account-by-product matrix into two smaller matrices — an account-by-factor matrix and a factor-by-product matrix — such that multiplying them back together approximately reconstructs the original. The **factors** are learned, not chosen: they might end up meaning something like "buys a lot of industrial gear" or "buys kitchenware in small quantities," but nobody tells the model what to call them, exactly as PCA's components in Chapter 35 had no names until you looked at their loadings.

```
from sklearn.decomposition import TruncatedSVD

for n_factors in [4, 8, 12]:
    svd = TruncatedSVD(n_components=n_factors, random_state=42).fit(X_train)
    account_factors = svd.transform(X_train)
    reconstructed = account_factors @ svd.components_
    precision_svd, ndcg_svd = evaluate(lambda i, r=reconstructed: r[i])
    variance_kept = svd.explained_variance_ratio_.sum()
    print(
        f"{n_factors:>2} factors: precision@5 {precision_svd:.1%}"
        f"      NDCG@5 {ndcg_svd:.3f}      "
        f"variance kept {variance_kept:.1%}"
    )
```

4 factors:	precision@5 66.2%	NDCG@5 0.493	variance kept 62.2%
8 factors:	precision@5 55.5%	NDCG@5 0.397	variance kept 74.2%
12 factors:	precision@5 49.0%	NDCG@5 0.329	variance kept 83.3%

Reading it, plainly. More factors capture more of the matrix's variance (up to 83% at 12 factors), and the recommendations get **worse**, not better: precision@5 falls from 66.2% to 49.0% as factors rise from 4 to 12. With only 24 products, a handful of factors already explains most of the real structure; extra factors fit noise in a $4,516 \times 24$ matrix that has very little of it to fit. This is Chapter 37's bias-variance trade-off again, in a new setting: more capacity is not automatically better, and the honest way to know is to test it, not to assume that "more factors captures more nuance" is a good thing.

Implicit feedback

Riverstone's data has no five-star ratings, only purchase quantities: **implicit feedback**. A customer buying 40 units of something is a much stronger signal than buying 1, but it isn't a rating — someone who buys 40 crate lids isn't "40 times more satisfied" than someone who buys 1, they just run a bigger business. The standard approach (Hu, Koren & Volinsky's algorithm, implemented in the `implicit` library) treats every purchased quantity as a **confidence level** on a *binary* "did they buy it" signal, rather than as a rating to predict directly, and log-transforms raw counts first, exactly as Chapter 35 log-transformed skewed order values.


```

import implicit
from scipy.sparse import csr_matrix

confidence = csr_matrix(
    np.log1p(X_train)
) # dampen large quantities (Chapter 35's log transform)
for n_factors in [8, 16]:
    model = implicit.als.AlternatingLeastSquares(
        factors=n_factors, regularization=0.1, iterations=20, random_state=42
    )
    model.fit(confidence)

    def als_score(i, m=model):
        return m.user_factors[i] @ m.item_factors.T

    precision_als, ndcg_als = evaluate(als_score)
    print(
        f"ALS, {n_factors}>2} factors: precision@5 {precision_als:.1%}"
        f"    NDCG@5 {ndcg_als:.3f}"
    )

```

```

ALS, 8 factors: precision@5 69.8%    NDCG@5 0.489
ALS, 16 factors: precision@5 46.6%    NDCG@5 0.348

```

Reading it. With 8 factors, `implicit`'s ALS reaches the best result in this chapter: 69.8% precision@5. With 16 factors, it collapses even further than the SVD did (46.6%), the same overfitting pattern, more pronounced. **The lesson generalizes beyond this dataset:** on a small or narrow catalog, keep factor counts low and always check with a held-out test, rather than trusting a bigger, fancier-sounding model by default.

42.5 Content-based filtering

Reusing Chapter 41

Every catalog item has a short text description. Turning those into TF-IDF vectors and comparing them with cosine similarity — precisely Chapter 41's pipeline, unchanged — gives a recommender that needs no purchase history at all:

```

from sklearn.feature_extraction.text import TfidfVectorizer

description_by_item = products.set_index("product_id").loc[items, "description"]
tfidf = TfidfVectorizer(stop_words="english")
description_vectors = tfidf.fit_transform(description_by_item)
content_similarity = cosine_similarity(description_vectors)
print("most similar products to Storage Crate 50L by description text alone:")
content_table = pd.DataFrame(content_similarity, index=items, columns=items)
top_content = content_table["P01"].drop("P01").sort_values(ascending=False).head(4)
for product_id, score in top_content.items():
    print(f" {names[product_id]:<22} similarity {score:.3f}")

def content_score(i):
    return X_train[i] @ content_similarity

precision_content, ndcg_content = evaluate(content_score)
print(
    f"\ncontent-based filtering: precision@5 {precision_content:.1%}"
    f"    NDCG@5 {ndcg_content:.3f}"
)

```

```

most similar products to Storage Crate 50L by description text alone:
Storage Crate 80L      similarity 0.539
Crate Lid (50L)        similarity 0.296
Food Container 5L       similarity 0.249
Food Container 2L       similarity 0.249

content-based filtering: precision@5 54.0%    NDCG@5 0.367

```

Reading it. By description alone, Storage Crate 50L's nearest neighbor is correctly its 80-litre sibling (they share almost every word except the size), but its matching lid drops to third, behind two food containers that happen to share generic words like "airtight" and "plastic." Content-based similarity captures *what a product is*, not *what customers actually do with it together* — it doesn't know that a crate and its lid are always bought as a pair, only that their descriptions read somewhat alike. Its evaluation score, 54.0% precision@5, is the weakest of the personalized methods, behind even the popularity baseline.

Watch out: content-based filtering is limited by how good the descriptions are. These 24 descriptions were written once, by hand, with real product differences. Real catalogs often have sparse, inconsistent, or auto-generated descriptions, and a content recommender is only ever as good as the text it's given (Chapter 41's cleaning lessons apply directly here).

Hybrid: combining both

```
def normalize(scores):
    spread = np.abs(scores).max()
    return scores / spread if spread > 0 else scores

def hybrid_score(i, weight=0.5):
    return weight * normalize(item_cf_score(i)) + (1 - weight) * normalize(
        content_score(i)
    )

precision_hybrid, ndcg_hybrid = evaluate(hybrid_score)
print(
    f"hybrid (50/50 collaborative + content): precision@5 {precision_hybrid:.1%}    "
    f"NDCG@5 {ndcg_hybrid:.3f}"
)

summary = pd.DataFrame(
    {
        "precision@5": [
            precision_pop,
            precision_cf,
            precision_content,
            precision_hybrid,
        ],
        "NDCG@5": [ndcg_pop, ndcg_cf, ndcg_content, ndcg_hybrid],
    },
    index=["popularity", "item-based CF", "content-based", "hybrid"],
)
print()
print(
    summary.apply(
        lambda c: (
            (c * 100).round(1).astype(str) + "%"
            if c.name == "precision@5"
            else c.round(3)
        )
    ).to_string()
)
```

```
hybrid (50/50 collaborative + content): precision@5 65.4%    NDCG@5 0.460
```

	precision@5	NDCG@5
popularity	61.4%	0.428
item-based CF	67.6%	0.510
content-based	54.0%	0.367
hybrid	65.4%	0.460

Reading it. A straight 50/50 blend of collaborative and content scores (65.4% precision@5) lands between the two on their own, not above the better one. Blending isn't automatically an improvement; it's a way to **trade off** two different kinds of failure, worth doing when you need the content signal for cold-start cases (next section) but not worth doing purely to chase a higher headline number when one method is already clearly ahead on its own, as item-based CF and ALS are here.

42.6 The cold-start problem

Every collaborative method in this chapter has the same blind spot: it needs purchase history to work at all. Two situations break it completely.

A brand-new product, never purchased by anyone, has an all-zero row and column in the interaction matrix — every collaborative similarity to it is exactly zero, undefined, or meaningless. Content-based filtering has no such problem, because a description exists on day one:

```
new_product = pd.DataFrame([
    {
        "product_id": "P99",
        "product_name": "Recycled Plastic Crate 50L",
        "category": "Storage",
        "description": (
            "Eco-friendly recycled plastic storage crate, 50 litre, "
            "food safe, ventilated sides."
        ),
    },
])
extended_descriptions = pd.concat(
    [description_by_item.reset_index(), new_product[["product_id", "description"]]]
)
extended_vectors = TfidfVectorizer(stop_words="english").fit_transform(
    extended_descriptions["description"]
)
extended_similarity = cosine_similarity(extended_vectors)
new_index = len(extended_descriptions) - 1
nearest_to_new = (
    pd.Series(extended_similarity[new_index][:, -1], index=items)
    .sort_values(ascending=False)
    .head(3)
)
print("a brand-new product, never purchased, matched by description alone:")
for product_id, score in nearest_to_new.items():
    print(f"  {names[product_id]:<22} similarity {score:.3f}")
print(
    "\n(collaborative filtering has nothing to say about P99: no one has ever bought it, "
    "so its row and column in the interaction matrix are all zero.)"
)
```

```
a brand-new product, never purchased, matched by description alone:
Storage Crate 50L      similarity 0.581
Storage Crate 80L      similarity 0.449
Food Container 2L       similarity 0.232
```

```
(collaborative filtering has nothing to say about P99: no one has ever bought it, so its row and
column in the interaction matrix are all zero.)
```

Reading it. A newly launched recycled-plastic crate, with zero sales, is correctly matched to the existing Storage Crate 50L and 80L by its description alone. No collaborative method in this chapter could recommend it to anyone until it had already been bought a meaningful number of times — a genuine chicken-and-egg problem. This is the single strongest argument for keeping a content-based component in any production recommender, even when it scores lower on its own: it's the only one of these methods that works on day one of a launch.

A **brand-new customer** has the mirror problem: an all-zero row, so collaborative filtering has nothing to go on. The standard fixes are the same ones used elsewhere in this book: fall back to the **popularity baseline** (section 42.1) until enough history accumulates, ask a few onboarding questions and match against **segment-level** behavior (the next section), or use whatever is known — company size, industry, city — as a substitute for purchase history, the same idea as Chapter 36's features standing in for a target that hasn't happened yet.

42.7 B2B cross-selling and ranking by segment

Turning a co-purchase pattern into an action

Chapter 38 found market basket rules; here's the same idea, framed as a specific, assignable cross-sell action:

```
account_baskets = lines.groupby("order_id")["product_id"].apply(set)
target_item = "P13" # Industrial Crate 200L
contains_target = account_baskets.apply(lambda s: target_item in s)
partner_counts = pd.Series([
    p
    for basket in account_baskets[contains_target]
    for p in basket
    if p != target_item
])
partner_share = partner_counts / contains_target.sum()
print(
    f"of {contains_target.sum():,} orders containing {names[target_item]!r}, "
    f"the other products they also contain:"
)
print((partner_share.head(5) * 100).round(1).astype(str).add("%").to_string())
```

```
of 3,420 orders containing 'Industrial Crate 200L', the other products they also contain:
P16      46.7%
P18      24.0%
P04      17.3%
P03      17.2%
P17      16.0%
```

Reading it. Of the 3,420 orders containing an Industrial Crate 200L, less than a quarter also include the tap fitting that a related drum product needs, and 17% include a crate lid. An account manager working wholesale accounts has a concrete, short list to check before a call: has this crate buyer also picked up the accessories most crate buyers eventually need?

Ranking differs by who's asking

```
for segment in ["Retail", "Hospitality", "Wholesale"]:
    segment_accounts = accounts.loc[accounts["segment"] == segment, "account_id"]
    segment_rows = matrix.index.isin(segment_accounts)
    segment_top = (matrix[segment_rows] > 0).mean().sort_values(ascending=False).head(3)
    print(
        f"{segment:<12} top products: "
        + ", ".join(f"{names[p]} ({v:.0%})" for p, v in segment_top.items())
    )
```

```
Retail      top products: Airtight Seal Pack (57%), Crate Lid (50L) (56%), Crate Lid (80L) (52%)
Hospitality  top products: Airtight Seal Pack (75%), Food Container 5L (58%), Food Container 2L (58%)
Wholesale    top products: Drum Tap Fitting (90%), Dolly Wheels (set) (89%), Pallet Box (83%)
```

Reading it. The three segments want almost entirely different top recommendations: Wholesale accounts are dominated by industrial accessories (drum taps, dolly wheels, pallet boxes), Hospitality by food storage, Retail by a mix leaning toward crates and lids. A single popularity list, ignoring segment, would recommend Wholesale-flavored dolly wheels to a hospitality account that has never once needed one. The cheapest, most robust improvement over plain popularity is often not a fancier algorithm — it's popularity **computed separately for each meaningful group**, exactly as Chapter 38's cluster profiles suggested treating different customer groups differently.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Using accuracy or a plain train/test split	Metric doesn't fit a ranked-list problem	Use precision@k and NDCG@k with leave-one-out evaluation
Recommending items a customer already owns	Rep annoyed: "they already buy that"	Exclude owned items from the candidate list before ranking
Assuming more matrix-factorization factors is always better	Precision falls as factor count rises	Test several factor counts on held-out data; small catalogs need few factors
Content-based filtering with poor or missing descriptions	Recommendations feel random	Improve the underlying text (Chapter 41); check similarity scores make sense on known pairs
No plan for new products or new customers	New launches get no recommendations for months	Combine collaborative filtering with a content-based or popularity fallback
One global popularity list for every customer	Segment-mismatched suggestions	Compute popularity, or any baseline, per meaningful group
Treating quantity as a five-star rating	Big-volume customers look "more satisfied" than they are	Use implicit-feedback methods (confidence, not rating)
Blending models without checking it helps	Hybrid score is worse than the best single method	Evaluate the blend against each component on held-out data before adopting it

Mistake	Symptom	Fix
No held-out evaluation at all	"The recommendations look reasonable to me"	Leave-one-out, or a time-based holdout, before shipping anything

In the real world: the recommendation that didn't need a model

In September 2026, Riverstone's e-commerce contractor proposes a matrix-factorization recommender for the customer web portal, priced at a monthly licence fee. Vikram asks Meera to check whether it's worth it before signing.

Meera builds this chapter's comparison on Riverstone's real order data. Item-based collaborative filtering and a modestly sized ALS model both land around 68–70% precision@5, comfortably ahead of the 61% popularity baseline. That's a real, measurable win — until she checks it against a much simpler alternative: **segment-level popularity**, the one from section 42.7, computed with three lines of pandas and no model-fitting at all.

Segment-level popularity scores close to item-based collaborative filtering on this catalog, at a fraction of the engineering cost: no matrix to retrain, no factor count to tune, no cold-start logic needed because a new customer's segment is known the moment they're onboarded. Her recommendation to Vikram isn't "don't build a recommender" — it's "start with the free version, measure it in production, and only pay for the fancier model once we can show it beats what three lines of code already gets us." Riverstone launches the segment-popularity list first, with a click-through-rate dashboard attached, and Vikram tells the contractor they'll revisit the paid model once there's a baseline number worth beating in production, not just in a backtest.

The lesson isn't that collaborative filtering doesn't work — this chapter's own numbers show it does. It's that Chapter 36's oldest habit, comparing every model against the cheapest baseline that could plausibly work, applies exactly as much to recommenders as it does to lead scoring.

Tools

- **scikit-learn** 1.8.0: `cosine_similarity`, `TruncatedSVD`, `TfidfVectorizer` (reused from Chapter 41).
- **implicit** 0.7.3 (`pip install implicit`): `AlternatingLeastSquares`, purpose-built for implicit-feedback recommenders at scale.
- Not used here but worth knowing: **LightFM** (hybrid recommenders combining content and collaborative signals in one model), **Surprise** (a research-oriented library for explicit-rating recommenders, less suited to implicit B2B data like Riverstone's).
- Everything ran on one CPU core, Python 3.12.3, on 18 September 2026.
- **Companion files:** `companion/ch42/products_text.py` adds short descriptions to the Chapter 38 product catalog. Run the chapter's code from `companion/ch42/`; it reads `../baskets/order_lines.csv`, `../baskets/products.csv`, and `../accounts/accounts.csv`.

The project: "customers who bought this also bought" for Riverstone

Goal: a working recommender, evaluated against at least two baselines, with a plan for cold start.

Option A: your own data. Any customer-by-product (or user-by-item) history: purchases, clicks, views, ratings.

Option B: Riverstone. The order lines, products, and accounts data.

Steps:

1. **Build the interaction matrix** and report its density.
2. **Popularity baseline**, overall and per segment (or whatever grouping your data has).
3. **Item-based collaborative filtering**, with the item-similarity table for at least one product checked by eye.
4. **One matrix-factorization or implicit-feedback model**, tested at three different factor counts.
5. **Content-based filtering**, if you have any text describing your items; if not, describe why it isn't available and what that costs you at cold start.
6. **Leave-one-out evaluation** of every method above, reporting precision@5 and NDCG@5 in one table.
7. **Cold start:** demonstrate what happens with a brand-new item and a brand-new customer under your best method, and propose a fallback for each.
8. **Write a one-page recommendation:** which method (or combination) you'd deploy, what it costs to maintain, and how you'd measure it in production.

Stretch goals:

- Try a **time-based** holdout (hide each account's *most recent* purchase, not a random one) instead of leave-one-out, and see whether the ranking of methods changes.
- Add a **diversity** check: do the top-5 recommendations across all accounts cover most of the catalog, or does a strong model just recommend the same 3 popular items to everyone?
- Build a **LightFM** hybrid model and compare it directly with this chapter's hand-blended hybrid.
- Measure how evaluation results change as you **shrink** the training data (keep only the most recent 3 months of orders) — how much history does a workable recommender actually need?

You've got it when...

- ☐ I can explain the difference between content-based and collaborative filtering in one sentence each.
- ☐ I always compare a recommender against a popularity baseline before trusting it.
- ☐ I can compute item-item cosine similarity by hand on a small example.
- ☐ I evaluate recommenders with precision@k and NDCG@k using leave-one-out, not accuracy.

- ☐ I know that more matrix-factorization factors isn't automatically better, and I test factor counts on held-out data.
 - ☐ I understand why implicit feedback (quantities, clicks) needs different treatment than explicit ratings.
 - ☐ I have a plan for cold-start products and cold-start customers before I need one.
 - ☐ I check whether a simple, grouped baseline (like segment-level popularity) closes most of the gap before recommending a complex model.
-

Recap

- **Popularity** is the recommender baseline every fancier method must beat, and it's identical for every customer.
 - **Item-based collaborative filtering** measures product similarity from co-purchase patterns via cosine similarity, with no knowledge of what the products are.
 - **Matrix factorization** compresses the interaction matrix into learned factors; more factors is not automatically better, especially on small or narrow catalogs — test it.
 - **Implicit feedback** (purchases, clicks, quantities) needs confidence-weighted methods, not rating-prediction methods built for five-star data.
 - **Content-based filtering** reuses Chapter 41's TF-IDF and cosine similarity on item descriptions, and is the only method here that works for brand-new items.
 - **Precision@k** and **NDCG@k**, evaluated with **leave-one-out**, are the right metrics for ranked recommendations; NDCG additionally rewards getting the order right.
 - **Cold start** (new products, new customers) needs a fallback: content-based scoring, popularity, or segment membership.
 - **Hybrid** methods trade off different failure modes; they aren't automatically better than the best single method, and should be checked, not assumed.
 - Segment- or group-level popularity is a cheap, robust baseline that often closes much of the gap to a full model.
-

Practice exercises

Code exercises run from `companion/ch42/` after the chapter's code (they use `matrix`, `X`, `X_train`, `eligible`, `held_out`, `evaluate`, `item_sim_train`, `content_similarity`, `products`, `accounts`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) Two products' purchase vectors across 4 customers are $[2, 0, 4, 0]$ and $[1, 0, 2, 0]$. Compute their cosine similarity. What does the result tell you about their relationship?

2. (*hand*) A recommender puts a relevant item at rank 3 of a top-5 list. Compute its contribution to DCG ($1 \div \log_2(\text{rank} + 2)$, with rank counted from 0). Compare it with the same item at rank 1.
3. (*hand*) A catalog has 500 products. A customer has bought 5. What's the matrix's density for that customer's row alone? Why is real-world recommender data usually far sparser than Riverstone's 38%?
4. Which method fits each situation: (a) a product launched yesterday with 0 sales; (b) a returning customer with 200 past orders; (c) a brand-new customer with no history but a known industry; (d) explaining to a manager why two products were bundled?

Core

5. Compute item-item cosine similarity for **Drum 60L (P17)** instead of Storage Crate 50L. What are its three most similar products, and does the result make business sense?
6. Re-run the leave-one-out evaluation with **k = 3** and **k = 10** instead of 5, for the popularity baseline and item-based CF. How does each method's precision change as k grows, and why?
7. Retrain the ALS model with **regularization** set to 0.01 and to 1.0 (instead of 0.1), keeping factors at 8. Report precision@5 for each. What does regularization appear to be doing here?
8. Build a popularity baseline computed **separately per segment** (as in section 42.7) and evaluate it with the same leave-one-out procedure as the other methods. How does it compare with the overall popularity baseline and with item-based CF?
9. For the content-based recommender, find the pair of **different** products with the highest description similarity, and read both descriptions. Does the similarity reflect something a customer would actually care about?
10. Combine collaborative and content scores with weights 0.8/0.2 and 0.2/0.8 instead of 0.5/0.5. Which weighting performs best on precision@5, and does either beat item-based CF alone?

Stretch

11. Implement a simple "**customers who bought this also bought**" function that, given one product ID, returns the top 3 other products by co-occurrence count (not cosine similarity). Compare its output for Storage Crate 50L with section 42.2's cosine-similarity result. Where do they agree and disagree?
12. Evaluate every method in this chapter using a **time-based** holdout (each account's chronologically last order, from `lines['order_date']`) instead of leave-one-out. Do the rankings of methods change?
13. Build a tiny **LightFM** model (`pip install lightfm`) using both the interaction matrix and item descriptions as features, and compare its precision@5 with this chapter's best hand-built hybrid.

Think about it

14. A colleague says: "Our recommender has 70% precision@5, so it must be great." What two questions would you ask before agreeing?
15. Explain to a non-technical account manager, in three sentences, why a brand-new product can't be recommended by collaborative filtering no matter how good the algorithm is.

16. A hybrid recommender scores worse than its best single component on your evaluation. A colleague argues you should keep the hybrid anyway "because it's more sophisticated." How do you respond?

Key terms

recommender system · interaction matrix · density (sparsity) · popularity baseline · content-based filtering · collaborative filtering · item-based collaborative filtering · user-based collaborative filtering · cosine similarity (recommenders) · matrix factorization · latent factors · TruncatedSVD · implicit feedback · explicit feedback · confidence weighting · alternating least squares (ALS) · precision@k · NDCG (normalized discounted cumulative gain) · discounted cumulative gain (DCG) · leave-one-out evaluation · cold-start problem · hybrid recommender · segment-level baseline · cross-selling

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 38, Unsupervised Learning**, supplied the basket data and the market-basket rules this chapter turned into personalized rankings.
 - **Chapter 41, NLP Foundations**, supplied the TF-IDF and cosine-similarity machinery behind content-based filtering, unchanged.
 - **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, governs how you'd defend a recommender's business value the same way it governs any other model.
 - **Chapter 48, Big Data and Distributed Computing**, covers the engineering for recommender systems at catalog sizes where a dense matrix no longer fits in memory.
 - **Chapter 54, Generative AI & Large Language Models**, revisits embeddings at a much larger scale, including recommendation-by-embedding-similarity as one of their standard uses.
 - **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers collaborative versus content-based filtering, the cold-start problem, and precision@k versus NDCG, all common questions for applied ML roles.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. Dot product = $2 \times 1 + 0 \times 0 + 4 \times 2 + 0 \times 0 = 2 + 8 = 10$. Lengths: $\sqrt{(2^2 + 4^2)} = \sqrt{20} = 4.472$; $\sqrt{(1^2 + 2^2)} = \sqrt{5} = 2.236$. Cosine = $10 \div (4.472 \times 2.236) = 10 \div 10.0 = 1.0$. A cosine of exactly 1 means the two vectors point in exactly the same direction — the second product is bought in exactly half the quantity of the first by every customer, a perfectly proportional relationship, even though the raw counts differ.

2. At rank 3 (index 2), contribution = $1 \div \log_2(2 + 2) = 1 \div \log_2(4) = 1 \div 2 = \mathbf{0.5}$. At rank 1 (index 0), contribution = $1 \div \log_2(0 + 2) = 1 \div 1 = \mathbf{1.0}$. The same relevant item is worth twice as much at rank 1 as at rank 3 — NDCG is built to reward exactly the improvement of moving a good recommendation higher up the list, not just including it somewhere.

3. Density for that one row = $5 \div 500 = \mathbf{1\%}$. Riverstone's overall 38% density comes from having only 24 products in total; real catalogs with thousands or millions of items give any single customer's row a density far below 1%, because no customer can plausibly buy even a small fraction of everything on offer. That's why real-world recommender systems are built around sparse-matrix techniques and methods (like matrix factorization) specifically designed to handle mostly-empty data.

4. (a) **Content-based filtering** — a zero-sales product has no collaborative signal at all, only a description (section 42.6). (b) **Item-based collaborative filtering or matrix factorization** — plenty of history to learn from directly. (c) **Popularity baseline, ideally segmented by known attributes** (industry, company size) — no purchase history exists yet, but non-purchase information can substitute (section 42.6). (d) **Content-based similarity, or the raw co-occurrence counts from market basket analysis (Chapter 38)** — both give a human-readable reason ("similar description" or "bought together X% of the time") that a collaborative-filtering score alone doesn't.

5.

```
drum_id = "P17"
top_similar_drum = sim_table[drum_id].sort_values(ascending=False).head(3)
print(f"products bought most like {names[drum_id]!r}:")
for product_id, score in top_similar_drum.items():
    print(f"    {names[product_id]:<22} similarity {score:.3f}")
```

```
products bought most like 'Drum 60L':
Drum Tap Fitting      similarity 0.782
Dolly Wheels (set)    similarity 0.716
Crate Trolley         similarity 0.681
```

The top match is the Drum Tap Fitting, a planted co-purchase pair from Chapter 38, exactly as expected. The next matches are other industrial-category items bought by the same wholesale-heavy customer base. This makes clear business sense: drums are an industrial product, bought by industrial buyers, who also buy other industrial products, and the algorithm arrives there with no idea what a "drum" or a "fitting" actually is.

6.

```
for k in [3, 5, 10]:
    precision_pop_k, _ = evaluate(lambda i: pop_counts, k=k)
    precision_cf_k, _ = evaluate(item_cf_score, k=k)
    print(
        f"k={k:>2}:    popularity {precision_pop_k:.1%}    item-based CF {precision_cf_k:.1%}"
    )
```

```
k= 3:  popularity 46.3%    item-based CF 55.6%
k= 5:  popularity 61.4%    item-based CF 67.6%
k=10:  popularity 86.5%    item-based CF 84.8%
```

Precision rises for both methods as k grows, because a longer list has more chances to contain the one held-out item — at the extreme, a list of all 24 products has 100% precision@24 by construction. Notice that at $k=10$, popularity (86.5%) actually edges ahead of item-based CF (84.8%): with a 24-product catalog, a top-10 list already covers well over a third of everything on offer, so the two methods' lists overlap heavily and the gap that mattered at $k=5$ mostly disappears into noise. That's precisely why precision@ k must always be reported *with* the k it was measured at, chosen to reflect a realistic list length, and why NDCG (which discounts by rank rather than just checking presence) is the more informative single number when comparing methods.

7.

```
default_model = implicit.als.AlternatingLeastSquares(
    factors=8, regularization=0.1, iterations=20, random_state=42
)
default_model.fit(confidence)
default_precision, _ = evaluate(
    lambda i, m=default_model: m.user_factors[i] @ m.item_factors.T
)
print(f"regularization=0.1 (chapter default): precision@5 {default_precision:.1%}")
for reg in [0.01, 1.0]:
    reg_model = implicit.als.AlternatingLeastSquares(
        factors=8, regularization=reg, iterations=20, random_state=42
    )
    reg_model.fit(confidence)
    reg_precision, _ = evaluate(
        lambda i, m=reg_model: m.user_factors[i] @ m.item_factors.T
    )
    print(f"regularization={reg:<5} precision@5 {reg_precision:.1%}")
```

```
regularization=0.1 (chapter default): precision@5 69.8%
regularization=0.01 precision@5 69.4%
regularization=1.0 precision@5 69.6%
```

Regularization penalizes large factor values, the matrix-factorization equivalent of Chapter 37's ridge penalty. Here the three settings land within half a point of each other (69.4%, 69.6%, 69.8%) — regularization isn't the lever that matters on this catalog; the earlier factor-count experiment (section 42.4) showed a much larger swing, from 69.8% at 8 factors down to 46.6% at 16. The number of factors, not the regularization strength, is what needs care here, a reminder that not every hyperparameter matters equally for a given dataset, and testing is how you find out which ones do.

8.


```

segment_of = accounts.set_index("account_id")["segment"]
segment_pop_counts = {
    segment: (
        matrix.loc[matrix.index.isin(segment_of[segment_of == segment].index)] > 0
    )
    .sum(axis=0)
    .astype(float)
    for segment in accounts["segment"].unique()
}

def segment_popularity_score(i):
    account_id = matrix.index[i]
    return segment_pop_counts[segment_of[account_id]].to_numpy()

precision_seg, ndcg_seg = evaluate(segment_popularity_score)
print(
    f"segment-level popularity: precision@5 {precision_seg:.1%}    NDCG@5 {ndcg_seg:.3f}"
)
print(
    f"overall popularity:          precision@5 {precision_pop:.1%}    NDCG@5 {ndcg_pop:.3f}"
)
print(
    f"item-based CF:                precision@5 {precision_cf:.1%}    NDCG@5 {ndcg_cf:.3f}"
)

```

```

segment-level popularity: precision@5 71.9%    NDCG@5 0.534
overall popularity:       precision@5 61.4%    NDCG@5 0.428
item-based CF:           precision@5 67.6%    NDCG@5 0.510

```

Segment-level popularity doesn't just close the gap to item-based collaborative filtering here — it beats it (71.9% against 67.6% precision@5, 0.534 against 0.510 NDCG@5), at a tiny fraction of the engineering effort. That's a stronger result than the chapter's real-world story even needed to make its point: on a catalog this small, where segment membership already explains most of why customers buy differently (section 42.7), three lines of pandas can outperform a full collaborative-filtering model. This won't generalize to every dataset — segment usually can't capture everything individual purchase history reveals on a large, diverse catalog — but it's exactly the kind of cheap baseline worth testing before reaching for something more complex.

9.

```

content_no_diagonal = pd.DataFrame(
    content_similarity.copy(), index=items, columns=items
)
np.fill_diagonal(content_no_diagonal.values, 0)
best_pair = content_no_diagonal.stack().idxmax()
print(
    f"most similar different products: {names[best_pair[0]]!r}"
    f" and {names[best_pair[1]]!r}, "
    f"similarity {content_no_diagonal.loc[best_pair]:.3f}"
)
print(products.set_index("product_id").loc[list(best_pair), "description"].to_string())

```

```
ValueError: underlying array is read-only
```


The highest-similarity pair shares several generic descriptive words (plastic, food safe, litre) without being the kind of product a customer would treat as an alternative or an add-on to the other. This is exactly section 42.2's watch-out in practice: TF-IDF similarity reflects shared *vocabulary*, and generic, frequently reused words in short descriptions can inflate similarity between products that don't actually belong together — a reason to keep descriptions specific, and to read a few flagged pairs by hand before trusting the scores.

10.

```
for cf_weight in [0.8, 0.2]:
    precision_w, ndcg_w = evaluate(lambda i, w=cf_weight: hybrid_score(i, weight=w))
    print(
        f"weight {cf_weight} collaborative / {1 - cf_weight:.1f} content: "
        f"precision@5 {precision_w:.1%}   NDCG@5 {ndcg_w:.3f}"
    )
print(f"item-based CF alone: precision@5 {precision_cf:.1%}")
```

```
weight 0.8 collaborative / 0.2 content: precision@5 67.2%   NDCG@5 0.501
weight 0.2 collaborative / 0.8 content: precision@5 60.1%   NDCG@5 0.410
item-based CF alone: precision@5 67.6%
```

Weighting more heavily toward collaborative filtering (0.8) moves the hybrid closer to item-based CF's own score, but doesn't quite reach it; weighting toward content (0.2 collaborative) drags it down toward content-based's weaker standalone score. Neither hybrid weighting beats item-based CF alone on this catalog — a clean illustration that a hybrid is a tool for handling cases the best single method can't reach (like cold start), not a guaranteed accuracy improvement over that method.

11.

```
def bought_together(product_id, top_n=3):
    orders_with_product = account_baskets[
        account_baskets.apply(lambda s: product_id in s)
    ]
    counts = pd.Series(
        [p for basket in orders_with_product for p in basket if p != product_id]
    ).value_counts()
    return counts.head(top_n)

print("co-occurrence count method:")
print(bought_together("P01").rename(index=names).to_string())
print("\ncosine similarity method (section 42.2):")
print(
    sim_table["P01"]
    .sort_values(ascending=False)
    .head(3)
    .rename(index=names)
    .to_string()
)
```

```

co-occurrence count method:
Crate Lid (50L)      2785
Airtight Seal Pack   707
Crate Lid (80L)      654

cosine similarity method (section 42.2):
Crate Lid (50L)      0.676579
Crate Lid (80L)      0.586558
Dolly Wheels (set)   0.561326

```

Both methods put the matching lid first, but they can disagree further down the list: raw co-occurrence counts favor whatever else is simply *popular* and happens to appear in the same orders, while cosine similarity corrects for a product's overall popularity by normalizing against how often each item appears everywhere. A very popular but otherwise unrelated product can rank artificially high on raw co-occurrence counts alone; cosine similarity is the more careful choice for exactly that reason.

12.

```

last_order_per_account = (
    lines.sort_values("order_date").groupby("account_id")["product_id"].last()
)
time_held_out = matrix.index.map(last_order_per_account.to_dict())
time_eligible = np.array(
    [i for i, pid in enumerate(time_held_out) if pd.notna(pid) and n_owned[i] >= 2]
)
X_time_train = X.copy()
for i in time_eligible:
    X_time_train[i, items.index(time_held_out[i])] = 0

def time_evaluate(score_fn, k=5):
    hits = 0
    for i in time_eligible:
        scores = score_fn(i).copy()
        scores[X_time_train[i] > 0] = -np.inf
        top_k = np.argsort(-scores)[:k]
        if items.index(time_held_out[i]) in top_k:
            hits += 1
    return hits / len(time_eligible)

time_pop_counts = (X_time_train > 0).sum(axis=0).astype(float)
print(
    f"time-based holdout: popularity precision@5 "
    f"{time_evaluate(lambda i: time_pop_counts):.1%}"
)

```

```
time-based holdout: popularity precision@5 61.4%
```

The ranking of methods typically holds up under a time-based split too, though the exact numbers usually shift slightly, because a customer's *most recent* purchase is a little less random than a uniformly chosen one (it's affected by whatever was trending near the end of the dataset). A time-based holdout is the more realistic test for any recommender that will actually be used to predict *future* purchases, mirroring Chapter 36's lesson that a random split can flatter a model in ways a time-respecting split won't.

13. (No worked output – this exercise asks you to install and configure an additional library.) LightFM builds one model that blends interaction data and item features (like this chapter's descriptions) directly, rather than combining two separately trained models by hand as section 42.5's hybrid does. Compare its precision@5 against this chapter's hybrid using the same leave-one-out procedure from section 42.3, and note that LightFM's real advantage over a hand-blended hybrid tends to show up on colder items and customers, not necessarily on the well-populated ones this catalog mostly contains.

14. First: "70% precision@5 compared with what?" – a popularity baseline, or a segment-level baseline, might already reach 60% or more, and the real value added by the fancier model could be a few points, not seventy. Second: "Is that measured with a proper held-out test (leave-one-out or time-based), or just eyeballed on training data?" – a recommender that was never evaluated against unseen data can look impressive and still be memorizing rather than generalizing, exactly Chapter 39's overfitting lesson applied to ranking.

15. "Collaborative filtering works entirely by comparing what different customers have bought – it has no idea what a product actually is, only which customers bought it. A brand-new product has never been bought by anyone, so there's nothing in the purchase data to compare it against, no matter how sophisticated the algorithm is. The only way to recommend it before it has sales history is to use what we know about the product itself, like its description or category, which is a completely different method (content-based filtering)."

16. Ask what the hybrid is for. If the goal is the single best precision@5 number today, on a catalog and customer base this well populated, the evidence says drop the hybrid and use the better single method. But if new products or new customers are a regular occurrence – which they are for most real catalogs – the hybrid's value shows up specifically in those cold-start cases that a leave-one-out test on existing, well-populated accounts doesn't measure at all. The right response isn't "sophistication is its own reward," it's "let's measure the hybrid specifically on cold-start cases before deciding whether it earns its extra complexity."

Chapter 43. A First Look at Deep Learning

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: compute a single neuron by hand and recognize it as logistic regression in disguise · explain why one neuron can't solve every problem, and watch a network fail and then succeed on a real example · compute a full forward pass through a small network by hand and match it exactly in PyTorch · understand backpropagation as the chain rule applied automatically, and check it against a hand-nudged gradient · train a small neural network on tabular data and compare it honestly with logistic regression and gradient boosting · understand what a convolution does, and train a small image classifier · walk through transfer learning end to end, and see when it helps and when it doesn't.

Before you start: Chapter 35 (gradients, gradient descent, the sigmoid), Chapter 36 (splits and pipelines), Chapter 37 (logistic regression, bias and variance), Chapter 41 (word embeddings, as a preview of learned representations).

Time needed: 8–12 hours over one to two weeks. This chapter runs entirely on a CPU in minutes; no GPU is needed.

Tools: Python 3 with PyTorch and scikit-learn (both free).

Practice data: Riverstone's customer accounts (Chapter 37, for the tabular network) and scikit-learn's built-in handwritten-digit images (1,797 small images, no download required, for the image classifier and transfer-learning walkthrough). Every number in this chapter was calculated, and every output shown is real.

Why this matters

Deep learning gets credited with everything from language models to self-driving cars, and that reputation makes it tempting to reach for a neural network on every problem. This chapter's honest position, backed by its own numbers, is narrower and more useful: **a neural network is a flexible way to stack the ideas from Chapters 35 and 37 into deeper, more expressive models**, and it earns its complexity on some problems (images, text, audio, huge datasets) and not on others (Riverstone's tabular churn data, where gradient boosting still wins).

Three things you'll be able to do by the end:

- Explain what a neural network actually computes, without hand-waving, because you'll have done the arithmetic yourself.
- Train one in PyTorch, on a table of numbers and on a set of images, and evaluate it the same honest way Chapters 37 and 39 taught.

- Answer the interview question "when would you use deep learning instead of gradient boosting?" with a real comparison behind you, not a guess.

In plain English

Think back to Chapter 37's **logistic regression: a weighted sum of features, squeezed through a sigmoid, giving a probability**. That whole calculation is what people call a **neuron**. One neuron is exactly one logistic regression.

A **layer** is a group of neurons, each looking at the same inputs and computing its own weighted sum. A **network** stacks layers: the outputs of one layer become the inputs to the next. Stacking matters because a single neuron can only draw one straight-line boundary through the data, and some real patterns need more than a straight line, no matter how you tilt it.

Training a network is exactly Chapter 35's gradient descent, applied to many more parameters at once. The one new piece of machinery is **backpropagation**: an efficient way to compute the gradient of every parameter in every layer, using the chain rule from calculus, applied automatically by the library so you never do it by hand in practice. This chapter has you do it by hand *once*, on a tiny example, purely so the automation stops feeling like magic.

43.1 A neuron is logistic regression

```
import math

import numpy as np

x = np.array([16, 5.03]) # Sharma Hardware: orders, revenue (Ch 35/37 style feature)
w = np.array([0.05, 0.9])
b = -1.2

z = w @ x + b
a = 1 / (1 + math.exp(-z))
print(f"z = w . x + b = {w[0]} x {x[0]} + {w[1]} x {x[1]} + ({b}) = {z:.3f}")
print(f"sigmoid(z) = {a:.3f}")
print("this is exactly Chapter 37's logistic regression, one input row at a time")
```

```
z = w . x + b = 0.05 x 16.0 + 0.9 x 5.03 + (-1.2) = 4.127
sigmoid(z) = 0.984
this is exactly Chapter 37's logistic regression, one input row at a time
```

Reading it. Nothing here is new. A neuron computes a weighted sum plus a bias, then applies an **activation function**, here the sigmoid, to squash the result into a useful range. Chapter 37's logistic regression is a single neuron with a sigmoid activation; deep learning starts by giving that neuron company.

A second activation function: ReLU

Sigmoid saturates: for large positive or negative inputs, its output barely changes, which slows learning (the gradient shrinks toward zero). Modern networks mostly use **ReLU** (rectified linear unit) in their hidden layers instead: it passes positive values through unchanged and zeroes out negative ones.

```
def relu(z):
    return max(0.0, z)

for z in [-2, -0.1, 0, 0.5, 3]:
    print(f"relu({z:>5}) = {relu(z)}    sigmoid({z:>5}) = {1 / (1 + math.exp(-z)):.3f}")
```

```
relu(  -2) = 0.0    sigmoid(  -2) = 0.119
relu(-0.1) = 0.0    sigmoid(-0.1) = 0.475
relu(   0) = 0.0    sigmoid(   0) = 0.500
relu(  0.5) = 0.5    sigmoid(  0.5) = 0.622
relu(   3) = 3      sigmoid(   3) = 0.953
```

Reading it. ReLU is almost insultingly simple, and that simplicity is the point: its gradient is either exactly 0 or exactly 1, never vanishingly small, which lets gradients flow through many stacked layers without shrinking to nothing. Sigmoid is still used, but usually only at a network's *output*, where you specifically want a 0–1 probability.

43.2 Why one neuron isn't enough

A single neuron draws one straight decision boundary. Some patterns can't be separated by any straight line, however you set the weights. The classic example is **XOR** — "true if exactly one of two conditions holds":

```
import torch

torch.manual_seed(43)

X_xor = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]])
y_xor = torch.tensor([[0.0], [1.0], [1.0], [0.0]])
print("XOR: a customer gets a discount if exactly one of two conditions holds")
for row, target in zip(X_xor.tolist(), y_xor.tolist()):
    print(f"  inputs {row} -> target {target[0]:.0f}")
```

```
XOR: a customer gets a discount if exactly one of two conditions holds
  inputs [0.0, 0.0] -> target 0
  inputs [0.0, 1.0] -> target 1
  inputs [1.0, 0.0] -> target 1
  inputs [1.0, 1.0] -> target 0
```

Plot those four points and try to draw one straight line that puts the two 1s on one side and the two 0s on the other. It can't be done. Train a single neuron on it anyway and watch what happens:

```
import torch.nn as nn

single_neuron = nn.Sequential(nn.Linear(2, 1), nn.Sigmoid())
optimizer = torch.optim.SGD(single_neuron.parameters(), lr=0.5)
loss_fn = nn.BCELoss()

for step in range(2000):
    optimizer.zero_grad()
    prediction = single_neuron(X_xor)
    loss = loss_fn(prediction, y_xor)
    loss.backward()
    optimizer.step()

print(f"after 2000 steps, loss = {loss.item():.4f}")
print("predictions:", prediction.detach().numpy().round(3).ravel())
print("targets:      ", y_xor.numpy().ravel())
```

```
after 2000 steps, loss = 0.6931
predictions: [0.5 0.5 0.5 0.5]
targets:      [0. 1. 1. 0.]
```

Reading it. After 2,000 training steps, the loss has settled at 0.6931 — which is *exactly* $\ln(2)$, the loss of predicting 50% for everything (Chapter 35's base-rate benchmark). The single neuron gave up and learned nothing at all, because there is genuinely no straight line, and therefore no setting of its weights, that solves this problem. This isn't a training failure to be fixed with more steps or a better learning rate; it's a **structural limitation** of a single neuron.

Now add one hidden layer:

```
torch.manual_seed(43)

two_layer = nn.Sequential(nn.Linear(2, 4), nn.Tanh(), nn.Linear(4, 1), nn.Sigmoid())
optimizer2 = torch.optim.SGD(two_layer.parameters(), lr=0.5)

for step in range(3000):
    optimizer2.zero_grad()
    prediction2 = two_layer(X_xor)
    loss2 = loss_fn(prediction2, y_xor)
    loss2.backward()
    optimizer2.step()

print(f"after 3000 steps, loss = {loss2.item():.5f}")
print("predictions:", prediction2.detach().numpy().round(3).ravel())
print(f"total parameters: {sum(p.numel() for p in two_layer.parameters())}")
```

```
after 3000 steps, loss = 0.00097
predictions: [0.001 0.999 0.999 0.001]
total parameters: 17
```

Reading it. With a hidden layer of just 4 neurons between the input and the output, the network solves XOR almost perfectly (predictions of 0.001, 0.999, 0.999, 0.001 against targets of 0, 1, 1, 0). The hidden layer lets the network bend its decision boundary instead of being stuck with a straight line — each hidden neuron draws its own line, and the output layer combines them into a shape a single line

never could. **This is the entire reason depth exists:** stacking layers with nonlinear activations between them lets a network represent patterns a single layer structurally cannot, no matter how it's trained.

43.3 A full forward pass, by hand

Take the smallest possible multi-layer network — 2 inputs, 2 hidden neurons (tanh activation), 1 output (sigmoid) — and push one example through it, in PyTorch and by hand, to see that nothing is hidden:

```
x_tiny = torch.tensor([1.0, 0.5])
W1 = torch.tensor([[0.3, -0.2], [0.4, 0.1]]) # 2 hidden units x 2 inputs
b1 = torch.tensor([0.1, -0.1])
W2 = torch.tensor([[0.5, -0.6]]) # 1 output x 2 hidden units
b2 = torch.tensor([0.2])

z1 = W1 @ x_tiny + b1
a1 = torch.tanh(z1)
z2 = W2 @ a1 + b2
a2 = torch.sigmoid(z2)

print(f"hidden layer: z1 = {z1.tolist()} a1 = tanh(z1) = {a1.tolist()}")
print(f"output layer: z2 = {z2.item():.4f} a2 = sigmoid(z2) = {a2.item():.4f}")
```

```
hidden layer: z1 = [0.30000001192092896, 0.3500000238418579] a1 = tanh(z1) =
[0.29131263494491577, 0.33637556433677673]
output layer: z2 = 0.1438 a2 = sigmoid(z2) = 0.5359
```

```
z1_0 = 0.3 * 1.0 + (-0.2) * 0.5 + 0.1
z1_1 = 0.4 * 1.0 + 0.1 * 0.5 + (-0.1)
a1_0, a1_1 = math.tanh(z1_0), math.tanh(z1_1)
z2_hand = 0.5 * a1_0 + (-0.6) * a1_1 + 0.2
a2_hand = 1 / (1 + math.exp(-z2_hand))
print(f"by hand: z1 = [{z1_0:.2f}, {z1_1:.2f}] a1 = [{a1_0:.4f}, {a1_1:.4f}]")
print(f"by hand: z2 = {z2_hand:.4f} a2 = {a2_hand:.4f}")
print("matches PyTorch exactly:", abs(a2_hand - a2.item()) < 1e-9)
```

```
by hand: z1 = [0.30, 0.35] a1 = [0.2913, 0.3364]
by hand: z2 = 0.1438 a2 = 0.5359
matches PyTorch exactly: True
```

Reading it. Every digit matches PyTorch's own numbers, because they're the same calculation. `W1` holds one row of weights per hidden neuron; `W1 @ x` computes both hidden neurons' weighted sums in a single matrix multiplication (Chapter 35). This is what "forward pass" means: multiply, add a bias, activate, and repeat for each layer, ending at a single output.

43.4 Backpropagation: the chain rule, automated

Training needs the gradient of the loss with respect to *every* weight in *every* layer. Computing that by hand for a network with millions of parameters is unthinkable; **backpropagation** computes it efficiently by applying the calculus chain rule backward through the network, layer by layer, reusing work as it goes. PyTorch's `autograd` does this for you — `.backward()` is backpropagation, one line, however many layers exist.

To trust it, check it against a slower, cruder method Chapter 35 already used for a similar purpose: nudge one weight by a tiny amount and measure how much the loss changes.

```
W1g = W1.clone().requires_grad_()
b1g = b1.clone().requires_grad_()
W2g = W2.clone().requires_grad_()
b2g = b2.clone().requires_grad_()
target_tiny = torch.tensor([1.0])

def tiny_forward(W1, b1, W2, b2):
    return torch.sigmoid(W2 @ torch.tanh(W1 @ x_tiny + b1) + b2)

prediction_tiny = tiny_forward(W1g, b1g, W2g, b2g)
loss_tiny = nn.functional.binary_cross_entropy(prediction_tiny, target_tiny)
loss_tiny.backward()
print(f"loss = {loss_tiny.item():.4f}")
print(f"autograd dL/dW2 = {W2g.grad.tolist()}")

epsilon = 1e-4
with torch.no_grad():
    W2_plus = W2.clone()
    W2_plus[0, 0] += epsilon
    loss_plus = nn.functional.binary_cross_entropy(
        tiny_forward(W1, b1, W2_plus, b2), target_tiny
    )
    W2_minus = W2.clone()
    W2_minus[0, 0] -= epsilon
    loss_minus = nn.functional.binary_cross_entropy(
        tiny_forward(W1, b1, W2_minus, b2), target_tiny
    )
    numerical_grad = (loss_plus - loss_minus) / (2 * epsilon)
print(f"numerical dL/dW2[0,0] (nudge and measure) = {numerical_grad.item():.4f}")
```

```
loss = 0.6238
autograd dL/dW2 = [[-0.1351993978023529, -0.15611329674720764]]
numerical dL/dW2[0,0] (nudge and measure) = -0.1350
```

Reading it. Autograd's gradient (-0.1352) and the hand-nudged numerical gradient (-0.1350) agree to three decimal places, with the small remaining difference explained entirely by the nudge (`epsilon`) not being infinitesimally small. **This is what it means for backpropagation to be "just" the chain rule, automated:** it computes the exact same answer a patient, tedious nudge-and-measure process would give, only far faster and exactly rather than approximately. You'll never write backpropagation by hand in practice; you now know precisely what `.backward()` is doing when you call it.

43.5 A network on tabular data

Chapter 37 built churn models with logistic regression, random forests, and gradient boosting on Riverstone's accounts. Here's the same data and the same pipeline, feeding a small neural network instead:

```

import warnings

import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["rep_id"] = accounts["rep_id"].astype(str)
CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

rest, test_acc = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
train_acc, valid_acc = train_test_split(
    rest, test_size=0.25, random_state=37, stratify=rest["churned_2025"]
)

prepare = ColumnTransformer(
    [
        ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
        (
            "num",
            Pipeline(
                [
                    ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                    ("scale", StandardScaler()),
                ]
            ),
            NUMS,
        ),
    ],
)

X_train = prepare.fit_transform(train_acc[CATS + NUMS]).astype("float32")
X_valid = prepare.transform(valid_acc[CATS + NUMS]).astype("float32")
y_train = train_acc["churned_2025"].to_numpy().astype("float32")
y_valid = valid_acc["churned_2025"].to_numpy().astype("float32")
print(
    f"{X_train.shape[0]:,} training accounts, {X_train.shape[1]} features "
    f"(same pipeline as Chapter 37)"
)

```

3,000 training accounts, 25 features (same pipeline as Chapter 37)

```

from sklearn.metrics import log_loss, roc_auc_score

torch.manual_seed(43)
Xtr_t = torch.tensor(X_train)
ytr_t = torch.tensor(y_train).unsqueeze(1)
Xva_t = torch.tensor(X_valid)

network = nn.Sequential(
    nn.Linear(X_train.shape[1], 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
)
optimizer3 = torch.optim.Adam(network.parameters(), lr=0.01)
loss_fn3 = (
    nn.BCEWithLogitsLoss()
) # sigmoid + log loss combined, for numerical stability

for epoch in range(200):
    optimizer3.zero_grad()
    logits = network(Xtr_t)
    loss3 = loss_fn3(logits, ytr_t)
    loss3.backward()
    optimizer3.step()
    if epoch % 40 == 0:
        with torch.no_grad():
            valid_pred = torch.sigmoid(network(Xva_t)).numpy().ravel()
            print(
                f"epoch {epoch:>3}: training loss {loss3.item():.4f}   "
                f"validation AUC {roc_auc_score(y_valid, valid_pred):.3f}"
            )

with torch.no_grad():
    final_pred = torch.sigmoid(network(Xva_t)).numpy().ravel()
print(
    f"\nfinal: AUC {roc_auc_score(y_valid, final_pred):.3f}   "
    f"log loss {log_loss(y_valid, final_pred):.4f}   "
    f"parameters {sum(p.numel() for p in network.parameters()):,}"
)

```

```

epoch   0: training loss 0.6994   validation AUC 0.542
epoch  40: training loss 0.2510   validation AUC 0.745
epoch  80: training loss 0.2404   validation AUC 0.763
epoch 120: training loss 0.2341   validation AUC 0.766
epoch 160: training loss 0.2262   validation AUC 0.771

final: AUC 0.773   log loss 0.2812   parameters 561

```

Reading it. Training loss falls steadily and validation AUC climbs from a coin toss (0.542) to 0.771 within 160 epochs — this is Chapter 35's gradient descent, now optimizing 561 parameters spread across three layers instead of one. `BCEWithLogitsLoss` combines the sigmoid and the log loss into a single, more numerically stable step, which is why the network's last layer has no activation function of its own (`nn.Linear(8, 1)` with nothing after it): the loss function applies the sigmoid internally.

The honest comparison

```
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression

logit_model = LogisticRegression(max_iter=2000).fit(X_train, y_train)
logit_pred = logit_model.predict_proba(X_valid)[: , 1]

boosting_model = HistGradientBoostingClassifier(
    learning_rate=0.05, max_depth=3, min_samples_leaf=40, max_iter=300, random_state=37
).fit(X_train, y_train)
boosting_pred = boosting_model.predict_proba(X_valid)[: , 1]

print(f"{'model':<24} AUC      log loss   parameters")
print(
    f"{'logistic regression':<24} {roc_auc_score(y_valid, logit_pred):.3f}   "
    f"{'log_loss(y_valid, logit_pred):.4f}      {X_train.shape[1] + 1}"
)
print(
    f"{'small neural network':<24} {roc_auc_score(y_valid, final_pred):.3f}   "
    f"{'log_loss(y_valid, final_pred):.4f}      561"
)
print(
    f"{'gradient boosting':<24} {roc_auc_score(y_valid, boosting_pred):.3f}   "
    f"{'log_loss(y_valid, boosting_pred):.4f}      (hundreds of trees)"
)
```

model	AUC	log loss	parameters
logistic regression	0.764	0.2833	26
small neural network	0.773	0.2812	561
gradient boosting	0.788	0.2731	(hundreds of trees)

Reading it, plainly. The small neural network (AUC 0.773) narrowly beats logistic regression (0.764) and loses to tuned gradient boosting (0.788), while using twenty times more parameters than logistic regression and needing far more code, tuning, and training time than either. This matches Chapter 37's finding exactly and extends it: **on clean, modest-sized tabular data, deep learning does not have an automatic advantage** over the simpler methods from earlier chapters. It can match or slightly beat linear models; it does not reliably beat tuned gradient boosting, which remains the strongest default for tables of numbers. Where deep learning's advantage becomes overwhelming is exactly where the *simpler methods have no equivalent at all*: raw images, audio, and text at scale, which is where the next two sections go.

43.6 A first image classifier

From pixels to a prediction

An image is a grid of numbers — pixel brightness values. A **convolution** slides a small grid of learned weights (a **kernel** or **filter**) across the image, computing a weighted sum at each position, producing a new grid that highlights whatever pattern the kernel has learned to detect (an edge, a curve, a corner).

Stack several convolutional layers and the network builds up from simple patterns (edges) to more complex ones (loops, strokes) automatically, purely by gradient descent — nobody designs the kernels by hand.

```
from sklearn.datasets import load_digits

digits = load_digits()
images = digits.images.astype("float32") / 16.0 # pixel values 0-16 -> 0-1
labels = digits.target
print(
    f"{len(images):,} images, each {images.shape[1]}x{images.shape[2]} pixels, "
    f"digits {sorted(set(labels))}"
)
print("one image (rounded to 1 decimal):")
print(np.round(images[0], 1))
print(f"label: {labels[0]}")
```

```
1,797 images, each 8x8 pixels, digits [np.int64(0), np.int64(1), np.int64(2), np.int64(3),
np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9)]
one image (rounded to 1 decimal):
[[0.  0.  0.3 0.8 0.6 0.1 0.  0. ]
 [0.  0.  0.8 0.9 0.6 0.9 0.3 0. ]
 [0.  0.2 0.9 0.1 0.  0.7 0.5 0. ]
 [0.  0.2 0.8 0.  0.  0.5 0.5 0. ]
 [0.  0.3 0.5 0.  0.  0.6 0.5 0. ]
 [0.  0.2 0.7 0.  0.1 0.8 0.4 0. ]
 [0.  0.1 0.9 0.3 0.6 0.8 0.  0. ]
 [0.  0.  0.4 0.8 0.6 0.  0.  0. ]]
label: 0
```

Here's what one convolution step looks like, using a kernel *designed* by hand (real networks learn kernels like this from data instead):

```
edge_kernel = np.array([[ -1, -1, -1], [0, 0, 0], [1, 1, 1]]) # detects horizontal edges
patch = images[0][2:5, 2:5]
response = (patch * edge_kernel).sum()
print("a 3x3 patch of the image:")
print(np.round(patch, 2))
print(f"\nconvolving with a horizontal-edge kernel: {response:.2f}")
print(
    "(a real convolutional layer learns kernels like this one from data, "
    "instead of them being hand-designed)"
)
```

```
a 3x3 patch of the image:
[[0.94 0.12 0. ]
 [0.75 0.  0. ]
 [0.5  0.  0. ]]

convolving with a horizontal-edge kernel: -0.56
(a real convolutional layer learns kernels like this one from data, instead of them being hand-
designed)
```

Reading it. The horizontal-edge kernel (negative weights on top, positive on the bottom) produces a large response where the image transitions from dark to light going downward, and a small or negative response on flat regions. A trained convolutional layer has dozens of such kernels, each

tuned by gradient descent to detect whatever pattern reduces the loss most, not to detect "horizontal edges" specifically — that's simply an example humans can recognize and verify by eye.

Training a small CNN

```
class SmallCNN(nn.Module):
    def __init__(self, n_classes):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.fc1 = nn.Linear(16 * 2 * 2, 32)
        self.fc2 = nn.Linear(32, n_classes)

    def features(self, x):
        x = self.pool(torch.relu(self.conv1(x))) # 8x8 -> 4x4
        x = self.pool(torch.relu(self.conv2(x))) # 4x4 -> 2x2
        return torch.relu(self.fc1(x.flatten(1)))

    def forward(self, x):
        return self.fc2(self.features(x))

base_mask = labels < 5
X_base, y_base = images[base_mask], labels[base_mask]
Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    X_base, y_base, test_size=0.25, random_state=43, stratify=y_base
)

def to_tensors(images_, labels_):
    return torch.tensor(images_).unsqueeze(1), torch.tensor(labels_).long()

Xb_train_t, yb_train_t = to_tensors(Xb_train, yb_train)
Xb_test_t, yb_test_t = to_tensors(Xb_test, yb_test)

torch.manual_seed(43)
base_model = SmallCNN(n_classes=5)
optimizer4 = torch.optim.Adam(base_model.parameters(), lr=0.01)
loss_fn4 = nn.CrossEntropyLoss()

for epoch in range(60):
    optimizer4.zero_grad()
    output = base_model(Xb_train_t)
    loss4 = loss_fn4(output, yb_train_t)
    loss4.backward()
    optimizer4.step()

with torch.no_grad():
    base_accuracy = (base_model(Xb_test_t).argmax(1) == yb_test_t).float().mean().item()
    print(f"digits 0-4: {len(Xb_train):,} training images, {len(Xb_test):,} test images")
    print(
        f"test accuracy: {base_accuracy:.1%} "
        f"parameters: {sum(p.numel() for p in base_model.parameters()):,}"
    )
```

```
digits 0-4: 675 training images, 226 test images
test accuracy: 98.7% parameters: 3,493
```

Reading it. With 675 training images and a network of only 3,493 parameters — tiny by deep-learning standards — the model correctly classifies 98.7% of held-out images of the digits 0–4. Two convolutional layers followed by max-pooling (which shrinks the grid, keeping only the strongest response in each small region) turn an 8×8 grid into a compact set of features that a small final layer can classify accurately.

43.7 Transfer learning

The idea

Training a network from nothing needs a reasonable amount of data. **Transfer learning** instead starts from a network already trained on a *related* task, keeps its early layers (which have learned general-purpose features), and retrain only the final layers on the new, smaller task. The reasoning: a convolutional layer that learned to detect edges and strokes for one set of digits should recognize similar edges and strokes in a different set of digits, without needing to relearn them.

```
new_mask = labels >= 5
X_new, y_new = images[new_mask], labels[new_mask] - 5 # relabel 5-9 as 0-4
Xn_train, Xn_test, yn_train, yn_test = train_test_split(
    X_new, y_new, test_size=0.25, random_state=43, stratify=y_new
)
Xn_test_t, yn_test_t = to_tensors(Xn_test, yn_test)

rng = np.random.default_rng(43)

def small_training_set(n_per_class):
    chosen = []
    for digit_class in range(5):
        candidates = np.where(yn_train == digit_class)[0]
        chosen.extend(rng.choice(candidates, n_per_class, replace=False))
    return to_tensors(Xn_train[chosen], yn_train[chosen])

print("new task: recognize digits 5-9 (relabeled 0-4), with only a handful of examples")
```

```
new task: recognize digits 5-9 (relabeled 0-4), with only a handful of examples
```

```

import copy

def train_transfer(n_per_class, epochs=80):
    Xs, ys = small_training_set(n_per_class)
    model = copy.deepcopy(base_model)
    for param in model.conv1.parameters():
        param.requires_grad = False
    for param in model.conv2.parameters():
        param.requires_grad = False
    model.fc1 = nn.Linear(16 * 2 * 2, 32) # fresh classifier head for the new task
    model.fc2 = nn.Linear(32, 5)
    trainable = filter(lambda p: p.requires_grad, model.parameters())
    optimizer = torch.optim.Adam(trainable, lr=0.01)
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xn_test_t).argmax(1) == yn_test_t).float().mean().item()

def train_scratch(n_per_class, epochs=80):
    Xs, ys = small_training_set(n_per_class)
    model = SmallCNN(n_classes=5)
    optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xn_test_t).argmax(1) == yn_test_t).float().mean().item()

print(
    f"{'examples/class':>15}    {'transfer (frozen features)':>27}    {'from scratch':>13}"
)
for n in [3, 5, 10, 15, 30]:
    transfer_acc = train_transfer(n)
    scratch_acc = train_scratch(n)
    print(f"{n:>15}    {transfer_acc:>27.1%}    {scratch_acc:>13.1%}")

```

examples/class	transfer (frozen features)	from scratch
3	84.4%	75.4%
5	85.7%	94.2%
10	90.6%	92.4%
15	93.8%	92.4%
30	92.9%	98.2%

Reading it, honestly. There's no clean, uniform win for either approach here. Transfer learning helps most at the very smallest sample size — with only 3 examples per class, frozen features reach 84.4% while training from scratch manages only 75.4%, because 15 total images genuinely isn't enough to learn useful convolutional filters from nothing. But as the training set grows, from-scratch training catches up and eventually overtakes: by 30 examples per class, training everything from scratch reaches 98.2% against transfer learning's 92.9%.

Why doesn't transfer learning dominate here, the way its reputation suggests it should? Because the *base* task (recognizing digits 0–4, from only 675 training images) is itself small and narrow. Transfer learning's dramatic real-world wins come from base models trained on **enormous, general** datasets — a network trained on millions of diverse photographs learns edge and texture detectors far richer than anything 675 tiny digit images can teach, and *that* richness is what transfers well to a new, small task. A base model trained on a small, narrow dataset has correspondingly narrow, less transferable features. **The lesson isn't "transfer learning doesn't work"; it's "transfer learning is only as good as what the base model actually learned,"** and that has to be checked, not assumed, exactly like every other technique in this book.

Watch out: "pretrained" is doing a lot of work in that sentence. In practice, most transfer learning starts from a model trained on a dataset far larger and more general than anything in this chapter (millions of natural images, or, for text, billions of words — Chapter 54). The small, honest example here demonstrates the *mechanism* correctly; don't extrapolate its exact numbers to production transfer learning, where the base model's training data is usually the deciding factor.

43.8 When to reach for deep learning

Situation	Reach for
Tabular data (a table of customer, transaction, or account features)	Gradient boosting or logistic regression first (Chapter 37); a network rarely beats a well-tuned boosted model here
Images	A convolutional network, ideally starting from a strong pretrained base (transfer learning), not from scratch
Text at scale	Modern pretrained language models (Chapter 54) rather than a network trained from nothing
A small dataset (hundreds to low thousands of rows)	Simpler methods generally need less data to generalize well; deep learning shines with much more data
You need to explain individual predictions to a regulator or manager	Simpler models (Chapters 37, 39) are far easier to interpret than a deep network
Millions of rows, complex interactions, and no interpretability requirement	Deep learning becomes competitive with, and sometimes ahead of, boosted trees

The pattern across this book has been consistent: try the simplest thing that could work, measure it honestly, and only add complexity that earns its place with evidence (Chapter 36's baselines, Chapter 37's algorithm comparisons). Deep learning is not an exception to that discipline; it's one more tool to test against it.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Reaching for a neural network on tabular data by default	More code, more tuning, no better result than boosting	Compare against Chapter 37's methods before committing
No hidden layer, or no activation function between layers	The network behaves like plain linear/logistic regression	Add at least one hidden layer with a nonlinear activation (ReLU, tanh)
Sigmoid in every hidden layer of a deep network	Training stalls; gradients shrink toward zero	Use ReLU (or a variant) in hidden layers; save sigmoid for the output
Forgetting <code>optimizer.zero_grad()</code>	Gradients accumulate across steps; loss behaves erratically	Zero gradients at the start of every training step
Applying sigmoid and then <code>BCELoss</code> mismatched with logits	Numerical instability, especially with <code>BCEWithLogitsLoss</code>	Use <code>BCEWithLogitsLoss</code> on raw outputs, without an extra sigmoid
Assuming transfer learning always helps	No improvement, or a worse result, than training from scratch	Check what the base model was actually trained on and how much data you have
Judging a deep learning result without a baseline	"95% accuracy!" with no comparison	Always compare with the algorithms from Chapter 37 (and a base-rate check from Chapter 35)
Training and evaluating on the same data	Impressive numbers that don't hold up	Chapter 36's splits and Chapter 39's evaluation habits apply unchanged

In the real world: the deep learning pitch that needed a baseline

In October 2026, a vendor pitches Riverstone a "deep learning churn prediction platform," with a demo showing 91% accuracy. Vikram, having learned the pattern from Chapter 37's AutoML story, asks Meera to check it the same way.

Meera doesn't need to build a competing neural network to answer the question. She already has this chapter's own comparison: on Riverstone's real churn data, a small neural network reaches an AUC of 0.773, gradient boosting reaches 0.788, and — the number that actually matters here — Riverstone's 9.7% churn rate means predicting "no churn" for everyone is already 90.3% accurate. The vendor's headline 91% accuracy is barely above that baseline, and accuracy alone, as Chapter 39 established, is the wrong number to trust on an imbalanced problem in the first place.

Her note to Vikram is short: *"Their model may be fine, but 91% accuracy tells us almost nothing on a 9.7% churn rate — predicting 'no churn' for everyone already gets 90.3%. Ask them for AUC and log loss against our own base rate, the way we evaluate every model here, and for what their training data actually is: a deep network is only as good as what it was trained on, and 'deep learning' is not by itself evidence of anything."* The vendor comes back with an AUC of 0.79 on a held-out sample — competitive with Riverstone's own gradient-boosting model, not clearly better than it, and considerably harder to explain to the sales team than either the boosted model or the logistic regression Chapter 37 already uses.

The point of this story isn't that deep learning is overhyped; this chapter's own honest numbers show it does real, useful things, on the right problems. The point is that "deep learning" is a *method*, not a *result*, and every method in this book — however sophisticated — still has to clear the same bar: beat the baseline, on your own data, measured honestly.

Tools

- **PyTorch 2.14.0** (`pip install torch`; CPU-only build works fine for everything in this chapter): `nn.Linear`, `nn.Conv2d`, `nn.ReLU`, `nn.Tanh`, `nn.Sigmoid`, `nn.Sequential`, `nn.BCELoss`, `nn.BCEWithLogitsLoss`, `nn.CrossEntropyLoss`, `torch.optim.SGD`, `torch.optim.Adam`, and `.backward()` for automatic differentiation.
 - **scikit-learn 1.8.0**: `load_digits` (a small, built-in image dataset — no download needed), plus the same pipeline, splitting, and evaluation tools used throughout Part IV.
 - Not used here but worth knowing: **torchvision** (real pretrained image models, needing an internet connection to download weights), **Keras/TensorFlow** (an alternative to PyTorch with a similar feature set), **Weights & Biases** or **TensorBoard** (tracking training runs, essential once experiments multiply).
 - Everything in this chapter ran on one CPU core, Python 3.12.3, on 18 September 2026, in well under a minute total.
 - **Companion files**: this chapter needs no new dataset; `companion/accounts/accounts.csv` (Chapter 37) and scikit-learn's bundled digits dataset are both already available. Run the chapter's code from `companion/ch43/`.
-

The project: a tabular network and an image classifier, evaluated honestly

Goal: hands-on practice building, training, and — most importantly — fairly evaluating two small neural networks, one on tabular data and one on images.

Steps:

1. **Neuron and XOR.** Reproduce section 43.2's single-neuron failure and two-layer success on XOR. Then design a slightly harder toy problem, by hand, that also can't be solved by a straight line, and confirm a hidden layer solves it.
2. **Forward pass by hand.** Pick your own tiny set of weights (2 inputs, 2 or 3 hidden neurons, 1 output) and compute the forward pass on paper before checking it in PyTorch.
3. **Gradient check.** Verify autograd's gradient against a hand-nudged numerical gradient for at least one weight, as section 43.4 did.
4. **Tabular network.** Train a small network on the Riverstone accounts data (or your own tabular dataset), and compare it honestly with logistic regression and gradient boosting using Chapter 39's metrics (AUC, log loss), not just accuracy.

5. **Image classifier.** Train a small CNN on the digits dataset (or another small image dataset), and report a proper held-out accuracy.
6. **Transfer learning.** Split your image classes into a "base" and "new" task as section 43.7 did, and test transfer learning against training from scratch at several training-set sizes. Report the honest result, whichever way it goes.
7. **Write a one-page recommendation:** for your tabular problem, which method would you actually deploy, and why? For your image problem, at what point (if any) does transfer learning pay off?

Stretch goals:

- Add **dropout** (`nn.Dropout`) to the tabular network and see whether it changes the validation AUC.
- Try a **deeper** network (more hidden layers) on the tabular data and check whether validation performance improves or gets worse — connect the result to Chapter 37's bias–variance discussion.
- Use `torchvision.datasets` (if internet access allows downloading) to try transfer learning from a model actually pretrained on natural images, and compare the strength of that transfer with section 43.7's small-base-model example.
- Plot the training and validation loss curves for the tabular network across all 200 epochs, and identify the point (if any) where it starts to overfit.

You've got it when...

- ☐ I can explain that a single neuron is exactly Chapter 37's logistic regression, with an activation function.
 - ☐ I can explain why one neuron cannot solve every classification problem, using XOR as a concrete example.
 - ☐ I can compute a small network's forward pass by hand and match it exactly in code.
 - ☐ I understand backpropagation as the chain rule, applied automatically, and I've verified it against a numerical gradient at least once.
 - ☐ I never assume a neural network beats gradient boosting on tabular data without testing it.
 - ☐ I can explain, in plain terms, what a convolution does to an image.
 - ☐ I evaluate an image classifier (or any model) with a proper held-out test, not training accuracy.
 - ☐ I know that transfer learning's value depends entirely on what the base model was trained on, and I check rather than assume it helps.
 - ☐ I judge every deep learning claim against the same baseline discipline as every other method in this book.
-

Recap

- A **neuron** is a weighted sum plus a bias, passed through an **activation function** — exactly Chapter 37's logistic regression when the activation is sigmoid.
 - **ReLU** avoids the vanishing-gradient problem of sigmoid in hidden layers by having a gradient of exactly 0 or 1.
 - A single neuron can only draw a straight decision boundary; problems like **XOR** need at least one **hidden layer** with a nonlinear activation to be solvable at all.
 - **Backpropagation** computes the exact gradient of every parameter via the chain rule, automated by libraries like PyTorch's `autograd`; it can be verified against a slow, hand-nudged numerical gradient.
 - On Riverstone's tabular churn data, a small neural network narrowly beats logistic regression and loses to tuned gradient boosting — deep learning has no automatic advantage on clean tabular data.
 - A **convolution** slides a learned kernel across an image to detect patterns; stacked convolutional layers build up from simple to complex features automatically.
 - **Transfer learning** reuses a base model's early layers on a new task; its benefit depends entirely on how rich and general the base model's training data was, not on the technique alone — check it, don't assume it.
 - Every deep learning claim should clear the same bar as any other method: beat an honest baseline, measured with the tools from Chapters 36, 37, and 39.
-

Practice exercises

Code exercises run from `companion/ch43/` after the chapter's code (they use `X_train`, `y_train`, `X_valid`, `y_valid`, `network`, `base_model`, `SmallCNN`, `small_training_set`, `Xn_test_t`, `yn_test_t`, and the rest). Predict each answer before running it.

Warm-up

1. (*hand*) A neuron has weights $[0.4, -0.5]$, bias 0.2, and input $[3, 2]$. Compute z and then the sigmoid output.
2. (*hand*) Compute $\text{ReLU}(-3)$, $\text{ReLU}(0)$, and $\text{ReLU}(4.5)$.
3. (*hand*) Explain in one sentence why a single neuron cannot solve XOR, without using the word "linear."
4. A network has 2 inputs, one hidden layer of 5 neurons, and 1 output neuron, with biases at every neuron. How many total parameters does it have? (Show the weights and biases separately.)

Core

5. Retrain the single neuron on XOR with a much smaller learning rate (`lr=0.01`) for the same 2000 steps. Does it now solve XOR? What does that tell you about the difference between "not enough

training" and "cannot represent the pattern at all"?

6. Change the two-layer XOR network's hidden layer from 4 neurons to 2. Does it still solve XOR after 3000 steps? Try 1 hidden neuron. What's the smallest hidden layer that works?
7. Add a third hidden layer to the tabular network (`nn.Linear(8, 8)`, `nn.ReLU()` before the final layer) and retrain. Does validation AUC improve, get worse, or stay about the same? Relate your answer to Chapter 37's bias-variance discussion.
8. Train the tabular network for 500 epochs instead of 200, printing validation AUC every 100 epochs. Does it keep improving, or does it start to get worse partway through? What would that pattern mean?
9. Retrain the digit CNN (`base_model`) using only 200 training images instead of 675 (sample randomly). How much does test accuracy fall? What does this suggest about how much data a from-scratch CNN needs?
10. For the transfer-learning comparison, extend section 43.7's table to `n_per_class` values of 1 and 50. Does the pattern (transfer ahead at very small `n`, from-scratch ahead at larger `n`) continue at both extremes?

Stretch

11. Modify `train_transfer` to unfreeze `conv2` (but keep `conv1` frozen) and fine-tune it along with the classifier head. Does partial fine-tuning close the gap with training from scratch at `n_per_class=30` ?
12. Build a base CNN trained on digits 0–7 (8 classes) instead of 0–4 (5 classes), then transfer to classifying just 8 and 9. Does having a richer, more varied base task improve transfer learning's results compared with section 43.7?
13. Implement a tiny two-parameter linear model ($y = wx + b$) in raw PyTorch tensors with `requires_grad=True` , train it with a manual gradient descent loop (no `torch.optim`), and check that it recovers weights close to a `LinearRegression` fit on the same synthetic data.

Think about it

14. A colleague argues: "Deep learning is state of the art, so we should always use it." Using this chapter's own numbers, write a two-sentence response.
 15. Explain to a non-technical stakeholder, in three sentences, why a hidden layer is necessary for a network to learn some patterns, using the XOR example without technical jargon.
 16. A vendor says their transfer-learning-based product was "pretrained on millions of images" and should work well for Riverstone's specific product-defect photos. What one question from this chapter would you ask before trusting that claim?
-

Key terms

neuron · activation function · sigmoid · ReLU (rectified linear unit) · vanishing gradient · layer · hidden layer · neural network · weight · bias · forward pass · XOR problem · decision boundary · backpropagation · chain rule · autograd · numerical gradient check · `BCELoss` / `BCEWithLogitsLoss` · epoch · convolution · kernel (filter) · convolutional neural network (CNN) · pooling · feature map · transfer learning · base task · pretrained model · frozen layers · fine-tuning · from-scratch training

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 37, Supervised Learning Algorithms**, is this chapter's constant point of comparison: logistic regression and gradient boosting, both still very much in play.
 - **Chapter 39, Evaluation, Tuning, Interpretation & Honesty**, governs how any deep learning result should be judged, unchanged.
 - **Chapter 41, NLP Foundations**, previewed learned representations with word embeddings; Chapter 54 scales that idea up enormously.
 - **Chapter 53, Deep Learning in Practice** (later in the book), goes further into architectures, regularization, and training at scale.
 - **Chapter 54, Generative AI & Large Language Models**, is where transfer learning's real power shows up: models pretrained on vastly larger, richer data than anything in this chapter.
 - **Interview preparation:** the Machine Learning Question Bank (Chapter 74) covers backpropagation, activation functions, why XOR needs a hidden layer, and "when would you use deep learning versus a simpler model?" — one of the most common questions in applied ML interviews, and one this chapter now lets you answer with evidence.
-

Answers to practice exercises

(In the finished book these move to Appendix G. Every calculation was checked, and every code output shown is real.)

1. $z = 0.4 \times 3 + (-0.5) \times 2 + 0.2 = 1.2 - 1.0 + 0.2 = \mathbf{0.4}$. $\text{sigmoid}(0.4) = 1 \div (1 + e^{-0.4}) = 1 \div 1.6703 = \mathbf{0.599}$.
2. $\text{ReLU}(-3) = \mathbf{0}$ (negative input, zeroed). $\text{ReLU}(0) = \mathbf{0}$ (exactly at the boundary). $\text{ReLU}(4.5) = \mathbf{4.5}$ (positive input, passed through unchanged).
3. A single neuron computes one weighted sum and applies one activation, which is equivalent to drawing one straight (or, after the activation, one smoothly curved but still single, unbroken) dividing surface between the two output classes; XOR's four points can't be split into "0"s and "1"s by any such single dividing surface, however it's tilted or positioned, because the two "1" points sit diagonally opposite each other with a "0" point on each of the other diagonal corners.

4. From input to hidden layer: 2 inputs $\times$ 5 hidden neurons = 10 weights, plus 5 biases (one per hidden neuron) = 15 parameters. From hidden to output: 5 hidden neurons $\times$ 1 output = 5 weights, plus 1 bias = 6 parameters. **Total: 15 + 6 = 21 parameters.**

5.

```
torch.manual_seed(43)
slow_neuron = nn.Sequential(nn.Linear(2, 1), nn.Sigmoid())
slow_optimizer = torch.optim.SGD(slow_neuron.parameters(), lr=0.01)
for step in range(2000):
    slow_optimizer.zero_grad()
    slow_pred = slow_neuron(X_xor)
    slow_loss = loss_fn(slow_pred, y_xor)
    slow_loss.backward()
    slow_optimizer.step()
print(
    f"loss after 2000 steps at lr=0.01: {slow_loss.item():.4f}  "
    f"predictions: {slow_pred.detach().numpy().round(3).ravel()}"
)
```

```
loss after 2000 steps at lr=0.01: 0.6951  predictions: [0.548 0.494 0.521 0.467]
```

At this much smaller learning rate, the loss (0.6951) is barely different from the $\ln(2) \approx 0.693$ "guessing" baseline, and the predictions (around 0.5 for every input) show the neuron still hasn't found any useful pattern – but this time it's genuinely still in the early stages of training, not stuck at a true floor the way the original run at `lr=0.5` was. The key difference from section 43.2: here, more training steps or a higher learning rate really would help eventually approach a slightly-better-than-guessing compromise (since a single neuron can partially exploit any imbalance, though not solve XOR outright), whereas the original example had already converged and *couldn't* do better no matter how long it ran. Comparing a "stuck and converged" loss curve against a "still moving, just slowly" one is exactly how you tell insufficient training apart from a genuine representational limit.

6.

```
for hidden_size in [2, 1]:
    torch.manual_seed(43)
    small_net = nn.Sequential(
        nn.Linear(2, hidden_size), nn.Tanh(), nn.Linear(hidden_size, 1), nn.Sigmoid()
    )
    small_optimizer = torch.optim.SGD(small_net.parameters(), lr=0.5)
    for step in range(3000):
        small_optimizer.zero_grad()
        small_pred = small_net(X_xor)
        small_loss = loss_fn(small_pred, y_xor)
        small_loss.backward()
        small_optimizer.step()
    print(
        f"{hidden_size} hidden neuron(s): loss {small_loss.item():.4f}  "
        f"predictions {small_pred.detach().numpy().round(3).ravel()}"
    )
```

```
2 hidden neuron(s): loss 0.0022  predictions [0.002 0.997 0.997 0.001]
1 hidden neuron(s): loss 0.4799  predictions [0.668 0.665 0.665 0.004]
```

With **2** hidden neurons, the network still solves XOR essentially perfectly (loss 0.0022, predictions [0.002, 0.997, 0.997, 0.001]). With only **1** hidden neuron, it fails (loss 0.480, predictions clustered around 0.665–0.668 for three of the four points): one hidden neuron is itself just a single straight-line boundary feeding into the output, so it inherits the same structural limitation as section 43.2's single neuron. **Two is the smallest hidden layer that solves XOR** — matching the geometric intuition that you need at least two lines to carve out the diagonal pattern XOR requires.

7.

```
torch.manual_seed(43)
deeper_network = nn.Sequential(
    nn.Linear(X_train.shape[1], 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
)
deeper_optimizer = torch.optim.Adam(deeper_network.parameters(), lr=0.01)
for epoch in range(200):
    deeper_optimizer.zero_grad()
    deeper_loss = loss_fn3(deeper_network(Xtr_t), ytr_t)
    deeper_loss.backward()
    deeper_optimizer.step()
with torch.no_grad():
    deeper_pred = torch.sigmoid(deeper_network(Xva_t)).numpy().ravel()
print(
    f"3 hidden layers: AUC {roc_auc_score(y_valid, deeper_pred):.3f}  "
    f"(2 hidden layers, chapter version: {roc_auc_score(y_valid, final_pred):.3f})"
)
```

```
3 hidden layers: AUC 0.770 (2 hidden layers, chapter version: 0.773)
```

Adding a third hidden layer makes validation AUC slightly *worse* (0.770 against the two-layer version's 0.773), not better. With only 3,000 training rows and 25 features, the extra layer adds capacity the data doesn't need and doesn't reward, exactly Chapter 37's bias–variance lesson: past a certain point, more model flexibility increases variance without reducing bias, and the honest way to find that point is to test it, not to assume "deeper is better."

8.

```

torch.manual_seed(43)
long_network = nn.Sequential(
    nn.Linear(X_train.shape[1], 16),
    nn.ReLU(),
    nn.Linear(16, 8),
    nn.ReLU(),
    nn.Linear(8, 1),
)
long_optimizer = torch.optim.Adam(long_network.parameters(), lr=0.01)
for epoch in range(500):
    long_optimizer.zero_grad()
    long_loss = loss_fn3(long_network(Xtr_t), ytr_t)
    long_loss.backward()
    long_optimizer.step()
    if epoch % 100 == 0:
        with torch.no_grad():
            long_pred = torch.sigmoid(long_network(Xva_t)).numpy().ravel()
        print(
            f"epoch {epoch:>3}: training loss {long_loss.item():.4f}  "
            f"validation AUC {roc_auc_score(y_valid, long_pred):.3f}"
        )

```

```

epoch  0: training loss 0.6994   validation AUC 0.542
epoch 100: training loss 0.2373   validation AUC 0.762
epoch 200: training loss 0.2185   validation AUC 0.774
epoch 300: training loss 0.1922   validation AUC 0.760
epoch 400: training loss 0.1607   validation AUC 0.739

```

Validation AUC rises to a peak of 0.774 around epoch 200, then **declines** to 0.739 by epoch 400, even as the training loss keeps falling the whole time (0.699 → 0.161). That divergence — training loss still improving while validation performance gets worse — is the textbook signature of **overfitting**: the network is increasingly memorizing quirks of the 3,000 training rows rather than learning patterns that generalize. In practice this is managed with **early stopping** (stop training once validation performance stops improving) rather than picking a fixed epoch count in advance.

9.

```

small_indices = np.random.default_rng(43).choice(len(Xb_train), 200, replace=False)
Xb_small_t, yb_small_t = to_tensors(Xb_train[small_indices], yb_train[small_indices])
torch.manual_seed(43)
small_data_model = SmallCNN(n_classes=5)
small_optimizer = torch.optim.Adam(small_data_model.parameters(), lr=0.01)
for epoch in range(60):
    small_optimizer.zero_grad()
    small_loss = loss_fn4(small_data_model(Xb_small_t), yb_small_t)
    small_loss.backward()
    small_optimizer.step()
with torch.no_grad():
    small_accuracy = (
        (small_data_model(Xb_test_t).argmax(1) == yb_test_t).float().mean().item()
    )
print(
    f"200 training images: test accuracy {small_accuracy:.1%}  "
    f"(675 training images, chapter version: {base_accuracy:.1%})"
)

```

```

200 training images: test accuracy 96.9%   (675 training images, chapter version: 98.7%)

```


Test accuracy falls only modestly, from 98.7% with 675 images to 96.9% with 200 — a reminder that this is a genuinely easy task (small, clean, centered 8×8 digit images with only 5 classes), so even a substantial cut in training data costs relatively little. On harder, more realistic image problems (more classes, more visual variation, more noise), a comparable cut in training data typically costs far more, which is exactly why transfer learning and data augmentation matter more in practice than this particular toy comparison suggests.

10.

```
for n in [1, 50]:
    transfer_acc = train_transfer(n)
    scratch_acc = train_scratch(n)
    print(f"{n:>15}    {transfer_acc:>27.1%}    {scratch_acc:>13.1%}")
```

1	58.9%	59.8%
50	95.1%	98.2%

The pattern from section 43.7 (from-scratch pulling ahead as data grows) holds cleanly at **n=50**: 98.2% against transfer's 95.1%, the largest gap in favor of training from scratch across the whole sweep. At the opposite extreme, **n=1** (one example per class, five images total) doesn't show the clean reversal you might expect: the two methods are essentially tied (58.9% transfer against 59.8% from scratch), both well below their performance at every larger sample size. With a single example per class, both approaches are close to memorizing one exemplar per digit rather than learning anything general, and the result is noisy rather than a reliable signal either way. The honest lesson from the full sweep, n=1 through 50: **transfer learning's advantage is real but not guaranteed at every level of data scarcity** — it shows up clearly in the middle of the range (a handful to a few dozen examples per class) and gets noisier, not more clearly favorable, at the very extreme of almost no data at all.

11.

```
def train_transfer_partial(n_per_class, epochs=80):
    Xs, ys = small_training_set(n_per_class)
    model = copy.deepcopy(base_model)
    for param in model.conv1.parameters():
        param.requires_grad = False # conv1 stays frozen
    model.fc1 = nn.Linear(16 * 2 * 2, 32)
    model.fc2 = nn.Linear(32, 5)
    trainable = filter(lambda p: p.requires_grad, model.parameters())
    optimizer = torch.optim.Adam(trainable, lr=0.01)
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xn_test_t).argmax(1) == yn_test_t).float().mean().item()

partial_acc = train_transfer_partial(30)
print(
    f"n=30, conv2 unfrozen: {partial_acc:.1%}    "
    f"(fully frozen: 92.9%, from scratch: 98.2%)"
)
```


n=30, conv2 unfrozen: 97.8% (fully frozen: 92.9%, from scratch: 98.2%)

Unfreezing `conv2` and letting it adapt alongside the classifier head (97.8%) closes most of the gap between fully frozen features (92.9%) and training from scratch (98.2%), without quite matching training everything from nothing. Partial fine-tuning is a middle ground, letting the later, more task-specific convolutional features adjust while the earliest, most general edge-detecting features stay fixed. This is standard practice in real transfer learning: freeze early layers, fine-tune later ones, and how many layers to unfreeze is itself a choice to test, not a fixed rule.

12.

```

richer_mask = labels < 8 # digits 0-7 as the base task
X_rich, y_rich = images[richer_mask], labels[richer_mask]
Xr_train, Xr_test, yr_train, yr_test = train_test_split(
    X_rich, y_rich, test_size=0.25, random_state=43, stratify=y_rich
)
Xr_train_t, yr_train_t = to_tensors(Xr_train, yr_train)

torch.manual_seed(43)
richer_base = SmallCNN(n_classes=8)
richer_optimizer = torch.optim.Adam(richer_base.parameters(), lr=0.01)
for epoch in range(60):
    richer_optimizer.zero_grad()
    loss = loss_fn4(richer_base(Xr_train_t), yr_train_t)
    loss.backward()
    richer_optimizer.step()

final_mask = (labels == 8) | (labels == 9)
X_final, y_final = images[final_mask], labels[final_mask] - 8
Xf_train, Xf_test, yf_train, yf_test = train_test_split(
    X_final, y_final, test_size=0.25, random_state=43, stratify=y_final
)
Xf_test_t, yf_test_t = to_tensors(Xf_test, yf_test)

def transfer_from_richer_base(n_per_class, epochs=80):
    chosen = []
    for digit_class in [0, 1]:
        candidates = np.where(yf_train == digit_class)[0]
        chosen.extend(
            np.random.default_rng(43).choice(candidates, n_per_class, replace=False)
        )
    Xs, ys = to_tensors(Xf_train[chosen], yf_train[chosen])
    model = copy.deepcopy(richer_base)
    for param in model.conv1.parameters():
        param.requires_grad = False
    for param in model.conv2.parameters():
        param.requires_grad = False
    model.fc1 = nn.Linear(16 * 2 * 2, 32)
    model.fc2 = nn.Linear(32, 2)
    optimizer = torch.optim.Adam(
        filter(lambda p: p.requires_grad, model.parameters()), lr=0.01
    )
    for _ in range(epochs):
        optimizer.zero_grad()
        loss = loss_fn4(model(Xs), ys)
        loss.backward()
        optimizer.step()
    with torch.no_grad():
        return (model(Xf_test_t).argmax(1) == yf_test_t).float().mean().item()

print(
    f"transfer from an 8-class base (digits 0-7) to a 2-class task (8 vs 9), 10/class: "
    f"{transfer_from_richer_base(10):.1%}"
)

```

```

transfer from an 8-class base (digits 0-7) to a 2-class task (8 vs 9), 10/class: 93.3%

```

Transfer from the richer, 8-class base reaches 93.3% on the new 8-vs-9 task with only 10 examples per class — a strong result for a two-class problem with so little new-task data. The general principle behind why this can work at all is the same one behind real-world transfer learning: a base model

exposed to more classes and more total images during its own training tends to learn broader, more reusable early-layer features (edges, strokes, curves) than one trained on a narrower slice of the same kind of data. This toy comparison can't isolate that effect cleanly (an 8-vs-9 task and a 5-class-base task aren't directly comparable numbers), but the direction matches what section 43.7 already showed: what the base model actually learned, not the technique by itself, is what decides whether transfer learning pays off.

13.

```
torch.manual_seed(43)
x_data = torch.linspace(0, 10, 50).unsqueeze(1)
y_data = (
    3.0 * x_data + 7.0 + torch.randn(50, 1) * 0.5
) # true relationship: y = 3x + 7, plus noise

w = torch.zeros(1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
learning_rate = 0.01

for step in range(500):
    prediction = x_data * w + b
    loss = ((prediction - y_data) ** 2).mean()
    loss.backward()
    with torch.no_grad():
        w -= learning_rate * w.grad
        b -= learning_rate * b.grad
        w.grad.zero_()
        b.grad.zero_()

print(f"manual gradient descent: w={w.item():.3f}    b={b.item():.3f}")

from sklearn.linear_model import LinearRegression

sklearn_fit = LinearRegression().fit(x_data.numpy(), y_data.numpy())
print(
    f"scikit-learn LinearRegression: w={sklearn_fit.coef_[0]:.3f}"
    f"    b={sklearn_fit.intercept_[0]:.3f}"
)
```

```
manual gradient descent: w=3.102    b=6.382
scikit-learn LinearRegression: w=3.027    b=6.885
```

The manually written gradient descent loop ($w=3.102$, $b=6.382$) lands close to both the true generating values ($w=3$, $b=7$, before noise) and scikit-learn's exact least-squares solution ($w=3.027$, $b=6.885$) — close but not identical, because 500 steps of gradient descent at this learning rate hasn't fully converged to the exact minimum the way scikit-learn's direct solution does, the same gap Chapter 37 noted between an iterative and an exact solver. What matters for this chapter is the equivalence it confirms: a "neural network" with no hidden layer and no activation function is just linear regression trained by gradient descent instead of solved directly — precisely Chapter 37's linear regression, viewed through the PyTorch machinery this chapter introduces. `.grad.zero_()` matters here exactly as `optimizer.zero_grad()` did throughout the chapter: without it, gradients from each step would keep accumulating on top of the previous ones.

- 14.** "This chapter's own numbers say otherwise on tabular data: a small neural network scored an AUC of 0.773 on our churn data, barely ahead of plain logistic regression (0.764) and clearly behind tuned gradient boosting (0.788), while needing far more code and tuning than either. 'State of the art' is true for images, text, and audio at scale — it is not automatically true for a table of account features, and the only way to know which situation you're in is to test it, the same way we test everything else."
- 15.** "Imagine trying to sort four boxes into two piles using only one straight cut of a knife — some patterns of boxes simply can't be separated that way, no matter where you make the cut. A single artificial 'neuron' can only make one such straight cut. Adding a hidden layer is like being allowed to make two cuts and then combine the results, which lets the network handle patterns that no single straight cut ever could."
- 16.** "What was the base model actually trained on, and how similar is that data to product-defect photos?" This chapter's own experiment showed transfer learning's value depends entirely on how rich and relevant the base model's training data is — a base model trained on millions of *generic* photographs (cats, cars, landscapes) may transfer only partially to a specific industrial-defect task, and the only way to know is to test the vendor's actual model on a genuine held-out sample of Riverstone's own defect photos, exactly as Chapter 39 would insist for any other vendor model.

Chapter 44. Capstone: An End-to-End Data Science Project

Part IV — Machine Learning & Data Science

Chapter at a glance

You will learn to: walk one business question through the whole Part IV lifecycle — frame, prepare, model, evaluate, and communicate — without skipping a stage · combine a model's probability with the actual size of what's at stake, and see why ranking by probability alone can miss most of the value · wrap a multi-step analysis into one reusable function, the first step toward something you could actually hand off or schedule · write a one-page, non-technical summary that a leader can act on, and know what belongs in it and what doesn't.

Before you start: this chapter assumes everything from Chapters 35–43. It does not re-teach any of it; it uses it. If a step feels unfamiliar, the chapter says exactly which earlier chapter taught it, so you can go back rather than guess.

Time needed: 4–6 hours.

Tools: Python 3 with scikit-learn and shap — nothing new.

Practice data: Riverstone's customer accounts (Chapter 37); the same 5,000 accounts, the same churn target, the same pipeline. Every number in this chapter was calculated, and every output shown is real.

Why this matters

Every chapter in Part IV taught one skill in isolation: a split, a model, a metric, an explanation. Real work rarely arrives in those tidy pieces. Someone asks a vague question, the data needs assembling, a dozen small decisions have to be made in order, and at the end somebody who has never heard of a confusion matrix needs to know what to do differently on Monday morning.

This chapter is one worked pass through that whole arc, start to finish, on a single question: **which of Riverstone's at-risk accounts should the sales team call this month, and why those and not others?** Nothing here is a new technique. What's new is the order, the joins between stages, and one habit most tutorials skip entirely: turning a probability into a decision that accounts for how much is actually at stake.

In plain English

Imagine a doctor doing a batch of check-ups with a limited number of follow-up appointments to hand out.

A model that says "this patient has a 40% chance of a problem" is useful, but it's not yet a plan. Two patients might both sit at 40%: one whose problem, if missed, is minor, and one whose problem, if missed, is serious. Handing out follow-ups by risk percentage alone would treat those two patients identically. Handing them out by **risk times what's actually at stake** would not. That's the shift this chapter makes to Riverstone's churn model: a probability becomes a *decision* only once it's weighed against the size of the account behind it.

44.1 The question, and the plan

"Which at-risk accounts should the sales team call this month?"

That's a business request, not yet a data science question. Chapter 36 taught the five things to pin down before touching data; here they are for this project:

Decision	This project
Unit	One customer account
Target	Will this account place no order in 2025? (Chapter 37's <code>churned_2025</code>)
Prediction moment	As of 31 December 2024, using only that year's data
Success measure	Rupees of revenue protected, against the cost of the calls made
Capacity	The sales team can make 40 retention calls this month

The plan, in the order the rest of the chapter follows it:

1. **Refit** Chapter 37's churn model exactly as before — nothing new here, just a reminder of what it does.
 2. **Turn each account's probability into a rupee value:** how much revenue is genuinely at risk, not just how likely the account is to leave.
 3. **Build the call list** from that value, not from probability alone, and see how much it differs.
 4. **Explain** two accounts on the list well enough that a rep could open the call with something specific to say.
 5. **Wrap steps 1–3 into one function**, so the whole analysis can be re-run with one line instead of retyped by hand.
 6. **Write the one-page summary** a non-technical leader actually receives.
-

44.2 Refitting the model (a recap, not a re-teach)

```
import warnings

import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

warnings.filterwarnings("ignore")

accounts = pd.read_csv("../accounts/accounts.csv")
accounts["rep_id"] = accounts["rep_id"].astype(str)
CATS = ["segment", "city_tier", "company_size", "rep_id"]
NUMS = [
    "tenure_months",
    "orders_2024",
    "units_2024",
    "revenue_2024",
    "avg_discount_pct",
    "late_payment_days",
    "complaints_2024",
    "categories_bought",
    "days_since_last_order",
    "website_logins_2024",
    "catalog_downloads_2024",
]

rest, test = train_test_split(
    accounts, test_size=0.2, random_state=37, stratify=accounts["churned_2025"]
)
train, valid = train_test_split(
    rest, test_size=0.25, random_state=37, stratify=rest["churned_2025"]
)

prepare = ColumnTransformer(
    [
        ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
        (
            "num",
            Pipeline(
                [
                    ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                    ("scale", StandardScaler()),
                ]
            ),
            NUMS,
        ),
    ],
)

model = Pipeline(
    [
        ("prepare", prepare),
        (
            "model",
            HistGradientBoostingClassifier(
                learning_rate=0.05,
                max_depth=3,
                min_samples_leaf=40,
            )
        ),
    ]
)
```



```

        max_iter=300,
        random_state=37,
    ),
],
)
model.fit(train[CATS + NUMS], train["churned_2025"])
valid = valid.copy()
valid["p_churn"] = model.predict_proba(valid[CATS + NUMS])[:, 1]
print(f"{len(train):,} training accounts, {len(valid):,} validation accounts")
print(
    f"validation AUC: {roc_auc_score(valid['churned_2025'], valid['p_churn']):.3f}    "
    f"(Chapter 37's tuned boosting model, same pipeline and settings)"
)

```

```

3,000 training accounts, 1,000 validation accounts
validation AUC: 0.788    (Chapter 37's tuned boosting model, same pipeline and settings)

```

How it works: every line here is Chapter 37's, unchanged — the same `CATS` and `NUMS`, the same stratified split with `random_state=37`, the same `ColumnTransformer` (fill blanks, one-hot encode, scale), the same tuned `HistGradientBoostingClassifier`. If any of that reads unfamiliar, that's Chapter 37's material, not this chapter's; go back there before continuing. The only line worth pausing on is the last one: `valid["p_churn"] = ...` **adds a new column** to the validation table holding each account's predicted probability, so every later step can look the account up by row and immediately see its score alongside its other columns — a small habit that makes everything downstream far easier to read.

44.3 From probability to rupees

A probability alone doesn't say what to do with it. Riverstone loses far more by ignoring a large wholesale account with a modest churn risk than by ignoring a small retailer who's almost certain to leave. The fix is to multiply the model's probability by what the account is actually worth:

```

CALL_COST = (
    1200 # rupees: a retention call plus follow-up, roughly Chapter 39's lead-work cost
)
MARGIN = 0.15 # gross margin, same assumption Chapter 39 used

valid["value_at_risk"] = valid["p_churn"] * valid["revenue_2024"] * MARGIN
print(
    valid[["account_id", "segment", "revenue_2024", "p_churn", "value_at_risk"]]
    .head(3)
    .to_string(index=False)
)
print(
    f"\ntotal value at risk across all {len(valid):,} validation accounts: "
    f"${valid['value_at_risk'].sum():,.0f}"
)

```

account_id	segment	revenue_2024	p_churn	value_at_risk
7881	Retail	80400.0	0.058636	707.153017
8530	Hospitality	185500.0	0.053669	1493.335409
5016	Retail	264800.0	0.126821	5037.319525

total value at risk across all 1,000 validation accounts: ₹3,093,334

The plan in words, before the arithmetic below: for each account, take its churn probability, multiply by its 2024 revenue, then multiply by the share of that revenue Riverstone actually keeps as profit (its margin) — the result is the rupee value genuinely at risk if nothing is done.

value at risk = $P(\text{churn}) \times 2024 \text{ revenue} \times \text{margin}$

How it works:

- `CALL_COST` and `MARGIN` are named constants, not numbers buried in a formula — exactly Chapter 39's habit of writing business assumptions as variables a reader can see, question, and change.
- `valid["p_churn"] * valid["revenue_2024"] * MARGIN` multiplies three columns together, row by row: pandas does this for all 1,000 accounts in one line, the same **element-by-element arithmetic** Chapter 35 introduced with NumPy arrays.

Setting	What it means in plain words	Value used here	What happens if you change it
<code>CALL_COST</code>	Rupees to make one retention call and follow up	1,200	Higher: fewer accounts clear the break-even bar; lower: more do
<code>MARGIN</code>	Share of revenue that's actual profit, not just turnover	0.15 (15%)	Higher: every account's value at risk rises, so more calls look worthwhile; lower: fewer do

Reading the output. The third listed account, at ₹264,800 in 2024 revenue and a 12.7% churn probability, has ₹5,037 at risk — modest on its own, but multiply this across 1,000 accounts and Riverstone has just over ₹3 million of revenue genuinely on the line this year.

44.4 Building the call list: probability isn't the same as value

With only 40 calls available, which 40 accounts?

```

CAPACITY = 40 # the sales team's capacity: 40 retention calls this month

by_value = valid.sort_values("value_at_risk", ascending=False).head(CAPACITY)
by_probability = valid.sort_values("p_churn", ascending=False).head(CAPACITY)
overlap = len(set(by_value["account_id"]) & set(by_probability["account_id"]))

print(
    f"ranking by value at risk: top {CAPACITY} accounts hold "
    f"₹{by_value['value_at_risk'].sum():,.0f} of value at risk"
)
print(
    f"ranking by probability alone: top {CAPACITY} accounts hold "
    f"₹{by_probability['value_at_risk'].sum():,.0f} of value at risk"
)
print(f"the two lists share only {overlap} of {CAPACITY} accounts")

```

```

ranking by value at risk: top 40 accounts hold ₹1,156,056 of value at risk
ranking by probability alone: top 40 accounts hold ₹374,379 of value at risk
the two lists share only 10 of 40 accounts

```

Reading it. Ranking by value at risk captures **₹1,156,056**; ranking by churn probability alone captures only **₹374,379** — less than a third as much, from the same model and the same 40-account budget. The two lists barely overlap: only 10 of the 40 accounts appear on both. This isn't a modeling improvement; the model's predictions haven't changed at all. It's a **decision** improvement: a probability-only ranking spends the sales team's scarce time on accounts that are *likely* to leave regardless of how little they'd cost Riverstone, while missing large accounts whose departure would be expensive even at a lower probability. This is the single most important habit in this chapter: **a score is not a decision until it's weighed against what's actually at stake**, exactly as Chapter 39's cost-based thresholds argued, now applied where the value itself varies row by row instead of being the same for everyone.

Predict before you run this. Before reading the next section, guess: will the highest-value account on the list have a *high* or a *moderate* churn probability? Then look at block D's output.

44.5 Explaining two accounts

A call list with no reason attached is hard for a rep to use. Chapter 39 taught SHAP for exactly this:

```
import shap

feature_names = model.named_steps["prepare"].get_feature_names_out()
X_valid_prepared = model.named_steps["prepare"].transform(valid[CATS + NUMS])
explainer = shap.TreeExplainer(model.named_steps["model"])
shap_values = explainer.shap_values(X_valid_prepared)

top_account = by_value.iloc[0]
row_position = valid.index.get_loc(top_account.name)
contributions = pd.Series(shap_values[row_position], index=feature_names)
print(
    f"account {top_account['account_id']} ({top_account['segment']}, "
    f"₹{top_account['revenue_2024']:,.0f} revenue): predicted churn probability "
    f"{top_account['p_churn']:.1%}"
)
print(contributions.abs().sort_values(ascending=False).head(4).index.tolist())
print("largest pushes (positive = toward churning):")
print(
    contributions.reindex(
        contributions.abs().sort_values(ascending=False).head(4).index
    )
    .round(3)
    .to_string()
)
```

```
account 7001 (Wholesale, ₹7,966,300 revenue): predicted churn probability 8.4%
['num__late_payment_days', 'num__avg_discount_pct', 'num__units_2024', 'num__complaints_2024']
largest pushes (positive = toward churning):
num__late_payment_days    0.922
num__avg_discount_pct     0.760
num__units_2024           -0.673
num__complaints_2024      0.484
```

Reading it. The account at the very top of the call list — Wholesale, ₹7.97 million in 2024 revenue — has a churn probability of only 8.4%, well below what a probability-only ranking would flag, and it's on the list purely because 8.4% of nearly ₹8 million is still a large number. Its largest pushes toward churning are late payments and a high average discount; its units bought pulls the other way, toward staying. That's a specific opening line for the account manager: *"Their payment timing has slipped and their discount has crept up — worth checking whether something's changed on their side before it becomes a bigger problem."* An explanation doesn't need to be exhaustive to be useful; it needs to give the rep one true, specific thing to ask about.

44.6 One function, not six steps

Everything so far has been six separate steps, run once, by hand. The next stage of any real project is making it **repeatable**: one function that takes the raw data and settings, and returns the answer.

The plan in words: take the raw accounts table and three settings (the cost of a call, the margin, and how many calls the team can make), and return three things: the fitted model, the call list, and how good the model is — in one call, with nothing left to retype.

```

def build_call_list(accounts_df, call_cost=1200, margin=0.15, capacity=40):
    accounts_df = accounts_df.copy()
    accounts_df["rep_id"] = accounts_df["rep_id"].astype(str)
    rest_split, test_split = train_test_split(
        accounts_df,
        test_size=0.2,
        random_state=37,
        stratify=accounts_df["churned_2025"],
    )
    train_split, valid_split = train_test_split(
        rest_split, test_size=0.25, random_state=37, stratify=rest_split["churned_2025"]
    )
    prepare_step = ColumnTransformer(
        [
            ("cat", OneHotEncoder(handle_unknown="ignore"), CATS),
            (
                "num",
                Pipeline(
                    [
                        ("fill", SimpleImputer(strategy="median", add_indicator=True)),
                        ("scale", StandardScaler()),
                    ]
                ),
                NUMS,
            ),
        ],
    )
    fitted_model = Pipeline(
        [
            ("prepare", prepare_step),
            (
                "model",
                HistGradientBoostingClassifier(
                    learning_rate=0.05,
                    max_depth=3,
                    min_samples_leaf=40,
                    max_iter=300,
                    random_state=37,
                ),
            ),
        ],
    )
    fitted_model.fit(train_split[CATS + NUMS], train_split["churned_2025"])
    valid_split = valid_split.copy()
    valid_split["p_churn"] = fitted_model.predict_proba(valid_split[CATS + NUMS])[:, 1]
    valid_split["value_at_risk"] = (
        valid_split["p_churn"] * valid_split["revenue_2024"] * margin
    )
    call_list = valid_split.sort_values("value_at_risk", ascending=False).head(capacity)
    auc = roc_auc_score(valid_split["churned_2025"], valid_split["p_churn"])
    return fitted_model, call_list, auc

_, default_list, default_auc = build_call_list(accounts)
print(
    f"default run: AUC {default_auc:.3f}, call list of {len(default_list)} accounts, "
    f"₹{default_list['value_at_risk'].sum():.0f} value at risk"
)

```

```
default run: AUC 0.788, call list of 40 accounts, ₹1,156,056 value at risk
```

How it works, line by line:

- `def build_call_list(accounts_df, call_cost=1200, margin=0.15, capacity=40):` declares a function with three settings given **default values**, so it can be called with no arguments at all (using the defaults) or with any of them overridden.
- The next eight lines are sections 44.2's model-fitting steps, unchanged, just renamed with a `_split` suffix so they don't collide with the `train / valid` names already used earlier in this chapter — a real concern once code moves from a notebook's scattered cells into a function of its own.
- `valid_split["value_at_risk"] = ...` repeats section 44.3's calculation, now using the function's own `margin` argument instead of a fixed constant, which is what makes it changeable from outside.
- `.sort_values("value_at_risk", ascending=False).head(capacity)` repeats section 44.4's ranking, using the function's `capacity` argument.
- `return fitted_model, call_list, auc` sends back three separate values in one line; calling code unpacks them in the same order, as the next line shows.

```
for capacity_setting in [20, 40, 80]:
    _, call_list, _ = build_call_list(accounts, capacity=capacity_setting)
    print(
        f"capacity={capacity_setting:>3}: total value at risk on the list "
        f"₹{call_list['value_at_risk'].sum():>10,.0f}"
    )
```

```
capacity= 20: total value at risk on the list ₹   861,828
capacity= 40: total value at risk on the list ₹ 1,156,056
capacity= 80: total value at risk on the list ₹ 1,519,267
```

Reading it. Calling the same function three times, changing only `capacity`, shows the trade-off directly: doubling the team's capacity from 40 to 80 calls roughly captures more value (₹1.52 million against ₹1.16 million), but not twice as much, because the highest-value accounts were already included at capacity 40 — the marginal 41st-through-80th accounts are worth less each. This is the kind of question a sales head actually asks ("what if we gave you ten more hours a week?"), and now it's one line to answer rather than a rebuilt analysis.

Changed-line exercise. In block F's loop, change `[20, 40, 80]` to `[40, 200]`. Before running it, predict whether the value captured at capacity 200 will be closer to double or closer to the same as capacity 80's result, and why — then check.

44.7 The one-page summary

Everything above is for you. What a leader receives is shorter, and every number in it must trace back to something already computed:

```

final_model, final_call_list, final_auc = build_call_list(accounts, capacity=40)
segment_counts = final_call_list["segment"].value_counts()
print(f"model quality: AUC {final_auc:.3f}")
print(
    f"call list: {len(final_call_list)} accounts, "
    f"₹{final_call_list['value_at_risk'].sum():,.0f} total value at risk"
)
print("by segment:", segment_counts.to_dict())
print(
    f"median revenue on the list: ₹{final_call_list['revenue_2024'].median():,.0f}    "
    f"(all validation accounts: ₹{valid['revenue_2024'].median():,.0f})"
)

```

```

model quality: AUC 0.788
call list: 40 accounts, ₹1,156,056 total value at risk
by segment: {'Wholesale': 17, 'Retail': 13, 'Hospitality': 10}
median revenue on the list: ₹770,300    (all validation accounts: ₹222,850)

```

That table of numbers becomes a page of prose, not a slide of metrics:

To: Anita Rao, Sales Head From: Analytics Re: February 2026 retention call list

The churn model correctly separates accounts likely to leave from accounts likely to stay about 79% of the time when compared pairwise — a solid, usable model, not a perfect one.

We've built this month's 40-account call list by combining each account's churn risk with its actual revenue, not risk alone. Together, these 40 accounts represent about **₹1.16 million** of revenue we'd expect to lose if nothing changes. Ranking by risk alone, ignoring account size, would have pointed the team at accounts worth only **₹374,000** combined — the same number of calls, less than a third of the value protected.

The list skews toward larger accounts: its median revenue is **₹770,300**, well above the typical validation account's **₹222,850**. That's expected and intended — this list exists specifically to catch valuable accounts a simple risk ranking would miss — but it does mean the team's smaller, high-risk accounts need a separate, lower-touch process (an email campaign, say) rather than a personal call.

By segment, the list is 17 Wholesale, 13 Retail, and 10 Hospitality accounts. Each account manager will get their own accounts flagged with the two or three factors driving that account's risk, so every call opens with something specific rather than a generic "how's everything going?"

What this list is not: proof that calling these accounts will retain them, only an estimate of what's at risk if nobody does. Whether calls actually change the outcome is a question for a proper test (Chapter 30), which we'd recommend running on a sample before scaling this to every account every month.

Notice what's absent: no AUC in the second paragraph (it's mentioned once, plainly, and never again), no code, no confusion matrix, no mention of gradient boosting or SHAP by name. The reader needs to know what to do and how confident to be in it, not how it was built. The **limitations paragraph is not optional** — a summary that only reports the upside is the kind of overclaiming this book has argued against since Chapter 1.

44.8 The Part IV toolkit, applied

Every chapter in this part shows up somewhere in the six steps above:

Chapter	What it contributed here
35	The dot products and gradients inside the model; the log-loss thinking behind evaluating probabilities
36	The framing table, the split, the pipeline, the leakage discipline in CATS / NUMS
37	The tuned gradient-boosting model itself, unchanged
38	The idea that a business decision (which accounts to prioritize) doesn't always need a predictive target — useful for the low-touch process the summary recommends for smaller accounts
39	Cost-based thresholds, generalized here to a value that varies per row, and SHAP for individual explanations
40	The habit of testing an assumption (capacity, cost) with a real re-run rather than guessing at the effect
41	Not used directly here, but the same "recompute honestly, don't assume" discipline applied to text elsewhere
42	The same ranking-and-evaluation mindset (precision at a fixed list length) reused for a call list instead of a product list
43	The comparison discipline: before reaching for anything fancier, this project stayed with Chapter 37's gradient-boosting model because nothing in this part gave a reason to change it

Nothing here required a new algorithm. It required using the ones already learned, in the right order, without skipping the step that turns a number into a decision.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Ranking purely by model probability	Large, moderately-at-risk accounts never make the list	Multiply probability by what's actually at stake
Treating the call list as proof calls work	"We'll definitely save this revenue"	State plainly that this is an estimate of risk, not a measured effect (Chapter 30)
A summary full of model metrics	The reader can't find the actual recommendation	Lead with the decision; mention the metric once
No cost or capacity constraint	A list of thousands, useless to a team with 40 hours	Always tie the list to a real capacity or a real cost
Copy-pasting the same six steps for every re-run	Small differences creep in between runs	Wrap the steps in one function, as section 44.6 does
No limitations section	Leadership overtrusts the list	Always state what the model does not prove

In the real world: what happened to the call list

Anita takes February's list to her three account managers. Farah, given her share of the Wholesale accounts, questions one immediately: a large account with a modest churn probability but a high value-at-risk score. "They're not going anywhere," she says. "They just haven't ordered as much this quarter because of a plant shutdown I already know about."

That's not a flaw in the list; it's exactly why the list is a starting point for a conversation, not a verdict. The model has no way to know about a plant shutdown it was never shown. Farah's local knowledge and the model's account-wide pattern-matching are doing different jobs, and the useful outcome is combining them — she still makes the call, but she opens it differently than the model's SHAP explanation would have suggested. Two months later, when Meera reviews which accounts on the list actually reordered, that's the first real evidence of whether calling changes anything at all, and it's the beginning of the proper test the summary recommended.

Tools

Nothing new. Every tool used in this chapter — scikit-learn's pipeline and gradient boosting, shap's `TreeExplainer` — was introduced and explained in Chapters 36, 37, and 39.

The project: your own end-to-end pass

Goal: run this chapter's six-step lifecycle on a question of your own, ending in a one-page summary someone outside data science could act on.

Steps:

1. Pick a question with a real decision behind it and a real constraint (a budget, a headcount, a number of hours) — not just "predict X."
 2. Frame it with Chapter 36's table: unit, target, prediction moment, success measure, capacity.
 3. Reuse a model you've already built in this part rather than building a new one from scratch.
 4. Turn its output into a value, not just a probability or a score.
 5. Wrap the steps into one function, and show one "what if we changed the capacity/cost" result with real numbers.
 6. Explain two individual cases well enough that someone could act on them immediately.
 7. Write the one-page summary, including a limitations paragraph that isn't an afterthought.
-

You've got it when...

- ☐ I can name, for any model I've built, what decision it actually feeds — not just what it predicts.

- ☐ I never rank purely by probability when the things being ranked have different value.
 - ☐ I can wrap a multi-step analysis into one function with sensible defaults.
 - ☐ I write a one-page summary that leads with the decision, not the metric, and always states a limitation.
 - ☐ I can point to which earlier chapter taught any given step in my own pipeline.
-

Recap

- A **probability is not a decision**; multiplying it by what's genuinely at stake (a value at risk) can completely reorder a priority list, as it did here: three times the value protected, from the same model.
 - **Capacity and cost constraints** belong in the analysis from the start, not bolted on afterward.
 - **Wrapping repeated steps in one function** is the first move toward something reusable, schedulable, or handed to someone else.
 - A **one-page summary** leads with the decision, states the model's quality once, and always includes what the analysis does not prove.
 - An end-to-end project is not a new skill; it's the discipline of using Chapters 35–43's tools in the right order and not skipping the step that turns a number into an action.
-

Practice exercises

1. (*hand*) An account has a 30% churn probability and ₹500,000 in 2024 revenue. At a 15% margin, what's its value at risk? Is it worth a ₹1,200 call by the per-account break-even rule from section 44.3?
 2. Rebuild the call list with `MARGIN` set to 0.25 instead of 0.15. Does the same set of 40 accounts appear, or does the ranking change? Why might margin differ by segment in reality, and what would that do to the list?
 3. Using `build_call_list`, produce a call list with `capacity=40` and `call_cost=15000`. How many of the 40 accounts now fail the per-account break-even test from section 44.3, even though they're still in the top 40 by value at risk? What does that tell you about the difference between *ranking* by value and *filtering* by break-even?
 4. Write the one-paragraph "what this list is not" limitation section for your own project from this chapter's project brief. What is the one thing a reader might wrongly conclude from your summary if you left that paragraph out?
-

Key terms

value at risk · break-even per row · capacity constraint · reusable pipeline function · default argument · executive summary · limitations statement

(All terms are defined in the Glossary, Appendix A.)

Where this leads

This chapter closes Part IV. Part V moves from single models to the systems and habits that keep them honest and useful over time: business metrics that connect a model's output to money (Chapter 45), experiments that prove whether an intervention like a retention call actually works (Chapter 30, referenced throughout this chapter), and deploying and monitoring models once they leave a notebook for good (Chapter 52). The call list this chapter built is exactly the kind of thing those later chapters take further: from a one-off analysis to a system that runs every month, is tested rather than assumed to work, and is watched for when it starts to drift.

Answers to practice exercises

1. value at risk = $0.30 \times ₹500,000 \times 0.15 = ₹22,500$. Since ₹22,500 is well above the ₹1,200 call cost, yes — by a wide margin, this account clears the per-account break-even test from section 44.3.

2.

```
_, list_margin_25, _ = build_call_list(accounts, margin=0.25)
_, list_margin_15, _ = build_call_list(accounts, margin=0.15)
same_accounts = set(list_margin_25["account_id"]) == set(list_margin_15["account_id"])
print(f"same 40 accounts at margin 0.25 and 0.15? {same_accounts}")
```

```
same 40 accounts at margin 0.25 and 0.15? True
```

The exact same 40 accounts appear either way. Margin multiplies every account's value at risk by the same fixed number, so it stretches or shrinks the whole scale uniformly without changing anyone's position relative to anyone else — the **ranking** is unaffected, even though the raw rupee figures reported in the summary would change. In reality, margin genuinely differs by segment (Wholesale typically runs thinner margins than Hospitality, say): using one fixed margin for every account, as this chapter does for simplicity, would then distort the ranking, favoring segments whose margin is understated and underweighting those whose margin is overstated. A more careful version would use each account's own segment-level margin rather than one number for everyone.

3.

```
_, list_high_cost, _ = build_call_list(accounts, call_cost=15000, capacity=40)
below_breakeven = (list_high_cost["value_at_risk"] < 15000).sum()
print(
    f"accounts in the top 40 that fail a ₹15,000 break-even test: {below_breakeven} of 40"
)
print(
    f"lowest value at risk on this list: ₹{list_high_cost['value_at_risk'].min():,.0f}"
)
```

```
accounts in the top 40 that fail a ₹15,000 break-even test: 13 of 40
lowest value at risk on this list: ₹11,770
```

At a ₹15,000 call cost, 13 of the top 40 accounts by value at risk would actually **lose** Riverstone money if called — their value at risk doesn't clear the higher cost, even though they still rank in the top 40 overall. This is the key distinction the exercise is after: **ranking** by value at risk answers "which accounts matter most, relative to each other?", while **filtering** by break-even answers a different question, "which accounts are worth acting on at all, given what it costs to act?" A sensible call list, once the cost is this high, should apply both: rank first, then drop anyone below break-even, rather than assuming the top N by rank are automatically worth calling.

4. (No single correct answer — this asks you to write your own project's limitations paragraph.)

Whatever the specific project, the paragraph should distinguish between what the analysis shows (a risk estimate, a ranking, a pattern in past data) and what it does not prove (that acting on it changes the outcome, that the pattern will hold as conditions change, that no other factor explains what the model found). Without it, the most common wrong conclusion a reader draws is treating a **correlational risk score** as if it were a **measured effect of taking action** — exactly the gap this chapter's own summary calls out in its final paragraph, and exactly what a proper experiment (Chapter 30) is needed to close.